TBdecompile

Reference Manual

ToolBook II 8.1

27 October 2026

v0.0.11

The record, the lookups, and the command cards.

One file, and the same book in parts. No part reaches 100 pages.

`who-is-who` is not in this volume.

Table of contents

Date: 27 October 2026

Print this sheet on its own. It is not inside the contents file. The chapter list is the record on disk. The page counts are the PDFs that already exist. If those two disagree, the record is ahead of the print, and this sheet is the one to replace.

Piece	File	Pages
Cover	`cover.pdf`	1
Table of contents	`toc.pdf`	2
Preface	`preface.pdf`	1
Foreword	`foreword.pdf`	1
Contents	`contents.pdf`	58

Inside the contents

These titles are the first heading of each file in `TBdecompile/record/`, in order. Their pages inside the contents file are not on this sheet. Counting those pages means typesetting the contents.

- How to read this record
- 1. The corpus
- 2. File formats
- 3. Runtime
- 4. Objects, file open, and the reader
- 5. Graphics
- 6. OpenScript
- 7. Debugging and hosts
- 8. Naming
- 9. The modern stack
- 10. Open, retired, and superseded
- 11. Keepers
- 12. The book-open spine
- 13. Specimen pages
- 14. OpenScript bytes
- 15. System books and the NIDA shell
- 16. The password cipher
- 17. Modules
- 18. Where every loose sheet went
- 19. Heap, stack, and the resource manager
- 20. CDB and OBJ
- 21. What the stress run broke
- 22. Value, VBX, and EXECSCRIPT

Appendices

Appendix	File	Pages
Appendix A — Export and Ordinal Maps	`appendix-a-exports.pdf`	11
Appendix B — Architecture and Gaps	`appendix-b-architecture.pdf`	6
Appendix C — TBX File Format	`appendix-c-tbx.pdf`	4

Appendix	File	Pages
Appendix D — Address Maps	`appendix-d-maps.pdf`	41
Appendix E — Numeric and System Tables	`appendix-e-tables.pdf`	5
Appendix F — Confirmed Function Names	`appendix-f-names.pdf`	3
Appendix G — SoftICE	`appendix-g-softice.pdf`	4
Appendix H — The Watcom Debugger	`appendix-h-watcom.pdf`	3
Appendix I — Reading the Assembly	`appendix-i-assembly.pdf`	4
Appendix J — OpenScript	`appendix-j-openscript.pdf`	5
Appendix K — Ghidra	`appendix-k-ghidra.pdf`	20
Appendix L — Bochs	`appendix-l-bochs.pdf`	3

Preface

Date: 27 October 2026

`TBdecompile-v0.0.11.pdf` is the whole book, in this order: cover, table of contents, preface, foreword, contents, then appendices A through L.

Four sheets sit in front of the contents. Each one is its own file, so any one of them can be printed without the others.

Sheet	File
Cover	`cover.pdf`
Table of contents	`toc.pdf`
Preface	`preface.pdf`
Foreword	`foreword.pdf`

The table of contents is the sheet that changes first. It lists the files in the edition and, inside the contents, the chapter titles taken from the record. It does not contain the chapters. Reprint it on its own when a chapter or an appendix has been added and the contents file has not been typeset yet. Replacing that sheet does not require printing the record again.

The contents file is the files in `record/`, copied whole, in order, beginning with the reading guide. A chapter was not shortened to fit a page, and it was not cut in the middle of a paragraph. The reading guide is part of the contents. It is not a second preface. The grades (Known, Thought, Conjecture, Not true, Superseded) are defined there.

If the contents do not fit under 100 pages, the break goes between chapters. The same rule applies to an appendix: the break goes between headings, and no part reaches 100 pages.

Appendices are not in the contents file. Each one is its own document. None of them is a slice of the narrative, and the narrative is not a slice of them.

File	What it holds
Appendix A	BAS, RTM, and W16 export maps, and the BAS ordinal map. Copied whole from the 12 October map notes.
Appendix B	Architecture and the gaps. Replaces the 15 October architecture appendix.
Appendix C	TBX, against the exported specimen. Replaces the 15 October TBX appendix.
Appendix D	NE-segment maps, one boot's WDW selectors, and the 20 October Bochs selectors. Each of those files, complete.
Appendix E	Decimal/hex 0–255, ASCII 0–127, and the Win16 KERNEL/USER/GDI ordinals from the 15 October lookups.
Appendix F	Confirmed function names, the 15 October table, carried forward whole.
Appendix G	SoftICE. Commands, the screen, and the book-open read from the night of 15 October.
Appendix H	The Watcom debugger. Commands, the screen, and what the three export traces did not find.
Appendix I	Assembly, read beside the Ghidra C, so a listing is not trusted just because it compiled into C.
Appendix J	OpenScript. The byte classes, the two decoder bugs, the handlers that expand, and what the player does not do.
Appendix K	How to disassemble. How Ghidra turns seeds into instructions, how a name differs from a purpose and an operation, and the passes this project uses to finish a listing.
Appendix L	Bochs. The command card for the host that opened the lesson on 20 October.

When this manual and `record/` disagree, the record wins.

Foreword

Date: 27 October 2026

This volume is the current statement on the ToolBook II 8.1 decompile. It replaces the 15 October 2026 operational notes. Those notes stay on disk under `trail/20261012/` and in the 15 October PDF. They are the trail. They are not a second current text.

The 15 October contents put `CDBSENDCREATE` (BAS ordinal 51) on the path that builds objects while a book opens. That placement is not true. Ordinal 51 is the Instructor create and rewrite path. A lesson open does not call it. Chapter 4 is the correction. Appendix B draws it.

The 15 October OpenScript note copied back-reference bytes `0xC0`–`0xFF` from the output stream and decoded the keyword table to the wrong words. The copy is from the compressed input. Chapter 14 is that encoding.

Absence from a Ghidra HTML export is not absence from the binary. The listings have undisassembled runs. Appendix K is how those listings are finished and then made readable. It does not name the record reader.

This print includes chapters 19 through 22 and Appendix L. Chapter 19 is the NearScript heap, the value stack, and `RESMGR*`. None of those is the `0x76`-byte object array. Chapter 20 is the split between `CDB*` and `OBJ*`. They are not a header and its definitions. Chapter 21 is the stress run of the Python that models those layers. Nested `step` and a reused heap index were the breaks. It is not a timing of `TB80BAS`. Chapter 22 is the value bus, a VBX control, and `EXECSCRIPT`. A VBX is a Visual Basic extension control. It is not the page object, and neither of the other two is a VBX. Appendix L is the Bochs card from 20 October.

The 16 October contents file that stopped at chapter 18 is not this edition. The table of contents is a separate sheet. The chapter titles on it are the record. The page counts are the PDFs in this edition.

The 15 October operational notes were compared with this volume. What that short file still had, and this volume had thinned, is in the contents: the author-path dispatch and the property-id list (chapter 4), the graphic property names (chapters 2 and 5), and the SoftICE standing orders (chapter 7, with the command cards in Appendices G, H, and I). What it got wrong is not put back. The unsifted symbol-map dumps are not put back either.

Chapter 16.5 is the author session in which `sssssssssss.tbk` became password-protected and `TBCBT` did not load. `nc81base.sbk` was refused. Writing that password is not a load of the cipher DLL. Chapter 15.7 records that the ToolBook 11.5 files which use the Instructor 8.1 system-book names are stubs.

TBWorldview is its own binder. It is not reprinted here. Where that binder still draws ordinal 51 on book open, this volume and Appendix B win.

`who-is-who` is not in this volume.

How to read this record

Date: 2026-10-16

Status: This directory is the official statement.

`TBdecompile/record/` is the consolidated record of the ToolBook II 8.1 work. The loose sheets under `tbworkingpapers/tbwpNNNN/` are the trail those findings were written on. They are not a second current record. When a loose sheet and this directory disagree, this directory wins. Chapter 18 names every loose sheet and the section that absorbed it.

The dated folders `trail/20261012/`, `trail/20261014/`, and `trail/20261015/` stay as the notes of those days. `trail/20261016/00-consolidated.md` is the short correction this volume expands. Read those folders for the trail. Do not read `trail/20261015/00-how-toolbook-operates.md` section 2, or `trail/20261015/01-theories-of-operation.md` section 1, or `trail/20261014/02-still-open.md` opening paragraph, as the runtime create path.

Grades

Grade	Meaning
Known	Seen in a binary, a live trace, or a matching export. A later observation would have to contradict the evidence, not just a reading of a listing.
Thought	Fits the evidence and is the working model. Not yet watched end to end.
Conjecture	Plausible. Not tested.
Not true	Was a working idea. Later evidence closed it. Do not search it again.
Superseded	An older note still says this. The later note wins. The older path is listed so it is not reopened by accident.

Order

File	Subject
01-corpus.md	What the books are, and which machines can run them
02-formats.md	`.tbk`, `.sbk`, `.tbx`, `.fbk`
03-runtime.md	Modules, launch seam, NIDA system layer
04-objects.md	Live object, file open, the unnamed reader
05-graphics.md	Painters and what is stored versus computed
06-openscript.md	Bytecode, handlers, the modern subset
07-debugging.md	SoftICE, WDW, Unicorn, Bochs, where the cipher lives
08-naming.md	What has been named, and what a name is worth
09-modern.md	Foolbook, Foolcourse, Foolversity, TBpipeline
10-open.md	Ranked gaps, and the notes that are closed
11-keepers.md	One-boot live addresses and decompile line numbers
12-wdw-spine.md	The book-open spine and what the three traces hit
13-specimen-pages.md	P1040, P430, P440 inside that record, P420, P1170, and the TBX draw inventory
14-openscript-bytes.md	Encoding, the back-reference fix, corpus counts
15-system-books.md	`.sbk` tiers, NIDA shell, type-name table
16-cipher.md	TBCBT segment 7, the keyed path, the empty-key path, and the author write that does not load TBCBT
17-modules.md	Module roles, including the ones that are not the reader
18-absorption.md	Every loose sheet, and where its claim went
19-heap-stack-resmgr.md	NearScript heap, the value stack, `RESMGR*`, and the callers the breakpass did not name
20-cdb-obj.md	`CDB*` is the book API. `OBJ*` is the object file and object memory. Not a header and its definitions.
21-stress.md	Stress of the Python models. Not a timing of `TB80BAS`.

File	Subject
22-value-vbx-execscript.md	The value bus, a VBX control, and `EXECSCRIPT`. Three mechanisms. None of them is the `0x76` page object.

Lookup tables that are too long to typeset live in `maps/`:

File	What it is
maps/rtm-ne-to-ghidra.md	TB80RTM NE segment to Ghidra space
maps/bas-ne-to-ghidra.md	TB80BAS NE segment to Ghidra space
maps/wdw-to-ghidra.md	One boot's WDW selector:offset against the decompile
maps/bochs-20261020.md	One boot's Bochs selectors. Dead after the next reboot.

NE segment numbers are not Ghidra `CodeN` names. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps. Live selectors change every boot. Offsets inside a segment do not.

What this record will not hold

Binary dumps, SoftICE console logs, and personal correspondence stay out of this repository. Names of correspondents are masked. The correspondent already used in the public notes is JohnDoe1. `who-is-who` is not part of this record.

Which older sentence to trust

If a note written on or before 2026-10-15 puts `CDBSENDCREATE` between file open and the first painted object, that sentence is superseded. The correction is chapter 4. Instructor calls ordinal 51 when it creates or rewrites an object. A lesson open does not.

If `tbworkingpapers/tbwp0001/openscript_notes.md` says the keyword table decodes to "italic" or "systime", that sentence is superseded. The bug was copying bytes `0xC0`–`0xFF` from the output stream. Chapter 14 is the encoding that actually decodes the handlers.

Absence from a Ghidra HTML export is not absence from the binary. The listings have undisassembled runs. A candidate is closed only when the disassembly or a live trace shows what the function actually does.

If a note says `HEAP*` or `TBHEAP*` allocates the `0x76`-byte object, or that `TBHEAPCLEARCRITSECT` has no definition, that sentence is superseded. Chapter 19.

The printable form of this directory is TBdecompile-v0.0.11.pdf. That file is the whole book. The same book is also the separate sheets in manual/20261027/: front matter, the contents, and one file per appendix. A sheet that would reach 100 pages is broken between headings. `python3 manual/build.py` reprints them.

1. The corpus

Claims

Id	Grade	Statement	Source
C-001	Known	The lesson files in this work are Nida Corporation electronics lessons, written as Click2learn ToolBook II 8.1 shells. The book version word is `01 00 00 08`. This Nida is not the drug-abuse institute that shares the letters.	TBWorldview WV-001

Id	Grade	Statement	Source
C-002	Known	The catalog is a Navy ATT menu export (`nida:m3search`): menu `0_MASTER3` plus 20 rating tracks. 611 `.tbk` files are on disk. 290 catalog ids are unique. 321 files are not in the catalog.	courses listing, project notes 2026-10-08
C-003	Known	The worked specimen is Logic Families, `9e954d01.tbk`. It is not self-contained. It names the NIDA system books and jumps into `nida81.tbk`.	lesson header at file `0xB1A96`
C-004	Known	Windows 98 can step the engine. 32-bit Windows XP can run it. 64-bit Windows cannot run the 16-bit NE images. Windows 11 is outside the range.	WV-028
C-005	Known	The engine calls 16-bit `USER`, `GDI`, and `KERNEL`. A startup log that reports `USER32`, `GDI32`, and `KERNEL32` missing is not a broken install.	WV-027
C-006	Not true	ToolBook 8.1, as held here, is a 32-bit product that needs Windows 10 or earlier. The images in hand are 16-bit NE. ToolBook 9.0 is the 32-bit product.	`TBdecompile/trail/20261014/03-tbx.md`

What the set is

The books are computer-aided instruction for Navy electronics ratings. The menu export lists tracks such as GSE, EM, IC, GM, FC, ET, STG, AE, ATI, AO, ATO, AS, TM, CTM, FT, ETSS, STS, CTT, MN, and MV, at versions V03.2 through V03.4. `courses.xml` is that menu. It is not a drug-abuse curriculum.

A lesson is a ToolBook II 8.1 book plus a shell. The shell is `nida81.tbk`. The system books are `nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`, and `NC81BANK.SBK`, with `nc81int.dll` and `nc81rec.dll` beside them. Removing the system books leaves a blue screen, breaks resize, and stops the menu. The lesson file itself still opens. Renaming `nida81.tbk` removes the splash and reports that the shell is not a book. The lesson can still be entered.

`9e954d01.tbk` is the book every later measurement is checked against. It is Logic Families. At file offset `0xB1A96` it names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and it calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`.

Who used it

Known. The published users of this particular set are the Navy lessons on disk and the training shops that shipped ToolBook courses. This is not a living authoring market. Someone outside the set who still holds a native 8.x book has the same file problem. That last sentence is a thought, not a census (WV-030).

Machines

The floor for a real run is Windows 98. That is the known-good debug host, not the top of the range. The ceiling is 32-bit Windows XP. The 16-bit NE images do not run on 64-bit Windows. A correspondent described 8.1 as 32-bit and 9.0 as the release that added 64-bit support. The binaries in this archive are 16-bit NE. Until a 32-bit 8.1 engine is actually in hand, the binaries win (C-006).

SoftICE work is closed as of 11 October 2026. Later tracing used the Watcom debugger on a fresh Windows 98 image. Those two debuggers do not share commands. See chapter 7.

2. File formats

Claims

Id	Grade	Statement	Source
F-001	Known	A lesson begins with a `0xC00` PE stub that has no useful code. The book stream is marked by `ZZZTBKBOOKLOC` and magic `0x22222222`, then `03 4A 42 4F` and version `01 00 00 08`.	WV-002
F-002	Known	Page records are the only objects that start `80 65 00 05 00 02 02`. Widgets inside a page start `14 00 00`.	WV-003
F-003	Known	`.tbk` and `.sbk` are the same container. The page name sits at offset `0x0B` of the page record.	WV-031
F-004	Known	Field text is a `Text\0\0` property: 8 bytes, a 16-bit length, then the string. The string often starts with CRLF.	page reader notes
F-005	Known	A ToolBook 11.5 Save To XML, enabled by `saveSpecial=true`, writes a folder of XML plus resources. A `.tbkx` file is a ZIP of that folder.	`TBdecompile/trail/20261014/03-tbx.md`
F-006	Known	Binary 8.1 stores many draw objects as bounds. The 11.5 XML expands those bounds into Vertices lists.	P1040 pin bounds `5055,4605,5205,4860` match the TBX Vertices start
F-007	Known	Foolbook `.fbk` / `.fsk` is a 4-byte magic (`FBK1` / `FSK1`), one flags byte, then JSON. Bit 0 means the payload is zlib-compressed.	Foolbook format notes

2.1 Native book

`tbload.exe`, through `MINSTB32.dll`, searches for `ZZZTBKBOOKLOC`. The signature check is a virtual call, not a literal compare sitting in `tbload`. On `9e954d01.tbk` the JBO body is at file offset `0xC00`. A later open measurement put the same book's first useful read at `0x9EC`. Both figures are about the same file. `0xC00` is the stub length. `0x9EC` is where one host's stage-2 open landed. Do not treat them as two different formats.

A page record holds a 32-byte name, usually `P` plus digits, then a property block. The property block at +43 of the early notes starts `14 00 00 01`. Widgets are not separate top-level objects on disk. They are length-prefixed chunks nested in the page.

Known. An ascending table of words inside a page, such as the 27 words at page `+0x8C3` on P1170, is an offset directory into a `14 00 00 83` content blob. It is not a Group. A Group is its own object and has its own member list.

Not true (WV-010). On-disk `14 00 00 01` does not become `14 00 00 11` when the book loads. One early dump was read that way. Later live objects used `14 00 00 02`. `9e954d01.tbk` has both `01` and `11` on disk. `93950101.tbk` is the other way around. That is a file difference, not a load-time mutation.

Property ids recovered from `TB80BAS` resource type `0x2D1`, id `0x2D0`, are `0x4000` plus a slot:

Id	Name
`0x4004`	bounds
`0x4015`	idNumber
`0x4023`	script
`0x402E`	text
`0x4034`	vertices
`0x4050`	Point-array setter. Runtime only. Not a record marker on disk.
`0x4059`	layer
`0x4097`	normalGraphic
`0x4098`	invertGraphic
`0x4099`	disabledGraphic
`0x409A`	stretchGraphic

Id	Name
0x409B	checkedGraphic

The graphic ids are the names on a Button in the TBX specimen (`bitmap id N`) and in the property resource. `N` is a resource index, not a file offset. Which bytes inside a `14 00 00` chunk hold `0x4097` is still open. An early carve that reported 641 BMP headers is not this binding. The count used against the specimen is about 75 BMPs in the export, and 67 markers paired with 72 of them.

2.2 System books

`.sbk` files use the same container. The NIDA `.sbk` files are resource containers: the object-type name table (ComboBox, Stage, Picture, Hotword, RecordField, PaintObject, Curve, AngledLine, Pie, Arc, IrregularPolygon, Polygon, Line, RoundedRectangle, Ellipse, Group, Field, Button, Rectangle, then Page, Background, Book) and menu resources. They are password-protected. The stored ciphertext and the empty-key decode are chapter 16.6. The decompile does not need the password to read the records.

Thought, narrowed. A search for the plain words `to handle` finds no OpenScript in those `.sbk` files. A later reader found compressed handler streams in `tb80r.sbk` (33 handlers), stored as keyword opcodes rather than the words `to handle`. “No script at all” is too strong. What those streams do when a lesson runs has not been traced. `nc81base`, `nc81recs`, `nc81test`, and `nc81trnr` still have no decoded `to handle` text.

`nida81.tbk` is a real book, not a resource bag. It has 43 objects and pages `P20000` through `P22130`. It does not name lesson ids. It has 100 `buttonClick` handlers in the same bytecode as a lesson. P310 is Minimum Requirements (586, 133 MHz, 12 MB RAM, 1 GB, mouse, Windows 98). P20001 is the small menu. P22130 is Test Console Setup.

2.3 The 11.5 export

Known, as of the exported specimen. In `Instructor115.ini`, under `[Options]`, set `saveSpecial=true`. Open a copy of the 8.1 book, answer the version warning, press F3 for Author level, then File → Save To XML. The result is a folder named `<book>.TBK TBX`. A `.tbkx` is a ZIP of that folder. Opening the ZIP in 11.5 imports it and rebuilds a book.

The schema says Bounds is required on a draw object and Vertices is optional. The schema spells IrregularPolygon wrong (`IrregularPloygon`). The binary stores bounds. The export expands them to Vertices. On P1040 the pin bounds `5055, 4605, 5205, 4860` match the start of the TBX Vertices list. That is the expansion direction: bounds in the 8.1 file, point list in the XML.

The exported specimen of `9e954d01` has 131 `Page.xml` files, 14 backgrounds, about 75 BMPs, and 302 `.script` files. Two of those pages, P440 and P2920, have no `80 65` page record of their own. The binary has 129 pages. All 129 names match a TBX page. They are not a missed page marker. P440's sentence and its house shapes are inside the P430 record (chapter 13). P2920 has not been searched the same way.

Saving from the 11.5 trial converts the book. An 8.1 runtime cannot open that converted file. 8.1 stores an imported picture as a basic blob and turns it into a BMP. 11.5 stores WMF, JPG, TIF, BMP, PNG, and GIF as themselves.

WV-009 called the export a conjecture on 12 October. The specimen exists. The conjecture is closed. WV-013’s description of the ZIP-plus-XML shape matches the specimen.

2.4 Foolbook envelope

`.fbk` and `.fsk` are not renamed JSON files with a new extension. The envelope is magic, one flags byte, then a JSON payload. The engine reads compressed, uncompressed, and the older plain-JSON working copies. `.json` remains the working copy. `.tbk` and `.sbk` stay supported so an existing ToolBook file can be opened. System books load core first, then user `.fsk` files as mods.

The philosophy written for that reader is discipline with a recorded deviance: use explicit points when they exist, then the TBX Vertices, then a bounds approximation, and say which one was used.

3. Runtime

Claims

Id	Grade	Statement	Source
R-001	Known	The intended launch is `TB80RUN` (32-bit) spawning `TB80NET.EXE` with the book on the command line. The seam is that spawn, not an in-process thunk.	module catalog, `TB80RUN` survey
R-002	Known	`TB80RTM.EXE` is the 16-bit shell. It owns `TBKMain`, `TBKClient`, `TBKChildWindow`, `TBKReserved_1`, `SchedulerWndTB80`, and the frame class created with `TB80MAIN`. A lesson click is delivered to `TBKClient`.	SoftICE class list
R-003	Known	`TB80MAIN.EXE` is a 23552-byte 32-bit stub. It calls `TB80RMT CreateRemoteCallServer` and pumps keys. It is not the book engine. Failure text: “TB80MAIN: Loading ActiveX support failed.”	`TB80MAIN` decompile
R-004	Known	`TB80BAS` is the book server (“ToolBook Server DLL”), 606 exports. `CDBINIT` is ordinal 1. `CDBFILEOPEN` is ordinal 8. `CDBVIEWPAGE` is 15. `CDBVIEWBKGND` is 18. `CDBPAINTCLIENT` is ordinal 26.	export table, WV-011
R-005	Known	`TB80RTM` `WM_PAINT` calls `CLIENTTOPAGE`, then ordinal 26. Shape drawing is in BAS, not in RTM.	`tbwp0018/rtm-paint-path.md`
R-006	Known	`TB80RMT` is not on the path that opens a lesson. The 1.5 `PHYSSIZE` index is not in these 8.1 books.	WV-014
R-007	Known	`nc81int.dll` exports 47 trainer calls. `IntCheckComm130E` opens a port at 9600 8O1, sends a probe, and returns `0x3D` on match or `0x3C` on failure. Families: 130E, TD2000, 5050, 205E.	`nc81int` exports
R-008	Known	`nc81rec.dll` writes `0x46`-byte session records to `SESSION3.REC`. Exports include `RecStartSession`, `RecCloseSession`, `RecBookmarkSession`, `RecWriteResponse`, `RecReadVariable`.	`nc81rec` exports
R-009	Known	On 2026-10-16 the `TB80RTM` exports were broken before the run, and again after one step once the program was running. Neither pass fired. The run went from start to end. At the end the Assembly window was `0000:0000` and `????`.	that session
R-010	Thought	Those symbols were not bound to the code that executed. "Set too early" does not cover the second pass. Why a break set after one live step still missed is not known.	R-009

3.1 Process

`tbload.exe` finds the book marker and hands off through a vtable at `FUN_00406a70` (virtual call at `00406aa1`, vtable+4). That vtable is not resolved. Thought: that call is the first handoff out of the 32-bit search.

`TB80RUN` does not open the book itself. It starts `TB80NET.EXE`. `TB80NET` is a 16-bit NE executable, not a DLL. It is the network access provider (zlib, `TBKNET3WNDPROC`, `TBK.INI`). It does not parse pages.

TB80W32.DLL (about 246 exports) and TB80W16.DLL (about 406) are the thunk halves. W16 is also the DoCmd* command bus (DoCmdDraw, DoCmdMouseDown). TBFILE32.DLL supplies long filenames to the 16-bit engine. TB80F32L.dll is the flat 32-bit entry and the common-dialog hook.

TB80RTM is the window. Classes seen live include SchedulerWndTB80 and TbkNet3TB80 (the latter owned by TB80NET). The lesson click does not arrive on TBKChildWindow. Break the TBKClient window procedure. BMSG does not see that 16-bit queue.

3.2 Book server and neighbors

Module	What it is for	What it is not
TB80BAS.DLL	Objects, properties, painters, twip scale, file open	Not the window
TB80CMP.DLL	OpenScript compile and expand. Resources 0x77 and 0x78	Not the page walker
TB80UTL.DLL	Type-name table, string manager, metrics, heaps	Not the 14 00 00 reader
TB80RCR.DLL	Linked beside the book server. Exports only partly named	Not classified
TB80ENU1.DLL / ENU2	English strings, 16-bit and 32-bit	Not code that opens a book
TB80BMP.DLL	DIB, metafile, FX blit. Ordinal 3 is the bitmap reader	Not a picture store
TB80JPG.DLL	JPEG. Version resource still says TB60JPG.DLL	
TB80LNL.DLL	MCI, wave, MMIO. The notify callback is Lionel	
TB80MM.DLL	tbkmmNotify, imageNotify, mciNotify, timerNotify	Does not classify objects
TB80OLE / OCX / ACX / VBX	Clipboard, controls, automation	Not on the Logic Families paint path

linkDLL libraries add commands only when a handler links them: TBDLG, TBWIN, TBDOS, TBREG, TBDB3, TBPDX, TBCBT. TBCBT is the CBT scoring extension. Its segment 7 holds the password cipher. See chapter 7. TBWIN is INIT_TBKWIN and TBM_EVALUATE. Neither TBWIN nor TB80NET walks page records.

Author binaries TB80EDT.DLL and TB80.EXE are not in this set. The runtime half of the author contract is visible: BAS exports CDBEDITBOOK, CDBVIEWBOOK, CDBISVIEWERINGENERICEDITMODE, CDBSENDEDITLEAVETOALL*, and TBXAUTHORMODE. RTM calls GETAUTHORLOCALIZEDDLL. That is bookmarked, not solved.

3.3 NIDA layer

A system book adds script above the book. A linkDLL adds commands. Most of what an author treats as “ToolBook” is one of those two, not a new entry point in BAS.

tb80r.sbk is the Asymetrix reader book (Help, Bookmarking, Logging, Path manager, Popup Page Manager). It is not the engine. tb80hyp.sbk is hyperlink and help. tb80anm.sbk is path animation.

The lesson calls initializebase, initializerecs, and initializetest by name. Thought. Those names have not been watched as a call log, and the order in toolbook.lst has not been watched live. version.ini in the set is 03.001.0006, dated 04-18-05. CAILT.DAT maps course ids to Navy CDs. 9E954 maps to DIE. STDAPPS.LST is a short encoded list, not a book.

nc81int.dll and nc81rec.dll are 32-bit PE. They are not Asymetrix. The trainer probe and the session file are the two behaviors that matter for getting a NIDA lesson to run as a lesson rather than as a picture of a page.

3.4 What startup has been watched

WDW on W16, before any book open, hits platform detection, OLE init, and the 32-bit thunk: GETPLATFORMREF, IS32BITENV, GETRUNTIMELOCALIZEDDLL, ACXINITIALIZETOOLBOOKSUPPORT, ACXOLEINITIALIZE, LOADLIBRARYEX32W, GETPROCADDRESS32W.

WDW on UTL, after INITTASK, hits string and task utilities: GETSTRMGRSTRING, plugin init, TBK.INI profile reads, metrics, GHEAPCREATE. No record walk.

The Unicorn host has not opened a book. Pass 06 asks for TB80.INI and exits. See chapter 7.

3.5 The RTM export list, and a dead task

Watcom's Functions window for TB80RTM is 43 exports. ?CALLPROC32W@@Z at the top of that window is the decorated CallProc32W import. The Ks are Watcom's size marks, not part of a name.

EGYPT is the old window prefix. The frame is MAINWNDPROC. The client is EGYPTCLIENTWNDPROC, the child is EGYPTCHILDWNDPROC, and EGYPTNOTIFYPROC takes the notifies. STATUSBARWNDPROC is the status bar. SUBEDITCONTROLPROC subclasses an edit control. DDEWNDPROC is DDE. XDLLTRANSWINPROC is a window owned by another DLL.

The shell's own book calls are ENUMWINSFOROPEN, BOOKQUERYSAVE, EGCDBSAVEBOOK, and HANDLEREMOVESCRIPTPROC. EGDYNALINKERRORFUNC runs when a DLL did not load.

Around that frame the names are the product: ASCII file dialogs, cache-file and CD-ROM save, optimized-save progress, the image store, a palette broadcast, the print dialogs, find and its condition dialog, two ask dialogs whose PL stem is still unread, resource pickers, TOOLNOTIFY, and the remote pair XCTOLEREMOTECOMMAND / XCTOLEREMOTEGET.

The 12 October export map gives each name a line in Toolbook/_decompiles/TB80RTM.EXE.c. A first pass took every call in the next 400 lines and credited it to the export. That mixed in the next function. The following is only what is inside the export itself.

EGYPTCLIENTWNDPROC switches on the message. 0x0F is paint: FUN_1070_0be0, or GETUPDATERECT, or FUN_1060_0dca, depending on the flags at the window struct +0x124 and +0x126. 0x14 returns 1, so the default background erase does not run. 0x02 calls ordinal 76. 0x07 and 0x08 are focus. 0x100, 0x101, and 0x102 are key down, key up, and char. 0x104 and 0x105 return without acting when DAT_1450_0a41 is set. 0x113 is the timer. 0x4A is WM_COPYDATA. Help ids 0x7DF9, 0x7E02, and 0x7E03 go through CXTHELPSETCONTEXT.

MAINWNDPROC is 57 lines. The message is private: 0x41, 0x4AA, 0x4AB, 0x4D8, 0x4D9. 0x111 is WM_COMMAND. A fixed set of command ids goes to FUN_1098_0124, and the rest to FUN_1368_0000.

BOOKQUERYSAVE resolves the path with ASYM_GETLONGFILENAME when the process is 32-bit. If DAT_1450_0a41 or DAT_1450_0a45 is set it returns 7 and does not prompt. Otherwise it prompts through FUN_1068_0034 with style 0x31.

EGCDBSAVEBOOK is the same shape and it does not call CDBGETVALUEEX. An empty name, or either of those two flags, sets the result to 7. Otherwise it prompts with style 0x24 or 0x30 and returns whether the answer was 6.

HANDLEREMOVESCRIPTPROC is a dialog procedure. 0x110 sets the item text from ordinal 106. 0x111 with a zero id calls ENDDIALOG. EGDYNALINKERRORFUNC returns immediately unless the id is 0x1FAC or the _C_FILE_INFO_ id, then calls ordinal 161. POSTPALCHNGTOOTHERS is seven lines: SENDMESSAGE of 0x311, which is WM_PALETTECHANGED, and return 1. CHANGEPRINTERDIALOGPROC reads and writes the profile string. ASCIIFORMATDLGPROC is a dialog on 0x110 and 0x111, command ids 1, 2, 0x106B, and 0x106C. PLASKDIALOGPROC and PLREQUESTDIALOGPROC end the dialog. The PL stem is still not a word in the listing. The palette helpers TBKPALAPPEND and CREATEPALETTE sit in the next function, not in POSTPALCHNGTOOTHERS.

4. Objects, file open, and the reader

Claims

Id	Grade	Statement	Source
O-001	Known	A live object is `0x76` bytes. Bounds are four words at `+0x0F`, in twips. At 96 dpi, screen pixels are stored / 15, which is `(stored * 96 + 720) / 1440`.	live traces, `FUN_1008_01dd`
O-002	Known	`+0x1F` selects the BitBlt raster op. `+0x3B` is a GDI handle. `+0x30` is the point count. `+0x32` is the point-array far pointer, 4 bytes per point, stored twips.	polygon and BitBlt traces
O-003	Known	`CDBSENDCREATE` (BAS ordinal 51, `12c8:00BC`) creates or rewrites objects from the Instructor UI. The runtime book-open path does not call it. `FUN_12c8_0ef8` is downstream of that author path.	`tbwp0025/cdbsendcreate-is-author.md`, WV-042
O-004	Superseded	The chain `CDBSENDCREATE` → `FUN_12c8_0850` → `FUN_12c8_0e32` → `FUN_12c8_0ef8` is the runtime reader.	`TBdecompile/trail/20261015/00-how-toolbook-operates.md` §2, `tbwp0002`, `tbwp0008`
O-005	Known	SoftICE and WDW both reached `OBJFILEOPEN` and the buffered read in `FUN_1160_03de`. The raw DOS path at offset `04F3` is skipped when the buffer is already satisfied.	2026-10-16 consolidated note
O-006	Known	The W16, UTL, and BAS traces in `tbwp0026` do not contain a function that reads a `14 00 00` record. `CDBCHUNK*` walks text. `VALUENEWLONG` constructs a value. `OBJFILEOPEN` is downstream of the reader.	`tbwp0026/trace-review.md`, WV-044
O-007	Superseded	The runtime reader has no name and has not been watched.	open-question notes. The 20 October read is O-008.
O-008	Known	On a lesson open the object body is read by `1178:08B2` → `1188:04DA` → `1188:08D6` → `1160:02CC`. `HUGEGETDATA` is a different tree and was not on this path. Message `0xE` runs after the bytes are in memory.	Bochs, 2026-10-20. Section 4.6. Appendix L.
O-009	Known	A field's text handle is word 1 of the block behind object `+0x24`. On disk that word is a trailer-handle index, and the field class in the specimen is `0xA`. Word 0 is the style-run handle. `FUN_1440_0160` does not read `+0x42`.	XML match, and `research/store-20261022.md`.
O-010	Known	Class `0x3A` is the book's `ClassTbl`: `0x2C`-byte rows. Class `0x26` is `Viewer`. In the sample books the next free number is `0x3D`.	`research/class-table.md`.

Id	Grade	Statement	Source
O-011	Known	A class 5 page holds its background as a far reference at `+0x47` / `+0x49`. On `9e954d01.tbk` all 131 resolve to a class 4 record.	`research/store-20261022.md`.
O-012	Known	The ordered page list hangs off book `+0x10`. Each row is 13 bytes: background id, page id, page reference, and a zero.	Same note. 131/131 on the specimen.

4.1 The live object

These offsets are the ones the papers and the live traces agree on.

Offset	What	Grade
`-2`	Type byte. `0x11` is an arc. The other type bytes are not mapped.	Known for `0x11` only
`+0x0F`	Bounds: left, top, right, bottom. Written as twips.	Known
`+0x17` `+0x19` `+0x1B` `+0x1D`	Four words copied by `FUN_12c8_0ef8` from a source structure at `+0x3E` `+0x40` `+0x42` `+0x44`. The disk `14 00 00` record does not sit at those source offsets.	Known as a copy. The source is not the disk record.
`+0x1F`	BitBlt raster-op flag. An earlier note also said the bits are taken from a source structure at `+0x0A`, `+0x0C`, and `+0x0E`. That copy was not re-checked against the BitBlt trace.	Known as the raster-op byte. The source-bit copy is Thought.
`+0x30`	Point count (word)	Known
`+0x32`	Point array far pointer	Known
`+0x3B`	GDI handle	Known
`+0x52`	Per-pin index. Not applied by the current renderer. Pins that share point bytes draw on top of each other.	Known that the field exists. Not known how paint uses it.

`ScaleTwipsToScreen` is `FUN_1008_01dd`, BAS segment 2 offset `0x1DD`. With `DS:01A6` and `DS:01A8` set to 96 it turns stored twips into screen points. On the P1140 to P1130 change those words were 96 and 96, so the short form is division by 15.

`FUN_12c8_0ef8` copies four words into the object. It does not parse a page. The disk record and the structure it copies from are different shapes. The step between those two shapes is still unnamed. That step is not `CDBSENDCREATE`.

4.2 Author create, stated once

`CDBSENDCREATE` receives bounds that are already formed. It does not compare a buffer to `14 00 00`. Nothing inside BAS calls it by name. One external thunk from `FUN_13d0_0822` reaches ordinal 51. That caller is author UI state (it tests bytes such as `0x26`, `0x36`, `0x38` and stores coordinates around object `+0x638`). It is not the book-open loop.

Do not break on ordinal 51 to find the reader. The stop will fire when Instructor creates or rewrites an object. It will not fire because a lesson page was opened. The 15 October operational notes ended this section with “list the XREFs; the caller that passes a page blob is the reader.” That sentence is not true. The one caller that was found does not pass a page blob.

The function is still real. `CDBNEWOBJECT` indexes objects as `param_6 * 0x76`. The size matches the live object. The index expression is the decompile, not a second measurement of the size. The block that holds the array is `FUN_1278_3be6` (decompile line 121070): `GLOBALALLOC` or `GLOBALREALLOC` of `count * 0x76`. That is not `TBHEAPALLOC`. Chapter 19.

`param_5` selects the path. The string arithmetic in the decompile is Ghidra matching those constants against `s_CTbkSimpleOLEDispatch_CallMethod`. It is not a string compare in the program.

param_5	Path
0x1014, 0x1015	FUN_12c8_0850, then FUN_12c8_0e32 (allocator), then FUN_12c8_0ef8 (field copier)
0x1021, 0x1022	FUN_12c8_0a86
0x3018–0x301B	CDBCHECKBOUNDDTL, then FUN_12c8_0e08
0x3022	FUN_12c8_0d40
anything else	Ordinal 115, TOOLBOOKFATAL

param_3 and param_4 are the bounds pair. They are clamped against 32000 and 0x8300. param_6 and param_7 are compared to the current object id. param_9 is the error out-byte. FUN_12c8_0850 has one caller, uses ENTER 4,0, locks a context, and tears the object down on error. It is not the record reader.

FUN_12c8_0ef8 writes the four bounds words two ways. If its source pointer is null, it copies globals that were set from the parameters (DAT_1630_01FE through DAT_1630_0204) into +0x0F, +0x11, +0x13, +0x15. Otherwise the default arm of its param_6 switch copies four words from that pointer into the same slots. The same function copies source +0x3E / +0x40 / +0x42 / +0x44 to object +0x17 / +0x19 / +0x1B / +0x1D, a byte from source +0x06 through a type table, and source +0x04 as the pattern lock. The disk 14 00 00 record is not that source structure.

param_6 in that switch is an object type, not the param_5 above:

param_6	What the decompile does
0x0F, 0x10, 0x13, 0x14	Polygon-like. OBJNEWNEAR(count << 2) for the point array.
0x11, 0x12	Left alone. 0x11 is the arc that was seen live.
0x24	Copies four words from the source pointer, and does something else the listing did not name.
default	Copies the four bounds words.

The property registrar is FUN_1058_0a98 at 1058:0A98. It writes property ids. It does not write painter pointers. The ids it writes are 0x4004 (bounds, nine times; the first is decompile line 30313), 0x4009, 0x400D, 0x4015 (idNumber), 0x401A–0x401C, 0x4020–0x4022, 0x4023 (script), 0x4027, 0x402B, 0x4031–0x4033, 0x4034 (vertices), and 0x4035. Names are only attached where the property resource or the TBX specimen has attached one. The others are slots, not captions.

Older notes that drew this chain as the runtime path:

- tbworkingpapers/tbwp0002/bas-create-path.md
- tbworkingpapers/tbwp0008/cdbsendcreate-sites.md
- TBdecompile/trail/20261015/00-how-toolbook-operates.md, section 2
- TBdecompile/trail/20261012/01-object-create-path.md, the “still open” paragraph
- TBWorldview/ToolBook_Architecture.md, the book-open flowchart that calls CDBSENDCREATE per widget

Those files still describe a real function. They mis-place it. The tables above are the part worth keeping.

4.3 File open, as far as it has been stepped

CDBFILEOPEN is BAS ordinal 8. Thought (WV-007). It is the export on the open path. It has not been shown to expand a 14 00 00 record itself.

Under it, OBJFILEOPEN has a large frame (0x25A in one trace). FUN_1160_03de is the read wrapper. It calls FUN_1258_00a8 (buffered path). If that call satisfies the request, a jump skips the raw DOS read at offset 04F3. Live dumps of a 4-byte read returned an internal pointer (FC B5 BD B0 at one buffer), not a page header and not the JBO signature. A wider dump showed 4F 46 nearby. That is not a page record.

CDBCHUNKFIND, CDBCHUNKLAST, CDBCHUNKEND, and CDBSKIPSEP walk characters: comma, CR, LF, tab, space. They are text chunkers. A decompile that shows ISDBCSLEADBYTE and a comma test is not the page walker.

4.4 Where the reader is not

Closed, so they are not searched again as the reader:

Candidate	What it actually is
`FUN_1028_2680`	Filter registrar. Calls `TBREGISTERFILTER`. The `0x14` is a type-code limit, not the record marker.
`FUN_1008_2d31` (switch at `1008:2DC7`)	OpenScript math. `+ - * /`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`.
`TB80RTM`	No named file open. Paint goes to BAS ordinal 26.
BAS segments `1018`, `1278`, `1478`, `14b8`, `1508`, `10d0`, `1030`, `14a8`, `1070`	Sampled. No stream walker.
`1160:03DE`	The buffered read seen on 16 October. The lesson-open body read is `1160:02CC`, section 4.6.
`FUN_10b0_00a6`	Dispatches on type codes including `0x14`. It is a type dispatcher, not a reader.
Code at `12e8:0000`	OLE object clone.
BAS segment 182	Real preload code, 39 far calls with a zero segment and no reloc. Not on the book-open path.

Known. No static search of the disassembled image has found the bytes `14 00 00` used as a compare against a far pointer that then builds an object. Thirteen undisassembled runs contain the word `14 00`. Those are far-call immediates, `MOV CX, 0x14`, or jump displacements.

Known. The 20 October read does not compare the marker. It uses the length in the handle-table entry. The marker is still in the bytes, at record offset 50 in the traced object and at 52 in 146 of 206. Section 4.6.

4.5 Page and background

Known. `80 65` records are the pages. `9e954d01.tbk` has 129 of them.

Known. A TBX page can name a background by an id, such as 23 on P10213, that does not appear as a word or a dword in the matching binary page. The on-disk link is still not matched. Parent and z-order are not in the page record that has been decoded. Absolute bounds are not stored for every shape. The P1170 bracket was computed from bounds relative to a parent. The parent offset has not been applied to the P420 rects.

Known, live record only. A class 5 page holds a far reference at `+0x47` / `+0x49` (O-011). Property `0x4016` asks that object for its size. Property `0x429d` is forwarded when the reference is nonzero. The class 4 record has the fill, the pattern, and the size. Property `0x4036` returns only the low word. Follow the dword. The note is `research/paint-20261021.md`.

4.6 The 20 October read

Bochs, Win98 SE, debugger outside the guest. Appendix L is the command card. Selectors in this section are that boot only.

Known. `1048:2174` opens the book. Each object then hits `1178:08B2`. That calls `1188:04DA` (allocate `header + body`) and `1188:08D6`. `08D6` calls `1160:02CC`. The traced call read `0x19C` bytes from file position `0xB7221` into the block at offset 6. A flag `0x40` on the entry caused one more read, 2 bytes, at `0xB73BD`. `HUGEGETDATA` was not hit.

Known. The on-disk table is 206 entries of `0x18` bytes at file `0x10D6`. Index `0x12` is this record: position `0xB7221`, length `0x19C`, type byte `0x25`. Handles `0x0C1E`, `0x0C36`, and `0x0C4E` are `0x18` apart in memory. A 20-byte dump of the live entry matched the file through `+0x13`, except status (0 in memory, 1 on disk). Bytes `+0x14` through `+0x17` were not dumped. On disk those four bytes are `00 21 25 00`: a zero, the nibble-swapped index, the type, and a zero.

Known. After the read, `1178:08B2` sends message `0xE` through `1180:02CE` and `1180:03C8`. Handle `0x0C1E` returned `10B0:0000`. There, `0xE` calls `FUN_10b0_54e0`, which calls `FUN_10c8_018e` only when the error byte is still 0. That is a near-handle fixup, not the file read. Handle `0x0C4E` returned `10F8:0000`, which has no `0xE` case. The handler for

handle 0x0C36 (this 412-byte record) was not captured. On the object that was dumped, message 0xE changed only block bytes 0 through 3, to a far pointer to the reader object. The 412 body bytes did not change.

Thought. Record byte 12 matches the table's type byte in 142 of 206 entries, so it is the class code. The 0x12 seen on the 03C8 stack was a different object. It is not yet that byte.

4.7 Scripts and groups

The detail is research/paint-20261021.md. These are live-record offsets.

Known. FUN_10d0_12e0 picks the script by the class word at record −2. Book (class 1): mode in the low two bits of +0x22, word at +0x24. Page and background: mode +0x30, word +0x2A. An ordinary object: mode +0x0A, word +0x0B. The top bits of book +0x22 are the color-screen flags. Mode 0 is inline. Mode 2 is a type 9 resource. The shared script is a separate request, type 0x13. Modes 1 and 3 have no branch.

Known. A group's count is +0x24. The entries at +0x2E are 10 bytes: the child, then its left, top, right, and bottom. No separate stream-in was found. Whether load rewrites the child word was not checked.

5. Graphics

Claims

Id	Grade	Statement	Source
G-001	Superseded	Polygon kind 04 00 with an inline point list at +0x22.	The specimen's class 0x10 keeps the count at +0x30 and a near pointer at +0x32. G-002. research/store-20261022.md.
G-012	Known	A resource id is the fourth word of the 8-byte directory entry. A button stores that id plus one, at graphics-block +0x15.	FUN_1128_1e44 accepts a match only when the scanned word is the fourth. 396/396 buttons.
G-002	Known	The polygon painter is BAS FUN_1408_0376 (segment 0x82 offset 0x376). It copies object +0x30 / +0x32 to DS:30AC and calls GDI!POLYGON.	SoftICE, 2026-10-10
G-003	Known	Content rectangles are bounds-only. In the page blob the tail is four twip words, a zero dword, a fill dword, then 01 00 00 04. 9e954d01.tbk has 961 of that marker.	Foolbook shapes.py count, P1040
G-004	Known	An arc object is type 0x11. Four twip pairs at +0x25, +0x29, +0x2D, +0x31. Bounds at +0x0F. Fill at +0x18. Painter is BAS segment 0x7B.	live arc, 2026-10-10
G-005	Known	ToolBook does not call GDI!PIE. Pie wedges are computed by GDIELLIPSEPOINTFROMANGLE (property 0x4123) and filled with the arc geometry.	import table and decompile
G-006	Known	A diagram bracket on P1170 is not stored as points. BAS segment 0x5B computes four polyline points from the object bounds.	live GDI!POLYLINE, 4 points
G-007	Thought	P420’s flame is several pen strokes in the record at page +0xA13, matching three 96-point red IrregularPolygons in the TBX export. The on-disk control structure is not known.	render attempts, TBX
G-008	Known	A live button holds its four picture ids in the block at +0x28. Normal id at block +0x15, invert +0x27, disabled +0x39, checked +0x4B.	research/paint-20261021.md
G-009	Known	A stored bitmap is an 8-byte prefix, then a Windows BMP. 9e954d01.tbk has 67. Use bfSize from the BM header. The prefix dword is not always bfSize + 8.	same note, file scan 2026-10-21
G-010	Known	Resource directories are one per type. FUN_1028_218a reads OBJGETHDRLONG((type + 5) * 4). Type 1 is a bitmap. TB80BMP is not the lookup.	same note. One Bochs hit returned an object reference for type 9

Id	Grade	Statement	Source
G-011	Known	Draw-object pattern `+0x25`, stroke `+0x17`, fill `+0x1B`. Background pattern `+0x4B`, fill `+0x5B`, size `+0x5F` / `+0x61`. `0xFD` is hollow, `0xFE` solid paper, `0xFF` solid ink.	same note

5.1 Who paints

The painters live in `TB80BAS`. They are reached through a type dispatcher, not by a named call from outside BAS. `TB80RTM` handles `WM_PAINT` only as far as `CLIENTTOPAGE` and `CDBPAINTCLIENT` (ordinal 26).

Painter	Where	GDI
`PaintPolygon`	`1408:0376`	`POLYGON`
`PaintPolyline`	`13e8:00E6`	`POLYLINE`
`PaintRectangle`	`1020:03E4`, and segment `0x83` (`FUN_1410_0066`)	`RECTANGLE`
`PaintArc` / alt	`10A0:0558`, `10A0:0458`	`ARC`
`PaintEllipse`	`1230:2F1C` and an alt	ellipse
`PaintRoundRect`	`1230:2278` and alts	round rect
`ScaleTwipsToScreen`	`1008:01DD`	none. Converts in place.

`FUN_1008_01dd` is that converter. For each pair it computes `(coord * factor + 0x2D0) / 0x5A0`. `0x5A0` is 1440, one inch in twips. `0x2D0` is 720, so the add is rounding. The x factor is `DAT_1630_01a6` and the y factor is `DAT_1630_01a8`. At 96 DPI the factor is 96 and the division is the same as dividing by 15. The `/15` rule is that case, not the function.

The polygon type table is not in the static image. Unpacked BAS still carries Optloader segment-table debris. The HTML listing has zero cross-references to `FUN_1408_0376`. Thought. The table is built at load, or fixed up by an NE relocation that the listing does not show as a pointer. A break on the painter, then one frame up, is how the table gets found. Searching the file for the bytes `76 03 08 14` will not find it. The selector is assigned at load.

`SetPointArray` (`FUN_1408_00d8`, property `0x4050`) stores the count at `+0x30`, allocates `count * 4` with `OBJNEWNEAR` into `+0x32`, and copies the points. That is the runtime setter. The file does not contain a `0x4050` record.

`LOCKPATTERN` (`1098:0876`) takes a pattern index at `[BP+0xC]`, accepts 1 through `0x80`, and calls `TOOLBOOKFATAL` outside that range. `UNLOCKPATTERN` is `1098:09FC`. It is a fill hatch, not a picture id. An earlier listing also saw a pattern index at object `+0x17` and `+0x5B`. `+0x17` is already the four-word copy in chapter 4. Do not treat those two notes as the same field until both are dumped on one object.

The October 12 field card puts the polygon's object pointer at `[BP+4]`. A far call's first argument, after `ENTER`, is `[BP+6]` (the manual's assembly card). The card did not record whether `PaintPolygon` was entered by a near call or a far call. Dump both slots once. The count at `+0x30` and the points at `+0x32` do not depend on that.

5.2 What is stored

Polygons. `9e954d01.tbk` has 96 kind-`0400` records on 13 pages. P430 has 22. A broader search that ignores the kind word produces false positives. P430 draws as a chip: silver pins (`0xC0C0C0`) and a light-gray body (`0xE0E0E0`). One live 11-point polygon is a pin tip, not the body. The captured screen points were `(218,335)` through `(218,341)`, a 10 by 17 notch. The same points exist in the file as twips near page offset `0x9CB` of P430: `(3147,3862)` through `(3147,3973)`.

Pins that share identical point bytes stack on top of each other until the index at `+0x52` is applied. That index is not applied in the current clean-room draw.

Rectangles. P1040 is the TTL intro page (TBX page id 880). Two DIP packages: 7400 on the right, 5400 on the left. The 8.1 file stores 28 pin boxes (fill `0xC0C0C0`) and two dark bodies (fill `0x404040` in the TBX, `64,64,64`). The pin

tip is not an 11-point list on this page. Paint expands the tip. The TBX export writes the expanded Vertices. Matching a TBX Vertices list back to the binary means matching the bounding box, not searching for 11 points.

Brackets and strokes. On P1170 the bracket's live polyline was `(354,201) (321,201) (321,238) (358,238)`. The object bounds in twips were left `4801`, top `3015`, right `5370`, bottom `3570`. Divided by 15 that is the same rectangle. The points are computed. Connecting lines and detail strokes of this kind are not extracted yet.

Arcs. Live geometry is known (G-004). Fill `0x808080` was seen at `+0x18`. No page record has been matched to type `0x11`, and the start and sweep angles are not located in the struct. Do not set a breakpoint on `GDI!PIE`. The call is not imported.

Curves. Curve's type id in `nc81base.sbk` is `0x05C7` (file offset `0x1810`). That id is not in the P420 page records. The point runs sit in the record at page `+0xA13`. A naive walk draws a tangle in screen space around 400–500. The TBX export of the flame is three 96-point red IrregularPolygons. Thought. The tangle is strokes concatenated that should be drawn separately. A length prefix or a zero terminator between groups has not been proven. There is no `Curve` name on the page.

Pictures. A button's four picture ids are in the block at live record `+0x28` (G-008). The bytes are an 8-byte prefix and then a BMP (G-009). `9e954d01.tbk` has 67 of them. The lookup is the per-type directory in G-010, not `TB80BMP`. A picture that is not a button still has no located id. `DRAWPICTURECALLBACK` writing a GDI handle at object `+0x0C` / `+0x0E` remains a reading of the listing, not a live dump. An earlier note said 641 BMPs carved. That count is a header scan, not a widget binding, and it is not the number to use.

The shared near-start graphic in the lesson books is a 581 by 157, 8 bpp gold plate, not a text banner. Two notes locate the `nida81` splash building: file `0x413FAB` (323 by 344, 8 bpp, off-bits 310) and a carved copy taken from file `0x487BE` (27874 bytes). This record does not treat those two offsets as the same byte. The live `BITMAP` header that was found was width `0x143`, height `0x158`, 8 bpp, stride `0x258`, `bmBits` null. Known that the splash is a DIB. Not known which property on the page object holds its id.

BAS segment `1150` builds the book palette (`CREATEPALETTE`, `SELECTPALETTE`, `REALIZEPALETTE`). An 8 bpp BMP needs that palette. Segment `1018` creates and deletes pens, brushes, and bitmaps. Who owns the lifetime is open. Fill color and stroke color are known (G-011). Pen width is not.

Field text is `DRAWTEXT` from a paint context at `param_5+0x4A`, `+0x4C`, and `+0x50`. The disk handle is word 1 of the block at object `+0x24` (O-009). Word 0 of that block is the style-run handle. The rest of the field layout, including the border, is open.

5.4 Classes, from the 21 October listing

`FUN_1058_2ae4` registers the draw segment. This does not retire G-002. The note is `research/paint-20261021.md`.

Class	Draw segment	Record	What it calls
8	`0x1410`	`0x28`	`FUN_1410_0066`, `RECTANGLE`
`0xC`	`0x13d8`	`0x28`	`FUN_13d8_0066`, `ELLIPSE`
`0xD`	`0x13e0`	`0x2C`	`FUN_13e0_015a`, `ROUNDRECT`. Corner radii at `+0x28` and `+0x2A`

The ellipse previously listed in segment `1230` is not this registration. Bounds for all three are `+0x0F`, `+0x11`, `+0x13`, `+0x15`. Class `0xF` is registered with a `0x34`-byte record. It is not yet named polygon.

5.3 What a render is allowed to claim

A clean-room picture of a page is not the engine (WV-005). The current honest draw is:

1 Explicit point records (kind `0400`) where the count and the pairs are in the file.
2 TBX Vertices where an export exists and the binary only stored bounds.
3 A bounds rectangle where neither points nor an export exist.

Say which of the three was used. P430 from kind `0400` is a check against the chip. P1040 from bounds is a check against the two packages, with the understanding that the pin tips are fatter on screen than the stored boxes. P420 from a raw point dump is not a flame until the stroke breaks are known.

6. OpenScript

Claims

Id	Grade	Statement	Source
S-001	Known	Handlers are compressed bytecode. The keyword table is NE resource `0x77` (type 100, id 119). The offset table is at byte 2 plus count times 4. There are 994 keywords.	decoder notes
S-002	Known	Bytes `0xC0`–`0xFF` copy from the compressed input stream, not from the output. An earlier decoder that copied from the output was wrong.	WV-032, decoder fix 2026-10-10
S-003	Known	The decoder used for NIDA navigation stops at the first standalone `end`, so trailing decode garbage is dropped.	engine notes
S-004	Known	`9e954d01.tbk` yields 76 handlers on 39 pages. `nida81.tbk` yields 100 `buttonClick` handlers. `tb80r.sbk` yields 33, with the `to handle` prefix also compressed.	corpus counts
S-005	Known	`FUN_1008_2d31` is the expression evaluator. The modern engine implements `+ - * /` and `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`, and otherwise leaves the original text.	`openscript_analysis` 1.0.0r6
S-006	Known	A `buttonClick` on `nida81.tbk` page `P20000` goes to `P20001` because the handler says so. Navigation is not “only a quoted page id.”	WV-033, WV-041
S-007	Thought	A short script compiled by Instructor 8 and expanded with the existing decoder would lock the keyword map. That compile has not been done.	`tbwp0014/instructor8-compile-test.md`

6.1 What the bytecode is

Resource `0x77` is a compiler emit routine as well as the keyword pool. Resource `0x78` (id 120) is another emit routine. `COMPILERPROPINFOFROMINDEX` locks resource `0x70`. The property-name table is a different resource (BAS type `0x2D1`, id `0x2D0`). Do not treat every locked resource in that range as the keyword list.

The byte classes, the back-reference formula, the corpus counts, and the handlers that have been expanded are chapter 14. The working description, including the two bugs and the compile test that has not been run, is Appendix J of the reference manual. An early reading of the same bytes produced "italic" and "systime" and concluded the keyword table was wrong. That reading copied `0xC0`–`0xFF` from the output stream and used the wrong offset-table stride. It is superseded. Do not cite it.

High-bit opcodes (`0xE0`, `0xF0`, `0xFF`, and neighbors) emit whitespace, CR/LF, or a repeated span. After the input-stream back-reference fix, the 76 handlers in Logic Families decode. Examples that are real source, not Latin-1 of the bytecode:

- `send next`
- `go to page "P400"`
- `go to page "P310" of book "nida81.tbk"`

The common subset the courses actually use is `go to`, `send`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, and `name of this book` / `this page`. The engine stops at the first standalone `end`.

`tb80r.sbk` uses the same bytecode. The difference is that the handler name prefix is compressed too. Encoding rules match. The byte histogram has more `0xFF` and `0xE0`.

6.2 What a lesson does with it

Logic Families is a menu of jumps, not a self-running program.

Page	What the handler does
P900	Lesson menu. TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` is the IBM PC Compatible Computer line. `buttonClick` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Assessment. Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Section end. Menu returns to P900.
P1290	Quiz. Correct choice 5408. Again → P1292. Next → P1300.

A retry page returns to the quiz whose `again` field names it. `initializeRecs` on the Maybe2 shell clears responses when a session loads. That behavior is inferred from the shell script, not from a live `nc81rec` trace.

The modern engine also accepts `send to`, a `get` that calls, `forward`, `break`, and `property of`. Predicates such as `is not in` are recorded rather than evaluated. Property writes are not applied to a live object. The TBX `.script` files are plain text and are easier to read than the bytecode. They are a playback path for the export, not a reader of the `.tbk`.

6.3 Expressions

The 148-case switch at `1008:2DC7` is `FUN_1008_2d31`. It stays in segment `1008`. It checks for names such as `log`, then uses the FPU (`FLD`, `FSTP`, `FCLEX`). It is not the record reader and not a paint path. The clean-room evaluator copies that set of operations and falls back to the original string when it does not recognize the right-hand side.

6.4 What is not done

The full keyword map is not locked. Resource `0x77` has 994 entries. The engine proves the ones the NIDA handlers use. A compile of a short script in Instructor 8, saved into a book, then expanded, would show whether the offset table and the names still match. That test is written down and not run.

Nested property expansion, the opcodes past the common subset, and `forward` into a system-book handler are unimplemented or only sketched. `COMPILERCOMPILETEXT` is CMP ordinal 1. `COMPILEREXPANDTEXT` is ordinal 3. `CFUNEXECUTE` is 8. `CFUNEVALUATE` is 9. Those ordinals are the stable breakpoints if the compile test is run.

7. Debugging and hosts

Claims

Id	Grade	Statement	Source
D-001	Known	SoftICE and the Watcom debugger do not share command syntax. SoftICE field work closed 11 October 2026. Later breaks are WDW.	field manual, `tbwp0026/wdw-break-syntax.md`
D-002	Known	WDW has no full automatic call trace. `break <symbol> resume on` plus `log <file>` logs and continues. `resume on` is a property of a breakpoint that already exists.	`tbwp0026/wdw-trace.md`
D-003	Known	In WDW, module qualification uses `@` (`TB80BAS@OBJFILEOPEN`). SoftICE's `!` does not work there.	same note
D-004	Known	Symbol maps are `NE-segment:offset name`. The segment index is not a live selector. SoftICE and WDW do not load the `.map` as a module.	`tbwp0003/TB80BAS.map` (3799 entries), `TB80RTM.map` (2669)
D-005	Known	The Unicorn host has not opened a lesson. Pass 06 exits after `TB80.INI`. The entry does not ask for the book.	WV-006, WV-036
D-006	Known	TBCBT NE segment 7 holds `KEYEDPASSWORDENCRYPTEX`, `MAKEKEYEDPASSWORDEX`, `ENCRYPTEX`, and `DECRYPTEX`.	WV-043
D-007	Known	NE segment numbers are not Ghidra's `CodeN` names. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps.	`TBdecompile/maps/rtm-ne-to-ghidra.md`
D-008	Known	From 2026-10-20 the live host is Bochs 3.1. The debugger sits outside the Win98 SE guest. SoftICE does not have to be loaded for a break.	Appendix L
D-009	Known	Offsets in the Ghidra listing equal offsets in the guest. Selectors do not. After each boot they are taken from the LDT and checked by segment size.	Appendix L, `maps/bochs-20261020.md`

7.1 SoftICE

The kit is `SI405w9x.exe` plus ICEDUMP 6.021. The loader that worked was `w9x/4.05.334/icedump.exe`. It loaded on the Windows 98 VM after both `kernel32.dll` and `user32.dll` exports were listed in `winice.dat`.

Ordinal breakpoints are stable across boots. Selectors are not. `TB80BAS.Ordinal_8` is `CDBFILEOPEN` on every boot. `36F7:03DE` is `FUN_1160_03de` only until the next reboot.

`/DUMP` writes the SoftICE console, not the address given, unless the dump experts are set. Several early `page.bin` and `rtm.bin` files are command logs or the DOS device chain (`CON`, `AUX`, `PRN`). A dump is real when it contains book strings at the file offsets already known, as `page.bin01511` did (`ASYM_TpID`, `Page#`, matching `9e954d01.tbk` at `0xB8B51`).

Do not mix these commands into a WDW session. The command card, the screen, and the 15 October book-open read are Appendix G of the reference manual. The assembly frame that card depends on is Appendix I. This chapter is the claim list. It is not a second copy of those cards.

Standing orders that stay true, and that the short operational notes had in one place: `BC *` before the next breakpoint, because a live `GDI!POLYGON` breakpoint freezes SoftICE when the page scrolls. Segment numbers in `MAP` output are hex. Do not search all of memory. A wide search for `80 65` GPFed the VM. `BPX BAS(82):0376` is not an address. `MAP`, then the live selector, then the offset.

7.2 WDW

On the Watcom debugger:

- Create the breakpoint first, then set `resume on`.
- Qualify with `@`, not `!`.
- Log with `log <file>`.
- There is no trace that prints every call. The three logs in `tbwp0026` are named-export hits: 11 in W16, 53 in UTL, 173 in BAS.

Those three logs are described in chapter 12. Appendix H is the Watcom screen and the command words. `!` still does not work there.

On 2026-10-16 every `TB80RTM` export was broken in the Functions window before the run, and broken again after one single-step once the program was running. The program ran from start to end. Nothing fired either time. At the end the Assembly window was `0000:0000` and `????`. The miss is not "the breaks were set too early." WDW was not attaching those names to the code that executed. The 15 October hits were unpacked W16, UTL, and BAS, with `break <symbol>` and `@`. That has not been shown to work for this RTM window.

GPF notes from the SoftICE sessions still apply in spirit: do not step into an unloaded segment, and do not single-step a far call into `KERNEL` unless that segment is present. The CMP expand host died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present). That fault is parked. It is not the book-open path.

7.3 Unicorn

`unfold.py` enters 16-bit protected mode, calls `BootApp`, and dumps each segment `LoadAppSeg` materializes. Pass 01 filled 3 of 139 RTM segments. Pass 02 filled 138, after the call stub was flushed from the Unicorn cache so every call did not replay segment 2. Passes 03 through 07 bind imports, including 461 ToolBook ordinals to real DLL segments. 339 imports stay Windows stubs. The host still dies at `FFFF:0158` and `0080:0098` before a DOS open of the book.

Thought (WV-008). A real Win16 return is what would get that host further. The three hosts (the RTM experiment, `newl`, `neww`) are not one program and do not share a Windows table (WV-035).

The path that does open a page is the pipeline host, which parses the file instead of running RTM. On `9e954d01.tbk` that open is at `0x9EC`, 129 pages. `render.html` is written only when the debug-render flag is set (WV-034).

7.4 Password cipher

`TBCBT` segment 7 (Ghidra `Code7`, base `1030`) holds the four cipher exports. A SoftICE module list that showed segments `01`, `0E`, `02`, `0B`, `08`, `0D`, `04`, `03` had not loaded segment 7. The dialog is an ordinary Windows dialog. The cipher is not in the dialog procedure.

After a failed password, the typed string and the constants `0xA01` and `0x300` remain in memory. Searching for those constants finds the call site even while segment 7 is unloaded. The three addition schemes, the with-password file delta, and the fact that the stand-in key does not decrypt it are chapter 16. Section 16.5 is the 16 October author session: `sssssssssss.tbk` became password-protected and `TBCBT` was not in the module list. The NIDA `.sbk` passwords are chapter 16.6. They are not required to read the records.

7.5 Ghidra

Two NE loader bugs were fixed for this work: the entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image with entry segment bytes at or above 128. The unpacker does not rewrite those bytes. The Ghidra patch does. An honest BAS image loads in the patched Ghidra 12.1.4 build. The procedure for finishing the listing and renaming over time is Appendix K of the reference manual.

Force-disassemble scripts recover functions the loader never reached. One walks undefined `C8` (`ENTER`) bytes. The next accepts a real `RET` / `RETF` before treating the following byte as code, and it only creates functions at entry-like symbols. Run `C8` first, then the prologue pass. They do not name anything, and they do not touch code that is already defined.

`SENDMESSAGE` (321 hits in one RTM listing) and `LSTRLEN` (282 hits) have mangled arguments. That is a decompiler problem, not a missing function. It wants a fresh decompile, not another reading of the same HTML.

7.6 Bochs

The debugger is outside the guest. A breakpoint does not need SoftICE, and it does not need a program inside Windows. The cost is speed.

The command card, the start line, and the stops from the 20 October lesson open are Appendix L. The selectors from that boot are `maps/bochs-20261020.md`. They are dead after the next reboot. Rebuild them before setting `vb`.

What that session watched is chapter 4.6. This section does not repeat it.

8. Naming

Claims

Id	Grade	Statement	Source
N-001	Known	Eight modules were counted for pattern coverage: 10,991 unique functions, 1,918 nameable by pattern, 17.5 percent.	`toolbook_naming_progress.md`, 2026-10-11
N-002	Known	Confirmed renames, after the later digs, are 112: BAS 20, then BMP, RTM, CMP, UTL, W16, LNL, MM, RCR, FLT, NET, JPG, and VBX.	`naming_summary.md` through 101, then JPG +1 and VBX +6
N-003	Known	A pattern name is not a confirmed name. The nameable lists are candidates. The rename scripts are the ones that were applied.	`TBdecompile/rename/`
N-004	Known	Coverage by module, pattern only: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6.	progress table

8.1 What the numbers mean

“Nameable” means the decompiler text matches a pattern: a GDI call, `GLOBALALLOC`, `SENDMESSAGE`, a string move. “Confirmed” means a person or a script renamed that one function because the body matches the name. The gap between 1,918 and 112 is mostly unreviewed candidates, not unknown programs.

The 30 percent target in the October 11 note was a counting goal, not a research goal. Naming another thousand `StrCpy` wrappers will not find the record reader.

8.2 Confirmed painters and neighbors

These are the names later chapters use. Addresses are Ghidra `segment:offset` on the image those scripts were run against. Selectors move. Offsets inside the NE segment do not, once the segment is identified.

Name	Address	Module
`PaintPolygon`	`1408:0376`	BAS
`PaintPolyline`	`13e8:00E6`	BAS
`PaintRectangle`	`1020:03E4`	BAS
`PaintArc`	`10A0:0558`	BAS
`PaintEllipse`	`1230:2F1C`	BAS
`PaintRoundRect`	`1230:2278`	BAS
`ScaleTwipsToScreen`	`1008:01DD`	BAS
`SetProperty`	`12D0:03EC`	BAS
`SetPointArray`	`1408:00D8`	BAS
`SetupPaintRect`	`12D0:086C`	BAS
`SetScriptProperty`	`1008:03F6`	BAS
`Lz77Expand`	`1000:0031`	BMP
`CreateDibBitmap`, `BitBltBmp`, `SelectPaletteBmp`	BMP rename script	BMP
`LoadCompilerResource`, `ExpandTextCore`	CMP rename script	CMP
`PaintPageRect`	`1398:05C6`	RTM

Ellipse, round-rect, and a second arc painter were added in the same BAS pass. They are real GDI wrappers. They are not evidence that every shape in a lesson is stored as that GDI call’s arguments. Chapter 5 is the rule for storage.

8.3 Scripts

Java and Python rename scripts live in `TBdecompile/rename/`. Copies of the nameable TSV files live under `TBWorldview/Modules/tb80*_knownnames/`. The scripts rename only when the old `FUN_` name is still present. They do not invent names for functions that do not match the body.

Ghidra `CodeN` is not the NE segment number. Use `TBdecompile/maps/` before typing a segment into a debugger.

8.4 What is still thin

`TB80RCR` exports are only partly named. The `TB80UTL` classification strings are known. They are not yet tied to the kind bytes inside `14 00 00` chunks. CMP is 5.6 percent pattern-nameable because most of it is the compiler, not a wrapper around a Windows API. A name on a CMP function is useful when it is `COMPILEREXPANDTEXT`. A name on a CMP function that only moves a token is not.

9. The modern stack

Claims

Id	Grade	Statement	Source
M-001	Known	The clean-room reader is not the engine. It does not score quizzes, place every bitmap, or run the NE images.	WV-005
M-002	Known	Foolbook 1.0.2 opens `.tbk`, `.sbk`, `.fbk`, and `.fsk`. A polygon becomes the point array `Polygon` is given. Twips convert with `(coord * dpi + 720) / 1440`. Each object is placed from its twip bounds. Twip `0, 0` is the window origin.	Foolbook `v1.0.2`
M-003	Known	Foolcourse is the process that serves a book. Foolbook is a local window. If Foolbook is started by a course, it pulls the book and skips the splash.	release notes, 2026-10-13

Id	Grade	Statement	Source
M-004	Known	The dependency order is Foolversity, then Foolcourse, then Foolbook, then `openscript_analysis`. Foollege is the layer under Foolversity that holds courses. It is not a running server in the current release.	hierarchy note
M-005	Known	TBpipeline 1.0.0r4 parses `9e954d01.tbk` (129 pages) without running RTM. A known bug is an 18-versus-23 render count.	pipeline release note
M-006	Thought	A reader that prefers explicit points, then TBX Vertices, then bounds, will look like the lesson often enough to study it. It will not be a substitute for the Win98 runtime until the reader in chapter 4 has a name and the `0x4097` slot is located in the page blob. The property's name is already known.	project philosophy

9.1 Foolbook

The engine parses the 8.1 container. It does not load `TB80BAS`. Polygons use the kind `0400` layout (count at `+0x1A`, pairs at `+0x22`). Rectangles use the bounds-only tail (`01 00 00 04`). On Logic Families that is on the order of 165 shapes from the early polygon-and-rect pass, and 961 rectangles from the later marker scan. Both counts can be true. They are different patterns.

Foolbook 1.0.2 places those shapes before the viewer sees them. The conversion is the one in chapter 5: `(coord * dpi + 720) / 1440`. At 96 dpi that is the `/15` case, with the 720 added so the division rounds. The placed polygon is a pixel point list. The placed rectangle is `x, y, w, h` in the window. Twip `0, 0` is the origin. The tag is `v1.0.2`. Foolcourse `v1.0.0r3` and Foolversity `v1.0.0r3` depend on that tag.

System books: core `.fsk` first, user `.fsk` after, as mods. `.sbk` is accepted. It is not required for the v1 player.

The window is native. A direct start opens `using_foolbook.fbk` and a splash that asks whether to resume the last page. A start from Foolcourse, or a start on a named file, skips the splash and uses the last page the course recorded. The splash text is a joke (`2+2=5` / "For greater values of 2."). It is not part of the book format.

OpenScript in this player is the subset in chapter 6. ActiveX, Python, and C++ as replacement engines are a stated direction. They are not implemented.

The book API and the object layer are separate modules in Foolbook (`engine/cdb.py`, `engine/obj.py`). A property get locks the `0x76` slot, runs the behavior, and unlocks. File open reaches the object file through the unnamed helpers, not by a direct call. That is chapter 20. It does not load `TB80BAS`, and it is not the reader.

9.2 Foolcourse and Foolversity

Foolcourse is the only server in the current release. Foolbook does not listen. The earlier idea that Foolbook itself served on port 1337 was withdrawn: a local viewer should not be the thing that stays up after the tab closes.

Foolversity writes students, schools, and curricula. Foolcourse reads them. The hierarchy, as stated, is: a Foollege sits under Foolversity and has Foolcourses; a Foolcourse has Foolbooks; a Foolbook can run OpenScript. Foollege is not the current server. A later Foolcollege was discussed as the place that would hold several courses for several students. That program does not exist yet.

Install is a virtual environment and `pip install` of the Foolversity git URL. A system Python on Debian will refuse the install. That is PEP 668, not a missing package. If a dependency has not changed, a reinstall should not download it again unless a fresh-start flag is set. That rule was written after several reinstalls pulled `openscript_analysis` for no content change.

9.3 TBpipeline and the unpacker

TBpipeline's useful stage is the file parser: load order recorded from a SoftICE log, then a TBK open, then a page walk, then HTML. The Unicorn RTM stage still stops before the book is opened, so a render from this package is a parse, not a playback. Curves, including an attempt at the P420 flame, are emitted as separate files. They are not placed on the page (WV-004's falsifier still stands for placement).

`toolbookunpacker` produces an honest NE image. It clears the Optloader self-load flag. It does not patch the entry table for Ghidra. That patch lives beside the unpacker as an optional post-step. Only BAS, on the current set, has entry segment bytes at or above 128.

9.4 What "getting NIDA working" means

Three different finishes are easy to mix up.

Finish	Done when
A page can be read	Field text, jumps, and the decoded shapes for that page match the Win98 screen closely enough to study the lesson.
A lesson can be walked	Next, Back, menus, and quiz flow follow the handlers, including the jumps into `nida81.tbk`.
A lesson can be run	The system books load, `initializebase` / `initializerecs` / `initializetest` do what the DLL exports imply, pictures land on their widgets, and the unnamed reader is no longer unnamed because the clean-room code does the same thing for the same bytes.

The first is partly done for Logic Families. The second is partly done for the menu and the quiz ids that have been transcribed. The third is not done. Authorship and a modern `TB80CMP` are side work. They help the next person. They do not open P1040.

10. Open, retired, and superseded

10.1 High

These block a faithful page.

Gap	Grade	Why it is still open	Next step that would close it
Runtime record reader	Known as the read in chapter 4.6: `1178:08B2` → `1188:04DA` → `1188:08D6` → `1160:02CC`.	The bytes are in memory before message `0xE`. Still open: the dispatcher for handle `0x0C36`, and which fields of the 412-byte body are the page.	One Bochs stop at the return of `1180:03C8` for that handle. Appendix L. Do not break ordinal 51.
Picture id	Known for a button and the directory (G-012).	The id is the fourth word. The button stores it plus one. A background picture is the block at `+0x37` when that block is present.	Which of the non-bitmap huge entries are palettes.
Property expansion	Known that `FUN_12c8_0ef8` copies four words from a structure that is not the disk record.	Offsets `+0x3E` and friends do not match a `14 00 00` header.	Only useful on the author path. For runtime, fold this into the reader break. Do not keep it as a separate `CDBSENDCREATE` hunt.
Tbload vtable	Thought. `FUN_00406a70` calls vtable+4 after the marker search.	The 32-bit search is found. The callee is not.	One break at `00406aa1` with the book path in a register. Low value until the reader is the goal again.
Polygon type table	Known for the specimen.	Class `0xF` and class `0x10` both call `POLYGON`. This book uses `0x10`, count at `+0x30`, points at `+0x32`.	Class `0x14` curves, and the eight class `0x19` records in `nida81.tbk`.

10.2 Medium

Gap	Grade	Note	
P420 stroke breaks	Thought	Three 96-point groups in TBX. One tangled dump from `+0xA13`. Look for length prefixes between runs before inventing a new curve format.	
Arc on disk	Known live, unknown on disk	Type `0x11`, four pairs, fill at `+0x18`. Angles not located. No `GDI!PIE`.	
Stroke color and pen width	Stroke known (G-011). Width is the pen table	Fill is `+0x1B` on a draw object and `+0x5B` on a background. Styles 1 through 9 are none, dot, dash, solid, then inside-frame widths 2, 3, 4, 6, and 8.	The table is `DS:0x1708` in the unpacked DLL. Not re-checked on a painted line.
Palette on an 8 bpp BMP	Known that segment `1150` builds a palette	Not known which palette a given carved BMP must use.	
`DRAWTEXT` beyond three offsets	Open	`param_5+0x4A`, `+0x4C`, `+0x50` are the paint path. The disk handle is word 1 at `+0x24` (O-009).	
Per-pin index `+0x52`	Known the field exists	Not applied. Shared point bytes stack.	
Background, parent, z-order	Background known on disk	Page `+0x47` names the class 4 record, 131/131 (O-011). Parent is the near pointer at `+0x04`. Z-order is the list at `+0x2C`, not a byte.	The on-disk bytes of that list header were not dumped.
`initializebase` and `toolbook.lst` order	Thought	Names are in the lesson. The call log is not.	
OpenScript keyword lock	Thought	Instructor 8 compile, then expand. Not run.	
`SENDMESSAGE` / `LSTRLEN` arguments	Known mangled	Re-decompile. Not a new algorithm.	

10.3 Retired

Idea	Grade	Why it stays closed
On-disk `14 00 00 01` becomes `14 00 00 11` in memory	Not true	Both values exist on disk. A later live object used `14 00 00 02`. WV-010.
`FUN_1028_2680` is the reader	Not true	Filter registrar.
`FUN_1008_2d31` is the reader	Not true	Math.
RTM contains the reader	Not true	Paint handoff only.
`CDBCHUNK*` is the reader	Not true	Text separators.
`GDI!PIE` draws pies	Not true	No import.
The reader must contain the literal bytes `14 00 00`	Thought, not a requirement	Length-prefixed walks would not. Do not close a function for lacking the literal, and do not keep scanning for the literal as the only method.
HTML listing absence	Not evidence	`??` runs and forced-disassembly passes exist.
`TB80BMP` resolves a stored bitmap id	Not true	The id is a type directory in `TB80BAS` segment `1028` (G-010). BMP's palette notify can still run.
`TB80FLT` reads a bitmap out of a book	Not true	It is the import filter. A picture already in the book does not go through it.

10.4 Superseded notes

Read these for the function bodies. Do not read them for the runtime path.

Note	What it got wrong
`tbworkingpapers/tbwp0002/bas-create-path.md`	Put `CDBSENDCREATE` on book open.
`tbworkingpapers/tbwp0008/cdbsendcreate-sites.md`	Same chain, more callers. The callers are author-side.
`TBdecompile/trail/20261015/00-how-toolbook-operates.md` §2	Same chain, written as “the create path that has been identified.”
`TBdecompile/trail/20261015/01-theories-of-operation.md` §1	Still calls the reader “the caller of `CDBSENDCREATE`.”
`TBdecompile/trail/20261014/02-still-open.md`	Same wording. The rest of that file’s open list is still the open list.

Note	What it got wrong
`TBWorldview/ToolBook_Architecture.md` book-open flowchart	Draws `CDBSENDCREATE` once per widget.

The correction, in one paragraph: Instructor calls `CDBSENDCREATE` when it creates or rewrites an object. A lesson open does not. The function that turns a `14 00 00` chunk into a live object during that open has not been named. It does not call ordinal 51. `FUN_12c8_0ef8` is on the author side of the same line.

10.5 Side quests that are real and not the reader

Kept because they will matter after the reader has a name.

- Object pool: `FUN_1478_014a` allocates near objects. `FUN_1478_00a0` is the pool.
- RTM dialog chrome: `GETDLGITEM` id `0x65` (55 hits), `GETWINDOWLONG` index 10 (196 hits). `FUN_1068_0098` is `GLOBALALLOC` with an out-of-memory retry.
- Author mode exports on BAS, and `GETAUTHORLOCALIZEDDLL` on RTM. The author binaries themselves are not in the archive.
- Password cipher in TBCBT segment 7. Constants `0xA01` and `0x300` after a failed password.
- CMP expand, parked at the W16 segment-not-present fault. Not on the NIDA open path.

10.6 Order of work

1. One lesson open under WDW. Break `TB80BAS@CDBFILEOPEN`, resume, then break the next BAS export that runs before the first paint. Log it.
2. When an object exists, dump `+0x0F` and the page offset it came from. That pair names the reader, whatever the export happens to be called.
3. Only then place bitmaps, split the P420 strokes, and apply `+0x52`.
4. Do not schedule another full-module naming pass, another Unicorn boot, or another break on ordinal 51 in service of step 1.

10.7 The loose sheets are not a second record

Chapter 18 names every markdown sheet under `tbworkingpapers/` and the section of this directory that replaced it. A sheet can still describe a real function and still be wrong about when that function runs. Leaving the sheet on disk keeps the trail. It does not keep the old claim alive. The printable form of the claim is the PDF next to these chapters.

11. Keepers

Addresses in this chapter are from the 2026-10-15 Windows 98 boots, DGROUP `347F`. Match a later boot on the `enter` size and the next two or three instructions, not on the selector. Decompile line numbers are from `TB80BAS.DLL.c` and do not move when the selector does.

`CDBSENDCREATE` is not in this list. It is author-only. See chapter 4.

11.1 File open and close

Function	Live (one boot)	Enter	Decompile line	Role
`CDBFILEOPEN`	`4C7F:017A`	`000A`	27592	Book-open export. Ordinal 8.
`CDBFILECLOSE`	`4C7F:0948`	`0002`	27965	Close. Same caller cluster as open.
`CDBFILEPATH`	`4C7F:09E0`	`0002`	28002	Path query.

Function	Live (one boot)	Enter	Decompile line	Role
CDBFILESTATUS	3ACF:0000	0004	30210	Status.
CDBQUERYMULTISAVE	4C7F:0B90	0018	28108	Multi-save query.
CDBMULTISAVE	4C7F:0E24	000C	28272	Multi-save.
OBJFILEOPEN	3147:177C	025A	76162	Large frame. Buffered read. Downstream of the reader.
OBJFILECLOSE	3147:10EA	0002	75887	Object-file close.
OBJGETFILE	3147:0094			Object file access. Seen on the page path.
OBJGETHDRLONG	3147:0242			Header long. A 4-byte read here was an internal pointer, not a page header.

CDBFILEOPEN callers seen: 322F:3252 (jump), 3D6F:0416 / 0F79 / 0E8E, and the 3247 spine. OBJFILEOPEN callers include 3D6F, 3247, and OBJGETFILE.

11.2 Object lifetime

Function	Live	Enter	Decompile line
OBJFREE	3147:0CCC		75619
OBJFREENEAR	3147:0D70		75662
OBJUNLOCK		004A	call site, not a definition
OBJREALLOCNEAR	3147:08D6	0018	
CDBCLEARBOOKREFS	4C7F:10A6	00B8	28451
CDBINIT		00C8	30304

HEAP* / TBHEAP* do not hand out the 0x76-byte object. That sentence, and the sentence that TBHEAPCLEARCRITSECT has no definition, are not true. TBHEAPCLEARCRITSECT is decompile line 93984. The object array is GLOBALALLOC of count * 0x76 in FUN_1278_3be6. Chapter 19 is the heap, the value stack, and RESMGR*. TBHEAPCLEARCRITSECT was a stop on the open path. It was not the last stop before the window. VALUEFREE was.

11.3 Page, instance, and the value bus

These are closer to the unnamed reader than ordinal 51. On the 2026-10-15 captures, CDBSENDCREATE was listed in the Functions window and did not appear in the Calls stacks. CDBCREATEPAGEREF and CDBSETVALUEEX did.

Function	Live	Enter	Decompile line	Note
CDBCLASSTABLEGETBEHAVIOR	44F7:121A	0008	125154	Splash gate. Unbreaking it let the splash run.
CDBCREATEINSTANCE	480F:0000	0002	24974	Instance constructor.
CDBCREATEPAGEREF	483F:03B2		25913	Page reference. Next break.
CDBVIEWNONE	483F:02F8	0002	25877	View state.
CDBVIEWPAGE			25726	Ordinal 15.
CDBVIEWBKGND			25825	Ordinal 18.
CDBSETVALUEEX	3397:147C			Extended property set. Next break.
CDBGETVALUE	3397:0D54		38475	Property bus.
CDBGETVALUEEX			38488	Extended get.
CDBPARENTVALUE	3397:07B2			Parent value.
CDBDEREFVALUE	3397:0400			Deref.
CDBLOCKVALUE	3397:02CA	0002	37826	Shifts 0B[bp]. Not a record parser.
CDBCLIENTSTATUS	480F:04C6			Client status.
CDBREGISTERINSTWINDOW	480F:0522			Instance window.
CDBDISPLAYSTATE	4877:0000			Display state.
CDBSYNCSCREENS	4877:0276			End of the author create path. Seeing it does not mean ordinal 51 fired.
CDBSENDSELECT	44CF:009E			Select. Not create.

Value constructors in segment `4267` fire often and are not the stream walker:

Function	Live	Enter
CDBVALUENEWBOOKREF	4267:0000	0004
CDBVALUEBOOKFROMSTRING	4267:03F4	0018
CDBVALUELAYERFROMSTRBOOK	4267:05F8	004C
CDBVALUERESOURCEFROMSTRING	4267:08B2	004A
CDBVALUEGRAPHICFROMSTRING	4267:0AA0	0074
VALUENEWBOOK	4267:0D90	0006
VALUENEWRESOURCE	4267:0EFC	000E
VALUENEWGRAPHIC	4267:1494	0008
CDBVALUENEWGRAPHICREF	4267:02D4	
CDBVALUENEWLAYERREF	4267:00C2	

`VALUENEWLONG` builds a long. It is not a record parser.

11.4 Chunk walkers

These walk characters: comma, CR, LF, tab, space, quote. A decompile that shows `ISDBCSLEADBYTE` and a comma test is this family.

Function	Live	Enter	Decompile line	Callers
CDBCHUNKEND	0D37:0000	0004	138456	3187:0210, self from FIND
CDBCHUNKFIND	0D37:0146	0008	138575	4857:0C68, 3187:0210, 322F:0577
CDBCHUNKLAST	0D37:0334	0010	138676	4857:090B, 3187:0210

11.5 Paint and resource binding

Function	Live	Decompile line	Note
CDBPAINTCLIENT	389F:0506	13056	Ordinal 26. RTM `WM_PAINT` ends here.
IMBITBLT		13277	BitBlt path.
IMDRAWLEVELS		13353	Draw levels. `IMDRAW` as a bare name is not a definition.
IMBLACKOUTBMP		13995	
IMINVERTBMP		14021	
IMSHOWBMP		14047	
RESMGREXTRACT		21513	Resource manager.
RESMGREXTRACTGDI	4D27:1730	21523	Extract for GDI.
GETDIBCOLORCOUNT	35D7:053A		Enter `0006`.
GETDIBSIZE	35D7:05DA	not in the BAS or BMP listing	Enter `0008`.
HUGEGETDATA	33BF:0D6C		Enter `0018`. Large fetch.
COLORHLSTORGB	44B7:0B84	155991	Menu color. Enter `0034`. Not the page painter.

Painter offsets that do not move inside the segment are in chapter 8. `PaintPolygon` is BAS segment `0x82` offset `0x376`.

11.6 Incidental on open

Seen, and not the reader: `CDBREGISTEROLE2WINDOW` (`3767:0000`), `CDBREGISTERVBXFORM` (`4437:0000`), `VBXNEWVIEW` (`4437:0216`), `VALUEARRAYINFO` (`4357:02F2`), `VALUEARRAYSET` (`4357:0932`), `VALUEARRAYGET` (`4357:05D8`), `HEAPVALUETOPOINTER` (`450F:0F28`), `HEAPVALUETOFRAME` (`450F:0F4E`), `HEAPENTERCITSECT` (`450F:0DCE`), `HEAPLEAVECRITSECT` (`450F:0DF6`). What the heap stops are is chapter 19.

`?SENDPROC@TBXBOBJ@@` was seen at `29AF:0000` (`cmp ax, 3014`). The decompile does not contain `SENDPROC@TBXBOBJ` as that name. Related symbols are `_SETPROC_TBXBOBJ` at line 2175, `_SENDPROC_TBXLAYER`, `_SENDPROC_TBXBASE`, and `_SENDPROC_TBXUSER`.

12. The book-open spine

12.1 The frames that repeat

On the 2026-10-15 book-open and page walk, these frames were in nearly every WDW Calls window. Selectors are that boot only.

Selector:offset	Note
`3497:03EF`	Early frame.
`3497:4A6E`	The planned step point.
`3197:0379`	
`3167:0CB1`	Near call.
`3167:14BC`	Far call.
`3247:02BF`	Near.
`3247:05B7`	Near.
`3247:0B7A`	Far.

Everything else in those captures hangs off this spine. The full selector-to-decompile table for that boot is `maps/wdw-to-ghidra.md` (485 lines). It is a lookup, not a finding. Do not treat a selector from that file as valid on the next boot.

12.2 What the three export traces contained

WDW has no full call trace. `break <symbol> resume on` plus `log <file>` logs named exports and continues. `resume on` is a property of a breakpoint that already exists. Module qualification is `TB80BAS@OBJFILEOPEN`. SoftICE `!` does not work in WDW.

Trace	Shots	What fired	What did not
W16	11	`GETPLATFORMREF`, `IS32BITENV`, `GETRUNTIMELOCALIZEDDLL`, `ACXINITIALIZETOOLBOOKSUPPORT`, `ACXOLEINITIALIZE`, `ASYM_STRINGFROMCLSID`, `SETTHUNKINGLAYERSERRORFUNC`, `LOADLIBRARYEX32W`, `GETPROCADDRESS32W`	Book open. Record walk.
UTL	53	`GETCHARINFO`, `GETSTRMGRSTRING`, task collections, `PLUGINSVCINIT`, `LOADRESSTR`, `CXTHELPINIT`, `GETTBKINIPROFILEINT` / `STRING`, `INITMETRICS`, `SETCONVERSIONDC`, `CXTTOPXT`, `GHEAPCREATE`, `GHEAPALLOC`, `SYMLOOKUPNUMBER`	Record walk. The type-name table lives here and was not the thing being stepped.
BAS	173	`VALUETOSTRING`, `VALUENEWSTACK`, the `VALUENEW*` family, `OBJFILEOPEN` (`enter 025A`), `STACKGETSIZE`, `STACKPUSH`, `CDBCHUNKLAST`, `CDBCHUNKFIND`, `CDBCHUNKEND`, `RESMGR*`	A compare of a far pointer to `0x14`. Ordinal 51.

What those stack, heap, and `RESMGR*` hits do is chapter 19. They are still not the reader.

`OBJFILEOPEN` in that BAS trace is the file-open path. The buffered reader is `FUN_1160_03de`, reached through `FUN_1258_00a8`. When the buffer satisfies the request, the raw DOS path at offset `04F3` is skipped. Live 4-byte reads returned an internal pointer (`FC B5 BD B0` in one dump), not a page header and not `03 4A 42 4F`.

12.3 Where to break next

1. `TB80BAS@CDBFILEOPEN` on a lesson open. Resume.
2. Then `CDBCREATEPAGEREF` or `CDBSETVALUEEX` (export names, not the selectors above).
3. When an object exists, dump `+0x0F` and the page offset it came from. That pair names the reader.
4. Do not break ordinal 51. Do not schedule another pass over the same W16 and UTL exports in order to find the reader.

The reader is still unnamed. The working model is in chapter 4: a length-prefixed walk in BAS after `CDBFILEOPEN` and before the first paint, with no literal `14 00 00` compare required.

12.4 Debugger facts that keep wasting a session

SoftICE field work closed 11 October 2026 (`SI405w9x` plus ICEDUMP 6.021). Ordinal breakpoints survive a reboot. Selectors do not. `TB80BAS.Ordinal_8` is `CDBFILEOPEN` on every boot.

`/DUMP` writes the SoftICE console unless the dump experts are set. Several early `page.bin` and `rtm.bin` files are command logs or the DOS device chain (`CON`, `AUX`, `PRN`). A dump is real when it contains book strings at offsets already known, as `page.bin01511` did (`ASYM_TpID`, `Page#`, matching `9e954d01.tbk` at `0xB8B51`).

Symbol maps are `NE-segment:offset` names (`TB80BAS.map`, 3799 entries; `TB80RTM.map`, 2669). SoftICE and WDW do not load a `.map` as a module.

The CMP expand host died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present). That fault is parked. It is not the book-open path.

The Unicorn host has not opened a book. Passes 01–07 fill segments and bind 461 ToolBook ordinals, then die at `FFFF:0158` and `0080:0098`. Pass 06 exits after `TB80.INI`. The three hosts (the RTM experiment, `newl`, `neww`) are not one program.

13. Specimen pages

Every number in this chapter is from `9e954d01.tbk` (Logic Families) or from its ToolBook 11.5 Save To XML, unless a path says otherwise. Screen pixels are stored twips / 15 at 96 dpi.

The binary has 129 pages, each a record starting `80 65 00 05 00 02 02`. The export has 131 `Page.xml` files. P440 and P2920 have no `80 65` record of their own. All 129 binary names match a TBX page. P440's sentence and the house shapes that sit on its TBX bounds are inside the P430 record. See 13.5. P2920 has not been searched the same way.

TBX folder names (`Page880`) are export ids, not the runtime page name (`P1040`). IdNumber 880 is page `P1040`.

13.1 P1040 — two DIP packages, bounds only

File offset `0x19C03A` (1687098). Page blob 7764 bytes. Background id 2. Next goes to `P1050`. Back goes to `P1001`.

The 8.1 file does not store the 11-point pin tips that the XML writes. Each pin is four twip words, a zero dword, fill `0xC0C0C0`, then marker `01 00 00 04`. The tip is expanded at paint, and the export writes that expansion as Vertices. The book has 961 copies of the `01 00 00 04` marker. P1040 uses 28 of them for pins and 2 for bodies.

7400, right package

Body at blob 0x09F4: bounds 4875, 3555, 7830, 4605, screen about 325,237–522,307, fill 64,64,64 (0x404040). Label field text 7400.

Bottom pins, top 4605, bottom 4860:

Blob offset	Bounds (twips)	Screen about
0x03E4	5055–5205	337,307–347,324
0x044C	5460–5610	364,307–374,324
0x04B4	5880–6030	392,307–402,324
0x051C	6315–6465	421,307–431,324
0x0584	6705–6855	447,307–457,324
0x05EC	7125–7275	475,307–485,324
0x0654	7560–7710	504,307–514,324

Top pins, top 3300, bottom 3555:

Blob offset	Bounds (twips)	Screen about
0x071C	5055–5205	337,220–347,237
0x0784	5460–5610	364,220–374,237
0x07EC	5880–6030	392,220–402,237
0x0854	6315–6465	421,220–431,237
0x08BC	6705–6855	447,220–457,237
0x0924	7110–7260	474,220–484,237
0x098C	7530–7680	502,220–512,237

The first pin's bounds 5055, 4605, 5205, 4860 are the start of the TBX Vertices list. That is the expansion direction: bounds in the 8.1 file, point list in the XML.

5400, left package

Body at blob 0x117C: bounds 1500, 3555, 4455, 4605, screen about 100,237–297,307, same fill 64,64,64. Label 5400.

Bottom pins, y 4605–4860: 0x0B6C 1680–1830, 0x0BD4 2085–2235, 0x0C3C 2505–2655, 0x0CA4 2940–3090, 0x0D0C 3330–3480, 0x0D74 3750–3900, 0x0DDC 4185–4335.

Top pins, y 3300–3555: 0x0EA4 1680–1830, 0x0F0C 2085–2235, 0x0F74 2505–2655, 0x0FDC 2940–3090, 0x1044 3330–3480, 0x10AC 3735–3885, 0x1114 4155–4305.

TBX group ids: 53 is the 7400 package, 52 is the 5400 package, 58 is the discussion title (gold plate button, graphic "bitmap id 149", plus a white "DISCUSSION" field and a blue drop shadow). The widget still does not name that bitmap inside the 8.1 object. See chapter 5.

Intro field bounds 600, 600, 8805, 2490. Next button 8907, 6210, 9342, 6630. Back button -3, 6210, 432, 6630.

13.2 P430 — kind 04 00 polygons, including a live pin tip

9e954d01.tbk has 96 kind-0400 records on 13 pages. P430 has 22. A search that ignores the kind word produces false positives.

Record layout: fill dword at +4, kind 04 00 at +0x0C, count word at +0x1A, twip pairs at +0x22. The painter is FUN_1408_0376. It copies object +0x30 / +0x32 to DS:30AC and calls GDI!POLYGON.

P430 draws as a chip: silver pins (0xC0C0C0) and a light-gray body (0xE0E0E0). One live 11-point polygon is a pin tip, not the body. Captured screen points ran (218,335) through (218,341), a 10 by 17 notch. The same points are in the

file as twips near page offset 0x9CB: (3147,3862) through (3147,3973).

Pins that share identical point bytes stack until the index at object +0x52 is applied. The clean-room draw does not apply it.

13.5 P440 — no page of its own

The TBX page is Page876, Name="P440", background id 2. The text field says the ICs sentence: classified into logic families by voltage, speed, and how many circuits they can drive. Four houses are irregular polygons. Next and Back are buttons with bitmap ids. The background field named Page# stores the text P1270. That is the background's stored sample, not this page's name.

The 8.1 file has no page named P440. The loader read 129 pages. The sentence is at file offset 834271, inside P430. That page starts at 0xC3312 and its blob is 42512 bytes, so the sentence is 34765 bytes into the blob.

The house shapes are in that same blob. Drawing every shape in it draws the chip and the houses on one window, plus false hits from the scan, including a yellow polygon that covers the page. Keeping only shapes whose bounds overlap the TBX page, and dropping a box larger than the page, leaves the four houses, some chip shapes that share those bounds, and one yellow false polygon. It does not draw the blue background, the sentence, or the button bitmaps. Those are not in the shape extract.

Known. P440 is not a missed 80 65 record, and it is not absent from the file. The sentence and the overlapping shapes are in the P430 record. Thought. The 11.5 export split that one record into a chip page and a house page. That split has not been watched in 11.5. P2920 is the other export page with no 80 65 record, and this search was not repeated for it.

13.3 P420 — the curve that is not yet a flame

Points sit at page +0xA13. Curve's type id in nc81base.sbk is 0x05C7 (file offset 0x1810). That id is not in the P420 page records. A naive walk draws a tangle in screen space around 400–500. The TBX export is three 96-point red IrregularPolygons. There is no Curve name on the page.

Thought. The tangle is strokes concatenated that should be drawn separately. A length prefix or a zero terminator between groups has not been proven. Do not treat a raw dump of +0xA13 as the flame.

13.4 P1170 — the bracket is computed, and the word table is an index

Live polyline: (354,201) (321,201) (321,238) (358,238). Object bounds in twips: left 4801, top 3015, right 5370, bottom 3570. Divided by 15 that is the same rectangle. BAS segment 0x5B computes the four points. They are not stored.

The 27 ascending words at page +0x8C3 are an offset directory into a 14 00 00 83 blob, not a Group. On this page the blob is file 0x1AB651, 1844 bytes, and the word table is file 0x1ABE97. The words begin 0x14, 0x16, 0x40, 0x52, 0x66, 0x7A, 0xC2 and are relative to the blob. Script-like markers inside the blob (buttonClick, "P1180", "P1140") sit at blob +0xA9, +0xB9, +0x17B, +0x18B. After those regions are set aside, the table still points at short structured runs (type bytes, small integers, 0xFFFFFF). A Group is its own object and has its own member list.

13.5 P400 record sizes

P400 is 7046 bytes and 12 records. The early walk, kept because the sizes show what a "page of widgets" actually is:

#	Page offset	Length	Kind	Note
0	`+0x2B`	33	`0001`	Small.
1	`+0x4C`	31	`0000`	Small.
2	`+0x6B`	18	`0D00`	Small.
3	`+0x7D`	1458	`D900`	Content blob: scripts and colors.
4	`+0x62F`	2828	`0021`	Large. Not a polygon.
5	`+0x113B`	894	`0000`	Count word at `+0x1A` is 21, not a kind-`0400` polygon.
6	`+0x14B9`	156	`1600`	Small.

13.6 Lesson jumps that have been read

Page	Handler
P900	TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Menu returns to P900.
P1290	Correct choice 5408. Again → P1292. Next → P1300.
P20000 of `nida81.tbk`	`buttonClick` goes to P20001 because the handler says so.

13.7 TBX draw inventory

Tag counts across the `9e954d01` export. These are XML tags, not kind-`0400` records. A page can have hundreds of IrregularPolygons that the 8.1 file stored as bounds.

Kind	Count
IrregularPolygon	654
Line	528
Field	497
Button	382
Group	371
Rectangle	243
AngledLine	66
Arc	62
Ellipse	45
Hotword	5

Pages whose export folder contains at least one IrregularPolygon. Columns are IrregularPolygon, Rectangle, Line, AngledLine, Arc, Ellipse, Group, Field.

TBX folder	Poly	Rect	Line	Angled	Arc	Ell	Group	Field
Page872	14	2	2	0	1	1	4	7
Page873	44	5	0	0	2	2	2	4
Page874	8	2	2	18	0	0	2	2
Page875	96	8	2	1	8	0	8	1
Page876	12	32	108	0	0	0	48	5
Page880	28	2	0	0	2	0	5	5
Page885	28	4	0	0	2	0	4	5
Page887	14	1	0	0	1	0	2	8
Page891	14	1	0	0	1	0	2	8
Page897	14	1	0	0	1	0	2	8
Page900	14	1	0	0	1	0	2	8

TBX folder	Poly	Rect	Line	Angled	Arc	Ell	Group	Field
Page901	42	3	0	0	3	0	3	4
Page909	14	1	0	0	1	0	2	8
Page912	42	3	0	0	3	0	5	7
Page914	28	4	0	0	2	0	5	6
Page915	42	3	0	0	3	0	5	7
Page917	42	3	0	0	3	0	4	8
Page918	42	3	0	0	3	0	4	8
Page919	14	1	0	0	1	0	1	4
Page923	28	2	0	0	2	0	5	5
Page931	3	8	27	0	0	0	13	4
Page932	9	28	81	0	0	1	37	5
Page934	14	1	0	0	1	0	2	2
Page940	6	22	54	0	0	2	24	5
Page941	14	1	2	0	1	0	2	4
Page943	14	1	2	0	1	0	2	4
Page949	2	0	7	0	0	0	0	2
Page961	12	32	108	0	0	0	49	7

Page880 is the P1040 export: 28 polygons, 2 rectangles, 2 arcs. The 28 polygons are the paint-time expansion of the 28 bounds records in section 13.1. Page875 (96 polygons, 8 arcs) is the dense drawing page in the export. It is not, by itself, proof of 96 kind-`0400` records.

13.8 Pictures, stated with the two offsets kept apart

Windows BMPs can be carved. One match against the export paired 67 markers with 72 TBX bitmaps. The widget does not name the bitmap. Compressed pictures go through `TB80BMP` ordinal 3 and have been followed as far as a far call at `0x1FCD`. `Lz77Expand` is BMP `FUN_1000_0031`.

The shared near-start graphic is a 581 by 157, 8 bpp gold plate, not a text banner.

Two notes locate the `nida81` splash and they are not the same byte: file `0x413FAB` (323 by 344, 8 bpp, off-bits 310), and a carved copy taken from file `0x487BE` (27874 bytes). The live `BITMAP` header that was found was width `0x143`, height `0x158`, 8 bpp, stride `0x258`, `bmBits` null. The splash is a DIB. Which property on the page object holds its id is not known.

Lesson picture counts from the early carve: 71 BM and 5 JPEG in the lesson, 76 BM and 12 JPEG in `nida81`. BAS segment `1150` builds the book palette. An 8 bpp BMP needs that palette. Which palette a given carved BMP must use is open.

14. OpenScript bytes

14.1 What is current

Known. Handlers are compressed bytecode. The keyword pool is `TB80CMP` NE resource `0x77` (type 100, id 119). Resource `0x78` (id 120) is the other emit routine. `COMPILERPROPINFOFROMINDEX` locks resource `0x70`. Property names are a different table: BAS resource type `0x2D1`, id `0x2D0` (`bounds` `0x4004`, `idNumber` `0x4015`, `script` `0x4023`, `text` `0x402E`, `vertices` `0x4034`).

An early string dump of resource `0x77` counted 992 strings in 15776 bytes, and of `0x78` counted 980 in 15616 bytes. The decoder that actually produced the 76 Logic Families handlers counts 994 keywords. Use 994. The offset table starts at byte 2 plus count times 4. An earlier loader used count times 2 and was wrong even before the back-reference

bug.

`COMPILERCOMPILETEXT` is CMP ordinal 1. `COMPILEREXPANDTEXT` is ordinal 3. `CFUNEXECUTE` is 8. `CFUNEVALUATE` is 9.

14.2 Encoding

From `FUN_1040_0970`. This is the encoding the handlers decode with.

Range	Meaning	
`0x00`–`0x7F`	Literal character.	
`0x80`–`0xBF`	Keyword. Index is `byte & 0xF`, or `((byte & 0xF) << 8) \	next` when bit 4 is set. A space is appended when bit 5 is clear.
`0xC0`–`0xFF`	Back-reference. Copy from the compressed input stream, not from the output.	
`0xE0`–`0xEF`	CRLF plus spaces.	
`0xF0`–`0xFE`	Spaces.	
`0xFF`	Repeat.	

The `0xC0`–`0xFF` range is also where the whitespace classes sit. The expander classifies the byte before it treats it as a generic back-reference. Do not implement "every byte above `0xBF` is a back-reference" and also implement the whitespace rows as a second pass over the same byte without the classification `FUN_1040_0970` uses.

Back-reference, as fixed on 2026-10-10:

```
if bit 4 is clear:
    length = (byte & 0xF) + 3
    src = (pos + 1) + (-4 - next_byte)
else:
    length = (byte & 0xF) + 4
    src = (pos + 1) + (-1 - word_offset)
copy `length` bytes from input[src]
```

A windowed decoder that only looks inside the slice under test will mark a back-reference that lands before the slice. On `tb80r.sbk`, reading those from the whole book dropped the `?` marks from 14256 to 66. The 66 that remain are question marks in the script.

The NIDA navigation decoder stops at the first standalone `end`. Trailing garbage is dropped. That is a corpus tool, not a claim that the expander itself stops there.

14.3 The superseded mismatch

`tbworkingpapers/tbwp0001/openscript_notes.md` says, in its first conclusion, that the decoder emits "italic", "third", "fourth", and "systime" where the script should say something else, and that the bytecode might not be OpenScript. That conclusion is superseded by the same file's later sections, and by this chapter.

Two bugs produced it. The keyword offset table was read as count times 2. Back-references copied from the output stream. After both fixes the same bytes are:

```
system lastbook
  lastbook = name of this book
  system nextpage
  nextpage = name of this page
  go to page "P20001" of book "nida81.tbk"
end
```

Other handlers that are real source: `send next`, `send previous`, `go to page "P400"`, `go to page "P310" of book "nida81.tbk"`.

Do not reopen "the keyword table is the wrong resource" from the early half of that note. Keywords live in `TB80CMP`. `TB80BAS` has no `handle` or `buttonClick` strings. `TB80RTM` has a small table at file `0xAACAD` (`handle`, `get`, `set`, `notifybefore`, `notifyafter`, `true`, `false`, `to`) and not the compiler keyword pool.

14.4 Corpus

Book	Handlers	How they are stored
`9e954d01.tbk`	76, on 39 pages	Plain `to handle` prefix, compressed body. An intermediate extract that required a plain `END` found 22. 76 is the count after the input-stream fix.
`nida81.tbk`	100	All `buttonClick`. Same encoding.
`tb80r.sbk`	33	The `to handle` / `to get` prefix is compressed too. A scan that keeps windows whose output starts with those words is how they were found. More `0xFF` and `0xE0` than the lesson books.
`nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`	0 decoded	High-byte runs do not decode as this bytecode. Early pattern counts (61, 17, 94, 62) were not handlers.
`tb80anm.sbk`, `tb80hyp.sbk`	0 decoded	Same. Strings and menus, not this handler format.

Of the 76 handlers in `9e954d01.tbk`, 75 are `buttonClick` and one is `EnterPage`. The 100 in `nida81.tbk` are all `buttonClick`. A handler whose name is not stored as plain text is not in those counts. The 11.5 export of `nida81.tbk` has eleven handler names. Ten of them are absent from this scan.

`openscript_analysis v1.0.0r9` reads this resource the same way: count at byte 0, id index, back-reference from the input, `0xFF` as a new character plus a count. It does not follow a back-reference that lands on another back-reference. The `EnterPage` handler does not need that chase. The release before it read the count from byte 2, skipped `0xC0`–`0xDF`, and repeated the previous output byte.

Lesson books store the handler name as plain text and compress the body. `tb80r.sbk` compresses the name as well. Encoding rules match. "No script at all in any system book" is too strong for `tb80r.sbk` and still true, as decoded text, for the four `nc81*` books.

14.5 What the courses actually do with it

The common subset is `go to`, `send`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, and `name of this book` / `this page`.

The clean-room engine also accepts `send NAME to TARGET`, a `get` that is a call, `forward to TARGET`, `break NAME`, and `PROP of TARGET`. Predicates such as `is not in` are recorded rather than evaluated. `local` and `system` declarations are ignored. Property writes are not applied to a live object. The expression evaluator copies `FUN_1008_2d31`: `+ - * /`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`, and otherwise leaves the original text. That function is not the record reader.

The page-by-page jumps are in chapter 13. Navigation is not "only a quoted page id." P20000 goes to P20001 because the handler says so.

14.6 Not done

A short script compiled in Instructor 8, saved, and expanded with ordinals 1 and 3 would lock the 994-name table against a known source. That test is written down and not run. Nested property expansion, opcodes past the common subset, and `forward` into a system-book handler are unimplemented or only sketched. The CMP expand host that died in `TB80W16` (`001c:00000015`, segment not present) is parked and is not the NIDA open path.

15. System books and the NIDA shell

`.sbk` and `.tbk` are the same container. Most of the Asymetrix `.sbk` files are resource containers: an object-type name table and menu resources. A few carry compressed handlers. The NIDA lesson still opens if the system books are removed. What breaks is the shell: blue screen, resize, and the menu. Renaming `nida81.tbk` removes the splash and reports that the shell is not a book. The lesson can still be entered.

15.1 Type names

The name table, in the order recovered from the system books, is: ComboBox, Stage, Picture, Hotword, RecordField, PaintObject, Curve, AngledLine, Pie, Arc, IrregularPolygon, Polygon, Line, RoundedRectangle, Ellipse, Group, Field, Button, Rectangle, then Page, Background, Book.

In `nc81base.sbk` that table sits around file `0x15E4`–`0x1ADC`. Curve's type id is `0x05C7` at file `0x1810`. The schema spelling `IrregularPloygon` is the 11.5 XSD, not this table.

These names are not yet tied to the kind byte inside a `14 00 00` chunk. `TB80UTL` holds the strings. It has not been shown to be the function that classifies a chunk.

15.2 NIDA shell

`nida81.tbk` is a real book. 43 objects. Pages `P20000` through `P22130`. It does not name lesson ids. It has 100 `buttonClick` handlers in the same bytecode as a lesson. P310 is Minimum Requirements (586, 133 MHz, 12 MB RAM, 1 GB, mouse, Windows 98). P20001 is the small menu. P22130 is Test Console Setup.

`9e954d01.tbk` at file `0xB1A96` names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and it calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`.

Thought. Those initialize names have not been watched as a call log, and the order in `toolbook.lst` has not been watched live. `version.ini` in the set is `03.001.0006`, dated 04-18-05. `CAILT.DAT` maps course ids to Navy CDs. `9E954` maps to DIE. `STDAPPS.LST` is a short encoded list, not a book.

15.3 Asymetrix `.sbk` tiers

Sizes are the files that were triaged. JBO is at `0xC00` unless noted.

Tier	File	Size	What it is for
Core	`tb80r.sbk`	954 KB	Reader book. 66 objects. 33 compressed handlers (chapter 14). Help, bookmarking, logging, path manager, popup page manager. Not the engine.
Core	`tb80.sbk`	3.1 MB	Large core system book. Type table present. Not fully extracted.
Core	`tb80a.sbk`	2.4 MB	Author-side companion. Not on playback.
Core	`tb80i.sbk`	619 KB	Instructor system book.
Core	`INST80.SBK`	70 KB	Installer book. Referenced by Inst80. Names `tb80r`.
Useful	`tb80hyp.sbk`	141 KB	Hyperlink and help. The binary scan finds no `to handle` text. The TBKX export does.
Useful	`tb80anm.sbk`	64 KB	Path animation. The binary scan finds no `to handle` text. The TBKX export does.
Useful	`tb80aicc.sbk`	86 KB	AICC strings. Not shown to be required for a local page.
Useful	`tb80actr.sbk`	208 KB	ActiveX runtime.
Parked	`tb80acta.sbk`	2.1 MB	ActiveX author.
Parked	`tb80axf.sbk`	618 KB	ActiveX related. JBO at `0xE00`.

Tier	File	Size	What it is for
Not on the local path	`tb80web.sbk`	1.1 MB	Web.
Not on the local path	`tb80ftp.sbk`	243 KB	FTP. JBO at `0xE00`.
Not on the local path	`tb80x.sbk`	256 KB	Unclassified specialty.
Not on the local path	`tb80lmx.sbk`	62 KB	Small. JBO at `0xE00`.

An early triage said system `.sbk` files do not contain OpenScript. That is superseded for `tb80r.sbk`. The 21 October TBKX exports of `tb80r`, `tb80anm`, and `tb80hyp` also contain the handler source. A binary scan that looks for the words `to handle` still misses them. The other rows above still have no exported handler text.

The four NIDA books `nc81base`, `nc81recs`, `nc81test`, and `nc81trnr` are password-protected resource containers. Runtime opens them without the password. The author path is the one that asks. The password is not required to read the records. The ciphertext those books store is chapter 16.6.

15.4 Trainer and session DLLs

`nc81int.dll` and `nc81rec.dll` are 32-bit PE. They are not Asymetrix.

`nc81int.dll` exports 47 trainer calls. `IntCheckComm130E` opens a port at 9600 8O1, sends a probe, and returns `0x3D` on match or `0x3C` on failure. Families: 130E, TD2000, 5050, 205E.

`nc81rec.dll` writes `0x46`-byte records to `SESSION3.REC`. Exports include `RecStartSession`, `RecCloseSession`, `RecBookmarkSession`, `RecWriteResponse`, `RecReadVariable`. `initializeRecs` on the shell clears responses when a session loads. That last sentence is inferred from the shell script, not from a live trace of the DLL.

15.5 `linkDLL` versus a system book

A system book adds script above the book. A `linkDLL` adds commands, and only when a handler links it. The set that shows up that way is `TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, and `TBCBT`. `TBWIN` is `INIT_TBKWIN` and `TBM_EVALUATE`. Neither `TBWIN` nor `TB80NET` walks page records. `TBCBT` is the CBT scoring extension and the password cipher. It is not the record reader.

15.6 Providers are not `sysBooks`

Segment 1278 loads class providers: a TBX DLL, a VBX DLL, a 32-bit TBX DLL, or another ToolBook book. It does not push `sysBooks`. The names sit on the book's class table. `a613` names `TB80ANM.SBK` and `TB80R.SBK`. When the provider is a book, or a TBX calls `REGISTERSYSBOOK`, the book goes on a bound list. The Author program adds that list to `sysBooks` through `CDBADDBOUNDSYSBOOKS`. The note is `research/class-providers.md`.

15.7 The 11.5 names are not these books

The ToolBook 11.5 common-file set uses the same filenames as the Instructor 8.1 system books. The files are not those books. Do not replace the rows in 15.3 with them.

An inventory of that set on 24 October 2026 (opentoolbook8) found two stubs:

Bytes	Names	Version resource
29,147	62 names, one hash. Includes `tb105hyp.sbk` and `tb80actr.sbk`, `tb80aicc.sbk`, `tb80anm.sbk`, `tb80ftp.sbk`, `tb80hyp.sbk`, `tb80r.sbk`.	TB110 `11.00.37`
29,145	`tb110actr.sbk`, `tb110aicc.sbk`, `tb110anm.sbk`, `tb110ftp.sbk`, `tb110hyp.sbk`, `tb110r.sbk`.	TB115 `11.50.12`

On both stubs the 16 bytes at `0xC00` are zero. The object-store header appears once, near offset 16440, not at `0xC00`. Neither stub returned book script. The 29,145-byte file shares a prefix with `tb115hyp.sbk` and is not that book cut short. The Instructor copies of `tb80anm`, `tb80hyp`, and `tb80r` stay 64,023, 140,613, and 953,705 bytes. Those three are not in the 11.5 archives.

16. The password cipher

The NIDA system books are password-protected. A lesson runtime opens them without presenting the password. Instructor and the author path are what ask. This chapter records where the cipher is and how the addition works. Section 16.6 is the empty-key branch and the ciphertext stored on the NIDA system books. The keyed 10-byte path in 16.2 is still open. The short strings tried against a stand-in key are not that password.

16.1 Where it lives

`TBCBT.DLL`, NE segment 7. Ghidra's name for that segment is `Code7`, base `1030`. The usual map is `0x1000 + (N - 1) * 8`, so segment 7 is `1030`. NE segment numbers stay fixed. Live selectors change. Segment 7 was not mapped in the SoftICE session that listed TBCBT code segments `01`, `0E`, `02`, `0B`, `08`, `0D`, `04`, and `03`. Those loaded segments are stubs and KERNEL/USER imports, not the cipher.

Exports on this path: `MAKEKEYEDPASSWORD`, `MAKEKEYEDPASSWORDEX`, `KEYEDPASSWORDENCRYPT`, `KEYEDPASSWORDENCRYPTEX`, `ENCRYPT`, `ENCRYPTEX`, `DECRYPT`, `DECRYPTEX`, `DECRYPTFILE`, `CREATEENCRYPTEDFILE`, `READENCRYPTEDFILE`, `WRITEENCRYPTEDFILE`.

The book path is `MAKEKEYEDPASSWORDEX` then `KEYEDPASSWORDENCRYPTEX`.

`TB80RCA.DLL` is Asymetrix UI (ASYMBITMAP, STATIC, TABS, spell, palette, BMR). It is not on the book-open path and not on the password path. `TB80ATH.EXE` is the 16-bit author and runs under NTVDM. The 32-bit pieces on a lesson launch are the launcher and the thunk (`TB80RUN`, `TB80W32`, `TB80MAIN`), not the cipher.

16.2 Three addition schemes

From `MAKEKEYEDPASSWORDEX`, `KEYEDPASSWORDENCRYPTEX`, and `FUN_1030_1046`. Inverse of each is `plain = (cipher - key) & 0xFF`.

Scheme	Add	Key byte
`(0xA01, 0x300)`	`toupper` of the key byte	From a 10-byte table. Index is `local_16 % 10`, then `+ 0x32`.
`(1, 0x400)`	`toupper` of the key byte, wrap at `0xFF`	Same table.
`(1, 0x700)`	The raw key byte, wrap at `0xFF`	Same table.

The live key is the 10 bytes that table selects. It is not the ASCII the user types. A test that encrypts `"nida"` or `"adin"` with a stand-in key `A`–`J` only checks the adder. It does not recover a book.

After a failed password dialog, search memory for the typed ASCII and for the constants `0xA01` and `0x300`. That finds the call site even while segment 7 is unloaded. Clicking OK and accepting "Wrong password" is the intended setup for that search. The dialog procedure itself is an ordinary Windows dialog.

16.3 What a with-password file difference is

A small book saved with a password and the same book saved without one differ from file offset `0xCEC` onward. The tails do not reconverge. The extra length is 1526 bytes of shift, not 1526 bytes of cipher text sitting at the end.

The local blob at the first difference begins:

```
c9 18 03 00 00 00 00 00 14 00 00 f0 00 00 77 2e
18 00 77 2e 6e 00 00 00 0c 45 00 00 7a 45 00 00
54 00 16 0d 00 00 0b 00 98 05 56 0e 00 00 00 00
```

The stand-in key does not decrypt it. Brute-force hits that came back short (`vgah`, `ap1`, `db9`, `iud`, `lxg`, `t57`, `vga`, `3p1`, `3ro`, `9e4`) are not the book password. They are discarded.

16.4 What is still open

This section is the keyed path only. The empty-key branch is 16.6 and does not use this table. The 10-byte key used on a real NIDA `.sbk` has not been captured. Segment 7 has not been stepped on a failed password. Until that capture exists, do not treat a decrypted string from the stand-in key as a finding.

16.5 Author password does not load TBCBT

On 16 October 2026 the author asked for `C:\TBK\nc81base.sbk`: "Please enter the password to gain Author level access." The next box said that password was not correct. The window title stayed `(Untitled)`, then became `sssssssssss.tbk`. The system book did not open.

`sssssssssss.tbk` became a password-protected book in that same session. `TBCBT.DLL` was not in the module list. The Extensions dialog on that book is the machine's OLE registry (`LMRT.DLL` and the other registered controls). It is not the `linkDLL` set, and `TBCBT` is not an entry on it.

Writing the author password stores the property block in section 16.3. It does not map segment 7. `TBCBT` is present only when a handler `linkDLL`s it. A new book has no such handler. A refused system book never runs its script, so it does not link the DLL either.

The stops while `sssssssssss.tbk` was current:

Stop	What it did
`??0TASKCOLLECTION`	UTL constructor. One `0x3C`-byte block. The flags word pushed is `0x2042` (movable, zeroed, shared). Called twice, two incoming selectors.
`GETTBKINIPROFILESTRING`	Reads `tbk.ini`. `AX` held a pointer (`AEA0`, `AD62`) when a string was found, and `0000` when it was not.
`GETTBKINIPROFILEINT`	Same file. The default at `[bp+6]` was `0x32D0`, then `0x3304`. `AX` came back `0000` both times.
`TB80ATH 4E67:0642`	The author EXE. `AX` was `4E2F`, so the `je` was not taken. Stores ES, CX+`0x100`, SI, DI, BX (`0x80`), and DX at `DS:305C`.
`GETCHARINFO`	Returns the cached dword at UTL `DS:0518`. That dword was zero.
`GETSTRMGRSTRING`	String id at `[bp+6]` was `0001`, then the function decrements it. Id `0` is the empty case.
`GETDATA@TASKCOLLECTION`	Lookup. Arguments were `0000` and `42F7`.
`OBJFILEOPEN`	BAS, `enter 025A` (602-byte frame). This is the open of the `.tbk`. `BX` was `0x50`, `CX` was `0x32D`.

After the status line "The task has completed running", every register was zero and the assembly pane was `????`. The `OLEUI*`, `DATASTUB*`, and `COPENSCRIPT` names on that screen are the export list of a dead task. They did not run.

16.6 Empty key

ENCRYPTEX with an empty key does not use the 10-byte table in 16.2. The nine bytes at DS:0x28 are BMWBMWBMW, indexed by position modulo 9. A letter encrypts as ((K + c - 0x82) mod 58) + 0x41. The inverse is what DECRYPTEX does: subtract K, and if the result is negative add 58, then add 0x41. cipher/empty_key.py is that pair. It is not the keyed branch.

Known. 62 6d 77 62 decrypts to aaaa. One a encrypts to 62.

Known. nc81base.sbk stores 62 70 49 6a 7a at file 0x1F6C. Those bytes are the author password. The same five bytes are in NC81BANK.SBK, nc81recs.sbk, nc81test.sbk, and nc81trnr.sbk. The password is not required to read the records. It is not printed here.

The keyed path in 16.4 is unchanged.

16.7 The lesson books

9e954d01.tbk and nida81.tbk store 0 in the first word of the book record. Neither has the password block this chapter describes. The author program that asks for the password is not in this archive. Chapter 16.5 already recorded that the prompt does not load TBCBT.

17. Modules

Eight modules were counted for pattern coverage on 2026-10-11: 10,991 unique functions, 1,918 nameable by pattern (17.5 percent). Confirmed renames after the later digs are 112: BAS 20, BMP 14, RTM 9, and the rest across CMP, UTL, W16, LNL, MM, RCR, FLT, NET, JPG, and VBX. Pattern coverage: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6. A pattern name is a candidate. The scripts in rename/ are what was applied. Naming another thousand StrCpy wrappers will not find the record reader.

17.1 Launch

Piece	What it is	What it is not
TB80RUN	32-bit launcher. Spawns TB80NET.EXE with the book on the command line.	Not an in-process thunk, and not the parser.
TB80NET.EXE	16-bit NE. Network provider: zlib, TBKNET3WNDPROC, TBK.INI. Class TbkNet3TB80.	Does not parse pages.
TB80RTM.EXE	16-bit shell. TBKMain, TBKClient, TBKChildWindow, TBKReserved_1, SchedulerWndTB80. WM_PAINT calls CLIENTTOPAGE, then BAS ordinal 26.	Does not draw shapes. A lesson click arrives on TBKClient, not TBKChildWindow. BMSG does not see that queue.
TB80MAIN.EXE	23552-byte 32-bit stub. Calls TB80RMT CreateRemoteCallServer and pumps keys. Failure text: "TB80MAIN: Loading ActiveX support failed."	Not the book engine.
tbload.exe	Finds ZZZTBKBOOKLOC via MINSTB32.dll (FUN_00405460). Virtual call at 00406aa1 inside FUN_00406a70 (vtable+4).	The callee is unresolved. Thought: first handoff out of the 32-bit search.
TB80W32.DLL / TB80W16.DLL	Thunk halves, about 246 and 406 exports. W16 is also the DoCmd* bus (DoCmdDraw, DoCmdMouseDown).	W16 setup (platform, OLE, 32-bit LoadLibrary) is not book open.
TBFILE32.DLL	Long filenames for the 16-bit engine.	
TB80F32L.dll	Flat 32-bit entry and the common-dialog hook.	
TB80RMT	Remote-call server used by TB80MAIN.	Not on the path that opens a lesson.
PHYSSIZE	A 1.5-era index.	Not in these 8.1 books.

The engine calls 16-bit USER, GDI, and KERNEL. A log that says USER32 is missing is not a broken install. The images in hand are 16-bit NE. ToolBook 9.0 is the 32-bit product. A correspondent's claim that 8.1 is 32-bit and needs Windows 10 is rejected by the binaries (claim C-006).

17.2 Book server and neighbors

Module	Role	Not
TB80BAS.DLL	Objects, properties, painters, twip scale, file open. 606 exports. CDBINIT ordinal 1, CDBFILEOPEN 8, CDBVIEWPAGE 15, CDBVIEWBKGND 18, CDBPAINTCLIENT 26, CDBSENDCREATE 51 (author). The CDB* / OBJ* split is chapter 20.	Not the window.
TB80CMP.DLL	Compile and expand. Resources 0x77, 0x78, 0x70.	Not the page walker. 5.6 percent pattern-nameable because it is a compiler.
TB80UTL.DLL	Type-name strings, string manager, metrics, heaps.	Not the 14 00 00 reader.
TB80RCR.DLL	Controls, mouse hook, effect objects, button drawing. Init is FUN_1010_0000.	Not classified as the book reader. The init does not open a file.
TB80ENU1 / ENU2	English strings, 16-bit and 32-bit.	Not code that opens a book.
TB80BMP.DLL	DIB, metafile, FX blit. Ordinal 3 is the bitmap reader. Lz77Expand is FUN_1000_0031.	Not a picture store. The widget still does not name the bitmap.
TB80JPG.DLL	JPEG. Version resource still says TB60JPG.DLL.	
TB80LNL.DLL	MCI, wave, MMIO. The notify callback is Lionel.	
TB80MM.DLL	tbkmmNotify, imageNotify, mciNotify, timerNotify.	Does not classify objects.
TB80OLE / OCX / ACX / VBX	Clipboard, controls, automation.	Not on the Logic Families paint path.
TB80FLT	Filter registration. FUN_1028_2680 calls TBREGISTERFILTER. The 0x14 there is a type-code limit.	Not the reader.
TB80RCA.DLL	Author UI widgets.	Not book open, not the cipher.
TBCBT.DLL	CBT scoring. Segment 7 is the cipher (chapter 16).	Not the reader. Not loaded when the author password-protected ssssssssssss.tbk (section 16.5). Not loaded as segment 7 in the session that listed the other code segments.
TBDLG, TBWIN, TBDOS, TBREG, TBDB3, TBPDX	linkDLL commands.	Not page walkers.
DIBUTIL.DLL	DIB helper beside BMP.	Not the object format.

Author binaries TB80EDT.DLL and TB80.EXE are not in this set. The runtime half of the author contract is visible: BAS exports CDBEDITBOOK, CDBVIEWBOOK, CDBISVIEWERINGENERICEDITMODE, CDBSENDEDITLEAVETOALL*, and TBXAUTHORMODE. RTM calls GETAUTHORLOCALIZEDDLL. TB80ATH.EXE is the 16-bit author, seen under NTVDM. That is bookmarked, not solved.

17.3 Closed as the reader

Candidate	What it actually is
CDBSENDCREATE (ordinal 51, 12c8:00BC)	Author create and rewrite.
FUN_12c8_0ef8	Copies four words on that author path. The source is not the disk record.
FUN_1028_2680	Filter registrar.
FUN_1008_2d31 (switch 1008:2DC7)	OpenScript math.
TB80RTM	Paint handoff only.
CDBCHUNKFIND / LAST / END / CDBSKIPSEP	Text separators.
VALUENEWLONG and the 4267 constructors	Value constructors.
FUN_10b0_00a6	Type dispatcher. Code 0x14 is a type code.
12e8:0000	OLE object clone.
BAS segment 182	39 far calls with a zero segment and no reloc. Not book open.
BAS segments 1018, 1278, 1478, 14b8, 1508, 10d0, 1030, 14a8, 1160, 1070	Sampled. No stream walker. 1160 is debugging (CDBGETNEURONURL, CDBATTACHBREAKADDRESS). 1018 creates pens, brushes, and bitmaps.
A static search for the bytes 14 00 00	Thirteen undisassembled runs contain the word 14 00 as immediates, MOV CX, 0x14, or jump displacements. Not a far-pointer compare that builds an object.

17.4 Ghidra

Two NE loader bugs were fixed for this work: the entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image with entry segment bytes at or above 128. `toolbookunpacker` clears the Optloader self-load flag and does not patch the entry table. The Ghidra patch does. An honest BAS image loads in the patched Ghidra 12.1.4 build.

Force-disassemble: walk undefined `C8` (`ENTER`) bytes first, then the prologue pass. The prologue pass accepts a real `RET` / `RETF` before treating the following byte as code, and it only creates functions at entry-like symbols. They do not name anything.

`SENDMESSAGE` (321 hits in one RTM listing) and `LSTRLEN` (282) have mangled arguments. That is a decompiler problem. It wants a fresh decompile, not another reading of the same HTML.

`maps/bas-ne-to-ghidra.md` and `maps/rtm-ne-to-ghidra.md` are the segment crosswalks. `maps/wdw-to-ghidra.md` is one boot's selectors. None of the three is a narrative. Appendix K of the reference manual is the procedure those maps belong to.

17.5 RCR, from the 16 October stop

There is no separate HTML listing. The decompile is `Toolbook/_decompiles/TB80RCR.DLL.c`. It has 293 functions: 191 still named `FUN_`, 53 `ASYMTBKFX*`, 13 `ASYMCTL*`, 10 `BMR*`, 8 `ASYMMTRK*`, plus help, `WEP`, and `BTNPROCBITMAPBUTTONPROC`. The Watcom list is that export set. The names are definitions in this file, not unnamed functions waiting to be matched.

The stop was `TB80RCR.DLL`, selector `3C4F`. `ASYMTBKFXINIT` at `493A` is the decompile at line 25267: `FUN_1008_aa62`, then return 1. The far call `43EF:AA62` is that first call. `ASYMMTRKINIT` at `2E0A` is line 23315. The `cmp` of word `0044` is `DAT_1020_0044`. Zero means first time. The following `push 2042h` is the `GLOBALALLOC` flags. The listing allocates `0x84` bytes, locks them, registers two window messages, and calls `SETWINDOWSHOOKEX`.

Those two are not the first instructions of the DLL. `entry` (line 9587) does `LOCALINIT`, `GETVERSION`, and `int 21h`, then `FUN_1008_d7ca`, which calls `FUN_1010_0000`. That function's order is `LOCALINIT`, `HELPINIT`, `ASYMTBKFXINIT`, `ASYMMTRKINIT`, and if the mouse init returns 0, `FUN_1010_c79e`. `WEP` calls `ASYMTBKFXEXIT` and `FUN_1010_ca82`.

`BMRDRAWBUTTON`, `BMRDRAWBEVEL`, `BMRDRAWSHADOW1`, `BMRDRAWSHADOW2`, and `BMRDRAWSTATUSBAR` are the drawing. They are button faces and bevels. The init that was stopped does not open a book.

A later stop in the same session was `43EF:2992`, `ASYMCTLPAUSE` (line 3606), not `ASYMCTLPAUSED` (line 3656). The prologue has not run, so `DS` is still `3327`. The watch `3327:1B02 = 5341` is not the word the compare reads. After `mov ds, ax` the compare is `41FF:1B02`, which is `DAT_1020_1b02`. `ASYMCTLPAUSE` calls `FUN_1008_34de` and, on a hit, `UNHOOKWINDOWSHOOKEX`. `ASYMCTLPAUSED` only returns 0 or 1. `ASYMCTLISAPPREGISTERED` at `2ACE` is the same lookup: `push 0`, `push [bp+6]`, `call near 34DE`. `ASYMCTLGETPROPERTY` at `2D40` calls `GETCURRENTTASK` and then that lookup.

18. Where every loose sheet went

The absorbed sheets now live under `tbworkingpapers/history/sheets/`. A row that says `tbwp0001/…` means `history/sheets/tbwp0001/…`. The manual source stays at `tbworkingpapers/sheets/tbwp0030/`. `tbwp0019`'s notes, and the seventeen renamed JSON files that still read, are in `history/sheets/tbwp0019/`. The rest of that dump does not read. It is still `tbworkingpapers/tbwp0019/`. `who-is-who` is not in this index and is not in this repository. The shelf is `tbworkingpapers/history/miscellaneous/`. Binary password deltas stay in the working

papers, under `history/sheets/tbwp0027/`.

A row marked superseded means the sheet still contains a sentence this record closed. Read the sheet for the function body if you need it. Read this record for the claim.

Loose sheet	Fate	Where the claim lives
`tbwp0001/00-architecture.md`	kept	Chapters 3 and 17.
`tbwp0001/01-dll-catalog.md`	kept	Chapter 17.
`tbwp0001/02-open-questions.md`	superseded	Chapter 10. The reader is still unnamed; the create-path wording in that note is superseded.
`tbwp0001/ToolBook_Architecture.md`	superseded	Chapters 3–5. The book-open flowchart is superseded by chapter 4.
`tbwp0001/ToolBook_Chapter.md`	superseded	Chapters 3–5 for layout. Book-open flowchart that calls CDBSENDCREATE is superseded by chapter 4.
`tbwp0001/naming_summary.md`	kept	Chapter 8 and chapter 17. Counts 112 confirmed.
`tbwp0001/openscript_notes.md`	superseded	Chapter 14. The early italic/systime conclusion is superseded by the back-reference fix in the same file.
`tbwp0001/tbworkingpapers-pointer.md`	index	Chapter 0. Pointer only.
`tbwp0002/README.md`	kept	Chapter 7. Maps are NE-segment:offset and are not loaded as modules.
`tbwp0002/bas-create-path.md`	superseded	Chapter 4. CDBSENDCREATE is not the runtime reader.
`tbwp0003/p1040-breakdown.md`	kept	Chapter 13.1. TBX expansion of P1040.
`tbwp0003/p1040-tbk-raw.md`	kept	Chapter 13.1. Bounds, fills, blob offsets.
`tbwp0003/tb80r_compression.md`	kept	Chapters 14 and 15. 33 handlers, prefix also compressed.
`tbwp0003/tbx-vs-tbk-analysis.md`	kept	Chapters 2 and 13. Bounds in 8.1, Vertices in the export.
`tbwp0004/README.md`	kept	Chapters 2 and 5. Specimen graphics. Renders are not re-shipped here.
`tbwp0005/README.md`	kept	Chapters 6 and 9. Engine subset.
`tbwp0006/foollege-hierarchy.md`	kept	Chapter 9. Foollege is not a running server.
`tbwp0006/inventory.md`	kept	Chapter 13.7. Tag counts. Folder names are not runtime page names.
`tbwp0007/bounds-extractor.md`	superseded	Chapters 5 and 13. The 14 00 00 01 becomes 11 hypothesis in the header era is not true (WV-010).
`tbwp0007/content-rectangles.md`	kept	Chapters 5 and 13.1. Bounds-only, marker 01 00 00 04.
`tbwp0007/core-sbk-confirmation.md`	kept	Chapter 15.
`tbwp0007/denny-exchange-summary.md`	kept	Chapter 2 for the Save To XML steps. The 32-bit claim in that exchange is rejected (C-006). Personal names are not copied.
`tbwp0007/ghidra-research-list.md`	superseded	Chapters 10 and 17. Create-path items in that list are superseded.
`tbwp0007/minor-decompile-notes.md`	kept	Chapters 2, 5, and 17. Graphic property ids `0x4097`–`0x409B`, `LOCKPATTERN`, `layer` at `0x4059`. The page-blob slot for the bitmap id is still open.
`tbwp0007/new-dumps-analysis.md`	kept	Chapters 7 and 12. A dump is real only when it contains known book strings.
`tbwp0007/page880-comparison.md`	kept	Chapter 13. Page880 is P1040, IdNumber 880.
`tbwp0007/processing-log.md`	method	Chapter 18. A log, not a separate finding.

Loose sheet	Fate	Where the claim lives
tbwp0007/sbk-triage.md	superseded	Chapter 15. The 'no OpenScript in any sbk' line is superseded for tb80r.sbk only.
tbwp0007/schema-usefulness.md	kept	Chapter 2. Bounds required, Vertices optional, IrregularPloygon typo.
tbwp0007/tbk-tbx-comparison.md	kept	Chapters 2 and 13.
tbwp0007/tbx-schema-notes.md	kept	Chapter 2.
tbwp0007/template-triage.md	kept	Chapter 15. Templates are not the NIDA playback path.
tbwp0007/waiting.md	kept	Chapter 10. Waiting list folded into the ranked gaps.
tbwp0008/cdbsendcreate-sites.md	superseded	Chapter 4. Callers are author-side.
tbwp0009/bas-html-extracts.md	superseded	Chapter 4. Extracts that treat ordinal 51 as runtime are superseded.
tbwp0009/html-coverage.md	kept	Chapter 17. HTML ?? is not absence.
tbwp0010/group-vs-offset-table.md	kept	Chapter 13.4. The word table is an index, not a Group.
tbwp0011/bas-xref-walk.md	superseded	Chapter 4. Cross-references that start at ordinal 51 are the author path.
tbwp0011/cdbsendcreate-and-nida81.md	superseded	Chapters 4 and 15. Author-only correction wins.
tbwp0012/new-html-listings.md	kept	Chapter 17. TBCBT, TBDB3, TBDLG, tbload, TBWIN surveyed.
tbwp0012/scaletwips-and-dispatcher.md	kept	Chapters 4 and 5. FUN_1008_01dd and the type dispatcher.
tbwp0013/14-00-00-pattern-search.md	superseded	Chapter 4. No literal compare found. Author-path sentences superseded.
tbwp0013/new-html-survey.md	superseded	Chapter 17. Survey. Author-path readings superseded.
tbwp0013/paintpolygon-pointer-search.md	kept	Chapter 5. No static far pointer to 1408:0376.
tbwp0013/switch-1008-2dc7.md	kept	Chapters 6 and 14. Math, not the reader.
tbwp0013/tbload-zzztbk-walk.md	kept	Chapters 2 and 17. Marker search and the unresolved vtable+4.
tbwp0014/14-00-00-in-undisassembled.md	kept	Chapter 4. The hits are immediates and displacements.
tbwp0014/approach-correction-undisassembled.md	kept	Chapter 4. Do not close a function for lacking the literal.
tbwp0014/author-side-traces.md	kept	Chapter 4. Author exports are real and not book open.
tbwp0014/bas-seg182-leftover.md	kept	Chapter 4. Not book open.
tbwp0014/code94-ole-clone.md	kept	Chapter 4. 12e8:0000 is an OLE clone.
tbwp0014/fun-1028-2680-is-filter-registrar.md	kept	Chapter 4. Closed.
tbwp0014/high-priority-status.md	superseded	Chapter 10. The disk-to-11 expansion row is not true.
tbwp0014/instructor8-compile-test.md	kept	Chapter 14.6. Written, not run.
tbwp0014/low-priority-status.md	kept	Chapter 10.
tbwp0014/medium-priority-status.md	kept	Chapter 10.
tbwp0014/new-html-survey.md	superseded	Chapter 17.
tbwp0014/openscript-token-map.md	kept	Chapter 14. What compile-plus-expand would lock.
tbwp0014/paintpolygon-table-static-limit.md	kept	Chapter 5. Table is runtime-built.
tbwp0014/supporting-modules-content.md	kept	Chapter 17.
tbwp0014/supporting-modules-useful-for.md	kept	Chapter 17.
tbwp0014/undisassembled-regions.md	kept	Chapter 17. Forced disassembly does not name them.
tbwp0014/undisassembled-triage.md	superseded	Chapter 4. Author-path readings superseded.
tbwp0015/README.md	method	Chapter 17. C8 pass, then prologue pass.

Loose sheet	Fate	Where the claim lives
tbwp0015/win16-merged/README.md	method	Chapter 17. Duplicate of the win16 note.
tbwp0015/win16-merged/scripts/ForceDisassembleC8.md	method	Chapter 17. Duplicate.
tbwp0015/win16-merged/scripts/ForceDisassemblePrologues.md	method	Chapter 17. Duplicate.
tbwp0015/win16/README.md	method	Chapter 17. Win16 support notes for the unpacker.
tbwp0015/win16/scripts/ForceDisassembleC8.md	method	Chapter 17. Method. Run before the prologue pass.
tbwp0015/win16/scripts/ForceDisassemblePrologues.md	method	Chapter 17. Method.
tbwp0016/openscript-expression-path.md	kept	Chapters 6 and 14. FUN_1008_2d31.
tbwp0016/page-background-index.md	kept	Chapters 4.5 and 10. Background id is not a word in the page record that has been decoded.
tbwp0016/property-block-expansion.md	superseded	Chapter 4. FUN_12c8_0ef8 is author-side. The 01-to-11 mutation is not true.
tbwp0017/bas-segment-pass2.md	kept	Chapter 4. Segments sampled, no stream walker.
tbwp0017/bas-segment-triage.md	kept	Chapter 4.
tbwp0017/side-quests-expanded.md	kept	Chapter 10.5.
tbwp0017/side-quests.md	kept	Chapter 10.5. Real, and not the reader.
tbwp0018/bas-decompile-4.md	superseded	Chapter 4. Same correction.
tbwp0018/bas-html-4-analysis.md	superseded	Chapter 4. Header correction stands: ordinal 51 is author-only.
tbwp0018/bas-html-4.md	superseded	Chapter 4. Same correction.
tbwp0018/current-goals.md	kept	Chapter 10.6. Order of work.
tbwp0018/decompile-polish-todo.md	method	Chapter 17. Polish is not the reader.
tbwp0018/dibutil-analysis.md	kept	Chapter 17. DIB helper, not the object format.
tbwp0018/foolbook-research-notes.md	kept	Chapter 9.
tbwp0018/record-reader-candidates.md	kept	Chapter 4.4. Candidates closed.
tbwp0018/rtm-3-triage.md	kept	Chapter 17.
tbwp0018/rtm-decompile-2.md	kept	Chapter 17. SENDMESSAGE / LSTRLEN arguments want a fresh decompile.
tbwp0018/rtm-deep-analysis.md	kept	Chapters 5 and 17.
tbwp0018/rtm-html-4-analysis.md	kept	Chapter 17. RTM is the paint handoff.
tbwp0018/rtm-html-4.md	method	Chapter 17. Listing note.
tbwp0018/rtm-html-undisassembled.md	kept	Chapter 17. ?? runs.
tbwp0018/rtm-nameable.md	kept	Chapter 8.
tbwp0018/rtm-paint-path.md	kept	Chapters 3 and 5. CLIENTTOPAGE then ordinal 26.
tbwp0018/rtm-triage.md	kept	Chapter 17.
tbwp0018/today-summary.md	method	Chapter 10. Day log of 2026-10-14. Later corrections win.
tbwp0018/unanalyzed-functions.md	kept	Chapter 10. Still unnamed is not the same as still a candidate.
tbwp0018/undisassembled-triage-2.md	superseded	Chapter 4. Author-path readings superseded.
tbwp0019/keepers.md	kept	Chapter 11. Enter sizes. Selectors are that boot.
tbwp0019/wdw-running-map.md	kept	Chapter 12. The spine.
tbwp0019/wdw-to-ghidra.md	kept	maps/wdw-to-ghidra.md. Lookup, not typeset.
tbwp0023/rtm-ne-to-ghidra.md	kept	maps/rtm-ne-to-ghidra.md. Already in the repo. Lookup.
tbwp0024/keeper-list-decompile.md	kept	Chapter 11. Line numbers in TB80BAS.DLL.c.
tbwp0024/keeper-list.md	kept	Chapter 11. Live addresses. CDBSENDCREATE removed.

Loose sheet	Fate	Where the claim lives
tbwp0024/ne-to-ghidra.md	kept	maps/bas-ne-to-ghidra.md. Lookup, not typeset.
tbwp0025/cdbsendcreate-is-author.md	kept	Chapter 4. The correction.
tbwp0026/bas-trace.md	kept	Chapter 12. 173 BAS shots.
tbwp0026/trace-review.md	kept	Chapter 12. Three traces, no reader.
tbwp0026/utl-trace.md	kept	Chapter 12. 53 UTL shots. No record walk.
tbwp0026/w16-trace.md	kept	Chapter 12. 11 W16 shots. No book open.
tbwp0026/wdw-break-syntax.md	kept	Chapters 7 and 12. @ not !, resume on is not a creation flag.
tbwp0026/wdw-trace.md	kept	Chapter 12.
tbwp0027/withpass-diff.bin	kept	Chapter 16. Binary stays in the working papers. Not committed.
tbwp0028/00-scaffold.md	index	Chapter 0. The scaffold this volume implements.
tbwp0028/chapters/00-how-to-read.md	kept	Chapter 0. Draft only. It is not the statement, and it lags the record after the author-dispatch tables and the graphic property ids.
tbwp0028/chapters/01-corpus.md	kept	Chapter 1.
tbwp0028/chapters/02-formats.md	kept	Chapter 2.
tbwp0028/chapters/03-runtime.md	kept	Chapter 3.
tbwp0028/chapters/04-objects.md	kept	Chapter 4.
tbwp0028/chapters/05-graphics.md	kept	Chapter 5.
tbwp0028/chapters/06-openscript.md	kept	Chapter 6.
tbwp0028/chapters/07-debugging.md	kept	Chapter 7.
tbwp0028/chapters/08-naming.md	kept	Chapter 8.
tbwp0028/chapters/09-modern.md	kept	Chapter 9.
tbwp0028/chapters/10-open.md	kept	Chapter 10. The chapters in this folder are the frozen draft. Where they disagree with record/, the record wins.

tbwp0029/00-official-record.md and the README.md at the root of the working papers are pointers. They are not findings. There is no tbwp0020, tbwp0021, or tbwp0022.

Dated folders already in this repository

Folder	Fate
trail/20261012/	Earlier consolidation, also typeset as the 9-page 15 October contents. Create-path wording that calls the reader a caller of CDBSENDCREATE is superseded by chapter 4. The dispatch tables, the registrar's property ids, and the painter offsets are kept there and in chapter 5.
trail/20261014/	Condensed working papers. 02-still-open.md still calls the reader the caller of CDBSENDCREATE. That sentence is superseded. 01-findings.md, 03-tbx.md, and 04-shelf.md now point at the record under the title. The rest of the open list matches chapter 10.
maps/ and rename/	Lookups and name lists. A trail line under the title says they are not findings.
trail/20261015/	00-how-toolbook-operates.md section 2 and 01-theories-of-operation.md section 1 still teach the author chain as runtime. Superseded by chapter 4. The nested 20261012/ inside it is a duplicate of trail/20261012/.
trail/20261016/00-consolidated.md	The short correction. This volume is the expansion.
rename/	The scripts chapter 8 refers to.
maps/	Segment crosswalks. Not typeset in the PDF.
tbpipeline/tbpipeline_bf/nehost/	The Unicorn host. Not in this repository. Segment dumps stay there, not here.

19. Heap, stack, and the resource manager

Date: 2026-10-16

Status: Side reading of `TB80BAS.DLL.c`, `TB80UTL.DLL.c`, and the 2026-10-15 WDW breakpass in `maps/wdw-to-ghidra.md`. Line numbers are the decompile. Live selectors are one boot. None of these functions is the unnamed record reader.

Claims

Id	Grade	Statement
H-001	Known	Three allocators. UTL `GHEAP*` is one Windows global block turned into a local heap. BAS `HEAP*` / `TBHEAP*` is the NearScript frame heap. `GLOBALALLOC` of `count * 0x76` in `FUN_1278_3be6` is the live-object array. They do not call each other.
H-002	Not true	"`HEAP*` / `TBHEAP*` hand out the `0x76`-byte object" and "there is no standalone `TBHEAPCLEARCRITSECT` definition." Both sentences were chapter 11. `TBHEAPCLEARCRITSECT` is decompile line 93984. No `TBHEAPALLOC` site passes `0x76`.
H-003	Known	The only static caller of `HEAPREGISTER` is `RESMGRCONNECT`. It passes the second of two `MAKEPROCINSTANCE` thunks. That thunk is stored at heap `+0x20` / `+0x22` and is an indirect call.
H-004	Known	The OpenScript value stack is tag `0x10` on that heap. It is not the CPU stack. A non-scalar push goes through `VALUETOSTRING` and the `CDBCHUNK*` comma split. That is why the stack and the chunk walkers shared a WDW batch.
H-005	Known	`RESMGRSENDMSG` is the resource dispatcher. `RESMGRENUM` calls a `code *` argument. Five WDW return addresses that call `VALUENEWRESOURCE` and `RESMGRGETREFCNT` are still unnamed.
H-006	Thought	The `0x303E`-byte heap object is the header plus a frame table of depth `0x400`. The arithmetic fits. The listing does not comment it.

19.1 Three allocators

The `0x76`-byte live object is chapter 4. `CDBNEWOBJECT` indexes it as `param_6 * 0x76`. The block that holds the array is `FUN_1278_3be6` (line 121049). Empty, it `GLOBALALLOC`s `param_2 * 0x76`. Otherwise it `GLOBALREALLOC`s to the same product. The free walk at line 120992 steps `0x76`. That path does not call `TBHEAPALLOC`.

`GHEAP*` is UTL segment 9, lines 9261–9390.

Export	What the body does
`GHEAPCREATE`	`GLOBALALLOC` of `param_2 + param_3 + 0x10`, then `GLOBALHANDLE`, then `LOCALINIT`. A non-zero `param_3` zeroes that many bytes from the symbol the listing calls `DAT_10f8_0010`. Whether that symbol is the new block was not checked. Failure calls `GHEAPDELETE`.
`GHEAPALLOC`	`LOCALALLOC`.
`GHEAPFREE`	`LOCALFREE`.
`GHEAPLOCK` / `UNLOCK` / `SIZE` / `REALLOC`	The matching `LOCAL*` call.
`GHEAPDELETE`	`GLOBALFREE` of the `GLOBALHANDLE`.

The BAS caller is `FUN_11a8_0000` (line 82614), after `GETFREESPACE` and `GLOBALCOMPACT`. It creates two heaps, `GHEAPCREATE` with sizes `0x2002` and `0x800`, stored at `DAT_1630_03b0` and `DAT_1630_03aa`. The `0x16E0` that the decompile prints as the first argument is the same extra the listing puts on `GLOBALALLOC` and `LOCALALLOC` throughout this file. It is not a third size. `FUN_11a8_03f8` (line 82888) and `FUN_11a8_087c` (line 83240) call `GHEAPALLOC` with size `0x40`.

The UTL export trace of 53 hits included `GHEAPCREATE`, `GHEAPALLOC`, and `LOADRESSTR`. `LOADRESSTR` (UTL line 7943, segment 7) loads a Windows string resource. It is not `RESMGR*`. That trace was not a record walk. Chapter 12 already says so.

HEAPREGISTER itself uses GLOBALALLOC for the directory and for each heap object. That is the third use of the Windows allocator, and it is not GHEAP*.

19.2 The script heap

BAS segment comment "Segment 63". The functions are FUN_11f0_*, lines 93127–94370. The decompile passes the string NearScript into this family. Treat that as the module's name for the heap, not as a type check, until FUN_11f0_2440 is read.

HEAPREGISTER (93127). If DAT_1630_11ae is null, GLOBALALLOC a directory of 0x2E bytes. Word 0 is capacity, set to 10. Word 1 is the count. Word 2 is 0xFFFF, the free-list sentinel. Each heap is GLOBALALLOC(0x303E). Word 0 of the heap is magic 0xCCDD. The first arena is FUN_11f0_280c(0x1000). The caller's far pointer is stored at heap +0x20 and +0x22. When the directory is full, GLOBALREALLOC grows the capacity by 9. The only static call is RESMGRCONNECT line 20089.

HEAPDEREGISTER (93236). If heap +0x20 / +0x22 is set, FREEPROCINSTANCE on it. Frees the arenas, unlinks the heap from the directory, and GLOBALFREEs the heap. Callers: the connect failure path (20127) and RESMGRDISCONNECT (20155).

HEAPCREATEFRAME (93315). Fails when the depth word at heap +6 is 0x400. Otherwise it carves at least 0x24 bytes from the current arena, or asks FUN_11f0_280c for another 0x1000. The frame slot is heap + depth * 10:

Offset	Contents
+0x82E / +0x830	Handler far pointer. HEAPGETHANDLER (94051) returns it.
+0x832 / +0x834	Arena far pointer.
+0x836	Caller BP with the low bit cleared. HEAPGETBPFRAME (94063) returns it.

0x82E + 0x400 * 10 is 0x302E. The error word TBHEAPALLOC writes is at +0x303C. Both sit inside the 0x303E object. That is H-006.

TBHEAPALLOC(size, tag, frameIndex, heap) (93620). Adds 4 for a header. FUN_11f0_25f8 rounds the sum. Rejects a zero round or a round >= 0xFFC5, and rejects a size that overflows when 4 is added. Those failures store 0x1FFF at heap +0x303C and return null. The block's first word is the tag. The second word is the frame index. The pointer returned is past those four bytes. Allocation walks the frame free list (FUN_11f0_21c4), grows it (FUN_11f0_1b18), and may start a new block (FUN_11f0_2790). Out of memory stores 0x1F7A. If the crit depth at heap +0xE is not zero, FUN_11f0_15aa must accept the block; otherwise the block is TBHEAPFREEd and the error is also 0x1F7A.

TBHEAPFREE (93800). The block is the user pointer minus 4. The frame index is the word at user-minus-2. It unlinks a 4-byte entry from the table at heap +0x18 / +0x1A. The scan gives up past 0x3FFF entries. The free of the bytes is FUN_11f0_22da.

The crit "section" is a nesting counter, not a Win32 critical section.

Export	Line	Effect
HEAPENTERCRITSECT	93893	If depth is not already -1, increment heap +0xE. Return the 32-bit cursor at +0x10 / +0x12.
HEAPLEAVECRITSECT	93909	May write the cursor back. Decrement depth if it is not zero.
TBHEAPSAVECRITSECT	93933	Copy depth and the cursor, three words.
TBHEAPRESTORECRITSECT	93953	Write them back. A non-zero saved depth does not move the cursor backwards.
TBHEAPCLEARCRITSECT	93984	Zero all three words.

HEAPFINDCDBVALUE (94907) calls TBHEAPALLOC(0x5A, 0xCDBF, ...). The size matches the node it then fills: the last word it writes is at offset 0x58. Every other tag in the table below is a small integer. Do not treat 0xCDBF as a type

code until those bytes are re-read. The same function's value-packing expression contains `*(uint *)0x0`, which is the decompiler losing a register.

Tags at the `TBHEAPALLOC` sites. The owner is the function whose body contains the call.

Tag	Size argument	Owner	Line
`0`	caller size	`VALUENEW`	85175
`0`	`local_8 + 0xC`	`FUN_11d8_1a28`	88923
`0`	`(n + 3) * 4`	`FUN_1368_0de0`, the stack chunk	154913
`9`	`param_1 + 3`	`VALUENEWSTRING`	87415
`0xA`	`8`	`VALUENEWDOUBLE`	86406
`0xB`	`4`	`VALUENEWLONG`	86282
`0xC`	`4`	`VALUENEWPOINT`	153277
`0xD`	`8`	`VALUENEWRECT`	153446
`0xE`	`7`	`VALUENEWDATE`	151703
`0xF`	`3`	`VALUENEWTIME`	151839
`0x10`	`0xC`	`VALUENEWSTACK`	154118
`0x11`	`param_4 * 4 + 0x16`	`FUN_11d8_0030`	87720
`0x12`	`0x100`	`VALUENEWCOLORTABLE`	155612
`0x13`	`param_1 + 2`	`VALUENEWICON`	153717
`0x19`	`10`	`FUN_11e0_1010`, `FUN_11e0_10e0`, `FUN_11e0_11dc`	90735, 90819, 90882
`0x22`	caller size	`FUN_1390_054a`, then `HEAPADDTOCLEANUPLIST`	157234

`VALUENEWCSTR` (87576) is the export in front of `FUN_11d8_0030`. The tag `0x11` call is in the helper, not in the export's own body.

19.3 Values, marks, and the indirect calls

A script value is not a pointer. `HEAPVALUETOPOINTER` (94011) takes the low 10 bits of the value (`value & 0x3FF`), scales by 2, and reads a selector word at `heap + index * 2 + 0x2E`. The offset half is the other argument. WDW's first instructions on that export are `and si,03FF` / `add si,si`. `HEAPVALUETOFRAME` (94020) is that lookup, then the word at the block's `-2`, which is the frame index. `FUN_11f0_0f76` checks the index is `< 0x400` and that `(block[-4] & 0x7FFF)` equals `value >> 10`.

`HEAPPOINTERTOVALUE` (94000) shifts the header word at `-4` left by 10 and mixes in a 10-bit field. The mix is the same broken `*(uint *)0x0` expression as in `HEAPFINDCDBVALUE`. The shift is in the listing. The low 10 bits of this direction are not a clean reading.

Mark and sweep:

Function	Line	What it does
`HEAPMARKFRAME`	94169	Sets bit 7 of the byte at pointer `-3`.
`HEAPMARKPOINTER`	94193	Walks the frame's blocks. Calls `FUN_11f0_2440`.
`HEAPSWEEPFRAME`	94304	Marks the frame's pointer list, then walks the mark table through `FUN_11f0_135a`.
`HEAPADDTOCLEANUPLIST`	93510	Returns 0 unless the header word `& 0x7FFF` is `0x22`.
`FUN_11f0_12ac`	94227	If the mark bit is set, clears it and keeps the block. If the type is `0x22`, `HEAPREMOVEFROMCLEANUPLIST`, then an indirect call.
`FUN_1390_0596`	157249	The matching free: remove from the cleanup list, then `TBHEAPFREE`.

The type-`0x22` call is not "the word at the block plus `0x2C`". `FUN_11f0_12ac` reads a far pointer from the first four bytes of the user data and calls the code pointer at that object plus `0x2C`. `FUN_1390_054a` stores the heap far pointer at user `+4` / `+6` and does not show what sits at user `+0`. The callee is unnamed.

`FUN_11f0_103c` (94071) locks the directory at `DAT_1630_11ae` and walks every registered heap. For a frame whose handler segment matches the argument, if heap `+0x20` is set, it calls that proc-instance, then `FUN_11f0_1160` may

rewrite handler slots. Static callers: FUN_10d0_07ee (line 51677, argument 1) and FUN_10d0_08ba (line 51737, argument 0). The proc-instance body is the thunk RESMGRCONNECT registered. This pass did not name the code inside the thunk. MAKEPROCINSTANCE in that function is decompiled against the CTbkSimpleOLEDispatch string pool, which is the same artifact chapter 4 warns about. It is not evidence the thunk is an OLE method.

FUN_1368_04ee (154384) is the stack marker. It checks the value with FUN_11f0_0f76, VALUEMARKs each element, and HEAPMARKFRAMEs each chunk and the header.

19.4 The value stack

Segment comment "Segment 110", FUN_1368_*, from line 154087. The context object is the script frame: heap far pointer at +4 / +6, frame index at +0xC.

VALUENEWSTACK allocates the header with TBHEAPALLOC(0xC, 0x10, frame, heap). Failure is CDBSETPLERR(..., 0x1F7A, 2, ...). Chunks come from FUN_1368_0de0. A request of 0 or more than 0x19 elements becomes 0x19. The chunk allocation is tag 0, size (count + 3) * 4. Each empty slot is written as type 1, subtype 0x400. The header holds the count, the first chunk, and the current chunk. HEAPPOINTERTOVALUE writes the value back to the caller. A second chunk is linked when the fill reaches 0x19.

Export	Line	Behavior
STACKGETSIZE	154518	Type 1 and subtype 0x400 returns 0. That pair is the null value. Otherwise the count word.
STACKPUSH	154537	Switches on param_2 >> 10, the type field above the 10-bit index.
STACKPUSHONE	154643	If the current chunk's used count equals its capacity, FUN_1368_0de0 allocates another. Then VALUECOPY. A count of 0xFFFF is not incremented.
STACKPOP	154759	Empty current chunk: CDBSETPLERR(..., 0x1FA2, 2, ...). Otherwise decrement, copy the slot, call FUN_1368_1586. An emptied chunk steps back to the previous one.
STACKSLIP	154806	Same 0x1FA2 on an empty chunk. Removes the front element and slides the rest down. Also calls FUN_1368_1586.
STACKELEMENT	154343	Index from the end: count - index - 1, via FUN_1368_0d94. Out of range is CDBSETPLERR(..., 0x38, 2, ...). An empty slot comes back as type 1 / subtype 0x400.
STACKSETELEMENT	154486	Same end-relative index, then VALUECOPY.
STACKFREE	154435	VALUEFREE each slot, TBHEAPFREE each chunk, then the header.
FUN_1368_0ab8	154705	Push, rotate the new elements to the front, and STACKPOP back to the old size on failure.
FUN_1368_01d2	154205	A count word of 0xFFFF is CDBSETPLERR(..., 0x1FFF, 2, ...).

STACKPUSH cases 1, 2, 5, 7, 10, 0xB, 0x14, 0x15, 0x16 go straight to STACKPUSHONE. Cases 8 and 0xC go through VALUEVALUE, then push with the low field | 0x800. Case 0xD pushes three words from the pointed-to object, then one more. Case 0x10 is a stack: push each element. The default converts with VALUETOSTRING and calls FUN_1368_0236 (154232), which splits the string on commas through CDBCHUNKLAST and CDBCHUNKFIND.

FUN_1368_1586 (155393) decrements the count word unless it is already 0xFFFF, in which case it calls FUN_1368_1538. The 12 October map puts the name STACKSLIP at 1368:1586. This decompile has both that address-shaped function and a separate export named STACKSLIP. The map is a different analysis, as chapter 11 says about other offsets. Behavior above uses the decompile names. A live break uses the WDW name and the WDW offset.

The TBX wrappers that were read are one call and a return: TBXSTACKPOP (115944), TBXSTACKFREE (115955), TBXSTACKPUSH (116277), TBXVALUENEWSTACK (116288).

One boot, DGROUP 347F, from the 17:38 batch. Same selector, one segment. The breakpass marked the whole STACK* set as skip. The skip was right about the reader. It was not a description of the functions.

Export	That boot
VALUENEWSTACK	4537:0000
STACKELEMENT	4537:0444
STACKSETELEMENT	4537:06B0
STACKGETSIZE	4537:0738
STACKPUSH	4537:0778
STACKPUSHONE	4537:09A2

19.5 The resource manager

RESMGR* is the book-resource manager. It is not the NE loader and not the reader.

RESMGRCONNECT (20043). CDBINIT. LOCALALLOC of 0xE bytes, seven words. Two MAKEPROCINSTANCE thunks. CDBCREATEINSTANCE(3, ...) takes the first thunk. HEAPREGISTER takes the second. The session words are: [0] instance, [1]/[2] heap far pointer, [3]/[4] and [5]/[6] the two proc-instances. Failure bytes: 0x0F if the local alloc fails, 0x0D if HEAPREGISTER returns null, 0x39 ('9') if a proc-instance fails. The failure path undoes whatever was created. RESMGRDISCONNECT (20144) destroys the instance, deregisters the heap, frees both thunks, and LOCALFREEs the session.

RESMGROPENBOOK (20177) calls CDBCREATEBOOKREF. RESMGRCLOSEBOOK (20197) calls FUN_1048_102e and CDBCLEARBOOKREFS.

RESMGRSENDMSG (20219) switches on the word at message +10.

Case	Call
1	FUN_1028_12d4, then FUN_1030_06a8
2	RESMGREXTRACT
3	RESMGRREPLACE
4	RESMGRENUM
5	Sets a flag, then RESMGRSENDMESSAGE
6	RESMGRGETNAME
7	RESMGRSETNAME
default	Stores 1 at message +0xC

RESMGRSENDMESSAGE is a separate export (line 21836). On the restart boot it was 3817:1C82, enter 0012. Its body was not walked here.

RESMGREXTRACT (21513) is FUN_1028_23ec(0, ...). RESMGREXTRACTGDI (21523) is FUN_1028_23ec(1, ...). The 12 October map puts the name RESMGREXTRACTGDI on 1028:23EC, which is the address form of FUN_1028_23ec, the shared body. The WDW stop to keep is the name plus the WDW offset: RESMGREXTRACT at 4D27:1710 on the first boot. The keepers table has RESMGREXTRACTGDI at 4D27:1730 on that same selector. FUN_1028_23ec itself was not read in this pass.

RESMGRENUM (22381) takes a code *. Each step calls FUN_1008_0422, then (*param_4)(...). The loop stops when the callback returns 0, or when the status byte is nonzero. A status of 'B' is cleared before the stop. The return is the number of steps. The 12 October map names 1008:0422 RESMGRGETREFCNT. In this decompile that address is the helper the enum loop calls. The export RESMGRGETREFCNT is a different function. Do not merge them.

Static callers of RESMGRENUM: RESMGRSENDMSG case 4, FUN_10b0_52e8 (47305), FUN_1300_029e (139318 and 139350), FUN_1308_0c2e (140211, the decompile has the string ACCELERATOR at that call). The callback arguments at 47305 and 140211 are small integers in the listing (0x5358, 0xDEC). Those are not named functions in this pass.

RESMGRREPLACE (20829) builds a message whose first word is 0x1A and whose size word is 6, then calls RESMGRSENDMESSAGE. Types 0x13 and 0x10 copy two extra words into that message. FUN_1028_0d52 calls

RESMGRGETREFCNT only when the type word is 0x13.

RESMGRGETREFCNT (22579) calls FUN_1028_2e76, then FUN_1008_0438, then OBJUNLOCK. FUN_1008_0438 is the address the map labels CDROMOBJINFO_OHANDLECHECK. The label is the map's. The body of 0438 was not read here.

Two more indirect calls sit in this segment. FUN_1028_2680 (22469), when a type id is < 0x14, calls the code pointer at that type's table entry. FUN_1028_2770 (22535) walks 0x14 entries from DAT_1630_0154 and calls the code pointer at +4. The map names 1028:2770 RESMGROPENBOOK. This decompile's export of that name is the CDBCREATEBOOKREF function at line 20177. The loop is not the open-book export.

VALUENEWRESOURCE is defined once, at line 90660. Nothing else in the C file calls it by that name. The callers are the WDW Calls window.

19.6 What the breakpass actually hit

Startup order on the first 2026-10-15 boot, heap and resource rows only. The full table is maps/wdw-to-ghidra.md. TBHEAPCLEARCRITSECT was not the last stop before the window. VALUEFREE was, two stops later.

Order	That boot	Export
6	450F:0E3E	TBHEAPSAVECRITSECT
9	450F:0E70	TBHEAPRESTORECRITSECT
13	450F:0840	TBHEAPALLOC
14	450F:0EE2	HEAPPOINTERTOVALUE
16	4D27:1710	RESMGREXTRACT
24	450F:0EC2	TBHEAPCLEARCRITSECT
26	4457:0B38	VALUEFREE, before the main window

Later the same day, still DGROUP 347F: HEAPCREATEFRAME 450F:0350, HEAPGETHANDLER 450F:0FEE, HEAPRELEASEFRAME 450F:05D6, RESMGRENUM 4D27:2522. HEAPRELEASEFRAME was not read in this pass. The batch note said skip HEAP* and RESMGRENUM. That is still the right call for finding the reader.

The restart (DGROUP 2977) hit the same offsets on a new selector. HEAPREGISTER 343F:0000, enter 0014, compares the word at 11AE (DAT_1630_11ae). TBHEAPALLOC 343F:0840, enter 0014, add ax,4, which is the header. HEAPPOINTERTOVALUE 343F:0EE2.

VALUENEWRESOURCE (4267:0EFC, enter 000E) and RESMGRGETREFCNT (4D27:2838, enter 0008) showed the same unnamed return addresses: 4D87:0102, 0ED7:1E5B, 4C0F:1C72, 518F:4776, 322F:180D. The Calls note for VALUENEWRESOURCE also prints 4D87:01C72 and a second 0ED7:1E5B. 4D87:01C72 is one digit away from 4C0F:1C72 and does not repeat on RESMGRGETREFCNT. Treat it as a bad copy of that line, not as a sixth caller, until a screenshot says otherwise. RESMGRREPLACE is an additional named caller of RESMGRGETREFCNT, which matches FUN_1028_0d52.

322F is the unnamed segment already on CDBFILEOPEN (322F:3252) and CDBCHUNKFIND (322F:0577, 322F:0E1E). 4D87:00ED is a caller of VALUEARRAYCHKDIMS. Same selector as 4D87:0102, different offset. Do not give any of these a name.

19.7 What this does not name

The reader is still the unnamed walk after CDBFILEOPEN and before the first paint. Chapter 4. The next break is still CDBCREATEPAGEREF or CDBSETVALUEEX.

Not named here: the body of the HEAPREGISTER proc-instance, the type-0x22 callee at object +0x2C, FUN_1028_23ec, FUN_11f0_2440, HEAPRELEASEFRAME, and the five 322F / 4D87 / 0ED7 / 4C0F / 518F return addresses.

CDBSENDCREATE is still author-only and was not on these Calls stacks.

20. CDB and OBJ

Date: 2026-10-16

Status: The split between the CDB* and OBJ* exports of TB80BAS. Counts are the NE export table of the unpacked DLL (606 named entries). Call sites are TB80BAS.DLL.c. Relocation counts are the unpacked images. This does not name the record reader.

Claims

Id	Grade	Statement
S-001	Known	Ordinals 1 through 56 are CDB*, with no other name in between. Ordinal 57 is LOCKPATTERN. There are 128 CDB* exports in all, across 26 NE segments.
S-002	Known	There are 29 OBJ* exports. Twenty-six of them are NE segment 47 (Ghidra 1170). OBJARENACHECK is segment 44, OBJEXPORTMSGNEAR is 49, OBJINIT is 103. OBJECTNAME (ordinal 160, segment 61) is not one of them.
S-003	Not true	"CDB* is the header and OBJ* is the definition of the same functions." Both are exported functions with bodies. Three stems occur on both sides (INIT, FILEOPEN, FILECLOSE). Those are not the same function.
S-004	Known	CDBGETVALUE, CDBSETVALUE, and CDBSENDVALUE are one call each, into the matching *EX export, with 0 in the extra argument. CDBUNREGISTEROLE2WINDOW is CDBREGISTEROLE2WINDOW(0, 0, ...).
S-005	Known	CDBGETVALUEEX locks with OBJLOCKNEAR, calls the code pointer at the 0x76 slot, then OBJUNLOCK. The same three calls (lock near, the operation, unlock) are in CDBSETVALUEEX and CDBSENDVALUEEX.
S-006	Known	CDBFILEOPEN does not name OBJFILEOPEN. The decompile reaches it through FUN_1048_1cc4 and through FUN_1030_02fc. Those two call FUN_1048_1766 and FUN_1048_2174, and those call OBJFILEOPEN.
S-007	Known	Outside BAS, the modules whose import table names TB80BAS are RTM, CMP, NET, and FLT. RTM's BAS relocations are CDB*, VALUE*, STACK*, HEAP* / TBHEAP*, and RESMGR*. The only OBJ* name among them is OBJBOOKVERSION.
S-008	Thought	CDB* is the book API the other modules link. OBJ* is the object file and the object memory inside BAS. The DLL never expands the letters. The C++ objects are the mangled TBX* classes, which are a third layer.

20.1 Not a header and a C file

A header would declare a function whose body lived in one object file. That is not this split.

The DLL contains no .c or .h names. Copyright string: Asymetrix, 1990–1995. Both prefixes are in the resident and nonresident name tables, so both are linkable. CDBGETVALUE is one call, and the function it calls is CDBGETVALUEEX, not an OBJ* function. The short-name / EX-name pair is inside CDB*.

CDBINIT (line 30304) is 193 lines of device caps, pens, and colors. At line 30465 it calls OBJINIT(0x242, 7, param_1) and returns if the error byte was set. OBJINIT (line 142908, NE segment 103) runs once: a profile int via GETTBKINIPROFILEINT, then FUN_11a8_0000. The book init starts the object layer. It is not a prototype for it.

The three shared stems:

Stem	Book side	Object side	What the decompile shows
INIT	CDBINIT, ordinal 1	OBJINIT, ordinal 225	One call, above.
FILEOPEN	CDBFILEOPEN, ordinal 8, line 27592	OBJFILEOPEN, ordinal 227, line 76162	Not a direct call. Section 20.3.
FILECLOSE	CDBFILECLOSE, ordinal 11	OBJFILECLOSE, ordinal 246	CDBFILECLOSE does not call it. CDBFILESAVEAS does, once, and also calls OBJFILECOPY.

CDBFILENEW calls OBJLOCK, OBJTOUCH, and OBJUNLOCK. CDBSETFOCUS calls OBJPRINCIPALOO. CDBDELETEOBJECTVALUE calls OBJLOCK, OBJLOCKNEAR, OBJFREENEAR, and OBJUNLOCK. No OBJ* body calls a CDB* function. OBJECTNAME does: CDBVALUEBOOKTOSTRING and CDBSETPLERR. That export is not in segment 47.

The 2026-10-12 symbol map line `11A8:04F6 OBJFILEOPEN` is `FUN_11a8_04f6`. The decompile calls that `FUN_` by name a few lines above the real `OBJFILEOPEN`. The export offset is `177C`. Chapter 11 already has that offset. Do not use the map line.

20.2 What each side is

Ordinals 1 through 56 are the book surface, in order: instance and thread, file, view, paint, the ref constructors (`CDBCREATEBOOKREF`, `CDBCREATEPAGEREF`, `CDBCREATEBKGNDREF`, `CDBCREATEGRAPHICREF`), the value bus, `CDBSENDCREATE` (ordinal 51, still author-only, chapter 4), `CDBNEWOBJECT`, then the four text chunkers. Fonts and patterns start at 57. Later `CDB*` exports exist (`CDBSETPLERR`, the OLE forwards, the break addresses). They are not in that first run.

`OBJ*` in segment 47 is the object machinery:

Group	Exports
Allocate and free	`OBJNEWNEAR`, `OBJNEWNEAREX`, `OBJREALLOCNEAR`, `OBJREALLOCNEAREX`, `OBJFREENEAR`, `OBJFREENEAREX`, `OBJFREE`, `OBJNEWSEG`, `OBJFREESEG`, `OBJTOUCH`
Lock	`OBJLOCK`, `OBJUNLOCK`, `OBJLOCKSEG`, `OBJLOCKNEAR`
File	`OBJGETFILE`, `OBJGETHDRLONG`, `OBJFILEOPEN`, `OBJFILECLOSE`, `OBJFILECOPY`, `OBJFILETYPE`, `OBJREADFAR`, `OBJBOOKVERSION`
Handle and principal	`OBJOOTOOH`, `OBJOPTOOH`, `OBJPRINCIPALOO`, `OBJPRINCIPALOP`

`OBJNEWNEAR` (line 75226) is a body, not a macro: `FUN_1258_001c`, then `FUN_1178_0518`. About seventy functions in BAS call it. None of those callers is a `CDB*` export. The book exports lock and free. The unnamed functions allocate.

`CDBGETVALUEEX` (line 38488) is the pattern the value bus uses. The index expression at line 38671 is `(param_7 & 0xff) * 0x76` added to the word at `local_6 + 2`. The code pointer is the dword at `+4` of that slot. `OBJLOCKNEAR` runs, the pointer is called, then `OBJUNLOCK` (line 38690). When the other handle is set, the call is `FUN_1008_03f6` instead, still between the same lock and unlock. `CDBSETVALUEEX` and `CDBSENDVALUEEX` do the same lock. The stride is the live object from chapter 4. This is not a second measurement of the size, and it is not `TBHEAPALLOC`.

`CDROM*` and `CDROMOBJINFO_*` are a different prefix (self-test, drop target, object-info procs). They are not this `OBJ*` segment. `VALUEFROMCDBVALUE` is the converter RTM imports (17 relocation sites). A CDB value and a script `VALUE*` are not the same word. `HEAPFINDCDBVALUE` and the `0xCDBF` argument stay chapter 19. Do not read that tag as one of these exports.

20.3 File open

`CDBFILEOPEN` calls `FUN_1030_02fc`, `FUN_1030_0d08`, `FUN_1048_1cc4`, and `FUN_1048_0b46`. It does not mention `OBJ*`.

`FUN_1048_1cc4` (line 29115) is also called by `CDBFILEMERGE` and `CDBCREATEBOOKREF`. Its decompile calls `FUN_1048_2174` and `FUN_1048_1766`. Both of those call `OBJFILEOPEN`. `FUN_1030_02fc`, which `CDBFILEOPEN` also calls, calls `FUN_1048_1766` as well. `FUN_1048_1766` has other callers (`FUN_1028_3adc`, `FUN_1040_0ca8`, `FUN_1048_159a`, several `FUN_1078_*`, `FUN_10c8_144c`). It is a shared file helper, not the open path alone.

Two more callers of `OBJFILEOPEN` sit outside that segment: `FUN_1170_34cc` (inside segment 47) and `FUN_1350_1000`. The bodies of `1cc4`, `1766`, `2174`, `34cc`, and `1000` were not read. The claim is the call graph, not a description of what each one parses.

Chapter 4 stands. `OBJFILEOPEN` is the large file frame (`enter 0x25A` on the boot that stopped there). A 4-byte read on that path was an internal pointer, not a page record. Reaching it from `CDBFILEOPEN` does not make it the reader.

20.4 Who links which

Relocation sites against TB80BAS in the unpacked images:

Module	Sites	What they are
TB80RTM.EXE	983	373 CDB* (98 distinct names), plus VALUE*, STACK*, HEAP* / TBHEAP*, RESMGR*. OBJECTNAME is 9 sites. OBJBOOKVERSION is 6. No OBJLOCK, OBJNEWNEAR, OBJFILEOPEN, or OBJUNLOCK.
TB80CMP.DLL	44	CDBLOOKUPNUMBER (2), CDBSETPLERR, CDBGETVALUE. The rest are VALUE* and HEAP* / TBHEAP*.
TB80NET.EXE	1	CDBNETCOMINIT.
TB80FLT.DLL	1	CONVERTTORUNTIMEMEDIAPATH. Not CDB* or OBJ*.

TB80W16, TB80OCX, TB80VBX, TB80UTL, TB80OLE, TB80MAIN, and TB80RUN do not name TB80BAS in the module-reference table. The packed originals are the wrong file for this count: every segment but the stub has the relocation flag clear. The unpacked images are the ones with 134 relocating segments in RTM.

One hundred of the 128 CDB* names appear in some other module's relocation table. The object manager does not.

20.5 The third layer

TBX* is 117 exports, starting at ordinal 291 (TBX32MSGTHUNK). Some names are the other two prefixes with TBX glued on: TBXCDBGETVALUE, TBXCDBSENDVALUE, TBXCDBSETPLERR, TBXOBJPRINCIPALOO, TBXOBJSENDNEAR, TBXCURRENTCDBID, TBXENUMCDBIDINSTANCES. Those are still inside BAS. They are not a second DLL.

The C++ objects are the mangled exports: TBXBASE, TBXUSER, TBXLAYER, TBXOBJECT, TBXBOBJ, TBX32OBJECT, OCXSTUBOBJECT. Each publishes SENDPROC, GETPROC, and SETPROC. CDROPTARGET is the drop target. CVARIANTVALUEUTILITIES is the variant bridge. That is the C++ object layer. OBJ* is __stdcall16far C, not those methods.

GETCDBID and TBXCURRENTCDBID are exports. The binary does not say what the three letters stand for. Do not expand them.

20.6 What this does not change

The reader is still the unnamed walk after CDBFILEOPEN and before the first paint. Chapter 4. The next break is still CDBCREATEPAGEREF or CDBSETVALUEEX.

Not read for this chapter: the bodies of FUN_1048_1cc4, FUN_1048_1766, FUN_1048_2174, FUN_1170_34cc, and FUN_1350_1000; the principal-op callee; every CDB* export after the value bus. CDBSENDCREATE was not re-opened.

21. What the stress run broke

Date: 2026-10-16

Status: The Python in openscript_analysis and the Foolbook engine, run under a stress suite. This is not a timing of TB80BAS. It does not name the record reader.

Claims

Id	Grade	Statement	Source
P-001	Known	Four nested step loops from 1 to 100 do not finish. Each loop had its own cap of 10,000, so the caps multiplied.	engine.py before the shared budget
P-002	Known	Eighty nested if lines raised RecursionError.	The same runners

Id	Grade	Statement	Source
P-003	Known	`name = "a and b"` was split on the `and` inside the quotes, so the comparison was false.	`_eval`
P-004	Known	`ScriptHeap.free` keyed only on the 10-bit index. After the index was reused, freeing the old value freed the new block.	`heap.py`
P-005	Known	`GHeap.alloc` did not look at `data + zero + 0x10`. A request larger than the block was accepted.	`GHeap.alloc`
P-006	Known	One `0xFF` opcode appends up to 256 bytes after the output cap had already been checked.	`expand.decompress`, Foolbook `decode`
P-007	Known	A resource row whose `type` was the word `Bitmap`, or a row that was not a dict, aborted `adopt` before the later rows.	`resmgr.adopt`

21.1 What runs now

The runners share one budget of 200,000 line steps and stop nesting at 48. The four-deep step returns in the stress suite, which itself finishes in under two seconds, and the action list contains `stopped`. The deep `if` does the same. It does not walk off the Python stack.

`and`, `or`, and the comparisons are scanned outside quotes. `&&` in the TBX runner is the same. `if name = "a and b"` now stores `1`.

`free` and `pointer` require the offset as well as the index. A stale value does not resolve, and it does not free the block that reused the index. `GHeap` refuses a request that would pass the end of its block, and a second free of the same ident returns false.

`decompress` and Foolbook `decode` clip each opcode to the room that is left. A 4,000-opcode `0xFF` run with `max_out` of 1,000 stays at 1,000 characters.

A bad resource row is skipped. The row with `type` 1 is still adopted.

A push of 3,000 comma-separated pieces still stops at the index limit (`0x400`) with `0x1F7A`. That path was already right. It stays under a second. The index limit was not raised. Raising it would not match the 10-bit value.

21.2 What was not made faster

The heap's per-frame block list is still a list. At 1,024 indexes a free is not the cost. The cost was the nested step, which did not return.

`tbpipeline` `layer_of` is a prefix test. It was not on this path.

The suite after the change is `openscript_analysis.test_stress`, `openscript_analysis.test_machine`, `engine.test_stress`, and `engine.test_runtime`. 38 tests, all passing. None of them reads a page record.

22. Value, VBX, and EXECSCRIPT

These are three mechanisms. A name belongs to one of them, or to none. It does not belong to two.

A VBX is a Visual Basic extension control. It is not the `0x76` page object. The value bus is not a VBX. `EXECSCRIPT` is not a VBX. The page object is chapter 4. The heap tags are chapter 19. The book API that locks the page object is chapter 20. This chapter is the split between the three names that get confused with that object.

Claims

Id	Grade	Statement	Source
V-001	Known	A script value is a tag and a payload. `VALUENEWLONG` is tag `0xB`, four bytes. `VALUENEWDOUBLE` is `0xA`, eight. `VALUENEWSTACK` is `0x10`, a `0xC`-byte header.	Chapter 19, the `TBHEAPALLOC` sites
V-002	Known	`CDBGETVALUE`, `CDBSETVALUE`, and `CDBSENDVALUE` are one call into the `EX` form. The `EX` form locks the `0x76` object, calls the code pointer in the slot, and unlocks. The value is what is passed. The object is what is locked.	Chapter 20
V-003	Known	`VBXINIT` loads `TB80VBX.DLL` and calls `GetProcAddress` for `VbxInit`. It does not allocate the view.	Appendix K, the body that was read
V-004	Known	`VBXNEWVIEW` pushes `0x11C` and calls far. The view magic in that reading is `0xABC1`. The size is not `0x76`.	`maps/wdw-to-ghidra.md`; Appendix K
V-005	Known	`TBXEXECSCRIPT` is BAS ordinal 405 at `1268:060c`, decompile line 115148. The other five names in the family are separate exports.	BAS export map
V-006	Thought	`EXECSCRIPT` is the TBX edge that runs a script. The bodies were not read. The stack thunks that were read are one call and a return. That shape is not claimed for this family.	The names, and chapter 19 on the thunks that were read
V-007	Not true	A VBX view, a script value, and `EXECSCRIPT` are the page object, or are each other.	This chapter

22.1 Value

The constructors whose tags were read:

Export	Tag	Size
`VALUENEW`	`0`	The caller's
`VALUENEWSTRING`	`9`	`param_1 + 3`
`VALUENEWDOUBLE`	`0xA`	8
`VALUENEWLONG`	`0xB`	4
`VALUENEWPOINT`	`0xC`	4
`VALUENEWRECT`	`0xD`	8
`VALUENEWDATE`	`0xE`	7
`VALUENEWTIME`	`0xF`	3
`VALUENEWSTACK`	`0x10`	`0xC`
`VALUENEWCSTR`	`0x11`	In the helper, not in the export's own body
`VALUENEWCOLORTABLE`	`0x12`	`0x100`
`VALUENEWICON`	`0x13`	`param_1 + 2`

`VALUENEWBOOK`, `VALUENEWRESOURCE`, and `VALUENEWGRAPHIC` are constructors in segment `4267`. The `enter` sizes on the one-boot capture are frame sizes. They are not the tags in the table. A frame of `0xE` is not tag `0xE`, and tag `0xE` is a date.

`TBXVALUENEWSTACK` is a thunk onto the value constructor. It is still the value mechanism. The `TBX` stem does not make it `EXECSCRIPT`.

The bus is `CDBGETVALUE`, `CDBSETVALUE`, `CDBSENDVALUE`, the `EX` forms, and `CDBPARENTVALUE`, `CDBDEREFVALUE`, `CDBLOCKVALUE`. They use a value. They are not a value allocation, and they are not a VBX.

22.2 VBX

A VBX is a Visual Basic extension control. The host module is `TB80VBX.DLL`. A file that ends in `.vbx` is the control that host loads. BAS does not implement the control. BAS reaches the host.

Export	What the body is	What it is not
VBXINIT	Loads TB80VBX.DLL. GetProcAddress of VbxInit.	Not the allocator. Not a view.
VBXNEWVIEW	The view. 0x11C bytes. Magic 0xABC1.	Not 0x76. Not a script value.
CDBREGISTERVBXFORM	Registers a form. Seen on open. Not the reader.	Not VBXNEWVIEW.

TB80VBX does not name TB80BAS in the module-reference table. The call goes the other way: BAS loads the DLL. Opening TB80VBX.DLL is a different image. The decompiler will not follow GetProcAddress into it.

22.3 EXECSCRIPT

Export	Line	Address, when the map has one
TBXEXECSCRIPT	115148	Ordinal 405, 1268:060C
TBXEXECSCRIPT_CALLBACK	115128	1268:0488
TBXEXECSCRIPTDYNAMIC	115215	
TBXEXECSCRIPTVARIANT	115231	1268:0516
TBXAUTOEXECSCRIPTVARIANT	115293	12F8:0018
TBXEXECSCRIPTVARIANTSPCL	115358	1008:09F8

These are six exports. They are not one function with six names. The body grade is Thought, and only for the role: the TBX edge that runs a script. The operation is not Known until a body is read. Do not fill the gap with the stack thunks. Do not file the family under VBX because both words sit near "control" in a search.

22.4 What the suite does with the split

The classifier lives in openscript_analysis only. Foolversity does not keep a second copy. CDBGETVALUE and CDBREGISTERVBXFORM are the book API, in cdb.py. OBJFILEOPEN and OBJNEWNEAR are the object file, in obj.py. Those two are not the same record: the book API locks an object and does not allocate it, and the object layer is the 0x76 array. VALUENEWLONG stays a heap constructor. VBXNEWVIEW stays the control view. TBXEXECSCRIPT stays the script edge. A value constructor refuses a size of 0x76.

Appendix A — Export and Ordinal Maps

BAS, RTM, and W16 exports, then the BAS ordinals that have been tied to a segment.

Date: 16 October 2026

These four tables are the 12 October map notes, copied whole. A row was not dropped to shorten the file, and the tables were not interleaved with the numeric tables or the address maps. Those are Appendix E and Appendix D.

An address in this appendix is a location. It is not a claim about when the function runs. `CDBSENDCREATE` is in the BAS table because it is an export. It is the author create path. It is not the runtime reader. That correction is Appendix B and chapter 4 of the contents. The row itself is unchanged.

BAS export → decompile mapping

> Duplicate of `TBdecompile/trail/20261012/`. Do not edit this copy. The current statement is `record/`.

Export	Decompile line	Map address
ANSILSTRNCMPI	—	
ASYM_LOCALANDHELPCBK	—	
CDBBOOKOFVALUE	—	
CDBCHECKBOUNDDTL	—	
CDBCHUNKEND	—	
CDBCHUNKFIND	138575	
CDBCHUNKLAST	—	
CDBCLASSTABLEGETBEHAVIOR	125154	
CDBCLEARBOOKREFS	—	
CDBCLIENTSTATUS	—	
CDBCLONEINSTANCE	25024	1038:08FC
CDBCOPYOBJECTVALUE	41627	1088:27E0
CDBCREATEBKGNDREF	—	
CDBCREATEBOOKREF	28347	1048:0DDA
CDBCREATEGRAPHICREF	37711	1070:0000
CDBCREATEINSTANCE	24974	
CDBCREATEPAGEREF	25913	1010:211C
CDBCREATETHREAD	—	
CDBDEREFVALUE	—	
CDBDESTROYINSTANCE	—	
CDBDESTROYTHREAD	—	
CDBDISPLAYSTATE	12804	1008:4B4A
CDBEDITBOOK	—	
CDBEXCLUSIVELOCK	—	
CDBFILECLONE	—	
CDBFILECLOSE	—	
CDBFILEMERGE	—	
CDBFILENEW	27532	
CDBFILEOPEN	27592	1028:1226
CDBFILEPATH	28002	1048:0B46
CDBFILESAVE	27675	
CDBFILESAVEAS	27751	
CDBFILESTATUS	—	
CDBFORMATPLERR	—	
CDBGETEVENTLIST	124383	1278:1A72
CDBGETFOCUSPROP	32396	
CDBGETHANDLERBREAKCOUNT	52502	1028:203C

Export	Decompile line	Map address
CDBGETHANDLERBREAKS	52551	
CDBGETSELECTPROP	33826	1008:0A76
CDBGETVALUE	38475	
CDBGETVALUEEX	38488	
CDBIMAGESTORE	—	
CDBINIT	—	
CDBINTERNALTESTCOMMAND	—	
CDBLAYEROFVALUE	—	
CDBLOCKFONT	130353	
CDBLOCKVALUE	—	
CDBMETRICS	—	
CDBMULTISAVE	28272	
CDBNEWOBJECT	131992	12C8:0850
CDBPAINTCLIENT	13056	
CDBPARENTVALUE	—	
CDBQUERYFILEPATH	38322	1078:036C
CDBQUERYMULTISAVE	28108	1048:0B6A
CDBQUERYVALUE	—	
CDBREFVALUE	—	
CDBREGISTERINSTWINDOW	25296	
CDBREGISTEROLE2WINDOW	142287	1310:0292
CDBREGISTERVBXFORM	—	
CDBRESETPLERR	—	
CDBSAMEVALUEP	37953	1040:06D6
CDBSCRIPTSEARCH	—	
CDBSEARCHINIT	—	
CDBSENDCREATE	131896	
CDBSENDEDITLEAVETOALLFILEVIEWERS	—	
CDBSENDEDITLEAVETOALLVIEWERS	—	
CDBSENDFOCUS	32291	1058:2ACA
CDBSENDSELECT	33879	
CDBSENDUNDO	—	
CDBSENDVALUE	38701	
CDBSENDVALUEEX	38712	
CDBSETFOCUS	32439	1060:143E
CDBSETFOCUSPROP	32414	1060:11E8
CDBSETIMAGEMEMORYMODEL	—	
CDBSETINTERNALTESTSTATE	—	
CDBSETPLERR	81300	
CDBSETSELECTPROP	34107	1030:00B0
CDBSETVALUE	38889	
CDBSETVALUEEX	38900	1048:2F0E
CDBSYNCSCREENS	—	
CDBTHREADFROMINSTANCE	—	
CDBUNIONDAMAGE	—	
CDBUNREGISTERVBXFORM	—	
CDBVALUEBOOKFROMSTRING	90045	
CDBVALUEBOOKTOSTRING	90104	
CDBVALUEGRAPHICFROMSTRING	90406	
CDBVALUEGRAPHICTOSTRING	90549	
CDBVALUELAYERFROMSTRBOOK	90173	
CDBVALUELAYERFROMSTRING	90158	
CDBVALUELAYERTOSTRING	90393	1028:0466
CDBVALUENEWBOOKREF	—	
CDBVALUENEWGRAPHICREF	89968	

Export	Decompile line	Map address
CDBVALUENEWLAYERREF	89850	
CDBVALUERESOURCEFROMSTRING	90315	
CDBVIEWBKGND	—	
CDBVIEWBOOK	—	
CDBVIEWNONE	—	
CDBVIEWPAGE	25726	
CDBVIEWPAGEVALUE	25936	
CDROMOBJINFO_ADDPROC	148404	1350:370E
CDROM_RECORDERROR	145779	
CDROM_SELFTEST	—	
CLASSTABLEGETCLASSEXTRABYTES	124768	
CLASSTABLEGETCLASSEXTRABYTESSIZE	124742	1280:0AC8
CLASSTABLEGETCLASSID	124530	1280:2C66
CLASSTABLEGETCLASSIDFROMMANGLEDNAME	124585	1278:19B4
CLASSTABLEGETCLASSLIBNAME	124692	
CLASSTABLEGETOBJECTSIZE	124729	
COLORHLSTORGB	155991	
COLORSTRINGS	155560	1370:0256
COMBOLBOXWNDPROC	108089	
COMBOWNDPROC	—	
COMPAREASNAME	—	
CONVERTTORUNTIMEMEDIAPATH	114241	
COPYDITHEREDPALETTE_8BIT	—	1170:4098
CREATEDIBMASK	—	
CREATEDITHEREDHEADER_8BIT	—	
DRAWDDSCURVESEG	—	
GETBITMAPINFO	—	
GETCDBID	—	
GETCLASSDATA	—	
GETDATA	—	
GETDIBCOLORCOUNT	—	
GETDIBSIZE	—	
GETHDC	—	
HCACHEPURGE	—	
HDLRINFO_ISVALID	—	
HEAPADDTOCLEANUPLIST	—	
HEAPCREATEFRAME	—	
HEAPDEREGISTER	—	
HEAPENTERCRITSECT	—	
HEAPGETBPFRAME	—	
HEAPGETHANDLER	—	
HEAPLEAVECRITSECT	—	
HEAPMARKFRAME	—	
HEAPMARKPOINTER	—	
HEAPPOINTERTOVALUE	—	
HEAPREGISTER	—	
HEAPRELEASEFRAME	—	
HEAPREMOVEFROMCLEANUPLIST	—	
HEAPSWEEPFRAME	—	
HEAPVALUETOFRAME	94020	
HEAPVALUETOPOINTER	—	
HUGEGETDATA	74213	1168:118A
IMBITBLT	—	
IMBLACKOUTBMP	—	
IMDRAW	—	

Export	Decompile line	Map address
IMDRAWLEVELS	13353	1020:16DE
IMINVERTBMP	—	
IMSHOWBMP	—	
LIONELDEREFBOOK	—	
LIONELREFBOOK	—	
LOCALANDHELPHOOK	20418	
LOCKPATTERN	43779	
LOOKUPSTDPROPBYNAME	—	
LPSTRTODOUBLE	86527	11C8:058E
MATHERRFLOAT	—	
MATHERROR	—	
OBJARENACHECK	72286	
OBJFILECLOSE	—	
OBJFILECOPY	75987	
OBJFILEOPEN	76162	11A8:04F6
OBJFREE	75619	
OBJFREENEAR	75662	
OBJFREENEAREX	75710	1160:0774
OBJFREESEG	—	
OBJGETFILE	—	
OBJGETHDRLONG	—	
OBJINIT	—	
OBJLOCK	75082	
OBJLOCKNEAR	75136	
OBJLOCKSEG	—	
OBJNEWNEAR	75226	1178:114C
OBJNEWNEAREX	75270	
OBJNEWSEG	75325	
OBJOOTOOH	74975	1178:151E
OBJOPTOOH	—	
OBJPRINCIPALOO	—	
OBJPRINCIPALOP	—	
OBJREALLOCNEAR	75365	
OBJREALLOCNEAREX	75413	
OBJTOUCH	—	
OBJUNLOCK	—	
PALENTRYTORGBQUAD	—	
PALENTRYTORGBTRIPLE	—	
PATTERNDITHER_8BIT	—	1598:001A
PROFILE_SETLOGFILE	—	
PSTRTODATEWITHFMT	152434	1358:13B4
PSTRTOTIMEWITHFMT	152259	1358:126E
RESFONTINSTALL	—	
RESMGRCHOOSEFILE	—	
RESMGRCLOSEBOOK	—	
RESMGRCONNECT	—	
RESMGRCOPY	—	
RESMGRDELETE	—	
RESMGRDISCONNECT	—	
RESMGRENUM	22381	
RESMGREXPORT	20962	10D0:1F5C
RESMGREXPORTBYHANDLE	21076	
RESMGREXTRACTGDI	21523	1028:23EC
RESMGRGETNAME	22096	
RESMGRGETREFCNT	22579	1008:0422

Export	Decompile line	Map address
RESMGRIMPORT	20699	
RESMGROPENBOOK	20177	1028:2770
RESMGRQUERYINFO	—	
RESMGRREPLACE	—	
RESMGRSENDMESSAGE	—	
RESMGRSENDMSG	—	
RESMGRSETNAME	22107	1028:27D2
RGBQUADTOLOGPALETTE	—	
SAMPLETIMER	—	
SETUPLAYERSTATE	—	
STACKELEMENT	154344	
STACKFREE	—	
STACKGETSIZE	—	
STACKPOP	154759	
STACKPUSH	154537	
STACKPUSHONE	154643	
STACKSETELEMENT	154487	
STACKSLIP	154806	1368:1586
STARTTIMER	—	
STOPTIMER	—	
STRINGFROMDATEWITHFMT	152079	
STRINGFROMTIMEWITHFMT	151951	
TBHEAPALLOC	93620	
TBHEAPCLEARCRITSECT	—	
TBHEAPFREE	—	
TBHEAPRESTORECRITSECT	—	
TBHEAPSAVECRITSECT	—	
TBKVERSION	—	
TBX32MSGTHUNK	—	
TBXAUTHORMODE	—	
TBXAUTOEXECSCRIPTVARIANT	115293	12F8:0018
TBXBOOKGETFDESC	—	
TBXBOOKGETFINDEX	—	
TBXBOOKLOCKFDESC	115706	
TBXBOOKUNLOCKFDESC	—	
TBXCTOVALUE	115614	
TBXCTOVALUEEX	115625	
TBXDAMAGEDOBJECT	115789	12B8:1536
TBXEXECSCRIPT	115148	
TBXEXECSCRIPTDYNAMIC	115215	
TBXEXECSCRIPTVARIANT	115231	1268:0516
TBXEXECSCRIPTVARIANTSPCL	115358	1008:09F8
TBXEXECSCRIPT_CALLBACK	115128	1268:0488
TBXFIXPENRECT	114838	1198:0146
TBXFONTINFO	—	
TBXGET	128412	1288:070C
TBXGETBKGNDRGBFILLCOLOR	115489	
TBXGETBKGNDRGBFILLCOLOR_CALLBACK	115477	1398:125C
TBXGETCLASSID	114848	1008:01DD
TBXGETFONTFACEINFO	115739	1110:0764
TBXGETPOINTER	—	
TBXGETREADERRIGHTCLICK	—	
TBXHUGEALLOC	114915	15E8:021E
TBXHUGEFREE	114936	1168:04B4
TBXHUGEGETDATA	115015	

Export	Decompile line	Map address
TBXHUGEGETDATA_CALLBACK	114997	1168:0C7A
TBXHUGESETDATA	114975	
TBXHUGESETDATA_CALLBACK	114957	1168:0740
TBXNEARALLOC	114865	
TBXNEARFREE	—	
TBXNEARREALLOC	114883	15E8:01C2
TBXSEND	128378	1278:1FE2
TBXSENDINST	115536	
TBXSENDINSTFROMENVP	115550	
TBXSENDPOINTER	—	
TBXSET	128460	
TBXSETFOCUS	115773	
TBXSETPOINTER	—	
TBXSTREAMOUT	115826	1008:0417
TBXVALUECOPY	115603	
TBXVALUESTRINGLENGTH	115696	
TBXVALUETOC	115581	
TBXVALUETOCEX	115592	
TBXVERIFYID	—	
VALUEARRAYCHKDIMS	87865	
VALUEARRAYGET	87982	
VALUEARRAYGETVALUE	88046	
VALUEARRAYINFO	87815	
VALUEARRAYSET	88133	
VALUEAS	84658	
VALUEASC	84734	
VALUEASCEX	84760	
VALUEASEX	84684	
VALUEATOL	—	
VALUECHECK	92136	
VALUECOMPAREDATE	156391	1370:1312
VALUECOMPARETIME	156432	
VALUECONVERT	84809	
VALUECONVERTEX	84835	
VALUECONVERTTOC	84985	
VALUECONVERTTOCEX	85011	
VALUEDISPOSE	—	
VALUEFREE	—	
VALUEFROMSTRING	—	
VALUEFROMSTRINGEX	—	
VALUEINIT	—	
VALUEMARK	—	
VALUENEW	—	
VALUENEWARRAY	87648	
VALUENEWBOOK	90561	
VALUENEWCOLOR	155412	1368:1538
VALUENEWCOLORTABLE	155590	11C8:0E64
VALUENEWCONST	—	
VALUENEWCPOINT	—	
VALUENEWDATE	151679	
VALUENEWDOUBLE	86392	
VALUENEWFROMDOUBLE	86591	
VALUENEWFROMLONG	86648	
VALUENEWGRAPHIC	91027	11E0:1D9E
VALUENEWINTEGER	—	

Export	Decompile line	Map address
VALUENEWKEYWORD	96022	1200:016C
VALUENEWLAYER	90920	
VALUENEWLAYEREX	90904	
VALUENEWLITDBL	—	
VALUENEWLITSTR	—	
VALUENEWLONG	86271	11C8:012E
VALUENEWPOINT	—	
VALUENEWRECT	153433	1360:0298
VALUENEWRESOURCE	90660	
VALUENEWSTACK	—	
VALUENEWSTRING	87385	
VALUENEWSTRINGEX	87452	
VALUENEWTIME	151828	
VALUETOC	—	
VALUETOCEX	—	
VALUETOSTRING	—	
VALUEVALIDATE	—	
VBXDESTROYBOOK	—	
VBXDESTROYVIEW	—	
VBXINIT	129642	
VBXNEWBOOK	—	
VBXNEWVIEW	—	
WEP	—	
WEP	—	
XCOPYNEAROBJECT	—	

Matched in decompile: 174 / 337

Mangled C++ exports (OLE/COM, `?NAME@CLASS@@...`) are not listed. They correspond to the CTbkSimpleOLEDispatch / CDROPTARGET family and are not resolved to FUN_ names in the current decompile.

Ordinal assignments (from Ghidra listing)

Export	Ordinal	Segment	Offset	Notes
CDBCHUNKEND	53	95 (12f0)	0000	Called by CDBCHUNKFIND, CDBCHUNKLAST. DBCS-aware chunk scanner.
CDBCHUNKLAST	56	95 (12f0)	0334	Calls CDBCHUNKEND and CDBSKIPSEP.
CDBCHECKBOUNDDTL	355	90 (12c8)	0000	Bounds range check. Called from CDBSENDCREATE, CDBNEWOBJECT. Uses device caps (LOGPIXELSX/Y) cached in DAT_1630_016e / DAT_1630_0170.

CDBCHECKBOUNDDTL detail

- Caches horizontal/vertical twips-per-pixel from `GetDeviceCaps` (indices 0x58 / 0x5a).
- `param_1 == 'X'` checks against horizontal limit; `'Y'` against vertical.
- Returns 1 if the value is within range, 0 otherwise.
- Called at four sites inside CDBSENDCREATE and two inside CDBNEWOBJECT.
- This is the clamp, not the disk-to-object bounds reader.

| CDBSKIPSEP | 55 | 95 (12f0) | 04b4 | Separator skipper. Called by CDBCHUNKFIND, CDBCHUNKLAST, CDBCHUNKREMOVE. Modes: 1 = nonempty check, 2 = CRLF pair, 3 = whitespace. |

RTM export → decompile mapping

> Duplicate of `TBdecompile/trail/20261012/`. Do not edit this copy. The current statement is `record/`.

Matched: 43 / 43

Export	Decompile line
ASCIIFILEDLGHOOKPROC	86850
ASCIIFORMATDLGPROC	86994
ASYM_ASCIIFILEDLGHOOK	86812
ASYM_CDROMSAHPCBK	78334
BOOKQUERYSAVE	22225
CACHEFILEOPTIONSDIALOGPROC	78444
CDROMSAVEASHOOK	78356
CHANGEPRINTERDIALOGPROC	136971
CONDITIONDIALOGPROC	132432
DDEWNDPROC	28773
EGCDBSAVEBOOK	27357
EGDYNALINKERRORFUNC	40852
EGYPTCHILDWNDPROC	47953
EGYPTCLIENTWNDPROC	31331
EGYPTNOTIFYPROC	32680
ENUMWINSFOROPEN	46390
FONTDLGHOOKPROC	30737
FONTSTATUSWNDPROC	138995
HANDLEREMOVESCRIPTPROC	72783
HEADERDIALOGPROC	137807
HEADERNOTIFYPROC	138403
HEADFOOTWNDPROC	66993
IMAGESTORESTATUS	28163
MAINWNDPROC	37122
MMRESDLGCHOOSE	150899
OPTIMIZEDSAVESTATUSDIALOGPROC	78670
PARAGRAPHDIALOGPROC	83266
PLASKDIALOGPROC	34880
PLREQUESTDIALOGPROC	34410
POSTPALCHNGTOOTHERS	73918
PREVIEWNOTIFYPROC	138525
PRINTOPTIONSDIALOGPROC	135102
PRINTPAGEDIALOGPROC	134383
PRINTREPORTDIALOGPROC	133543
RESCHOOSEDLGPROC	148133
RESOURCELISTENUM	93087
SEARCHDIALOGPROC	84108
STATUSBARWNDPROC	54892
SUBEDITCONTROLPROC	55062
TOOLNOTIFY	59326
XCTOLEREMOTECOMMAND	103260
XCTOLEREMOTEGET	103345
XDLLTRANSWINPROC	88606

W16 export → decompile mapping

> Duplicate of `TBdecompile/trail/20261012/`. Do not edit this copy. The current statement is `record/`.

Matched: 71 / 72

Export	Decompile line
ACXCTBKSIMPLEOLEDISPATCH_DELETE	8697
ACXCTBKSIMPLEOLEDISPATCH_NEW	8667
ACXFREEDATAOBJECT	8583
ACXGETDATAOBJECT	8519
ACXGETTOOLBOOKSUPPORTCONFIGURATION	8202
ACXINIT	8446
ACXINITIALIZETOOLBOOKSUPPORT	8338
ACXOLEINITIALIZE	8612
ACXOLEUNINITIALIZE	8642
ACXSTARTUP	8427
ACXUNINITIALIZETOOLBOOKSUPPORT	8376
ASYMDIBUTILCONVERT16	22541
ASYMGETVDMPOINTER16	10205
ASYMRELEASEVDMPOINTER16	10236
ASYM_CGETCWD	18704
ASYM_CLSIDFROMPROGID	16879
ASYM_CLSIDFROMSTRING	16929
ASYM_CONTEXTOPENFILE	19821
ASYM_CONVERTTOTREATAS	17105
ASYM_CREATEBITMAPFROMOLEICON	16564
ASYM_DETERMINEGUIDTYPE	17198
ASYM_DLLREGISTERSERVER	16497
ASYM_ENUMICONDRAW	16804
ASYM_FREEOLEICON	16838
ASYM_GETOPENFILENAME	18945
ASYM_GETSAVEFILENAME	19315
ASYM_GETSAVEFILENAMEEXPLORER	19318
ASYM_IS32BITENV	18663
ASYM_OLEICONDRAW	16622
ASYM_OPENFILE	19652
ASYM_OPENFILEEX	19903
ASYM_STRINGFROMCLSID	16867
ASYM_URLDOWNLOAD	19972
ASYM_XFORMHEIGHTINHIMETRICTOPIXELS	16724
ASYM_XFORMHEIGHTINPIXELSTOHIMETRIC	16700
ASYM_XFORMWIDTHINHIMETRICTOPIXELS	16678
ASYM_XFORMWIDTHINPIXELSTOHIMETRIC	16654
ASYM__CHDIR	18747
ASYM__FULLPATH	18803
ASYM__GETCWD	18730
ASYM__LCREAT	19631
ASYM__LOPEN	19688
ASYM__MAKEPATH	19727
CALLPROC32W	4136
COMPILERCOMPILETEXT	12395
COPYDITHERED8BITPALETTE32	22264
CREATEDIBMASK32	22242
FINDORLAUNCHREMOTECALLSERVER	8115
FREELIBRARY32W	4074

Export	Decompile line
GETMODULEFILENAME32W	6913
GETPLATFORMREF	11072
GETPROCADDRESS32W	4094
GETRUNTIMELOCALIZEDDLL	10638
GETTHUNKINGLAYERSERRORFUNC	5226
GETVDMPOINTER32W	4114
IS32BITENV	3892
ISCOMMONDLGLAUNCHED	18475
ISWINDOWS95ORLATER	9922
LOADLIBRARYEX32W	4053
LOADLOCALIZEDSTRING	5032
MINIMIZETHUNKINGLAYERS	5133
OPENFILE32W	6943
REGQUERYINFOKEY32W	11759
REGQUERYVALUEEX32W	11713
REGSETVALUEEX32W	11679
SCRIPTED	13053
SETTHUNKINGLAYERSERRORFUNC	5217
SHUTDOWNTHUNKINGLAYERS	3919
SYSTEMPARAMETERSINFO32W	11355
WINHELP32W	11260
WOWHANDLE32	4418

Unmatched

- WEP

BAS ordinal map (from ExportList.java)

> Duplicate of `TBdecompile/trail/20261012/`. Do not edit this copy. The current statement is `record/`.

Source: Ghidra ExportList.java console output.

Ordinal	Name	Address
1	CDBINIT	1058:0000
2	CDBCREATEINSTANCE	1038:0000
3	CDBDESTROYINSTANCE	1038:040a
4	CDBCREATETHREAD	1038:0320
5	CDBDESTROYTHREAD	1038:047a
6	CDBCLIENTSTATUS	1038:04c6
7	CDBFILENEW	1048:0028
8	CDBFILEOPEN	1048:017a
9	CDBFILESAVE	1048:030a
10	CDBFILESAVEAS	1048:04ae
11	CDBFILECLOSE	1048:0948
12	CDBFILEPATH	1048:09e0
13	CDBFILEMERGE	1048:0a18
14	CDBVIEWBOOK	1050:00f0
15	CDBVIEWPAGE	1040:0000
16	CDBEDITBOOK	1050:007e
17	CDBVIEWPAGEVALUE	1040:041e
18	CDBVIEWBKGND	1040:01f2
19	CDBGETSELECTPROP	1068:0000
20	CDBCLONEINSTANCE	1038:009a

Ordinal	Name	Address
21	CDBTHREADFROMINSTANCE	1038:036c
22	CDBFILESTATUS	1050:0000
23	CDBVIEWNONE	1040:02f8
24	CDBSENDSELECT	1068:009e
25	CDBSETSELECTPROP	1068:0686
26	CDBPAINTCLIENT	1010:0506
27	CDBDISPLAYSTATE	1010:0000
28	CDBSENDUNDO	1068:07ca
29	CDBCREATEBOOKREF	1048:0f5c
30	CDBCREATEPAGEREF	1040:03b2
31	CDBCREATEBKGNDREF	1040:0666
32	CDBCREATEGRAPHICREF	1078:0076
33	CDBREFVALUE	1078:0304
34	CDBDEREFVALUE	1078:0400
35	CDBSAMEVALUEP	1078:049e
36	CDBPARENTVALUE	1078:07b2
37	CDBQUERYVALUE	1078:0958
38	CDBCLEARBOOKREFS	1048:10a6
39	CDBGETVALUE	1078:0d54
40	CDBSETVALUE	1078:1450
41	CDBSENDVALUE	1078:1110
42	CDBQUERYMULTISAVE	1048:0b90
43	CDBMULTISAVE	1048:0e24
44	CDBSENDFOCUS	1060:0000
45	CDBGETFOCUSPROP	1060:019a
46	CDBSETFOCUSPROP	1060:01d6
47	CDBSETFOCUS	1060:0244
48	CDBBOOKOFVALUE	1078:0612
49	CDBLAYEROFVALUE	1078:06fe
50	CDBQUERYFILEPATH	1078:0aaa
51	CDBSENDCREATE	12c8:00bc
52	CDBNEWOBJECT	12c8:02c2
53	CDBCHUNKEND	12f0:0000
54	CDBCHUNKFIND	12f0:0146
55	CDBSKIPSEP	12f0:04b4
56	CDBCHUNKLAST	12f0:0334
69	CDBCHUNKREMOVE	12f0:0550
228	OBJLOCK	1170:0308
233	OBJNEWNEAR	1170:0552
355	CDBCHECKBOUNDDTL	12c8:0000
405	TBXEXECSCRIPT	1268:060c

Key entries for bounds work:

- Ordinal 51 `CDBSENDCREATE` at `12c8:00bc`
- Ordinal 52 `CDBNEWOBJECT` at `12c8:02c2`
- Ordinal 355 `CDBCHECKBOUNDDTL` at `12c8:0000`

Full export list (606 ordinals) was provided by the user. This table holds the confirmed mappings relevant to the current research.

Appendix B — Architecture and Gaps

What is known, what is known to exist, and where the gaps are.

Date: 16 October 2026

Status: This appendix replaces the 15 October 2026 architecture appendix. The diagrams in that file that put `CDBSENDCREATE` between the page walk and the live object described a real function in the wrong place. Those drawings are not the runtime path.

This file is the architecture appendix only. The narrative record is the contents volume. Export maps, numeric tables, and address maps are their own appendices. Nothing in those files is repeated here in order to make this one longer.

How to read this

`[+]` is a recovered fact. A later observation would have to contradict the evidence.

`[?]` is a confirmed gap. The thing exists. The function or the offset does not have a name yet.

`[×]` was drawn in the 15 October appendix, or in an older operating note, and a later trace closed it. It is not a target.

1. Overall architecture

ToolBook II 8.1 is a 16-bit NE book engine (`TB80RTM.EXE`) with a 32-bit launcher (`TB80RUN.EXE`) and a satellite DLL set.

```
TB80RUN.EXE (32-bit)
        |
        | WinExec
        v
TB80NET.EXE (16-bit)
        |
        v
TB80RTM.EXE                         NE book engine
        loads TB80BAS.DLL           objects, paint, properties
        loads TB80CMP.DLL           OpenScript compile / expand
        loads TB80BMP.DLL           DIB / picture
        loads TB80W16.DLL           DoCmd bus, thunks
        loads TB80UTL.DLL           object type table
        loads system books          nc81base/recs/test/trnr.sbk
                                    tb80r.sbk, nida81.tbk
        |
        v
page records in memory              80 65
        |
        v
[?] record reader                   length-prefixed walk
                                    does not call ordinal 51
        |
        v
live objects                        bounds at +0x0F
        |
        v
WM_PAINT
        |
        v
CLIENTTOPAGE -> CDBPAINTCLIENT      BAS ordinal 26
        |
        v
BAS painters -> GDI
```

[+] TB80RTM is the engine. TB80BAS owns object storage and paint. TB80UTL owns the type-name table.

[×] The 15 October arrow from the unnamed reader into CDBSENDCREATE. Ordinal 51 creates or rewrites objects from the Instructor UI. A lesson open does not call it.

[?] The function that walks a page blob and builds objects. It sits in BAS after CDBFILEOPEN and before the first paint. It has no name.

2. Book open path

```
Tbload / TB80RUN
        |
        v
find ZZZTBKBOOKLOC                  file offset of the book data
        |
        v
JBO signature check                 virtual method, not a literal compare
        |
        v
[?] page structure allocation       virtual method
        |
        v
CDBFILEOPEN                         BAS ordinal 8
        |
        v
page objects appear                 80 65 records
        |
        v
[?] walk nested chunks              length-prefixed
                                    a literal compare to 14 00 00 is not required
        |
        v
live object                         bounds at +0x0F
        |
        v
paint                               ordinal 26, then the BAS painters
```

[+] ZZZTBKBOOKLOC is searched by Tbload. JBO is checked. CDBFILEOPEN is named. SoftICE and WDW both reached OBJFILEOPEN and the buffered read in FUN_1160_03de. When the buffer satisfies the request, the raw DOS path at offset 04F3 is skipped.

[×] CDBSENDCREATE (ordinal 51) as the step that turns those chunks into objects. The W16, UTL, and BAS export traces do not show it on book open. Do not break on ordinal 51 to find the reader. The stop fires when Instructor creates or rewrites an object.

[?] The virtual method that allocates page structures, and the walker that runs after CDBFILEOPEN.

3. Author create path

This tree is recovered. It is not the reader.

```
CDBSENDCREATE (12c8:00bc, ordinal 51)
        |
        +-- param_5 == 0x1014 / 0x1015 --> FUN_12c8_0850
        |                                       |
        |                                       v
        |                                  FUN_12c8_0e32   allocator
        |                                       |
        |                                       v
        |                                  FUN_12c8_0ef8   field copier
        |                                       |
```

```
        |                                   v
        |                                object+0x0F = bounds
        |
        +-- param_5 == 0x1021 / 0x1022 --> FUN_12c8_0a86
        +-- param_5 == 0x3018-0x301b   --> CDBCHECKBOUNDDTL + FUN_12c8_0e08
        +-- else                       --> ordinal 115
```

[+] A live object is `0x76` bytes. Bounds are four words at `+0x0F`, `+0x11`, `+0x13`, `+0x15`, in twips.

[+] `FUN_12c8_0ef8` copies source `+0x3E` / `+0x40` / `+0x42` / `+0x44` to object `+0x17` / `+0x19` / `+0x1B` / `+0x1D`.

[+] Nothing inside BAS calls `CDBSENDCREATE` by name. One external thunk, from `FUN_13d0_0822`, reaches ordinal 51. That caller tests author-UI bytes and stores coordinates around object `+0x638`.

[?] The source structure `FUN_12c8_0ef8` reads is not the on-disk `14 00 00` record. An expansion step sits between them on the author path. That step is still not the runtime reader.

4. Object layout

Offset	Size	Meaning	Status
`-2`	word	Type. `0x11` is an arc. Other type bytes are not mapped.	[+] for `0x11` only
`+0x0F`	four words	Left, top, right, bottom, twips	[+]
`+0x17`	four words	Copied by `FUN_12c8_0ef8` from a source structure	[+] as a copy
`+0x1F`	byte	BitBlt raster-op select	[+]
`+0x25`	pairs	Arc geometry, four twip pairs	[+]
`+0x30`	word	Polygon point count	[+]
`+0x32`	far ptr	Polygon point array, 4 bytes per point, stored twips	[+]
`+0x3B`	word	GDI handle	[+]
`+0x52`	word	Per-pin index. Not applied by the current clean-room draw.	[+] that the field exists
unknown		Picture resource id	[?]
unknown		Script / handler pointer	[?]
unknown		Parent / group link	[?]

`ScaleTwipsToScreen` is `FUN_1008_01dd`, BAS segment 2 offset `0x1DD`. With `DS:01A6` and `DS:01A8` at 96, screen pixels are stored twips / 15, which is `(stored * 96 + 720) / 1440`.

5. Paint path

```
WM_PAINT (RTM)
        |
        v
CLIENTTOPAGE
        |
        v
CDBPAINTCLIENT                      BAS ordinal 26
        |
        v
type dispatcher                     BAS, built at load or by relocation
        |
        +-- polygon    PaintPolygon     1408:0376
        |                 count at +0x30, points at +0x32
        |                 ScaleTwipsToScreen, then GDI POLYGON
        |
        +-- polyline   PaintPolyline    13e8:00E6
        |
        +-- rectangle  PaintRectangle   1020:03E4 and FUN_1410_0066
        |                 bounds at +0x0F, then GDI RECTANGLE
        |
        +-- arc        PaintArc         segment 0x7B, helpers at 10A0
        |                 four twip pairs at +0x25
```

```
        |
        +-- ellipse    PaintEllipse     1230:2F1C and an alt
        |
        +-- roundrect  PaintRoundRect   1230:2278 and alts
        |
        +-- curve      [?] stroke walker
        |                  points exist; control structure unknown
        |
        +-- picture    [?] property to TB80BMP ordinal 3, then a DIB
                           +0x3B has been observed as a GDI handle
```

`[+]` ToolBook does not call `GDI!PIE`. Pie wedges are computed and filled with the arc geometry.

`[+]` A diagram bracket is not stored as points. BAS segment `0x5B` computes four polyline points from the bounds.

`[?]` The curve control structure, and the property that holds a picture id.

6. On-disk records

A page object starts `80 65 00 05 00 02 02`. The page name is a 32-byte field. Widgets are not separate top-level objects. They are length-prefixed chunks nested in the page, beginning `14 00 00`.

Polygon, kind `04 00`:

- Fill dword at `+4`.
- `+0x0C` = `04 00`.
- Point count word at `+0x1A`.
- Twip pairs at `+0x22`.
- Screen = twips / 15.

`9e954d01.tbk` has 96 of those records on 13 pages. P430 has 22.

A content rectangle is bounds-only. The tail is four twip words, a zero dword, a fill dword, then `01 00 00 04`. The same book has 961 of that marker. The pin tip is computed at paint. The TBX export writes the expanded Vertices. Matching a Vertices list back to the binary means matching the bounding box.

`[+]` Polygons and content rectangles have been parsed from the page blob.

`[?]` The runtime reader that turns these chunks into objects is not the parser the clean-room tools use. It has not been located. A static search has not found the bytes `14 00 00` used as a compare against a far pointer that then builds an object. The working model is that the reader trusts the length prefix and does not search for the marker. That model is not proven.

7. OpenScript path

```
on disk: handler bytecode in the page blob
        |
        v
resource 0x77                       keyword offset table
        |
        v
high bytes and back-references      whitespace, repeats, copies
        |
        v
source text                         "go to page", "send next", ...
        |
        v
[?] expression evaluator            FUN_1008_2d31 is the math path
        |
```

```
        v
runtime dispatch                    goto, send, assign, if, step
```

[+] The keyword table is resource `0x77`. Back-references `0xC0`–`0xFF` copy from the compressed input stream. An earlier decoder copied those bytes from the output and produced the wrong words. That decoder is closed.

[+] The common subset (`go to page`, `send next` / `previous`, basic conditionals, system properties) decodes. `tb80r.sbk` uses the same bytecode. There the handler-name prefix is compressed as well. Thirty-three handlers have been extracted.

[?] Full expression evaluation, and the compile-time token map, are only partly recovered.

8. System books

File	What it is
`nc81base.sbk`	Resource container. Object-type name table. No decoded `to handle` text.
`nc81recs.sbk`	Resource container. No decoded `to handle` text.
`nc81test.sbk`	Resource container. No decoded `to handle` text.
`nc81trnr.sbk`	Resource container. No decoded `to handle` text.
`tb80r.sbk`	Same container. 33 compressed handlers. Not "no script at all."
`nida81.tbk`	A book. 43 objects, pages `P20000`–`P22130`, 100 `buttonClick` handlers.
`nc81int.dll`	47 trainer / comms exports.
`nc81rec.dll`	Session recording exports.

[+] The four `nc81*.sbk` files do not hold the lesson logic. Lessons call `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. Those names resolve to `nida81.tbk` or to the two DLL export sets.

[×] The 15 October sentence that all system books are scriptless resource containers and that the logic lives only in the DLLs. It is too strong for `tb80r.sbk`, and it skips `nida81.tbk`.

[?] What the `tb80r.sbk` streams do when a lesson runs. The exact handoff from a lesson script into `nc81int` / `nc81rec` is only partly traced.

9. Confirmed gaps

These are the places further debugger work is expected to pay off. The "where to look" column is the current one. A 15 October instruction that said to break on `CDBSENDCREATE` and walk the stack is not on this list.

Gap	What exists	Where to look
Page-blob walker	Builds objects after `CDBFILEOPEN`, before the first paint	`TB80BAS@CDBFILEOPEN` on a lesson open, then resume. Then `CDBCREATEPAGEREF` or `CDBSETVALUEEX`.
Disk record to live object	The `14 00 00` chunk and the `0x76`-byte object are different shapes	When an object exists, dump `+0x0F` and the page offset it came from.
Curve strokes	P420 point runs. Three 96-point red polygons in the TBX.	The on-disk control structure at the page. Not another W16 or UTL export pass.
Picture id	TB80BMP ordinal 3 reads a DIB	Break on that ordinal. Inspect the object.
Script pointer	Handlers live in the page blob as bytecode	Script dispatch, not ordinal 51.
Parent / group	Relative bounds become absolute. The P1170 bracket was computed from a parent.	`CLIENTTOPAGE` or the bounds setter. The parent offset is not in the decoded page record.
Expression eval	Math beyond the FPU path in `FUN_1008_2d31`	Compile and expand a short script.

Closed, so they are not searched again as the reader: `CDBCHUNK*` (text walkers), `VALUENEWLONG` (a constructor), `OBJFILEOPEN` (downstream of the reader), `TB80RTM` (no named file open), the OpenScript math switch, and the filter registrar whose `0x14` is a type-code limit.

10. Working model of the missing reader

Not recovered. Fits the evidence. Not a listing.

```
function LoadPage(page_blob):
    offset = 0
    while offset < length(page_blob):
        chunk = page_blob[offset:]
        # length-prefixed. A search for the bytes 14 00 00 is not required.
        if not length_prefixed(chunk):
            break
        if kind(chunk) == 0x0400:
            obj = make_object()          # not CDBSENDCREATE
            set_bounds(obj, bounds_of(chunk))
            if has_points(chunk):
                set_point_array(obj, count_at_+0x1A, pairs_at_+0x22)
            set_fill(obj, dword_at_+4)
        elif is_bounds_only_rect(chunk):
            obj = make_object()
            set_bounds(obj, four_twip_words(chunk))
            # the pin tip is computed later, at paint
        elif is_picture(chunk):
            obj = make_object()          # picture id property is unnamed
        elif is_script(chunk):
            attach_handler(obj, chunk.bytecode)
        offset += chunk.length
    return objects
```

The step that has a name on the author side (`CDBSENDCREATE`, then `FUN_12c8_0ef8`) is not this function. Calling the author path from this model was the 15 October mistake.

11. What this means for a clean-room reader

A modern reader does not need the unnamed function. It needs the disk layout, which is partly recovered.

`[+]` Page objects are findable by the `80 65` marker. `9e954d01.tbk` has 129 of them.

`[+]` Polygons and content rectangles can be drawn from the nested chunks. Where the binary stores bounds, paint or the TBX export expands the tip.

`[+]` OpenScript handlers can be expanded to source, using the input-stream back-reference rule.

`[+]` The `nc81*.sbk` files supply type names and menus. `nida81.tbk` supplies the shell.

`[?]` Absolute position of nested objects, picture binding, and curve stroke order still need either the missing reader or a theory that matches the TBX expansion.

The `[?]` rows are the research targets. The `[+]` rows are what a clean-room implementation can already stand on. The `[×]` rows are not targets.

Appendix C — TBX File Format

Schema, and what the exported specimen actually contains.

Date: 16 October 2026

Status: This appendix replaces the 15 October 2026 TBX appendix. The schema and the specimen are the same ones. Counts that the record has since settled are stated at the grade the record gives them. This file does not absorb the contents volume, and it does not absorb the export-map appendix.

1. What TBX is

TBX is the XML exchange format used by ToolBook 9.5 and 11.5. It is not the on-disk TBK format. It is the expanded, human-readable form produced by File → Save To XML when `saveSpecial=true` is set in `Instructor115.ini`.

A TBKX package is a ZIP of a TBX folder. The folder contains `Book.xml`, one `Page.xml` per page, `Background.xml` files, `.script` files, and resource files (bitmaps and the rest).

Namespace: `urn:sumtotalsystems:toolbook.tbkx`.

The 11.5 specimen exists. An earlier note that called the export a conjecture is closed. Saving from the 11.5 trial converts the book. An 8.1 runtime cannot open that converted file.

2. Schema hierarchy

Taken from the published schema, not from a guess at the binary.

```
Object
  +-- ToolBookObject (IdNumber, Name, SharedScript, Script)
        +-- GraphicObject (RgbFill, RgbStroke, Transparent, Visible)
              +-- DrawObject (Bounds required, LineStyle, Pattern)
              |     +-- VectorObject (Vertices optional)
              |           +-- LineObject (LineEndSize, LineEndStyle)
              +-- TextObject (Bounds, FontFace, FontSize, FontStyle)
                    +-- FieldClass (Text, BorderStyle, FieldType)
```

Bounds is required on every DrawObject. The pattern is left, top, right, bottom in twips, each an integer of at most five digits, optional leading minus.

Vertices is optional on VectorObject. Bounds-only is valid. The tip shape is expanded at paint or at export time. The binary stores the bounds. The XML writes the point list.

The schema spells IrregularPolygon wrong (`IrregularPloygon`). Official exports use that spelling. This appendix uses the schema's spelling when it is naming the element, and the ordinary spelling when it is naming the shape.

3. Book structure

```
Book
  +-- Book.Pages            PageLocator list (IdNumber, Source path)
  +-- Book.Backgrounds      BackgroundLocator list
  +-- Book.Resources        Bitmap, Cursor, Font, Icon, Menubar,
  |                         SharedScript, Clip
  +-- Book.Viewers
  +-- Book.UserProperties
  +-- Book.Actions
```

Size default on the schema is `8640,5760` (6 by 4 inches at 1440 twips per inch).

From the `9e954d01` export the book element is:

```
Book Script="Book.script" Caption="Nida Corporation - Logic Families"
     Size="9360,6624" MinorVersionNumber="664"
     xmlns="urn:sumtotalsystems:toolbook.tbkx"
```

4. Page structure

```
Page
  +-- Page.Objects          Image, Arc, Button, Curve, Ellipse, Field,
  |                         Group, IrregularPloygon, Line, PaintObject,
  |                         Picture, Pie, Polygon, Rectangle,
  |                         RoundedRectangle, Stage, and the rest
  +-- Page.UserProperties
  +-- Page.Actions
```

Attributes include Name, IdNumber (required), Background (required), Script, SkipNavigation, SharedScript, and DefaultAllowDrop.

The `9e954d01` export contains 131 `Page.xml` files. The 8.1 binary has 129 `80 65` page records. All 129 names match a TBX page. Two XML pages, P440 and P2920, have no page record in the binary. They are an export addition, not a page the binary walk missed.

5. Object examples from 9e954d01

P1040 (TBX page id 880) is the TTL intro. Two DIP packages.

Element	Count	Note
IrregularPloygon	28	Pins, all with Vertices
Rectangle	2	Bodies, fill `64,64,64`
Arc	2	
Field	5	
Button	3	
Group	5	Left package at 5400, right package at 7400

The first pin's Vertices begin `5055,4605,5205,4605` and continue through the tip, ending at the same outline. The binary record at that location stores the bounding box `5055, 4605, 5205, 4860`, then a zero dword and the fill. The numbers match at the start. The tip is computed, not stored. TBX expands it.

6. P430 and P420

P430 is the chip diagram. The TBX page carries 96 IrregularPloygon elements with Vertices, plus 8 Groups, 8 Rectangles, and 8 Arcs. The live `GDI!POLYGON` traces of this page include an 11-point notch. The binary does not store 96 kind-`0400` point records on this page. It stores 22. The rest of the TBX polygons are expansions of bounds-only records. Counting XML elements and counting kind-`0400` records answer different questions. Both counts stand.

P420 is the flame. The TBX page has IrregularPloygon elements with very long Vertices lists, one dark red and one brighter red, matching three 96-point strokes, plus 18 AngledLines. Taking point order from the raw record rendered a tangle. The on-disk control structure for those strokes is still unknown. The TBX list is the expanded form, not a dump of the chunk.

7. Scripts

`Book.script` contains the `enterSystem` handler that pushes `tb80r.sbk`, `nc81base.sbk`, `nc81test.sbk`, `nc81recs.sbk`, and `nc81trnr.sbk`, and calls the `initialize*` / `openSession` functions.

Per-object `.script` files are plain OpenScript:

```
to handle buttonClick
  go to page "P400"
end
```

In the TBK those handlers are compressed bytecode. In TBX they are already expanded. The export of this specimen has 302 `.script` files, counting the book script and the per-object scripts.

8. Resources

Bitmaps are external files (`Bitmap100.bmp` and the rest). The specimen has about 75 BMPs. Bitmap resources carry `KeyColor` and `UseChromaKey` for transparency. 8.1 stores an imported picture as a basic blob and turns it into a BMP. 11.5 stores WMF, JPG, TIF, BMP, PNG, and GIF as themselves.

SharedScript is a resource, not inline. No embedded OLE or other complex resources were found in this export.

Backgrounds are first-class objects with their own XML and `.script` files. The specimen has 14 backgrounds. A TBX page can name a background by an id that does not appear as a word or a dword in the matching binary page. The link is indirect. The field that holds it has not been located.

9. TBX against TBK

TBX attribute	Binary, in `9e954d01.tbk`
`Bounds="l,t,r,b"`	Four twip words, plus a fill
`Vertices="x,y,..."`	Kind `0400` point list at `+0x22`, or a bounds-only record whose tip the export expanded
`RgbFill="r,g,b"`	Fill dword
`Script="..."`	Bytecode in the page blob
`IdNumber`	Page or object id
`Background`	Not a direct word in the decoded page record. The link is indirect.

TBX is the expanded form. A clean-room renderer can use Vertices and Bounds directly instead of reconstructing every `14 00 00` layout, and it should say which one it used.

10. Counts from the 9e954d01 specimen

Item	Count
`Page.xml`	131
`80 65` page records in the 8.1 file	129
XML pages with no binary page	2 (P440, P2920)
Backgrounds	14
`.script` files	302
Bitmaps	about 75
IrregularPloygon elements	654, across all pages
Kind-`0400` point records	96, on 13 pages

The large XML polygon count is why a binary walk that only counts explicit point records under-counts shapes. Many pins are bounds-only in the TBK and become point lists only in the TBX.

11. What a reader can take from this

TBX is a convenient source for the expanded geometry. Vertices and Bounds can be used directly. Scripts can be used as source. No bytecode decode is required for this lesson if the TBX is the input.

A reader that must also handle a raw TBK:

- Use the TBX as the reference for what the expanded form should look like.
- Treat a bounds-only binary record as complete for the box and incomplete for the tip. The tip is computed.
- Groups in the TBX show which objects form a visual unit. A run of ascending words inside a binary page is an offset directory, not a Group.
- P440 and P2920 have no binary counterpart. Do not go looking for an `80 65` record that is not there.

Appendix D — Address Maps

NE segment to Ghidra space, and one boot's WDW selectors.

Date: 16 October 2026

Each map below is the current file, complete. NE segment numbers are not Ghidra `CodeN` names. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps. Live selectors change every boot. Offsets inside a segment do not. The symbol maps are not reprinted. They are unsifted decompiler names and were removed from this edition. They remain on disk under `TBdecompile/trail/20261015/20261012/`. The Bochs selectors from 20 October are the last map. They die at the next reboot. Appendix L is the command card for that host.

NE segment to Ghidra address

> Trail, 2026-10-16. Lookup. NE segment to the Ghidra base used for this load. Not a finding. A live selector is not this base. If this file and `TBdecompile/record/` disagree, the record wins.

NE segment number, file offset, length, and flags were checked against the unpacked binaries in `Toolbook/_unpacked`. They match, file for file, including BAS (199) and RTM (139).

The Ghidra base is not stored in the DLL. It is `0x1000 + (NE - 1) * 8`, and the block is `CodeN` or `DataN`. That is how these listings were loaded. The DLL confirms the segment index. The listing confirms the address Ghidra gave it.

A listing address `Code35:1110:04e8` is NE segment 35, offset `0x04E8`.

`tbwp0023/rtm-ne-to-ghidra.md` drifts by 8 bytes at NE `0x2D`. The binary and the RTM `-5` listing both have `Code45:1160`.

`TB80OLE.DLL` is in this table from the unpacked binary. Its HTML export is one raw block (`CODE_0:1000`) and has no segment table. The Ghidra names for OLE follow the same formula. They have not been seen in a listing.

`TB80ATH.EXE`, `DIBUTIL.DLL`, and `decrypt.exe` are listing-only. Those three binaries are not in `_unpacked`.

OCX segments 50 and 51 have file offset 0 and a length word of 0. Minimum allocation is `0x0E`. There are no file bytes. That is not a 64K segment.

PE files, no NE table: `TB80ACX`, `TB80ENU2`, `TB80F32L`, `TB80MAIN`, `TB80RMT`, `TB80RUN`, `TB80W32`, `Tbload`, `zlib1`.

Index

File	Segments	Entry	Auto data	Data segments	Source
DIBUTIL.DLL.html	5	NE 2 → `1008:961E`	NE 5 → `1020`	4,5	listing only
TB80ATH.EXE.html	184	NE 2 → `1008:063A`	NE 184 → `15B8`	184	listing only
TB80BAS.DLL-4.html	199	NE 2 → `1008:089E`	NE 199 → `1630`	193,194,195,196,197,198,199	DLL matches listing
TB80BMP.DLL.html	27	NE 2 → `1008:02D8`	NE 27 → `10D0`	26,27	DLL matches listing
TB80CMP.ghidra.dll.html	47	NE 2 → `1008:00BE`	NE 47 → `1170`	35,36,37,38,39,40,41,42,43,44,45,46,47	DLL matches listing

File	Segments	Entry	Auto data	Data segments	Source
TB80ENU1.DLL.html	8	NE 6 → `1028:0054`	NE 8 → `1038`	8	DLL matches listing
TB80FLT.DLL.html	28	NE 23 → `10B0:0484`	NE 28 → `10D8`	27,28	DLL matches listing
TB80JPG.DLL.html	3	NE 2 → `1008:A5C2`	NE 3 → `1010`	3	DLL matches listing
TB80LNL.DLL.html	39	NE 15 → `1070:0054`	NE 39 → `1130`	37,38,39	DLL matches listing
TB80MM.DLL.html	22	NE 3 → `1010:0054`	NE 22 → `10A8`	22	DLL matches listing
TB80NET.EXE.html	45	NE 2 → `1008:0120`	NE 45 → `1160`	44,45	DLL matches listing
TB80OCX.DLL.html	53	NE 1 → `1000:0054`	NE 53 → `11A0`	47,48,49,50,51,52,53	DLL matches listing
TB80RCR.DLL.html	5	NE 2 → `1008:A998`	NE 5 → `1020`	4,5	DLL matches listing
TB80RTM.EXE-5.html	139	NE 2 → `1008:035A`	NE 139 → `1450`	139	DLL matches listing
TB80UTL.DLL.html	32	NE 4 → `1018:0526`	NE 32 → `10F8`	30,31,32	DLL matches listing
TB80VBX.DLL.html	47	NE 2 → `1008:00DA`	NE 47 → `1170`	47	DLL matches listing
TB80W16.DLL.html	56	NE 3 → `1010:027A`	NE 56 → `11B8`	35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56	DLL matches listing
TBCBT.DLL.html	16	NE 2 → `1008:0054`	NE 16 → `1078`	16	DLL matches listing
TBDB3.DLL.html	72	NE 2 → `1008:0628`	NE 72 → `1238`	70,71,72	DLL matches listing
TBDLG.DLL.html	19	NE 17 → `1080:027A`	NE 19 → `1090`	19	DLL matches listing
TBDOS.DLL.html	7	NE 2 → `1008:027A`	NE 7 → `1030`	7	DLL matches listing
TBFILE32.DLL.html	9	NE 3 → `1010:0054`	NE 9 → `1040`	9	DLL matches listing
TBPDX.DLL.html	5	NE 4 → `1018:0068`	NE 5 → `1020`	5	DLL matches listing
TBREG.DLL.html	11	NE 2 → `1008:0064`	NE 11 → `1050`	11	DLL matches listing
TBWIN.DLL.html	28	NE 2 → `1008:0946`	NE 28 → `10D8`	28	DLL matches listing

File	Segments	Entry	Auto data	Data segments	Source
decrypt.exe.html	3	NE 2 → `1008:15A4`	NE 3 → `1010`	3	listing only
TB80OLE.DLL	32	NE 2 → `1008:0054`	NE 32 → `10F8`	31,32	DLL only

DIBUTIL.DLL.html

Entry NE 2 at `1008:961E`. Auto data NE 5 at `1020`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (`0x01`)	`Code1:1000`	Code	`0x440`	`0x9F2`	`0x0D50`
2 (`0x02`)	`Code2:1008`	Code	`0xE50`	`0xC3C9`	`0x1D50`
3 (`0x03`)	`Code3:1010`	Code	`0xDC50`	`0x6930`	`0x1D50`
4 (`0x04`)	`Data4:1018`	Data	`0x14950`	`0x16`	`0x0C71`
5 (`0x05`)	`Data5:1020`	Data	`0x14970`	`0x3478`	`0x0D71`

TB80ATH.EXE.html

Entry NE 2 at `1008:063A`. Auto data NE 184 at `15B8`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (`0x01`)	`Code1:1000`	Code	`0x1F60`	`0x9F2`	`0x0D50`
2 (`0x02`)	`Code2:1008`	Code	`0x2980`	`0x6800`	`0x0D40`
3 (`0x03`)	`Code3:1010`	Code	`0x9B00`	`0x1F50`	`0x1D50`
4 (`0x04`)	`Code4:1018`	Code	`0xBC60`	`0x2B2E`	`0x1D50`
5 (`0x05`)	`Code5:1020`	Code	`0xEA00`	`0x30C8`	`0x1D50`
6 (`0x06`)	`Code6:1028`	Code	`0x11D20`	`0x55E`	`0x1D50`
7 (`0x07`)	`Code7:1030`	Code	`0x122A0`	`0x50C`	`0x1D50`
8 (`0x08`)	`Code8:1038`	Code	`0x12880`	`0x38A`	`0x1D50`
9 (`0x09`)	`Code9:1040`	Code	`0x12CC0`	`0x9EE`	`0x1D50`
10 (`0x0A`)	`Code10:1048`	Code	`0x13780`	`0x792`	`0x1D50`
11 (`0x0B`)	`Code11:1050`	Code	`0x14060`	`0x1A92`	`0x1D50`
12 (`0x0C`)	`Code12:1058`	Code	`0x15C20`	`0xAF4`	`0x1D50`
13 (`0x0D`)	`Code13:1060`	Code	`0x168A0`	`0x1A46`	`0x1D50`
14 (`0x0E`)	`Code14:1068`	Code	`0x18600`	`0x430`	`0x1D50`
15 (`0x0F`)	`Code15:1070`	Code	`0x18AE0`	`0x21C4`	`0x1D50`
16 (`0x10`)	`Code16:1078`	Code	`0x1AFA0`	`0x1E96`	`0x1D50`
17 (`0x11`)	`Code17:1080`	Code	`0x1D040`	`0xEE4`	`0x1D50`
18 (`0x12`)	`Code18:1088`	Code	`0x1E060`	`0x3D2`	`0x1D50`
19 (`0x13`)	`Code19:1090`	Code	`0x1E4A0`	`0x638`	`0x1D50`
20 (`0x14`)	`Code20:1098`	Code	`0x1EB80`	`0x314A`	`0x1D50`
21 (`0x15`)	`Code21:10A0`	Code	`0x220E0`	`0x1066`	`0x1D50`
22 (`0x16`)	`Code22:10A8`	Code	`0x23240`	`0x250A`	`0x1D50`
23 (`0x17`)	`Code23:10B0`	Code	`0x25A80`	`0x340C`	`0x1D50`
24 (`0x18`)	`Code24:10B8`	Code	`0x29020`	`0x359E`	`0x1D50`
25 (`0x19`)	`Code25:10C0`	Code	`0x2C8A0`	`0x36CA`	`0x1D50`
26 (`0x1A`)	`Code26:10C8`	Code	`0x30340`	`0x22A6`	`0x1D50`
27 (`0x1B`)	`Code27:10D0`	Code	`0x32720`	`0x1EC`	`0x1D50`
28 (`0x1C`)	`Code28:10D8`	Code	`0x329A0`	`0x3558`	`0x1D50`
29 (`0x1D`)	`Code29:10E0`	Code	`0x36160`	`0x47E`	`0x1D50`
30 (`0x1E`)	`Code30:10E8`	Code	`0x366A0`	`0xEAA`	`0x1D50`
31 (`0x1F`)	`Code31:10F0`	Code	`0x376E0`	`0x153C`	`0x1D50`
32 (`0x20`)	`Code32:10F8`	Code	`0x38DC0`	`0x2200`	`0x1D50`
33 (`0x21`)	`Code33:1100`	Code	`0x3B340`	`0x5272`	`0x1D50`

NE	Ghidra	Kind	File offset	Length	Flags
34 (0x22)	Code34:1108	Code	0x408A0	0x994	0x1D50
35 (0x23)	Code35:1110	Code	0x41320	0x994	0x1D50
36 (0x24)	Code36:1118	Code	0x41D60	0x59F8	0x1D50
37 (0x25)	Code37:1120	Code	0x479A0	0x3DE	0x1D50
38 (0x26)	Code38:1128	Code	0x47DE0	0x127A	0x1D50
39 (0x27)	Code39:1130	Code	0x49200	0x102A	0x1D50
40 (0x28)	Code40:1138	Code	0x4A360	0x4134	0x1D50
41 (0x29)	Code41:1140	Code	0x4E680	0x1A1A	0x1D50
42 (0x2A)	Code42:1148	Code	0x501A0	0xB5A	0x1D50
43 (0x2B)	Code43:1150	Code	0x50DC0	0x67A	0x1D50
44 (0x2C)	Code44:1158	Code	0x514A0	0x258	0x1D50
45 (0x2D)	Code45:1160	Code	0x51720	0x101A	0x1D50
46 (0x2E)	Code46:1168	Code	0x527E0	0x1E68	0x1C50
47 (0x2F)	Code47:1170	Code	0x54660	0x15BC	0x1D50
48 (0x30)	Code48:1178	Code	0x55EA0	0x1394	0x1D50
49 (0x31)	Code49:1180	Code	0x57360	0x15E4	0x1D50
50 (0x32)	Code50:1188	Code	0x58B40	0x1B84	0x1D50
51 (0x33)	Code51:1190	Code	0x5A960	0x1486	0x1D50
52 (0x34)	Code52:1198	Code	0x5BF20	0x2C58	0x1D50
53 (0x35)	Code53:11A0	Code	0x5EF00	0x1AE0	0x1D50
54 (0x36)	Code54:11A8	Code	0x60B40	0xCE2	0x1D50
55 (0x37)	Code55:11B0	Code	0x61960	0xFAE	0x1D50
56 (0x38)	Code56:11B8	Code	0x62AC0	0x17A4	0x1D50
57 (0x39)	Code57:11C0	Code	0x64300	0x50E	0x1D50
58 (0x3A)	Code58:11C8	Code	0x648A0	0x5EC	0x1D50
59 (0x3B)	Code59:11D0	Code	0x64EC0	0x84	0x1D50
60 (0x3C)	Code60:11D8	Code	0x64F60	0x854	0x1D50
61 (0x3D)	Code61:11E0	Code	0x658E0	0xC84	0x1D50
62 (0x3E)	Code62:11E8	Code	0x66660	0x196E	0x1D10
63 (0x3F)	Code63:11F0	Code	0x68180	0x2FF8	0x1D10
64 (0x40)	Code64:11F8	Code	0x6B500	0x39B2	0x1D10
65 (0x41)	Code65:1200	Code	0x6F240	0x2CB8	0x1D10
66 (0x42)	Code66:1208	Code	0x72220	0x214	0x1D10
67 (0x43)	Code67:1210	Code	0x724A0	0x850	0x1D10
68 (0x44)	Code68:1218	Code	0x72DC0	0x16A	0x1D10
69 (0x45)	Code69:1220	Code	0x72F80	0x1674	0x1D10
70 (0x46)	Code70:1228	Code	0x74760	0x50A	0x1D10
71 (0x47)	Code71:1230	Code	0x74D40	0xF46	0x1D10
72 (0x48)	Code72:1238	Code	0x75DC0	0xC04	0x1D10
73 (0x49)	Code73:1240	Code	0x76AA0	0x22	0x1D10
74 (0x4A)	Code74:1248	Code	0x76AE0	0x4E6	0x1D10
75 (0x4B)	Code75:1250	Code	0x77080	0x3240	0x1D10
76 (0x4C)	Code76:1258	Code	0x7A4A0	0x71E	0x1D10
77 (0x4D)	Code77:1260	Code	0x7AD20	0xD8C	0x1D10
78 (0x4E)	Code78:1268	Code	0x7BC00	0x6D94	0x1D10
79 (0x4F)	Code79:1270	Code	0x82D20	0x4C	0x1D10
80 (0x50)	Code80:1278	Code	0x82D80	0x1B0	0x1D10
81 (0x51)	Code81:1280	Code	0x82F60	0x496	0x1D10
82 (0x52)	Code82:1288	Code	0x83440	0xACE	0x1D10
83 (0x53)	Code83:1290	Code	0x83FA0	0xC7E	0x1D10
84 (0x54)	Code84:1298	Code	0x84D80	0x2AEA	0x1D10
85 (0x55)	Code85:12A0	Code	0x87C60	0x2638	0x1D10
86 (0x56)	Code86:12A8	Code	0x8A4E0	0x11DE	0x1D10
87 (0x57)	Code87:12B0	Code	0x8B840	0x372E	0x1D10
88 (0x58)	Code88:12B8	Code	0x8F2A0	0x8EE0	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
89 (0x59)	Code89:12C0	Code	0x98700	0x2088	0x1D10
90 (0x5A)	Code90:12C8	Code	0x9A9A0	0xD28	0x1D10
91 (0x5B)	Code91:12D0	Code	0x9B7C0	0x32E	0x1D10
92 (0x5C)	Code92:12D8	Code	0x9BB60	0x17F0	0x1D10
93 (0x5D)	Code93:12E0	Code	0x9D400	0x1B00	0x1D10
94 (0x5E)	Code94:12E8	Code	0x9F060	0xA80	0x1D10
95 (0x5F)	Code95:12F0	Code	0x9FBC0	0xBE2	0x1D10
96 (0x60)	Code96:12F8	Code	0xA0820	0x35A8	0x1D10
97 (0x61)	Code97:1300	Code	0xA4020	0x1822	0x1D10
98 (0x62)	Code98:1308	Code	0xA59C0	0x592	0x1D10
99 (0x63)	Code99:1310	Code	0xA5FA0	0x96E	0x1D10
100 (0x64)	Code100:1318	Code	0xA69C0	0x138	0x1D10
101 (0x65)	Code101:1320	Code	0xA6B40	0x14E	0x1D10
102 (0x66)	Code102:1328	Code	0xA6CE0	0x5BE	0x1D10
103 (0x67)	Code103:1330	Code	0xA7320	0x6C6	0x1D10
104 (0x68)	Code104:1338	Code	0xA7AC0	0x464	0x1C10
105 (0x69)	Code105:1340	Code	0xA7F40	0x75E	0x1D10
106 (0x6A)	Code106:1348	Code	0xA8720	0x784	0x1D10
107 (0x6B)	Code107:1350	Code	0xA8EE0	0xD9E	0x1D10
108 (0x6C)	Code108:1358	Code	0xA9D80	0x258	0x1D10
109 (0x6D)	Code109:1360	Code	0xAA020	0x1EF0	0x1D10
110 (0x6E)	Code110:1368	Code	0xAC120	0xC4C	0x1D10
111 (0x6F)	Code111:1370	Code	0xACE40	0x2308	0x1D10
112 (0x70)	Code112:1378	Code	0xAF880	0xDAA	0x1D10
113 (0x71)	Code113:1380	Code	0xB0700	0xAE	0x1D10
114 (0x72)	Code114:1388	Code	0xB07C0	0x7FA	0x1D10
115 (0x73)	Code115:1390	Code	0xB1040	0x38D0	0x1D10
116 (0x74)	Code116:1398	Code	0xB5FC0	0x22E0	0x1D10
117 (0x75)	Code117:13A0	Code	0xB8460	0x854	0x1D10
118 (0x76)	Code118:13A8	Code	0xB8D60	0x91A	0x1D10
119 (0x77)	Code119:13B0	Code	0xB9780	0x19B6	0x1D10
120 (0x78)	Code120:13B8	Code	0xBB340	0x1A70	0x1D10
121 (0x79)	Code121:13C0	Code	0xBCFC0	0x118	0x1D10
122 (0x7A)	Code122:13C8	Code	0xBD120	0x2E	0x1C10
123 (0x7B)	Code123:13D0	Code	0xBD160	0xB1A	0x1D10
124 (0x7C)	Code124:13D8	Code	0xBDD60	0xFCA	0x1D10
125 (0x7D)	Code125:13E0	Code	0xBEE40	0x1510	0x1D10
126 (0x7E)	Code126:13E8	Code	0xC0560	0xE0A	0x1D10
127 (0x7F)	Code127:13F0	Code	0xC15A0	0xD54	0x1D10
128 (0x80)	Code128:13F8	Code	0xC2400	0xA84	0x1D10
129 (0x81)	Code129:1400	Code	0xC2F80	0x1E8	0x1D10
130 (0x82)	Code130:1408	Code	0xC31E0	0x56C	0x1D10
131 (0x83)	Code131:1410	Code	0xC3820	0x2D8	0x1D10
132 (0x84)	Code132:1418	Code	0xC3B80	0x5A0	0x1D10
133 (0x85)	Code133:1420	Code	0xC4220	0x122A	0x1D10
134 (0x86)	Code134:1428	Code	0xC56A0	0x63A	0x1D10
135 (0x87)	Code135:1430	Code	0xC5D40	0x1A98	0x1D10
136 (0x88)	Code136:1438	Code	0xC7980	0x498	0x1D10
137 (0x89)	Code137:1440	Code	0xC7E80	0xF2E	0x1D10
138 (0x8A)	Code138:1448	Code	0xC8E60	0x396	0x1D10
139 (0x8B)	Code139:1450	Code	0xC9260	0x197A	0x1D10
140 (0x8C)	Code140:1458	Code	0xCAE80	0x720	0x1D10
141 (0x8D)	Code141:1460	Code	0xCB700	0x140	0x1D10
142 (0x8E)	Code142:1468	Code	0xCB880	0x136	0x1D10
143 (0x8F)	Code143:1470	Code	0xCBA00	0x1D4	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
144 (0x90)	Code144:1478	Code	0xCBC00	0xDF8	0x1D10
145 (0x91)	Code145:1480	Code	0xCCAA0	0x9F2	0x1D10
146 (0x92)	Code146:1488	Code	0xCD560	0xA4A	0x1D10
147 (0x93)	Code147:1490	Code	0xCE040	0xCBE	0x1D10
148 (0x94)	Code148:1498	Code	0xCEDA0	0x48C	0x1D10
149 (0x95)	Code149:14A0	Code	0xCF240	0x8B6	0x1D10
150 (0x96)	Code150:14A8	Code	0xCFB60	0xC34	0x1D10
151 (0x97)	Code151:14B0	Code	0xD0820	0x112	0x1D10
152 (0x98)	Code152:14B8	Code	0xD0980	0x9BC	0x1D10
153 (0x99)	Code153:14C0	Code	0xD13C0	0x15D2	0x1D10
154 (0x9A)	Code154:14C8	Code	0xD2B20	0x974	0x1D10
155 (0x9B)	Code155:14D0	Code	0xD35E0	0x584	0x1D10
156 (0x9C)	Code156:14D8	Code	0xD3C00	0x162E	0x1D10
157 (0x9D)	Code157:14E0	Code	0xD53A0	0x1A62	0x1D10
158 (0x9E)	Code158:14E8	Code	0xD6FA0	0xCC6	0x1D10
159 (0x9F)	Code159:14F0	Code	0xD7DE0	0x202	0x1D10
160 (0xA0)	Code160:14F8	Code	0xD8080	0xF40	0x1D10
161 (0xA1)	Code161:1500	Code	0xD9140	0x840	0x1D10
162 (0xA2)	Code162:1508	Code	0xD9A20	0x41BC	0x1D10
163 (0xA3)	Code163:1510	Code	0xDDFA0	0x139C	0x1D10
164 (0xA4)	Code164:1518	Code	0xDF4A0	0x37E	0x1D10
165 (0xA5)	Code165:1520	Code	0xDF8A0	0x41EE	0x1D10
166 (0xA6)	Code166:1528	Code	0xE3D00	0x10BC	0x1D10
167 (0xA7)	Code167:1530	Code	0xE4F40	0x181A	0x1D10
168 (0xA8)	Code168:1538	Code	0xE68C0	0x766	0x1D10
169 (0xA9)	Code169:1540	Code	0xE7100	0x540C	0x1D10
170 (0xAA)	Code170:1548	Code	0xEC860	0x1552	0x1D10
171 (0xAB)	Code171:1550	Code	0xEDFE0	0x2D68	0x1D10
172 (0xAC)	Code172:1558	Code	0xF1020	0x24EA	0x1D10
173 (0xAD)	Code173:1560	Code	0xF3680	0x2242	0x1D10
174 (0xAE)	Code174:1568	Code	0xF5A80	0x14C6	0x1D10
175 (0xAF)	Code175:1570	Code	0xF70C0	0xC0C	0x1D10
176 (0xB0)	Code176:1578	Code	0xF7D60	0x1C6	0x1D10
177 (0xB1)	Code177:1580	Code	0xF7F60	0x974	0x1D10
178 (0xB2)	Code178:1588	Code	0xF8940	0x140	0x1D10
179 (0xB3)	Code179:1590	Code	0xF8AA0	0x8D8	0x1D10
180 (0xB4)	Code180:1598	Code	0xF93C0	0xE4	0x1C10
181 (0xB5)	Code181:15A0	Code	0xF94C0	0x26A	0x1D10
182 (0xB6)	Code182:15A8	Code	0xF97A0	0x1DA	0x1D10
183 (0xB7)	Code183:15B0	Code	0xF99A0	0x227	0x1C10
184 (0xB8)	Data184:15B8	Data	0xF9BE0	0x5940	0x0D51

TB80BAS.DLL-4.html

Entry NE 2 at `1008:089E`. Auto data NE 199 at `1630`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x4760	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x5180	0x4B4B	0x1D70
3 (0x03)	Code3:1010	Code	0xA5E0	0x5390	0x1D70
4 (0x04)	Code4:1018	Code	0xFB80	0x1B50	0x1D70
5 (0x05)	Code5:1020	Code	0x117C0	0x1C4C	0x1D70
6 (0x06)	Code6:1028	Code	0x13640	0x44EA	0x1D70
7 (0x07)	Code7:1030	Code	0x17D00	0xD22	0x1D70
8 (0x08)	Code8:1038	Code	0x18AA0	0xC34	0x1D70

NE	Ghidra	Kind	File offset	Length	Flags
9 (0x09)	Code9:1040	Code	0x19740	0x1E02	0x1D70
10 (0x0A)	Code10:1048	Code	0x1B5E0	0x2FF2	0x1D70
11 (0x0B)	Code11:1050	Code	0x1E7A0	0x162	0x1D70
12 (0x0C)	Code12:1058	Code	0x1E920	0x2C2C	0x1D70
13 (0x0D)	Code13:1060	Code	0x216C0	0x1B1E	0x1D70
14 (0x0E)	Code14:1068	Code	0x23260	0x1E3C	0x1D70
15 (0x0F)	Code15:1070	Code	0x251E0	0x2806	0x1D70
16 (0x10)	Code16:1078	Code	0x27AA0	0x2BA4	0x1D70
17 (0x11)	Code17:1080	Code	0x2A720	0x226	0x1D70
18 (0x12)	Code18:1088	Code	0x2A960	0x2CD8	0x1D70
19 (0x13)	Code19:1090	Code	0x2D780	0x454	0x1D70
20 (0x14)	Code20:1098	Code	0x2DC20	0xF04	0x1D70
21 (0x15)	Code21:10A0	Code	0x2EC40	0x7BE	0x1D70
22 (0x16)	Code22:10A8	Code	0x2F4C0	0x644	0x1D70
23 (0x17)	Code23:10B0	Code	0x2FB60	0x5B6E	0x1D70
24 (0x18)	Code24:10B8	Code	0x35880	0xD70	0x1D70
25 (0x19)	Code25:10C0	Code	0x36640	0xD6C	0x1D70
26 (0x1A)	Code26:10C8	Code	0x373E0	0x28E2	0x1D70
27 (0x1B)	Code27:10D0	Code	0x39E00	0x238E	0x1D70
28 (0x1C)	Code28:10D8	Code	0x3C240	0x1AAE	0x1D70
29 (0x1D)	Code29:10E0	Code	0x3DDC0	0x217C	0x1D70
30 (0x1E)	Code30:10E8	Code	0x3FFC0	0x34BE	0x1D70
31 (0x1F)	Code31:10F0	Code	0x435A0	0x289E	0x1D70
32 (0x20)	Code32:10F8	Code	0x45F20	0xB24	0x1D70
33 (0x21)	Code33:1100	Code	0x46A80	0x275E	0x1D70
34 (0x22)	Code34:1108	Code	0x492A0	0x1CE	0x1D70
35 (0x23)	Code35:1110	Code	0x494A0	0xAE4	0x1D70
36 (0x24)	Code36:1118	Code	0x49FE0	0xAF0	0x1D70
37 (0x25)	Code37:1120	Code	0x4AB40	0x2D0	0x1D70
38 (0x26)	Code38:1128	Code	0x4AE40	0x222A	0x1D70
39 (0x27)	Code39:1130	Code	0x4D0E0	0x2BA8	0x1D70
40 (0x28)	Code40:1138	Code	0x4FDE0	0x1CA0	0x1D70
41 (0x29)	Code41:1140	Code	0x51B20	0x738	0x1D70
42 (0x2A)	Code42:1148	Code	0x522A0	0x1156	0x1D70
43 (0x2B)	Code43:1150	Code	0x53480	0x383	0x1D70
44 (0x2C)	Code44:1158	Code	0x53820	0x1194	0x1D70
45 (0x2D)	Code45:1160	Code	0x54A20	0x1094	0x1D70
46 (0x2E)	Code46:1168	Code	0x55B40	0x18BA	0x1D70
47 (0x2F)	Code47:1170	Code	0x57460	0x4362	0x1D70
48 (0x30)	Code48:1178	Code	0x5B940	0x1FC8	0x1D70
49 (0x31)	Code49:1180	Code	0x5D9A0	0x4D6	0x1D70
50 (0x32)	Code50:1188	Code	0x5DEC0	0xB42	0x1D70
51 (0x33)	Code51:1190	Code	0x5EA40	0x3E8	0x1D70
52 (0x34)	Code52:1198	Code	0x5EE80	0x57E	0x1D70
53 (0x35)	Code53:11A0	Code	0x5F440	0xC7A	0x1D70
54 (0x36)	Code54:11A8	Code	0x60100	0xEA6	0x1D70
55 (0x37)	Code55:11B0	Code	0x61080	0xF7A	0x1D70
56 (0x38)	Code56:11B8	Code	0x62020	0x1AE	0x1D70
57 (0x39)	Code57:11C0	Code	0x621E0	0x1DB4	0x1D70
58 (0x3A)	Code58:11C8	Code	0x64080	0x13CA	0x1D70
59 (0x3B)	Code59:11D0	Code	0x65540	0x3C8	0x1D70
60 (0x3C)	Code60:11D8	Code	0x65960	0x2D8E	0x1D70
61 (0x3D)	Code61:11E0	Code	0x68780	0x27BC	0x1D70
62 (0x3E)	Code62:11E8	Code	0x6B000	0x148E	0x1D70
63 (0x3F)	Code63:11F0	Code	0x6C660	0x2C96	0x1D70

NE	Ghidra	Kind	File offset	Length	Flags
64 (0x40)	Code64:11F8	Code	0x6F3A0	0x4C	0x1D70
65 (0x41)	Code65:1200	Code	0x6F420	0xC52	0x1D70
66 (0x42)	Code66:1208	Code	0x70120	0x1E24	0x1D70
67 (0x43)	Code67:1210	Code	0x72000	0x606	0x1D70
68 (0x44)	Code68:1218	Code	0x72660	0x480	0x1D70
69 (0x45)	Code69:1220	Code	0x72B20	0x348C	0x1D70
70 (0x46)	Code70:1228	Code	0x760E0	0x746	0x1D70
71 (0x47)	Code71:1230	Code	0x768A0	0x5EE8	0x1D70
72 (0x48)	Code72:1238	Code	0x7CA20	0x3B9E	0x1D70
73 (0x49)	Code73:1240	Code	0x808C0	0x137E	0x1D70
74 (0x4A)	Code74:1248	Code	0x81D00	0x38FA	0x1D70
75 (0x4B)	Code75:1250	Code	0x85840	0x618	0x1D70
76 (0x4C)	Code76:1258	Code	0x85EA0	0x11A0	0x1D70
77 (0x4D)	Code77:1260	Code	0x87140	0xCAE	0x1D70
78 (0x4E)	Code78:1268	Code	0x87EC0	0x2312	0x1D70
79 (0x4F)	Code79:1270	Code	0x8A420	0x22	0x1D70
80 (0x50)	Code80:1278	Code	0x8A460	0x7B56	0x1D70
81 (0x51)	Code81:1280	Code	0x92220	0x4070	0x1D70
82 (0x52)	Code82:1288	Code	0x96380	0xA2E	0x1D70
83 (0x53)	Code83:1290	Code	0x96E80	0xF30	0x1D70
84 (0x54)	Code84:1298	Code	0x97E60	0x422	0x1D70
85 (0x55)	Code85:12A0	Code	0x98320	0x3C	0x0D60
86 (0x56)	Code86:12A8	Code	0x98380	0x2C	0x1D30
87 (0x57)	Code87:12B0	Code	0x983C0	0x68A	0x1D30
88 (0x58)	Code88:12B8	Code	0x98AC0	0x179A	0x1D30
89 (0x59)	Code89:12C0	Code	0x9A3A0	0x230	0x1D30
90 (0x5A)	Code90:12C8	Code	0x9A600	0x1646	0x1D30
91 (0x5B)	Code91:12D0	Code	0x9BD00	0x1C16	0x1D30
92 (0x5C)	Code92:12D8	Code	0x9D9C0	0x2412	0x1D30
93 (0x5D)	Code93:12E0	Code	0x9FFC0	0x2034	0x1D30
94 (0x5E)	Code94:12E8	Code	0xA20A0	0x26DA	0x1D30
95 (0x5F)	Code95:12F0	Code	0xA4880	0x654	0x1D30
96 (0x60)	Code96:12F8	Code	0xA4F00	0x3F4	0x1D30
97 (0x61)	Code97:1300	Code	0xA5320	0x4C2	0x1D30
98 (0x62)	Code98:1308	Code	0xA5840	0x25B0	0x1D30
99 (0x63)	Code99:1310	Code	0xA7EE0	0xD3A	0x1D30
100 (0x64)	Code100:1318	Code	0xA8C60	0x948	0x1D30
101 (0x65)	Code101:1320	Code	0xA95E0	0x4AA	0x1D30
102 (0x66)	Code102:1328	Code	0xA9B00	0x12	0x1C30
103 (0x67)	Code103:1330	Code	0xA9B20	0x60	0x1D30
104 (0x68)	Code104:1338	Code	0xA9BC0	0x1026	0x1D30
105 (0x69)	Code105:1340	Code	0xAAC60	0x25DC	0x1D30
106 (0x6A)	Code106:1348	Code	0xAD2A0	0xF06	0x1D30
107 (0x6B)	Code107:1350	Code	0xAE2A0	0x642A	0x1D30
108 (0x6C)	Code108:1358	Code	0xB4860	0x1974	0x1D30
109 (0x6D)	Code109:1360	Code	0xB6280	0xD86	0x1D30
110 (0x6E)	Code110:1368	Code	0xB7080	0x15B4	0x1D30
111 (0x6F)	Code111:1370	Code	0xB86E0	0x134E	0x1D30
112 (0x70)	Code112:1378	Code	0xB9D80	0x4D4	0x1D30
113 (0x71)	Code113:1380	Code	0xBA280	0x186	0x1D30
114 (0x72)	Code114:1388	Code	0xBA440	0x6A	0x1C30
115 (0x73)	Code115:1390	Code	0xBA4C0	0xC0C	0x1D30
116 (0x74)	Code116:1398	Code	0xBB120	0x3D50	0x1D30
117 (0x75)	Code117:13A0	Code	0xBF040	0xDCE	0x1D30
118 (0x76)	Code118:13A8	Code	0xBFE40	0x63A	0x1D30

NE	Ghidra	Kind	File offset	Length	Flags
119 (0x77)	Code119:13B0	Code	0xC0520	0x2D1C	0x1D30
120 (0x78)	Code120:13B8	Code	0xC3440	0x1EA8	0x1D30
121 (0x79)	Code121:13C0	Code	0xC53E0	0x7DE	0x1D30
122 (0x7A)	Code122:13C8	Code	0xC5C60	0xC86	0x1D30
123 (0x7B)	Code123:13D0	Code	0xC6B00	0x96E	0x1D30
124 (0x7C)	Code124:13D8	Code	0xC74E0	0x1D2	0x1D30
125 (0x7D)	Code125:13E0	Code	0xC76E0	0x322	0x1D30
126 (0x7E)	Code126:13E8	Code	0xC7A40	0x35C	0x1D30
127 (0x7F)	Code127:13F0	Code	0xC7DE0	0x3246	0x1D30
128 (0x80)	Code128:13F8	Code	0xCB1E0	0x654	0x1D30
129 (0x81)	Code129:1400	Code	0xCB8A0	0x1744	0x1D30
130 (0x82)	Code130:1408	Code	0xCD0C0	0x540	0x1D30
131 (0x83)	Code131:1410	Code	0xCD680	0x1D2	0x1D30
132 (0x84)	Code132:1418	Code	0xCD880	0xF30	0x1D30
133 (0x85)	Code133:1420	Code	0xCE840	0x120E	0x1D30
134 (0x86)	Code134:1428	Code	0xCFB60	0xF3C	0x1D30
135 (0x87)	Code135:1430	Code	0xD0B20	0xD9C	0x1D30
136 (0x88)	Code136:1438	Code	0xD1940	0xAF4	0x1D30
137 (0x89)	Code137:1440	Code	0xD2500	0xD9A	0x1D30
138 (0x8A)	Code138:1448	Code	0xD3340	0x11B6	0x1D30
139 (0x8B)	Code139:1450	Code	0xD45A0	0xA7E	0x1D30
140 (0x8C)	Code140:1458	Code	0xD50A0	0xAB6	0x1D30
141 (0x8D)	Code141:1460	Code	0xD5C40	0x12C6	0x1D30
142 (0x8E)	Code142:1468	Code	0xD6F80	0xC60	0x1D30
143 (0x8F)	Code143:1470	Code	0xD7CA0	0x169A	0x1D30
144 (0x90)	Code144:1478	Code	0xD93E0	0xBC6	0x1D30
145 (0x91)	Code145:1480	Code	0xDA000	0x15E	0x1D30
146 (0x92)	Code146:1488	Code	0xDA180	0x652	0x1D30
147 (0x93)	Code147:1490	Code	0xDA860	0x1322	0x1D30
148 (0x94)	Code148:1498	Code	0xDBC20	0xCC8	0x1D30
149 (0x95)	Code149:14A0	Code	0xDC980	0x82E	0x1D30
150 (0x96)	Code150:14A8	Code	0xDD240	0xD2A	0x1D30
151 (0x97)	Code151:14B0	Code	0xDDFE0	0x307C	0x1D30
152 (0x98)	Code152:14B8	Code	0xE1240	0x1156	0x1D30
153 (0x99)	Code153:14C0	Code	0xE2440	0x13FE	0x1D30
154 (0x9A)	Code154:14C8	Code	0xE3A00	0x1C5C	0x1D30
155 (0x9B)	Code155:14D0	Code	0xE5700	0x1924	0x1D30
156 (0x9C)	Code156:14D8	Code	0xE70C0	0x754	0x1D30
157 (0x9D)	Code157:14E0	Code	0xE7840	0x1638	0x1D30
158 (0x9E)	Code158:14E8	Code	0xE9000	0x17DC	0x1D30
159 (0x9F)	Code159:14F0	Code	0xEA920	0x1F34	0x1D30
160 (0xA0)	Code160:14F8	Code	0xEC9A0	0x10EE	0x1D30
161 (0xA1)	Code161:1500	Code	0xEDBC0	0xAE4	0x1D30
162 (0xA2)	Code162:1508	Code	0xEE700	0x49B8	0x1D30
163 (0xA3)	Code163:1510	Code	0xF3160	0x8C	0x1D30
164 (0xA4)	Code164:1518	Code	0xF3200	0x19C	0x1D30
165 (0xA5)	Code165:1520	Code	0xF33C0	0x1B8E	0x1D30
166 (0xA6)	Code166:1528	Code	0xF5080	0x4D2	0x1D30
167 (0xA7)	Code167:1530	Code	0xF5580	0xBF6	0x1D30
168 (0xA8)	Code168:1538	Code	0xF61C0	0x4BA	0x1D30
169 (0xA9)	Code169:1540	Code	0xF66C0	0x90E	0x1D30
170 (0xAA)	Code170:1548	Code	0xF7020	0x30C	0x1D30
171 (0xAB)	Code171:1550	Code	0xF7360	0x2A8	0x1D30
172 (0xAC)	Code172:1558	Code	0xF7660	0x1EA	0x1D30
173 (0xAD)	Code173:1560	Code	0xF7880	0x30E	0x1D30

NE	Ghidra	Kind	File offset	Length	Flags
174 (0xAE)	Code174:1568	Code	0xF7BC0	0x5C4	0x1D30
175 (0xAF)	Code175:1570	Code	0xF81E0	0x1D04	0x1D30
176 (0xB0)	Code176:1578	Code	0xF9FA0	0x3512	0x1D30
177 (0xB1)	Code177:1580	Code	0xFD5A0	0x754	0x1D30
178 (0xB2)	Code178:1588	Code	0xFDD80	0x300	0x1D30
179 (0xB3)	Code179:1590	Code	0xFE0E0	0x532	0x1D30
180 (0xB4)	Code180:1598	Code	0xFE660	0x1180	0x1D30
181 (0xB5)	Code181:15A0	Code	0xFF820	0x8D8	0x1D30
182 (0xB6)	Code182:15A8	Code	0x100140	0x610	0x1D30
183 (0xB7)	Code183:15B0	Code	0x100780	0x163A	0x1D30
184 (0xB8)	Code184:15B8	Code	0x101E40	0x16C	0x1D30
185 (0xB9)	Code185:15C0	Code	0x101FC0	0x140	0x1D30
186 (0xBA)	Code186:15C8	Code	0x102120	0xEE6	0x1D30
187 (0xBB)	Code187:15D0	Code	0x103060	0x956	0x1D30
188 (0xBC)	Code188:15D8	Code	0x1039E0	0x86	0x1D30
189 (0xBD)	Code189:15E0	Code	0x103A80	0x59CC	0x1D30
190 (0xBE)	Code190:15E8	Code	0x109620	0x23A	0x1D30
191 (0xBF)	Code191:15F0	Code	0x1098A0	0xE2	0x1D30
192 (0xC0)	Code192:15F8	Code	0x1099A0	0xE2E	0x1D30
193 (0xC1)	Data193:1600	Data	0x10A800	0x2A0	0x0C71
194 (0xC2)	Data194:1608	Data	0x10AAA0	0x55C	0x0D71
195 (0xC3)	Data195:1610	Data	0x10B040	0x3F0	0x0C71
196 (0xC4)	Data196:1618	Data	0x10B440	0x1E	0x0C71
197 (0xC5)	Data197:1620	Data	0x10B460	0x1A72	0x0C71
198 (0xC6)	Data198:1628	Data	0x10CEE0	0x80	0x0C71
199 (0xC7)	Data199:1630	Data	0x10CF60	0x3BB8	0x0D71

TB80BMP.DLL.html

Entry NE 2 at 1008:02D8. Auto data NE 27 at 10D0.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xB20	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1540	0x3343	0x1D70
3 (0x03)	Code3:1010	Code	0x4EE0	0x220A	0x1D70
4 (0x04)	Code4:1018	Code	0x72A0	0x32A	0x1D70
5 (0x05)	Code5:1020	Code	0x7620	0x1E6	0x1D70
6 (0x06)	Code6:1028	Code	0x7860	0x54	0x1D70
7 (0x07)	Code7:1030	Code	0x78E0	0xAC0	0x1D70
8 (0x08)	Code8:1038	Code	0x84C0	0x40C	0x1D70
9 (0x09)	Code9:1040	Code	0x88E0	0x1BE	0x1D70
10 (0x0A)	Code10:1048	Code	0x8B00	0x8DAA	0x1D70
11 (0x0B)	Code11:1050	Code	0x11F20	0x3C	0x0D60
12 (0x0C)	Code12:1058	Code	0x11F80	0xB0	0x1D30
13 (0x0D)	Code13:1060	Code	0x12060	0x3E4	0x1D30
14 (0x0E)	Code14:1068	Code	0x12480	0x65A	0x1D30
15 (0x0F)	Code15:1070	Code	0x12B80	0x126A	0x1D30
16 (0x10)	Code16:1078	Code	0x13F20	0x136C	0x1D30
17 (0x11)	Code17:1080	Code	0x153A0	0x996	0x1D30
18 (0x12)	Code18:1088	Code	0x15DE0	0x946	0x1D30
19 (0x13)	Code19:1090	Code	0x168A0	0xB76	0x1D30
20 (0x14)	Code20:1098	Code	0x174A0	0xBBC	0x1D30
21 (0x15)	Code21:10A0	Code	0x18100	0x1A8A	0x1D30
22 (0x16)	Code22:10A8	Code	0x19C20	0x3CC	0x1D30
23 (0x17)	Code23:10B0	Code	0x1A020	0x1C4	0x1D30

NE	Ghidra	Kind	File offset	Length	Flags
24 (0x18)	Code24:10B8	Code	0x1A240	0x2E1	0x1D30
25 (0x19)	Code25:10C0	Code	0x1A580	0x43B	0x1D30
26 (0x1A)	Data26:10C8	Data	0x1AB20	0x6	0x0C71
27 (0x1B)	Data27:10D0	Data	0x1AB40	0x23AE	0x0D71

TB80CMP.ghidra.dll.html

Entry NE 2 at 1008:00BE. Auto data NE 47 at 1170.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x520	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xF40	0x224D	0x1D50
3 (0x03)	Code3:1010	Code	0x3360	0x1CC4	0x1D50
4 (0x04)	Code4:1018	Code	0x51E0	0x3C	0x0D40
5 (0x05)	Code5:1020	Code	0x5240	0x12	0x1C10
6 (0x06)	Code6:1028	Code	0x5260	0x8EB6	0x1D10
7 (0x07)	Code7:1030	Code	0xE180	0xED8	0x1D10
8 (0x08)	Code8:1038	Code	0xF0A0	0x1166	0x1D10
9 (0x09)	Code9:1040	Code	0x10280	0x10D0	0x1D10
10 (0x0A)	Code10:1048	Code	0x113C0	0x1F4A	0x1D10
11 (0x0B)	Code11:1050	Code	0x13360	0xF0E	0x1D10
12 (0x0C)	Code12:1058	Code	0x142A0	0x632	0x1D10
13 (0x0D)	Code13:1060	Code	0x14940	0x2AE	0x1D10
14 (0x0E)	Code14:1068	Code	0x14C20	0xAC0	0x1D10
15 (0x0F)	Code15:1070	Code	0x15740	0x15BC	0x1D10
16 (0x10)	Code16:1078	Code	0x16D80	0x2320	0x1D10
17 (0x11)	Code17:1080	Code	0x19140	0x1AAE	0x1D10
18 (0x12)	Code18:1088	Code	0x1AC80	0x25FA	0x1D10
19 (0x13)	Code19:1090	Code	0x1D360	0x42E	0x1D10
20 (0x14)	Code20:1098	Code	0x1D7C0	0x1490	0x1D10
21 (0x15)	Code21:10A0	Code	0x1ECA0	0x1EE6	0x1D10
22 (0x16)	Code22:10A8	Code	0x20BE0	0x263A	0x1D10
23 (0x17)	Code23:10B0	Code	0x232A0	0x312C	0x1D10
24 (0x18)	Code24:10B8	Code	0x26460	0x23DA	0x1D10
25 (0x19)	Code25:10C0	Code	0x28880	0x150C	0x1D10
26 (0x1A)	Code26:10C8	Code	0x29DE0	0xF60	0x1D10
27 (0x1B)	Code27:10D0	Code	0x2ADA0	0x95E	0x1D10
28 (0x1C)	Code28:10D8	Code	0x2B740	0x1822	0x1D10
29 (0x1D)	Code29:10E0	Code	0x2CFA0	0x1CF0	0x1D10
30 (0x1E)	Code30:10E8	Code	0x2ECE0	0x138C	0x1D10
31 (0x1F)	Code31:10F0	Code	0x300A0	0x4FAC	0x1D10
32 (0x20)	Code32:10F8	Code	0x35080	0x7BC	0x1D10
33 (0x21)	Code33:1100	Code	0x35880	0x22	0x1C10
34 (0x22)	Code34:1108	Code	0x358C0	0x249C	0x1D10
35 (0x23)	Data35:1110	Data	0x37E20	0x9CFE	0x0CF1
36 (0x24)	Data36:1118	Data	0x41B20	0x14	0x0C71
37 (0x25)	Data37:1120	Data	0x41B40	0x28	0x0C71
38 (0x26)	Data38:1128	Data	0x41B80	0xF0	0x0C71
39 (0x27)	Data39:1130	Data	0x41C80	0xA	0x0C71
40 (0x28)	Data40:1138	Data	0x41CA0	0x4D4	0x0C71
41 (0x29)	Data41:1140	Data	0x42180	0x508	0x0C71
42 (0x2A)	Data42:1148	Data	0x426A0	0x6A0	0x0C71
43 (0x2B)	Data43:1150	Data	0x42D40	0xC40	0x0C71
44 (0x2C)	Data44:1158	Data	0x43980	0x8C0	0x0C71
45 (0x2D)	Data45:1160	Data	0x44240	0x1F3F	0x0C71

NE	Ghidra	Kind	File offset	Length	Flags
46 (0x2E)	Data46:1168	Data	0x46180	0x2	0x0C71
47 (0x2F)	Data47:1170	Data	0x461A0	0x74C8	0x0D71

TB80ENU1.DLL.html

Entry NE 6 at 1028:0054. Auto data NE 8 at 1038.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x8C0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x12E0	0x42	0x1D70
3 (0x03)	Code3:1010	Code	0x1340	0x24	0x1D70
4 (0x04)	Code4:1018	Code	0x1380	0x84	0x1C70
5 (0x05)	Code5:1020	Code	0x1420	0x2C	0x1C70
6 (0x06)	Code6:1028	Code	0x1460	0x74D	0x1D70
7 (0x07)	Code7:1030	Code	0x1C40	0x54	0x1D70
8 (0x08)	Data8:1038	Data	0x1CC0	0x172	0x0C71

TB80FLT.DLL.html

Entry NE 23 at 10B0:0484. Auto data NE 28 at 10D8.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xC40	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1660	0x3C	0x0D60
3 (0x03)	Code3:1010	Code	0x16C0	0x2F86	0x1D30
4 (0x04)	Code4:1018	Code	0x4800	0xF02	0x1D30
5 (0x05)	Code5:1020	Code	0x57C0	0x40	0x1D30
6 (0x06)	Code6:1028	Code	0x5840	0x2F64	0x1D30
7 (0x07)	Code7:1030	Code	0x8A40	0x746C	0x1D30
8 (0x08)	Code8:1038	Code	0x102A0	0x19C	0x1D30
9 (0x09)	Code9:1040	Code	0x10480	0x17B2	0x1D30
10 (0x0A)	Code10:1048	Code	0x11E20	0x2734	0x1D30
11 (0x0B)	Code11:1050	Code	0x14740	0xF1E	0x1D30
12 (0x0C)	Code12:1058	Code	0x157C0	0xF18	0x1D30
13 (0x0D)	Code13:1060	Code	0x16880	0x1B66	0x1D30
14 (0x0E)	Code14:1068	Code	0x184E0	0xC16	0x1D30
15 (0x0F)	Code15:1070	Code	0x191C0	0x31A	0x1D30
16 (0x10)	Code16:1078	Code	0x19540	0x1D0A	0x1D30
17 (0x11)	Code17:1080	Code	0x1B2E0	0x890	0x1D30
18 (0x12)	Code18:1088	Code	0x1BBC0	0x6C4	0x1D30
19 (0x13)	Code19:1090	Code	0x1C360	0x1180	0x1D30
20 (0x14)	Code20:1098	Code	0x1D520	0x64	0x1D30
21 (0x15)	Code21:10A0	Code	0x1D5C0	0xE6	0x1D30
22 (0x16)	Code22:10A8	Code	0x1D6C0	0xC0E	0x1D30
23 (0x17)	Code23:10B0	Code	0x1E360	0x24C5	0x1D70
24 (0x18)	Code24:10B8	Code	0x20A60	0x8D8	0x1D30
25 (0x19)	Code25:10C0	Code	0x21380	0x140	0x1D30
26 (0x1A)	Code26:10C8	Code	0x214E0	0x21F	0x1D30
27 (0x1B)	Data27:10D0	Data	0x21740	0x90	0x0C71
28 (0x1C)	Data28:10D8	Data	0x217E0	0x3218	0x0D71

TB80JPG.DLL.html

Entry NE 2 at `1008:A5C2`. Auto data NE 3 at `1010`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x250	0x3B	0x0D60
2 (0x02)	Code2:1008	Code	0x2A0	0xDB9C	0x1D70
3 (0x03)	Data3:1010	Data	0xE8A0	0xEA5	0x0D71

TB80LNL.DLL.html

Entry NE 15 at `1070:0054`. Auto data NE 39 at `1130`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xBC0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x15E0	0x21A	0x1D70
3 (0x03)	Code3:1010	Code	0x1840	0x454	0x1D70
4 (0x04)	Code4:1018	Code	0x1E20	0x5EC	0x1D70
5 (0x05)	Code5:1020	Code	0x2480	0x906	0x1D70
6 (0x06)	Code6:1028	Code	0x2DE0	0x52E	0x1D70
7 (0x07)	Code7:1030	Code	0x3340	0x160E	0x1D70
8 (0x08)	Code8:1038	Code	0x4A80	0x90E	0x1D70
9 (0x09)	Code9:1040	Code	0x5440	0x4C26	0x1D70
10 (0x0A)	Code10:1048	Code	0xA1C0	0x2004	0x1D70
11 (0x0B)	Code11:1050	Code	0xC280	0xEA2	0x1D70
12 (0x0C)	Code12:1058	Code	0xD280	0x1E3A	0x1D70
13 (0x0D)	Code13:1060	Code	0xF0E0	0x32B6	0x1D70
14 (0x0E)	Code14:1068	Code	0x12460	0x3C	0x0D60
15 (0x0F)	Code15:1070	Code	0x124C0	0x2181	0x1D70
16 (0x10)	Code16:1078	Code	0x14840	0x40	0x1C30
17 (0x11)	Code17:1080	Code	0x14880	0x136	0x1D30
18 (0x12)	Code18:1088	Code	0x149E0	0x204	0x1D30
19 (0x13)	Code19:1090	Code	0x14C00	0x1648	0x1D30
20 (0x14)	Code20:1098	Code	0x16300	0x68A	0x1D30
21 (0x15)	Code21:10A0	Code	0x169E0	0x3780	0x1D30
22 (0x16)	Code22:10A8	Code	0x1A280	0x3AC8	0x1D30
23 (0x17)	Code23:10B0	Code	0x1DE20	0xF34	0x1D30
24 (0x18)	Code24:10B8	Code	0x1EEA0	0xABE	0x1D30
25 (0x19)	Code25:10C0	Code	0x1FA00	0x392	0x1D30
26 (0x1A)	Code26:10C8	Code	0x1FDE0	0x45F2	0x1D30
27 (0x1B)	Code27:10D0	Code	0x244A0	0x858	0x1D30
28 (0x1C)	Code28:10D8	Code	0x24D60	0x1A08	0x1D30
29 (0x1D)	Code29:10E0	Code	0x26840	0x818	0x1D30
30 (0x1E)	Code30:10E8	Code	0x270C0	0xA16	0x1D30
31 (0x1F)	Code31:10F0	Code	0x27B40	0xDF6	0x1D30
32 (0x20)	Code32:10F8	Code	0x289A0	0x1E26	0x1D30
33 (0x21)	Code33:1100	Code	0x2A880	0xF8C	0x1D30
34 (0x22)	Code34:1108	Code	0x2B8A0	0x8D8	0x1D30
35 (0x23)	Code35:1110	Code	0x2C1C0	0xE2	0x1D30
36 (0x24)	Code36:1118	Code	0x2C2E0	0x140	0x1D30
37 (0x25)	Data37:1120	Data	0x2C440	0x10	0x0C71
38 (0x26)	Data38:1128	Data	0x2C460	0x10	0x0C71
39 (0x27)	Data39:1130	Data	0x2C480	0x31B8	0x0D71

TB80MM.DLL.html

Entry NE 3 at `1010:0054`. Auto data NE 22 at `10A8`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x480	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xEA0	0x26	0x0D60
3 (0x03)	Code3:1010	Code	0xEE0	0xC63	0x1D70
4 (0x04)	Code4:1018	Code	0x1C00	0x3CA	0x1D70
5 (0x05)	Code5:1020	Code	0x2060	0x5E2	0x1D70
6 (0x06)	Code6:1028	Code	0x26A0	0xBC	0x1D70
7 (0x07)	Code7:1030	Code	0x2780	0x1838	0x1D70
8 (0x08)	Code8:1038	Code	0x4080	0x8C8	0x1D70
9 (0x09)	Code9:1040	Code	0x49A0	0x40C	0x1D70
10 (0x0A)	Code10:1048	Code	0x4DC0	0x1BE	0x1D70
11 (0x0B)	Code11:1050	Code	0x4FE0	0x210	0x1D70
12 (0x0C)	Code12:1058	Code	0x5220	0x3C	0x0D60
13 (0x0D)	Code13:1060	Code	0x5280	0xA3E	0x1D30
14 (0x0E)	Code14:1068	Code	0x5D20	0x70A	0x1D30
15 (0x0F)	Code15:1070	Code	0x6460	0x53E	0x1D30
16 (0x10)	Code16:1078	Code	0x6A20	0x464	0x1D30
17 (0x11)	Code17:1080	Code	0x6F00	0x142	0x1D30
18 (0x12)	Code18:1088	Code	0x7080	0xB4A	0x1D30
19 (0x13)	Code19:1090	Code	0x7C80	0x446	0x1D30
20 (0x14)	Code20:1098	Code	0x8120	0xC9A	0x1D30
21 (0x15)	Code21:10A0	Code	0x8F00	0x1C4	0x1D30
22 (0x16)	Data22:10A8	Data	0x9120	0x13D2	0x0D71

TB80NET.EXE.html

Entry NE 2 at `1008:0120`. Auto data NE 45 at `1160`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x3A0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xDC0	0x17F4	0x1D50
3 (0x03)	Code3:1010	Code	0x2660	0x16A8	0x1D10
4 (0x04)	Code4:1018	Code	0x3F00	0x206	0x1D10
5 (0x05)	Code5:1020	Code	0x4140	0x1904	0x1D10
6 (0x06)	Code6:1028	Code	0x5B20	0xBA4	0x1D10
7 (0x07)	Code7:1030	Code	0x6700	0x7A	0x1D10
8 (0x08)	Code8:1038	Code	0x67A0	0xBEC	0x1D10
9 (0x09)	Code9:1040	Code	0x7400	0x930	0x1D10
10 (0x0A)	Code10:1048	Code	0x7DC0	0x8D8	0x1D10
11 (0x0B)	Code11:1050	Code	0x86E0	0x1BA	0x1D10
12 (0x0C)	Code12:1058	Code	0x8900	0x974	0x1D10
13 (0x0D)	Code13:1060	Code	0x92E0	0xB60	0x1D10
14 (0x0E)	Code14:1068	Code	0x9EA0	0x3F0	0x1D10
15 (0x0F)	Code15:1070	Code	0xA2C0	0xE4	0x1D10
16 (0x10)	Code16:1078	Code	0xA3E0	0x140	0x1D10
17 (0x11)	Code17:1080	Code	0xA540	0x26A	0x1D10
18 (0x12)	Code18:1088	Code	0xA820	0x1DA	0x1D10
19 (0x13)	Code19:1090	Code	0xAA20	0x698	0x1D10
20 (0x14)	Code20:1098	Code	0xB100	0x1C6	0x1D10
21 (0x15)	Code21:10A0	Code	0xB300	0x1040	0x1D10
22 (0x16)	Code22:10A8	Code	0xC3A0	0x9A4	0x1D10
23 (0x17)	Code23:10B0	Code	0xCD80	0x9A	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
24 (0x18)	Code24:10B8	Code	0xCE40	0xA28	0x1D10
25 (0x19)	Code25:10C0	Code	0xD8A0	0x1CA4	0x1D10
26 (0x1A)	Code26:10C8	Code	0xF560	0x9D6	0x1D10
27 (0x1B)	Code27:10D0	Code	0xFF60	0x64	0x1D10
28 (0x1C)	Code28:10D8	Code	0xFFE0	0x1D6E	0x1D10
29 (0x1D)	Code29:10E0	Code	0x11D60	0x25A	0x1D10
30 (0x1E)	Code30:10E8	Code	0x11FE0	0xDEC	0x1D10
31 (0x1F)	Code31:10F0	Code	0x12E00	0x1CE	0x1C10
32 (0x20)	Code32:10F8	Code	0x12FE0	0x6AA	0x1C10
33 (0x21)	Code33:1100	Code	0x136A0	0x918	0x1D10
34 (0x22)	Code34:1108	Code	0x13FE0	0x50A	0x1D10
35 (0x23)	Code35:1110	Code	0x14500	0x162	0x1D10
36 (0x24)	Code36:1118	Code	0x146C0	0x1E4C	0x1D10
37 (0x25)	Code37:1120	Code	0x16640	0x772	0x1D10
38 (0x26)	Code38:1128	Code	0x16DE0	0xEBD	0x1D10
39 (0x27)	Code39:1130	Code	0x17D00	0x45A	0x1D10
40 (0x28)	Code40:1138	Code	0x18200	0x80A	0x1D10
41 (0x29)	Code41:1140	Code	0x18A40	0xB9E	0x1D10
42 (0x2A)	Code42:1148	Code	0x196E0	0x1F1	0x1D10
43 (0x2B)	Code43:1150	Code	0x19920	0x662	0x1D10
44 (0x2C)	Data44:1158	Data	0x1A020	0xE	0x0C71
45 (0x2D)	Data45:1160	Data	0x1A040	0x2DB0	0x0D71

TB80OCX.DLL.html

Entry NE 1 at 1000:0054. Auto data NE 53 at 11A0.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x3360	0x2AC5	0x1D50
2 (0x02)	Code2:1008	Code	0x8420	0x6A7	0x1D10
3 (0x03)	Code3:1010	Code	0x8B60	0x10CC	0x1D10
4 (0x04)	Code4:1018	Code	0x9D00	0x536F	0x1D10
5 (0x05)	Code5:1020	Code	0xF2A0	0x336	0x1D10
6 (0x06)	Code6:1028	Code	0xF600	0x2CC	0x1D10
7 (0x07)	Code7:1030	Code	0xF900	0x8E3	0x1D10
8 (0x08)	Code8:1038	Code	0x10240	0x882	0x1D10
9 (0x09)	Code9:1040	Code	0x10B20	0x1D9C	0x1D10
10 (0x0A)	Code10:1048	Code	0x129E0	0x370	0x1D10
11 (0x0B)	Code11:1050	Code	0x12DA0	0x2511	0x1D10
12 (0x0C)	Code12:1058	Code	0x153E0	0x947	0x1D10
13 (0x0D)	Code13:1060	Code	0x15D80	0x231	0x1D10
14 (0x0E)	Code14:1068	Code	0x15FE0	0x5A6	0x1D10
15 (0x0F)	Code15:1070	Code	0x16600	0x1B67	0x1D10
16 (0x10)	Code16:1078	Code	0x182C0	0x21AE	0x1D10
17 (0x11)	Code17:1080	Code	0x1A5A0	0x121E	0x1D10
18 (0x12)	Code18:1088	Code	0x1B8A0	0x133D	0x1D10
19 (0x13)	Code19:1090	Code	0x1CC80	0x1C26	0x1D10
20 (0x14)	Code20:1098	Code	0x1E980	0x1112	0x1D10
21 (0x15)	Code21:10A0	Code	0x1FB20	0x755	0x1D10
22 (0x16)	Code22:10A8	Code	0x202C0	0x720	0x1D10
23 (0x17)	Code23:10B0	Code	0x20A40	0x1078	0x1D10
24 (0x18)	Code24:10B8	Code	0x21B60	0x9E0	0x1D10
25 (0x19)	Code25:10C0	Code	0x225A0	0x11F8	0x1D10
26 (0x1A)	Code26:10C8	Code	0x23880	0x760	0x1D10
27 (0x1B)	Code27:10D0	Code	0x24040	0x7BE	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
28 (0x1C)	Code28:10D8	Code	0x24880	0x15A	0x1D10
29 (0x1D)	Code29:10E0	Code	0x24A00	0x7C7	0x1D10
30 (0x1E)	Code30:10E8	Code	0x25240	0x444	0x1D10
31 (0x1F)	Code31:10F0	Code	0x256C0	0x3CE	0x1D10
32 (0x20)	Code32:10F8	Code	0x25AC0	0x2DA	0x1D10
33 (0x21)	Code33:1100	Code	0x25DE0	0x25C	0x1D10
34 (0x22)	Code34:1108	Code	0x26060	0x2E3	0x1D10
35 (0x23)	Code35:1110	Code	0x26360	0x4714	0x1D10
36 (0x24)	Code36:1118	Code	0x2AC00	0xE0	0x1D10
37 (0x25)	Code37:1120	Code	0x2AD00	0xCC6	0x1D10
38 (0x26)	Code38:1128	Code	0x2B9E0	0x2055	0x1D10
39 (0x27)	Code39:1130	Code	0x2DA80	0x41C	0x1D10
40 (0x28)	Code40:1138	Code	0x2DF00	0x14E	0x1D10
41 (0x29)	Code41:1140	Code	0x2E060	0x21F	0x1D10
42 (0x2A)	Code42:1148	Code	0x2E2C0	0x6C3	0x1D10
43 (0x2B)	Code43:1150	Code	0x2EA60	0x117F	0x1D10
44 (0x2C)	Code44:1158	Code	0x2FC20	0x14DA	0x1D10
45 (0x2D)	Code45:1160	Code	0x31120	0x53	0x1D10
46 (0x2E)	Code46:1168	Code	0x311A0	0xD4A	0x1D10
47 (0x2F)	Data47:1170	Data	0x6060	0x20	0x0C71
48 (0x30)	Data48:1178	Data	0x60A0	0x9C	0x0D71
49 (0x31)	Data49:1180	Data	0x6160	0x20	0x0C71
50 (0x32)	Data50:1188	Data	0x0	0x0	0x0C71
51 (0x33)	Data51:1190	Data	0x0	0x0	0x0C71
52 (0x34)	Data52:1198	Data	0x61A0	0x90	0x0C71
53 (0x35)	Data53:11A0	Data	0x6240	0x2148	0x0D71

TB80RCR.DLL.html

Entry NE 2 at `1008:A998`. Auto data NE 5 at `1020`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xCC0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x16D0	0xD7F7	0x1D50
3 (0x03)	Code3:1010	Code	0xF9E0	0xCD02	0x1D50
4 (0x04)	Data4:1018	Data	0x1CE70	0x6	0x0C71
5 (0x05)	Data5:1020	Data	0x1CE80	0x1C17	0x0D71

TB80RTM.EXE-5.html

Entry NE 2 at `1008:035A`. Auto data NE 139 at `1450`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x1420	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1E40	0x4A7C	0x0D40
3 (0x03)	Code3:1010	Code	0x7240	0x1AFA	0x1D50
4 (0x04)	Code4:1018	Code	0x8EE0	0x276E	0x1D50
5 (0x05)	Code5:1020	Code	0xB860	0x2ACC	0x1D50
6 (0x06)	Code6:1028	Code	0xE540	0x55E	0x1D50
7 (0x07)	Code7:1030	Code	0xEAC0	0x41A	0x1D50
8 (0x08)	Code8:1038	Code	0xEF80	0x38A	0x1D50
9 (0x09)	Code9:1040	Code	0xF3C0	0x9D0	0x1D50
10 (0x0A)	Code10:1048	Code	0xFE60	0x6AA	0x1D50
11 (0x0B)	Code11:1050	Code	0x10640	0x1A7C	0x1D50
12 (0x0C)	Code12:1058	Code	0x121E0	0xAD2	0x1D50

NE	Ghidra	Kind	File offset	Length	Flags
13 (0x0D)	Code13:1060	Code	0x12E40	0x16AA	0x1D50
14 (0x0E)	Code14:1068	Code	0x147C0	0x430	0x1D50
15 (0x0F)	Code15:1070	Code	0x14CA0	0x1D44	0x1D50
16 (0x10)	Code16:1078	Code	0x16C80	0x1510	0x1D50
17 (0x11)	Code17:1080	Code	0x18320	0x9E6	0x1D50
18 (0x12)	Code18:1088	Code	0x18E00	0x3B8	0x1D50
19 (0x13)	Code19:1090	Code	0x19220	0x638	0x1D50
20 (0x14)	Code20:1098	Code	0x19900	0x2C9A	0x1D50
21 (0x15)	Code21:10A0	Code	0x1C920	0x1066	0x1D50
22 (0x16)	Code22:10A8	Code	0x1DA80	0x22B8	0x1D50
23 (0x17)	Code23:10B0	Code	0x20040	0x3314	0x1D50
24 (0x18)	Code24:10B8	Code	0x234E0	0x351A	0x1D50
25 (0x19)	Code25:10C0	Code	0x26CE0	0x3612	0x1D50
26 (0x1A)	Code26:10C8	Code	0x2A6A0	0x1E18	0x1D50
27 (0x1B)	Code27:10D0	Code	0x2C5C0	0x3320	0x1D50
28 (0x1C)	Code28:10D8	Code	0x2FB40	0x44C	0x1D50
29 (0x1D)	Code29:10E0	Code	0x30060	0xEAA	0x1D50
30 (0x1E)	Code30:10E8	Code	0x310A0	0x153C	0x1D50
31 (0x1F)	Code31:10F0	Code	0x32780	0x2090	0x1D50
32 (0x20)	Code32:10F8	Code	0x34B40	0x43CE	0x1D50
33 (0x21)	Code33:1100	Code	0x39160	0x994	0x1D50
34 (0x22)	Code34:1108	Code	0x39BE0	0x994	0x1D50
35 (0x23)	Code35:1110	Code	0x3A620	0x58C0	0x1D50
36 (0x24)	Code36:1118	Code	0x40120	0x3DE	0x1D50
37 (0x25)	Code37:1120	Code	0x40560	0x127A	0x1D50
38 (0x26)	Code38:1128	Code	0x41980	0xA6C	0x1D50
39 (0x27)	Code39:1130	Code	0x424C0	0x40E2	0x1D50
40 (0x28)	Code40:1138	Code	0x467A0	0xB5A	0x1D50
41 (0x29)	Code41:1140	Code	0x473C0	0x2C	0x1C50
42 (0x2A)	Code42:1148	Code	0x47400	0xDD8	0x1D50
43 (0x2B)	Code43:1150	Code	0x48360	0x50A	0x1D50
44 (0x2C)	Code44:1158	Code	0x488E0	0x5EC	0x1D50
45 (0x2D)	Code45:1160	Code	0x48F00	0x84	0x1D50
46 (0x2E)	Code46:1168	Code	0x48FA0	0x854	0x1D50
47 (0x2F)	Code47:1170	Code	0x49920	0xC76	0x1D50
48 (0x30)	Code48:1178	Code	0x4A680	0x1022	0x1D10
49 (0x31)	Code49:1180	Code	0x4B7A0	0x1D00	0x1D10
50 (0x32)	Code50:1188	Code	0x4D6A0	0x35B6	0x1D10
51 (0x33)	Code51:1190	Code	0x50F80	0x1914	0x1D10
52 (0x34)	Code52:1198	Code	0x52AE0	0x204	0x1D10
53 (0x35)	Code53:11A0	Code	0x52D40	0x81A	0x1D10
54 (0x36)	Code54:11A8	Code	0x53620	0x16A	0x1D10
55 (0x37)	Code55:11B0	Code	0x537E0	0x50A	0x1D10
56 (0x38)	Code56:11B8	Code	0x53DC0	0xD54	0x1D10
57 (0x39)	Code57:11C0	Code	0x54C40	0x9F2	0x1D10
58 (0x3A)	Code58:11C8	Code	0x556E0	0x22	0x1D10
59 (0x3B)	Code59:11D0	Code	0x55720	0x106	0x1D10
60 (0x3C)	Code60:11D8	Code	0x55840	0xD72	0x1D10
61 (0x3D)	Code61:11E0	Code	0x56680	0x14	0x1C10
62 (0x3E)	Code62:11E8	Code	0x566A0	0xD72	0x1D10
63 (0x3F)	Code63:11F0	Code	0x57560	0x4C	0x1D10
64 (0x40)	Code64:11F8	Code	0x575C0	0x1B0	0x1D10
65 (0x41)	Code65:1200	Code	0x577A0	0x496	0x1D10
66 (0x42)	Code66:1208	Code	0x57C80	0xA82	0x1D10
67 (0x43)	Code67:1210	Code	0x587A0	0xC7E	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
68 (0x44)	Code68:1218	Code	0x59580	0x34B8	0x1D10
69 (0x45)	Code69:1220	Code	0x5CD60	0x8A74	0x1D10
70 (0x46)	Code70:1228	Code	0x65D20	0x2080	0x1D10
71 (0x47)	Code71:1230	Code	0x67FC0	0xCF4	0x1D10
72 (0x48)	Code72:1238	Code	0x68DA0	0x32E	0x1D10
73 (0x49)	Code73:1240	Code	0x69140	0x17F0	0x1D10
74 (0x4A)	Code74:1248	Code	0x6A9E0	0x1890	0x1D10
75 (0x4B)	Code75:1250	Code	0x6C3C0	0xA80	0x1D10
76 (0x4C)	Code76:1258	Code	0x6CF20	0xBE2	0x1D10
77 (0x4D)	Code77:1260	Code	0x6DB80	0x31FC	0x1D10
78 (0x4E)	Code78:1268	Code	0x70FC0	0x1742	0x1D10
79 (0x4F)	Code79:1270	Code	0x72880	0x138	0x1D10
80 (0x50)	Code80:1278	Code	0x72A00	0x14E	0x1D10
81 (0x51)	Code81:1280	Code	0x72BA0	0x57E	0x1D10
82 (0x52)	Code82:1288	Code	0x73160	0x784	0x1D10
83 (0x53)	Code83:1290	Code	0x73920	0xD9E	0x1D10
84 (0x54)	Code84:1298	Code	0x747C0	0xC4C	0x1D10
85 (0x55)	Code85:12A0	Code	0x754E0	0x96E	0x1D10
86 (0x56)	Code86:12A8	Code	0x75F00	0x22F6	0x1D10
87 (0x57)	Code87:12B0	Code	0x78920	0xDAA	0x1D10
88 (0x58)	Code88:12B8	Code	0x797A0	0x258	0x1D10
89 (0x59)	Code89:12C0	Code	0x79A40	0x1EF0	0x1D10
90 (0x5A)	Code90:12C8	Code	0x7BB40	0xAE	0x1D10
91 (0x5B)	Code91:12D0	Code	0x7BC00	0x7B6	0x1D10
92 (0x5C)	Code92:12D8	Code	0x7C440	0x38D0	0x1D10
93 (0x5D)	Code93:12E0	Code	0x813C0	0x20EA	0x1D10
94 (0x5E)	Code94:12E8	Code	0x83660	0x854	0x1D10
95 (0x5F)	Code95:12F0	Code	0x83F60	0x78A	0x1D10
96 (0x60)	Code96:12F8	Code	0x847A0	0x18DE	0x1D10
97 (0x61)	Code97:1300	Code	0x86260	0x185E	0x1D10
98 (0x62)	Code98:1308	Code	0x87C80	0x118	0x1D10
99 (0x63)	Code99:1310	Code	0x87DE0	0x2E	0x1C10
100 (0x64)	Code100:1318	Code	0x87E20	0xB12	0x1D10
101 (0x65)	Code101:1320	Code	0x88A00	0xFB0	0x1D10
102 (0x66)	Code102:1328	Code	0x89AC0	0x14FA	0x1D10
103 (0x67)	Code103:1330	Code	0x8B1C0	0xE0A	0x1D10
104 (0x68)	Code104:1338	Code	0x8C200	0xD4A	0x1D10
105 (0x69)	Code105:1340	Code	0x8D060	0xA7C	0x1D10
106 (0x6A)	Code106:1348	Code	0x8DBE0	0x1E8	0x1D10
107 (0x6B)	Code107:1350	Code	0x8DE40	0x56C	0x1D10
108 (0x6C)	Code108:1358	Code	0x8E480	0x2D8	0x1D10
109 (0x6D)	Code109:1360	Code	0x8E7E0	0x560	0x1D10
110 (0x6E)	Code110:1368	Code	0x8EE20	0x122A	0x1D10
111 (0x6F)	Code111:1370	Code	0x902A0	0x4D4	0x1D10
112 (0x70)	Code112:1378	Code	0x907E0	0x1A98	0x1D10
113 (0x71)	Code113:1380	Code	0x92420	0x498	0x1D10
114 (0x72)	Code114:1388	Code	0x92920	0xF2E	0x1D10
115 (0x73)	Code115:1390	Code	0x93900	0x197A	0x1D10
116 (0x74)	Code116:1398	Code	0x95520	0x720	0x1D10
117 (0x75)	Code117:13A0	Code	0x95DA0	0x140	0x1D10
118 (0x76)	Code118:13A8	Code	0x95F20	0x2FE	0x1D10
119 (0x77)	Code119:13B0	Code	0x962A0	0x136	0x1D10
120 (0x78)	Code120:13B8	Code	0x96420	0x1D4	0x1D10
121 (0x79)	Code121:13C0	Code	0x96620	0xDF8	0x1D10
122 (0x7A)	Code122:13C8	Code	0x974C0	0x9F2	0x1D10

NE	Ghidra	Kind	File offset	Length	Flags
123 (0x7B)	Code123:13D0	Code	0x97F80	0xB50	0x1D10
124 (0x7C)	Code124:13D8	Code	0x98B60	0x2C2A	0x1D10
125 (0x7D)	Code125:13E0	Code	0x9BA80	0x516A	0x1D10
126 (0x7E)	Code126:13E8	Code	0xA0F20	0x100	0x1D10
127 (0x7F)	Code127:13F0	Code	0xA1060	0x24EA	0x1D10
128 (0x80)	Code128:13F8	Code	0xA36C0	0x2242	0x1D10
129 (0x81)	Code129:1400	Code	0xA5AC0	0x14C6	0x1D10
130 (0x82)	Code130:1408	Code	0xA7100	0xC0C	0x1D10
131 (0x83)	Code131:1410	Code	0xA7DA0	0x1C6	0x1D10
132 (0x84)	Code132:1418	Code	0xA7FA0	0x974	0x1D10
133 (0x85)	Code133:1420	Code	0xA8980	0x140	0x1D10
134 (0x86)	Code134:1428	Code	0xA8AE0	0x8D8	0x1D10
135 (0x87)	Code135:1430	Code	0xA9400	0xE4	0x1C10
136 (0x88)	Code136:1438	Code	0xA9500	0x26A	0x1D10
137 (0x89)	Code137:1440	Code	0xA97E0	0x1DA	0x1D10
138 (0x8A)	Code138:1448	Code	0xA99E0	0x227	0x1C10
139 (0x8B)	Data139:1450	Data	0xA9C20	0x4250	0x0D51

TB80UTL.DLL.html

Entry NE 4 at `1018:0526`. Auto data NE 32 at `10F8`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x1000	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1A20	0x84C	0x0D60
3 (0x03)	Code3:1010	Code	0x2340	0x3C	0x0D60
4 (0x04)	Code4:1018	Code	0x23A0	0x3217	0x1D70
5 (0x05)	Code5:1020	Code	0x5DA0	0x844	0x1D70
6 (0x06)	Code6:1028	Code	0x6600	0x56	0x1C70
7 (0x07)	Code7:1030	Code	0x6660	0x488	0x1D70
8 (0x08)	Code8:1038	Code	0x6B60	0xFB0	0x1D70
9 (0x09)	Code9:1040	Code	0x7BA0	0x1CE	0x1D70
10 (0x0A)	Code10:1048	Code	0x7DC0	0x7DA	0x1C70
11 (0x0B)	Code11:1050	Code	0x85A0	0x18	0x1C70
12 (0x0C)	Code12:1058	Code	0x85C0	0x11E	0x1D70
13 (0x0D)	Code13:1060	Code	0x8700	0x48A	0x1D70
14 (0x0E)	Code14:1068	Code	0x8EA0	0x17C	0x1D70
15 (0x0F)	Code15:1070	Code	0x9040	0x168	0x1D70
16 (0x10)	Code16:1078	Code	0x9280	0xCBA	0x1D70
17 (0x11)	Code17:1080	Code	0x9FA0	0xA30	0x1D70
18 (0x12)	Code18:1088	Code	0xAA40	0x88E	0x1D70
19 (0x13)	Code19:1090	Code	0xB360	0x3A	0x1D70
20 (0x14)	Code20:1098	Code	0xB3C0	0x132	0x1D30
21 (0x15)	Code21:10A0	Code	0xB520	0xBA	0x1D30
22 (0x16)	Code22:10A8	Code	0xB620	0x5CE	0x1D30
23 (0x17)	Code23:10B0	Code	0xBC80	0x25C	0x1D30
24 (0x18)	Code24:10B8	Code	0xBF00	0xEC6	0x1D30
25 (0x19)	Code25:10C0	Code	0xCE60	0x3E2	0x1D30
26 (0x1A)	Code26:10C8	Code	0xD280	0x8D8	0x1D30
27 (0x1B)	Code27:10D0	Code	0xDBA0	0x140	0x1D30
28 (0x1C)	Code28:10D8	Code	0xDD00	0x1C0	0x1D30
29 (0x1D)	Code29:10E0	Code	0xDF00	0x3A	0x1D30
30 (0x1E)	Data30:10E8	Data	0xDF60	0x200	0x0C71
31 (0x1F)	Data31:10F0	Data	0xE160	0x6	0x0C71
32 (0x20)	Data32:10F8	Data	0xE180	0x1AB0	0x0D71

TB80VBX.DLL.html

Entry NE 2 at `1008:00DA`. Auto data NE 47 at `1170`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xC60	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1680	0x2123	0x1D50
3 (0x03)	Code3:1010	Code	0x39C0	0x3C	0x0D40
4 (0x04)	Code4:1018	Code	0x3A20	0x179	0x1D10
5 (0x05)	Code5:1020	Code	0x3C40	0x153A	0x1D10
6 (0x06)	Code6:1028	Code	0x5340	0x1EA	0x1D10
7 (0x07)	Code7:1030	Code	0x5560	0x664	0x1D10
8 (0x08)	Code8:1038	Code	0x5C60	0x1BF0	0x1D10
9 (0x09)	Code9:1040	Code	0x7960	0xD86	0x1D10
10 (0x0A)	Code10:1048	Code	0x87C0	0xA94	0x1D10
11 (0x0B)	Code11:1050	Code	0x9280	0x3B8	0x1D10
12 (0x0C)	Code12:1058	Code	0x9660	0x1D6A	0x1D10
13 (0x0D)	Code13:1060	Code	0xB640	0x4F6	0x1D10
14 (0x0E)	Code14:1068	Code	0xBBA0	0x27C	0x1D10
15 (0x0F)	Code15:1070	Code	0xBE40	0x41A	0x1D10
16 (0x10)	Code16:1078	Code	0xC2C0	0x5DC	0x1D10
17 (0x11)	Code17:1080	Code	0xC8E0	0x160	0x1D10
18 (0x12)	Code18:1088	Code	0xCA60	0x1242	0x1D10
19 (0x13)	Code19:1090	Code	0xDDC0	0x6E4	0x1D10
20 (0x14)	Code20:1098	Code	0xE4E0	0x132	0x1D10
21 (0x15)	Code21:10A0	Code	0xE640	0x9AA	0x1D10
22 (0x16)	Code22:10A8	Code	0xF0C0	0xC77	0x1D10
23 (0x17)	Code23:10B0	Code	0xFDC0	0x1A2	0x1D10
24 (0x18)	Code24:10B8	Code	0xFFA0	0x130	0x1D10
25 (0x19)	Code25:10C0	Code	0x10120	0x1B4	0x1D10
26 (0x1A)	Code26:10C8	Code	0x10300	0x56	0x1D10
27 (0x1B)	Code27:10D0	Code	0x10360	0x9A	0x1D10
28 (0x1C)	Code28:10D8	Code	0x10420	0x38FE	0x1D10
29 (0x1D)	Code29:10E0	Code	0x13EC0	0x12FE	0x1D10
30 (0x1E)	Code30:10E8	Code	0x15260	0x46	0x1D10
31 (0x1F)	Code31:10F0	Code	0x152E0	0x4C	0x1C10
32 (0x20)	Code32:10F8	Code	0x15340	0x68E	0x1D10
33 (0x21)	Code33:1100	Code	0x15A80	0x2C4	0x1D10
34 (0x22)	Code34:1108	Code	0x15DA0	0xB74	0x1D10
35 (0x23)	Code35:1110	Code	0x169E0	0x3AC	0x1D10
36 (0x24)	Code36:1118	Code	0x16E20	0x528	0x1D10
37 (0x25)	Code37:1120	Code	0x173E0	0xE88	0x1D10
38 (0x26)	Code38:1128	Code	0x18320	0x78C	0x1D10
39 (0x27)	Code39:1130	Code	0x18AE0	0x107A	0x1D10
40 (0x28)	Code40:1138	Code	0x19BE0	0x248	0x1D10
41 (0x29)	Code41:1140	Code	0x19E60	0x3A	0x1C10
42 (0x2A)	Code42:1148	Code	0x19EA0	0xA20	0x1D10
43 (0x2B)	Code43:1150	Code	0x1A920	0x19E	0x1D10
44 (0x2C)	Code44:1158	Code	0x1AAE0	0x1C20	0x1D10
45 (0x2D)	Code45:1160	Code	0x1C720	0x370	0x1D10
46 (0x2E)	Code46:1168	Code	0x1CAA0	0xF0	0x1D10
47 (0x2F)	Data47:1170	Data	0x1CBE0	0x2778	0x0D71

TB80W16.DLL.html

Entry NE 3 at `1010:027A`. Auto data NE 56 at `11B8`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x3CE0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x4700	0x3C	0x0D60
3 (0x03)	Code3:1010	Code	0x4760	0x1FF9	0x1D70
4 (0x04)	Code4:1018	Code	0x6800	0x6E	0x1D70
5 (0x05)	Code5:1020	Code	0x68A0	0x654	0x1D70
6 (0x06)	Code6:1028	Code	0x6F40	0x234	0x1D70
7 (0x07)	Code7:1030	Code	0x71C0	0x916	0x1D70
8 (0x08)	Code8:1038	Code	0x7B40	0xD30	0x1D30
9 (0x09)	Code9:1040	Code	0x8940	0x50A	0x1D30
10 (0x0A)	Code10:1048	Code	0x8EA0	0x49C	0x1D30
11 (0x0B)	Code11:1050	Code	0x9380	0x14FE	0x1D30
12 (0x0C)	Code12:1058	Code	0xA8E0	0x2AF0	0x1D30
13 (0x0D)	Code13:1060	Code	0xD4C0	0xFE	0x1D30
14 (0x0E)	Code14:1068	Code	0xD600	0xA42	0x1D30
15 (0x0F)	Code15:1070	Code	0xE0C0	0x1D6	0x1D30
16 (0x10)	Code16:1078	Code	0xE2C0	0x53A	0x1D30
17 (0x11)	Code17:1080	Code	0xE860	0xCA2	0x1D30
18 (0x12)	Code18:1088	Code	0xF560	0xB7E	0x1D30
19 (0x13)	Code19:1090	Code	0x10120	0x68A	0x1D30
20 (0x14)	Code20:1098	Code	0x107E0	0x4CBE	0x1D30
21 (0x15)	Code21:10A0	Code	0x15500	0x1CE8	0x1D30
22 (0x16)	Code22:10A8	Code	0x17380	0x770	0x1D30
23 (0x17)	Code23:10B0	Code	0x17B40	0x3470	0x1D30
24 (0x18)	Code24:10B8	Code	0x1B0E0	0x25C	0x1D30
25 (0x19)	Code25:10C0	Code	0x1B380	0x3CC	0x1D30
26 (0x1A)	Code26:10C8	Code	0x1B7A0	0x2D2	0x1D30
27 (0x1B)	Code27:10D0	Code	0x1BAC0	0x4E6	0x1D30
28 (0x1C)	Code28:10D8	Code	0x1C000	0x4E8	0x1D30
29 (0x1D)	Code29:10E0	Code	0x1C520	0x582	0x1D30
30 (0x1E)	Code30:10E8	Code	0x1CB20	0x8D8	0x1D30
31 (0x1F)	Code31:10F0	Code	0x1D440	0x268	0x1D30
32 (0x20)	Code32:10F8	Code	0x1D720	0x140	0x1D30
33 (0x21)	Code33:1100	Code	0x1D880	0x1C0	0x1D30
34 (0x22)	Code34:1108	Code	0x1DA80	0x1234	0x1D30
35 (0x23)	Data35:1110	Data	0x1ED60	0x6	0x0C71
36 (0x24)	Data36:1118	Data	0x1ED80	0xA	0x0C71
37 (0x25)	Data37:1120	Data	0x1EDA0	0x8	0x0C71
38 (0x26)	Data38:1128	Data	0x1EDC0	0xA	0x0C71
39 (0x27)	Data39:1130	Data	0x1EDE0	0x8	0x0C71
40 (0x28)	Data40:1138	Data	0x1EE00	0x8	0x0C71
41 (0x29)	Data41:1140	Data	0x1EE20	0xA	0x0C71
42 (0x2A)	Data42:1148	Data	0x1EE40	0x24	0x0C71
43 (0x2B)	Data43:1150	Data	0x1EE80	0x10	0x0C71
44 (0x2C)	Data44:1158	Data	0x1EEA0	0x14	0x0C71
45 (0x2D)	Data45:1160	Data	0x1EEC0	0xA	0x0C71
46 (0x2E)	Data46:1168	Data	0x1EEE0	0xA	0x0C71
47 (0x2F)	Data47:1170	Data	0x1EF00	0x8	0x0C71
48 (0x30)	Data48:1178	Data	0x1EF20	0x8	0x0C71
49 (0x31)	Data49:1180	Data	0x1EF40	0x1C	0x0C71
50 (0x32)	Data50:1188	Data	0x1EF60	0x8	0x0C71
51 (0x33)	Data51:1190	Data	0x1EF80	0xA	0x0C71
52 (0x34)	Data52:1198	Data	0x1EFA0	0x10	0x0C71

NE	Ghidra	Kind	File offset	Length	Flags
53 (0x35)	Data53:11A0	Data	0x1EFC0	0xA	0x0C71
54 (0x36)	Data54:11A8	Data	0x1EFE0	0xA	0x0C71
55 (0x37)	Data55:11B0	Data	0x1F000	0x8	0x0C71
56 (0x38)	Data56:11B8	Data	0x1F020	0x2194	0x0D71

TBCBT.DLL.html

Entry NE 2 at 1008:0054. Auto data NE 16 at 1078.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x760	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1180	0x232B	0x1D70
3 (0x03)	Code3:1010	Code	0x3880	0x80	0x1D30
4 (0x04)	Code4:1018	Code	0x3940	0xE8	0x1D30
5 (0x05)	Code5:1020	Code	0x3A60	0x296	0x1D30
6 (0x06)	Code6:1028	Code	0x3D60	0xFD8	0x1D30
7 (0x07)	Code7:1030	Code	0x4F80	0x11F0	0x1D30
8 (0x08)	Code8:1038	Code	0x6200	0x7A6	0x1D30
9 (0x09)	Code9:1040	Code	0x6A40	0x748	0x1D30
10 (0x0A)	Code10:1048	Code	0x7200	0x1996	0x1D30
11 (0x0B)	Code11:1050	Code	0x8D00	0x8A2	0x1D30
12 (0x0C)	Code12:1058	Code	0x9620	0x59E	0x1D30
13 (0x0D)	Code13:1060	Code	0x9C00	0xDD6	0x1D30
14 (0x0E)	Code14:1068	Code	0xAC00	0x4D4	0x1D30
15 (0x0F)	Code15:1070	Code	0xB160	0x1C4	0x1D30
16 (0x10)	Data16:1078	Data	0xB380	0x296E	0x0D71

TBDB3.DLL.html

Entry NE 2 at 1008:0628. Auto data NE 72 at 1238.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xA00	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1420	0x32DC	0x1D70
3 (0x03)	Code3:1010	Code	0x4C40	0x3C	0x0D60
4 (0x04)	Code4:1018	Code	0x4CA0	0x76C	0x1D30
5 (0x05)	Code5:1020	Code	0x5460	0x96C	0x1D30
6 (0x06)	Code6:1028	Code	0x5EA0	0x90A	0x1D30
7 (0x07)	Code7:1030	Code	0x6820	0x126	0x1D30
8 (0x08)	Code8:1038	Code	0x6980	0x2C	0x1C30
9 (0x09)	Code9:1040	Code	0x69C0	0x82	0x1D30
10 (0x0A)	Code10:1048	Code	0x6A60	0x6E	0x1D30
11 (0x0B)	Code11:1050	Code	0x6AE0	0x38	0x1C30
12 (0x0C)	Code12:1058	Code	0x6B20	0x142	0x1D30
13 (0x0D)	Code13:1060	Code	0x6CA0	0x280	0x1D30
14 (0x0E)	Code14:1068	Code	0x6FA0	0x7AE	0x1D30
15 (0x0F)	Code15:1070	Code	0x7800	0x336	0x1D30
16 (0x10)	Code16:1078	Code	0x7BA0	0xABA	0x1D30
17 (0x11)	Code17:1080	Code	0x86C0	0x318	0x1D30
18 (0x12)	Code18:1088	Code	0x8A40	0x152	0x1D30
19 (0x13)	Code19:1090	Code	0x8C00	0xE6	0x1D30
20 (0x14)	Code20:1098	Code	0x8D00	0x46	0x1C30
21 (0x15)	Code21:10A0	Code	0x8D60	0x716	0x1D30
22 (0x16)	Code22:10A8	Code	0x9500	0xC4A	0x1D30
23 (0x17)	Code23:10B0	Code	0xA220	0x106	0x1D30

NE	Ghidra	Kind	File offset	Length	Flags
24 (0x18)	Code24:10B8	Code	0xA360	0x43E	0x1D30
25 (0x19)	Code25:10C0	Code	0xA820	0x352	0x1D30
26 (0x1A)	Code26:10C8	Code	0xABE0	0x258	0x1D30
27 (0x1B)	Code27:10D0	Code	0xAE80	0x23A	0x1D30
28 (0x1C)	Code28:10D8	Code	0xB140	0x3FA	0x1D30
29 (0x1D)	Code29:10E0	Code	0xB5A0	0x116	0x1D30
30 (0x1E)	Code30:10E8	Code	0xB6E0	0x1098	0x1D30
31 (0x1F)	Code31:10F0	Code	0xC860	0xEA	0x1D30
32 (0x20)	Code32:10F8	Code	0xC960	0x63C	0x1D30
33 (0x21)	Code33:1100	Code	0xCFC0	0x848	0x1D30
34 (0x22)	Code34:1108	Code	0xD8E0	0xF0	0x1D30
35 (0x23)	Code35:1110	Code	0xDA20	0xF6	0x1D30
36 (0x24)	Code36:1118	Code	0xDB60	0x168	0x1D30
37 (0x25)	Code37:1120	Code	0xDD20	0xAA	0x1D30
38 (0x26)	Code38:1128	Code	0xDDE0	0x8A	0x1D30
39 (0x27)	Code39:1130	Code	0xDEA0	0xCA	0x1D30
40 (0x28)	Code40:1138	Code	0xDFA0	0x21C	0x1D30
41 (0x29)	Code41:1140	Code	0xE200	0xAA	0x1D30
42 (0x2A)	Code42:1148	Code	0xE2C0	0x20C	0x1D30
43 (0x2B)	Code43:1150	Code	0xE560	0x94	0x1D30
44 (0x2C)	Code44:1158	Code	0xE620	0x24	0x1C30
45 (0x2D)	Code45:1160	Code	0xE660	0x76E	0x1D30
46 (0x2E)	Code46:1168	Code	0xEE80	0x47C	0x1D30
47 (0x2F)	Code47:1170	Code	0xF340	0xBC	0x1D30
48 (0x30)	Code48:1178	Code	0xF420	0x660	0x1D30
49 (0x31)	Code49:1180	Code	0xFAE0	0x192	0x1D30
50 (0x32)	Code50:1188	Code	0xFCA0	0xA6C	0x1D30
51 (0x33)	Code51:1190	Code	0x107A0	0xC6	0x1D30
52 (0x34)	Code52:1198	Code	0x10880	0x19E	0x1D30
53 (0x35)	Code53:11A0	Code	0x10A40	0x16A	0x1D30
54 (0x36)	Code54:11A8	Code	0x10BE0	0x82	0x1D30
55 (0x37)	Code55:11B0	Code	0x10C80	0xCE	0x1D30
56 (0x38)	Code56:11B8	Code	0x10D80	0x264	0x1C30
57 (0x39)	Code57:11C0	Code	0x11000	0x110	0x1D30
58 (0x3A)	Code58:11C8	Code	0x11140	0x16	0x1C30
59 (0x3B)	Code59:11D0	Code	0x11160	0x50	0x1C30
60 (0x3C)	Code60:11D8	Code	0x111C0	0x16	0x1C30
61 (0x3D)	Code61:11E0	Code	0x111E0	0x99A	0x1D30
62 (0x3E)	Code62:11E8	Code	0x11BC0	0xEE6	0x1D30
63 (0x3F)	Code63:11F0	Code	0x12C80	0xB4E	0x1D30
64 (0x40)	Code64:11F8	Code	0x13820	0x94	0x1C30
65 (0x41)	Code65:1200	Code	0x138C0	0xE34	0x1D30
66 (0x42)	Code66:1208	Code	0x147A0	0x4E	0x1D30
67 (0x43)	Code67:1210	Code	0x14800	0x1C4	0x1D30
68 (0x44)	Code68:1218	Code	0x14A20	0x8A2	0x1D30
69 (0x45)	Code69:1220	Code	0x15340	0x59E	0x1D30
70 (0x46)	Data70:1228	Data	0x15920	0x107	0x0C71
71 (0x47)	Data71:1230	Data	0x15A40	0x114	0x0C71
72 (0x48)	Data72:1238	Data	0x15B60	0x17D8	0x0D71

TBDLG.DLL.html

Entry NE 17 at `1080:027A`. Auto data NE 19 at `1090`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xA60	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1480	0x3C	0x0D40
3 (0x03)	Code3:1010	Code	0x14E0	0x1A7C	0x1D50
4 (0x04)	Code4:1018	Code	0x30E0	0x5BE	0x1D50
5 (0x05)	Code5:1020	Code	0x3700	0x18C	0x1D50
6 (0x06)	Code6:1028	Code	0x38C0	0x336	0x1D50
7 (0x07)	Code7:1030	Code	0x3C40	0x78A	0x1D50
8 (0x08)	Code8:1038	Code	0x4420	0x374	0x1D50
9 (0x09)	Code9:1040	Code	0x4820	0x1DC	0x1D50
10 (0x0A)	Code10:1048	Code	0x4A40	0xC56	0x1D50
11 (0x0B)	Code11:1050	Code	0x5780	0x263A	0x1D50
12 (0x0C)	Code12:1058	Code	0x7F60	0x4A2	0x1D50
13 (0x0D)	Code13:1060	Code	0x8440	0x290	0x1D50
14 (0x0E)	Code14:1068	Code	0x87A0	0xAC	0x1D50
15 (0x0F)	Code15:1070	Code	0x8860	0x8A2	0x1D50
16 (0x10)	Code16:1078	Code	0x9180	0x59E	0x1D50
17 (0x11)	Code17:1080	Code	0x9760	0xC75	0x1D50
18 (0x12)	Code18:1088	Code	0xA4A0	0x748	0x1D50
19 (0x13)	Data19:1090	Data	0xAC60	0x9821	0x0C71

TBDOS.DLL.html

Entry NE 2 at `1008:027A`. Auto data NE 7 at `1030`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x560	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xF80	0xCE3	0x1D70
3 (0x03)	Code3:1010	Code	0x1D40	0x3C	0x0D60
4 (0x04)	Code4:1018	Code	0x1DA0	0x2C1A	0x1D30
5 (0x05)	Code5:1020	Code	0x4B00	0x59E	0x1D30
6 (0x06)	Code6:1028	Code	0x50E0	0x8A2	0x1D30
7 (0x07)	Data7:1030	Data	0x5A00	0x1792	0x0D71

TBFILE32.DLL.html

Entry NE 3 at `1010:0054`. Auto data NE 9 at `1040`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x760	0x1F82	0x1D70
2 (0x02)	Code2:1008	Code	0x27C0	0x54	0x0D60
3 (0x03)	Code3:1010	Code	0x2840	0xAB8	0x1D70
4 (0x04)	Code4:1018	Code	0x36C0	0x4A	0x1D30
5 (0x05)	Code5:1020	Code	0x3720	0xC0E	0x1D30
6 (0x06)	Code6:1028	Code	0x43C0	0x8D8	0x1D30
7 (0x07)	Code7:1030	Code	0x4CE0	0x13F	0x1D30
8 (0x08)	Code8:1038	Code	0x4E40	0x1C3	0x1D30
9 (0x09)	Data9:1040	Data	0x33A0	0x2E6	0x0C71

TBPDX.DLL.html

Entry NE 4 at `1018:0068`. Auto data NE 5 at `1020`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0xC00	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1620	0x3C	0x0D40
3 (0x03)	Code3:1010	Code	0x1680	0x6E56	0x1D50
4 (0x04)	Code4:1018	Code	0x8A20	0x1E85	0x1D50
5 (0x05)	Data5:1020	Data	0xAAA0	0x940	0x0D71

TBREG.DLL.html

Entry NE 2 at `1008:0064`. Auto data NE 11 at `1050`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x2A0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xCC0	0x1D21	0x1D70
3 (0x03)	Code3:1010	Code	0x2AA0	0x3C	0x0D60
4 (0x04)	Code4:1018	Code	0x2B00	0x224	0x1D30
5 (0x05)	Code5:1020	Code	0x2D40	0x3EBE	0x1D30
6 (0x06)	Code6:1028	Code	0x6D20	0x18A	0x1D30
7 (0x07)	Code7:1030	Code	0x6EE0	0x510	0x1D30
8 (0x08)	Code8:1038	Code	0x7420	0x8A	0x1D30
9 (0x09)	Code9:1040	Code	0x74E0	0x1DA	0x1D30
10 (0x0A)	Code10:1048	Code	0x76E0	0x29C	0x1D30
11 (0x0B)	Data11:1050	Data	0x79A0	0x5D2	0x0D71

TBWIN.DLL.html

Entry NE 2 at `1008:0946`. Auto data NE 28 at `10D8`.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x920	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1340	0x2597	0x1D70
3 (0x03)	Code3:1010	Code	0x3D00	0x3C	0x0D60
4 (0x04)	Code4:1018	Code	0x3D60	0x4D4	0x1D30
5 (0x05)	Code5:1020	Code	0x42C0	0xBEC	0x1D30
6 (0x06)	Code6:1028	Code	0x51A0	0x968	0x1D30
7 (0x07)	Code7:1030	Code	0x5B60	0x124	0x1D30
8 (0x08)	Code8:1038	Code	0x5CA0	0x87C	0x1D30
9 (0x09)	Code9:1040	Code	0x65A0	0x7C2	0x1D30
10 (0x0A)	Code10:1048	Code	0x6DE0	0x224	0x1D30
11 (0x0B)	Code11:1050	Code	0x7060	0x5CE	0x1D30
12 (0x0C)	Code12:1058	Code	0x76E0	0xDD6	0x1D30
13 (0x0D)	Code13:1060	Code	0x86E0	0x7F4	0x1D30
14 (0x0E)	Code14:1068	Code	0x8F60	0x5D4	0x1D30
15 (0x0F)	Code15:1070	Code	0x9600	0x7A	0x1D30
16 (0x10)	Code16:1078	Code	0x96A0	0x8A2	0x1D30
17 (0x11)	Code17:1080	Code	0x9FC0	0x59E	0x1D30
18 (0x12)	Code18:1088	Code	0xA5A0	0xAC	0x1D30
19 (0x13)	Code19:1090	Code	0xA660	0x268	0x1D30
20 (0x14)	Code20:1098	Code	0xA940	0x1C0	0x1D30
21 (0x15)	Code21:10A0	Code	0xAB40	0x8D8	0x1D30
22 (0x16)	Code22:10A8	Code	0xB460	0x140	0x1D30
23 (0x17)	Code23:10B0	Code	0xB5C0	0x1C4	0x1D30

NE	Ghidra	Kind	File offset	Length	Flags
24 (0x18)	Code24:10B8	Code	0xB7E0	0xFD8	0x1D30
25 (0x19)	Code25:10C0	Code	0xCA00	0x748	0x1D30
26 (0x1A)	Code26:10C8	Code	0xD1C0	0xE8	0x1D30
27 (0x1B)	Code27:10D0	Code	0xD2E0	0x7A6	0x1D30
28 (0x1C)	Data28:10D8	Data	0xDB20	0x3EAE	0x0D71

decrypt.exe.html

Entry NE 2 at 1008:15A4. Auto data NE 3 at 1010.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x2A0	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0xCC0	0x205A	0x1D50
3 (0x03)	Data3:1010	Data	0x2EA0	0xFA0	0x0C51

TB80OLE.DLL

Entry NE 2 at 1008:0054. Auto data NE 32 at 10F8.

NE	Ghidra	Kind	File offset	Length	Flags
1 (0x01)	Code1:1000	Code	0x1040	0x9F2	0x0D50
2 (0x02)	Code2:1008	Code	0x1A60	0xAD9	0x1D50
3 (0x03)	Code3:1010	Code	0x25E0	0x3C	0x0D40
4 (0x04)	Code4:1018	Code	0x2640	0x10	0x1C10
5 (0x05)	Code5:1020	Code	0x2660	0x60	0x1D10
6 (0x06)	Code6:1028	Code	0x2700	0x8D6	0x1D10
7 (0x07)	Code7:1030	Code	0x30C0	0x25A	0x1D10
8 (0x08)	Code8:1038	Code	0x3360	0x2EBC	0x1D10
9 (0x09)	Code9:1040	Code	0x63C0	0x270	0x1D10
10 (0x0A)	Code10:1048	Code	0x66A0	0x384	0x1D10
11 (0x0B)	Code11:1050	Code	0x6A40	0x490	0x1D10
12 (0x0C)	Code12:1058	Code	0x6F40	0x6CE	0x1D10
13 (0x0D)	Code13:1060	Code	0x7640	0x594	0x1D10
14 (0x0E)	Code14:1068	Code	0x7C40	0x1F2	0x1D10
15 (0x0F)	Code15:1070	Code	0x7E60	0x7B0	0x1D10
16 (0x10)	Code16:1078	Code	0x8700	0x219A	0x1D10
17 (0x11)	Code17:1080	Code	0xAAA0	0xE6A	0x1D10
18 (0x12)	Code18:1088	Code	0xBA60	0x1D96	0x1D10
19 (0x13)	Code19:1090	Code	0xD980	0x18A	0x1D10
20 (0x14)	Code20:1098	Code	0xDB80	0xDC4	0x1D10
21 (0x15)	Code21:10A0	Code	0xEA60	0x44A	0x1D10
22 (0x16)	Code22:10A8	Code	0xEF80	0x21C	0x1D10
23 (0x17)	Code23:10B0	Code	0xF200	0x77E	0x1D10
24 (0x18)	Code24:10B8	Code	0xFA20	0x3CC	0x1D10
25 (0x19)	Code25:10C0	Code	0xFEA0	0x436	0x1D10
26 (0x1A)	Code26:10C8	Code	0x103C0	0x228	0x1D10
27 (0x1B)	Code27:10D0	Code	0x10680	0xC0E	0x1D10
28 (0x1C)	Code28:10D8	Code	0x11320	0x8D8	0x1D10
29 (0x1D)	Code29:10E0	Code	0x11C40	0x140	0x1D10
30 (0x1E)	Code30:10E8	Code	0x11DA0	0x108	0x1D10
31 (0x1F)	Data31:10F0	Data	0x11EE0	0x6	0x0C71
32 (0x20)	Data32:10F8	Data	0x11F00	0xC8E	0x0D71

TB80RTM NE segment to Ghidra space

> Trail, 2026-10-16. Lookup. NE segment to the Ghidra base used for this load. Not a finding. A live selector is not this base. If this file and `TBdecompile/record/` disagree, the record wins.

NE	Ghidra
0x01	1000
0x02	1008
0x03	1010
0x04	1018
0x05	1020
0x06	1028
0x07	1030
0x08	1038
0x09	1040
0x0A	1048
0x0B	1050
0x0C	1058
0x0D	1060
0x0E	1068
0x0F	1070
0x10	1078
0x11	1080
0x12	1088
0x13	1090
0x14	1098
0x15	10A0
0x16	10A8
0x17	10B0
0x18	10B8
0x19	10C0
0x1A	10C8
0x1B	10D0
0x1C	10D8
0x1D	10E0
0x1E	10E8
0x1F	10F0
0x20	10F8
0x21	1100
0x22	1108
0x23	1110
0x24	1118
0x25	1120
0x26	1128
0x27	1130
0x28	1138
0x29	1140
0x2A	1148
0x2B	1150
0x2C	1158
0x2D	1168
0x2E	1170
0x2F	1178
0x30	1180
0x31	1188
0x32	1190

NE	Ghidra
0x33	1198
0x34	11A0
0x35	11A8
0x36	11B0
0x37	11B8
0x38	11C0
0x39	11D0
0x3A	11D8
0x3B	11E0
0x3C	11E8
0x3D	11F0
0x3E	11F8
0x3F	1200
0x40	1208
0x41	1210
0x42	1218
0x43	1220
0x44	1228
0x45	1230
0x46	1238
0x47	1240
0x48	1248
0x49	1250
0x4A	1258
0x4B	1260
0x4C	1268
0x4D	1270
0x4E	1278
0x4F	1280
0x50	1288
0x51	1290
0x52	1298
0x53	12A0
0x54	12A8
0x55	12B0
0x56	12B8
0x57	12C0
0x58	12D0
0x59	12D8
0x5A	12E0
0x5B	12E8
0x5C	12F0
0x5D	12F8
0x5E	1300
0x5F	1310
0x60	1318
0x61	1320
0x62	1328
0x63	1330
0x64	1338
0x65	1340
0x66	1348
0x67	1350
0x68	1358
0x69	1360

NE	Ghidra
0x6A	1368
0x6B	1370
0x6C	1378
0x6D	1380
0x6E	1388
0x6F	1390
0x70	1398
0x71	13A0
0x72	13A8
0x73	13B0
0x74	13B8
0x75	13C0
0x76	13C8
0x77	13D0
0x78	13D8
0x79	13E0
0x7A	13E8
0x7B	13F0
0x7C	13F8
0x7D	1400
0x7E	1408
0x7F	1410
0x80	1418
0x81	1420
0x82	1428
0x83	1430
0x84	1438
0x85	1440
0x86	1448

WDW selector:offset → Ghidra segment:offset

> Trail, 2026-10-16. Lookup. One boot's WDW selectors. They change on the next boot. If this file and `TBdecompile/record/` disagree, the record wins.

Source: WDW screenshots 2026-10-15. Live selectors are temporary (this boot). Offsets within the segment are stable. Map is tbworkingpapers/tbwp0002/TB80BAS.map. C decompile is Toolbook/_decompiles/TB80BAS.DLL.c.

Resolved (in map)

WDW (this boot)	Export	Ghidra	Notes
4877:0000	CDBDISPLAYSTATE	1008:4B4A	display state
483F:03B2	CDBCREATEPAGEREF	1010:211C	create page ref; calls FUN_1048_159a(...,2,...)
3397:147C	CDBSETVALUEEX	1048:2F0E	set property value; cmp [bp+0E],42 then sub ax,4000
3767:0000	CDBREGISTEROLE2WINDOW	1310:0292	register OLE2 window
—	CDBNEWOBJECT	12C8:0850	object create (FUN_12c8_0850)
—	CDBFILEOPEN	1028:1226	file open

In C decompile, not in map (look up by name in Ghidra)

Export	C line	Distinctive pattern from screenshot
CDBSYNCSCREENS	12922	DAT_1630_3048 = 0; DAT_1630_304a = param
CDBCLEARBOOKREFS	28451	enter 0xB8; lea ax,-2E[bp]
CDBCLIENTSTATUS	25257	push [bp+08]; push [bp+06]; push 1; call far
CDBREGISTERINSTWINDOW	25296	mov bx,[bp+0E]; mov byte [bx],0
CDBREGISTERVBXFORM	129508	enter 6; mov bx,[bp+08]; push 1
VBXNEWVIEW	129713	enter 8; push 011C; call far
VALUEARRAYINFO	87815	mov al,[bp+0D]; and al,FC; cmp al,44
VALUEARRAYGET	87982	mov al,[bp+0A]; sub ah,ah
VALUEARRAYSET	88133	les bx,[bp+14]; mov ax,es:[bx+0C]
HEAPVALUETOPOINTER	(calls)	and si,03FF; add si,si
HEAPVALUETOFRAME	(calls)	push [bp+0C]; push [bp+0A]; push [bp+08]; push [bp+06]
OBJGETFILE	(calls)	enter 14; mov bx,[bp+0A]; mov byte [bx],0
OBJGETHDRLONG	(calls)	enter 2; mov word 2862,ax

Call stack spine (book-open / page path)

These frames appear in nearly every Calls window. Selectors are this boot.

```
0x3497:03EF
0x3497:4A6E
0x3197:0379
0x3167:0CB1 → 0x3167:14BC
0x3247:02BF → 0x3247:05B7 → 0x3247:0B7A
```

Practical

- Break on the Ghidra address by offset within the segment. The live selector is whatever WDW shows for TB80BAS that boot.
- CDBSENDCREATE is listed in the Functions window but did not appear in the Calls stacks. Object creation on this path is going through CDBCREATEPAGEREF / CDBSETVALUEEX / CDBVALUENEW*.
- Clear unused breakpoints. The "Device memory is nearly full" stop is the debugger.

Next

Set the break on CDBCREATEPAGEREF (Ghidra 1010:211C) or CDBSETVALUEEX (Ghidra 1048:2F0E). Step into the call that builds the page. That is closer to the record reader than waiting for CDBSENDCREATE.

Startup sequence (this batch)

Stops in order. VALUEFREE fired for the first time before the main window. Next batch ends at VALUETOSTRING before the file dialog.

Order	WDW this boot	Export	Ghidra (if known)	Notes
1	483F:041E	CDBVIEWPAGEVALUE	—	view page value
2	343F:0876	LOCKPATTERN	—	lock pattern
3	3397:0958	CDBQUERYVALUE	—	query value
4	483F:0666	CDBCREATEBKGNDREF	—	create background ref
5	3397:113C	CDBSENDVALUEEX	—	send value
6	450F:0E3E	TBHEAPSAVECRITSECT	—	heap critical section
7	336F:0096	CDBSCRIPTSEARCH	—	script search

Order	WDW this boot	Export	Ghidra (if known)	Notes
8	3397:0304	CDBREFVALUE	—	ref value
9	450F:0E70	TBHEAPRESTORECRITSECT	—	restore crit
10	4267:04E0	CDBVALUEBOOKTOSTRING	—	book to string
11	3397:0AAA	CDBQUERYFILEPATH	1078:036C	query file path
12	44D7:0000	VALUENEWSTRING	—	new string
13	450F:0840	TBHEAPALLOC	—	heap alloc
14	450F:0EE2	HEAPPOINTERTOVALUE	—	pointer to value
15	4877:090E	CDBSETIMAGEMEMORYMODEL	—	image memory model
16	4D27:1710	RESMGREXTRACT	—	resource extract
17	3397:0612	CDBBOOKOFVALUE	—	book of value
18	3397:1110	CDBSENDVALUE	—	send value
19	4877:0506	CDBPAINTCLIENT	—	paint client (sets 3048/304A)
20	4877:0B6C	IMDRAWLEVELS	1020:16DE	draw levels
21	3AB7:0244	CDBSETFOCUS	1060:143E	set focus
22	44CF:07CA	CDBSENDUNDO	—	send undo
23	4877:09CE	IMBITBLT	—	bitblt
24	450F:0EC2	TBHEAPCLEARCRITSECT	—	clear crit
25	381F:0254	CDBRESETPLERR	—	reset error
26	4457:0B38	VALUEFREE	—	free value (before main window)

Observations

- CDBPAINTCLIENT (4877:0506) sets DAT_1630_3048 = 0 and DAT_1630_304A from the stack, same pattern as CDBSYNCSCREENS. Paint is starting before the main window.
- RESMGREXTRACT (4D27:1710) is the resource extractor. It runs before VALUEFREE.
- VALUEFREE at 4457:0B38 is the last first-time stop before the main window. The next batch ends at VALUETOSTRING before the file dialog.
- CDBSENDCREATE still has not appeared in the Calls windows. Object creation on this path is going through CDBCREATEBKGNDREF / CDBCREATEPAGEREF / CDBSETVALUEEX / CDBSENDVALUE.
- Clear unused breakpoints. The "memory and was forced to stop" message is the debugger running out of room.

Next

The next batch ends at VALUETOSTRING (before the file dialog). Capture that sequence the same way. The record reader is still upstream of CDBSENDCREATE; these stops are the property and resource path that runs while the book is opening.

Selector → segment mapping

The WDW selector (483F, 3397, 450F, etc.) is temporary. Windows assigns it when the NE segment is loaded. The Ghidra segment number is the NE segment index. They are not the same number.

How to map this boot

1. In WDW, open the Module window (Window → Module, or the module list).
2. Find TB80BAS.DLL. It shows the selector for each loaded segment.
3. Or: set a break on a known function, then look at the Calls window. The return address selector is the caller's segment.
4. Once you know one selector maps to one Ghidra segment, every other function with that same selector is in the same Ghidra segment.

Known this boot (from the screenshots)

These selectors appeared. The map offsets do not match the WDW offsets, so the map is from a different Ghidra pass. Use the WDW offset as the stable address.

WDW selector	Seen on	Notes
483F	CDBVIEWPAGEVALUE, CDBCREATEBKGNDREF	same segment
3397	CDBQUERYVALUE, CDBSENDVALUEEX, CDBREFVALUE, CDBQUERYFILEPATH, CDBBOOKOFVALUE, CDBSENDVALUE	same segment
450F	TBHEAP* family	heap segment
4877	CDBSETIMAGEMEMORYMODEL, CDBPAINTCLIENT, IMDRAWLEVELS, IMBITBLT	paint/image segment
4D27	RESMGREXTRACT	resource segment
4267	CDBVALUEBOOKTOSTRING	
44D7	VALUENEWSTRING	
44CF	CDBSENDUNDO	
3AB7	CDBSETFOCUS	
381F	CDBRESETPLERR	
4457	VALUEFREE	
336F	CDBSCRIPTSEARCH	
343F	LOCKPATTERN	

Practical

- The offset is what matches Ghidra. If the map offset does not match the WDW offset, the map is from a different analysis. Use the WDW offset and the function name.
- To get a fresh map, re-export the Ghidra symbol map from the current analysis.
- Clear unused breakpoints. The debugger is running out of memory.

Method (this boot)

For each first-time stop we have:

- WDW selector (temporary, changes next boot)
- Function name (stable)
- Offset within the segment (stable)
- First instructions (stable pattern)

Match later by name + offset + instruction bytes. Do not re-scan the whole Functions list. The Calls window already shows the return address of the caller; that is the next place to look if the function is not named.

If the session is restarted, the selector changes. The offset and the instruction bytes do not. Match on those.

What we have this boot

See the table above. The map offsets do not match the WDW offsets, so the map is from a different analysis. Use the WDW offset and the function name. Re-export the Ghidra symbol map from the current analysis to get fresh offsets.

Batch 2026-10-15 17:38–17:46

Function	WDW selector:offset	Segment group	Notes
VALUETOSTRING	4457:0DE8	4457	value convert
OBJFILEOPEN	3147:177C	3147	file open
STACKGETSIZE	4537:0738	4537	stack
VALUENEWSTACK	4537:0000	4537	stack

Function	WDW selector:offset	Segment group	Notes
STACKPUSH	4537:0778	4537	stack
CDBCHUNKLAST	0D37:0334	0D37	chunk walk
CDBCHUNKEND	0D37:0000	0D37	chunk walk
STACKPUSHONE	4537:09A2	4537	stack
STACKELEMENT	4537:0444	4537	stack
STACKSETELEMENT	4537:06B0	4537	stack
VALUENEW	4457:0C6E	4457	value
VALUEAS	4457:0038	4457	value
VALUEASEX	4457:00A0	4457	value
RESMGRENUM	4D27:2522	4D27	resource enum
CDBLOCKVALUE	3397:02CA	3397	value lock
HEAPCREATEFRAME	450F:0350	450F	heap
VALUEASC	4457:01B8	4457	value
VALUEASCEX	4457:0220	4457	value
VALUECONVERTTOC	4457:0818	4457	value
VALUECONVERTTOCEX	4457:0880	4457	value
VALUEFROMSTRING	4457:0E1A	4457	value
CDBSETPLERR	381F:0010	381F	error
LPSTRTODOUBLE	4477:05B4	4477	string→double
HEAPGETHANDLER	450F:0FEE	450F	heap
VALUECHECK	44FF:0000	44FF	value
CDBCHUNKFIND	0D37:0146	0D37	chunk walk
HEAPRELEASEFRAME	450F:05D6	450F	heap
VALUENEWLAYEREX	4267:127A	4267	value
VALUEATOL	443F:0000	443F	value
ANSILSTRNCMPI	2A8F:00D3	2A8F	string compare
?SENDPROC@TBXBOBJ@@...	29AF:0000	29AF	ActiveX/OLE
VALUENEWTIME	449F:01C0	449F	value

Selector groups this batch

- 4457: VALUETOSTRING, VALUENEW, VALUEAS, VALUEASEX, VALUEASC, VALUEASCEX, VALUECONVERTTOC, VALUECONVERTTOCEX, VALUEFROMSTRING (value conversion cluster)
- 4537: STACKGETSIZE, VALUENEWSTACK, STACKPUSH, STACKPUSHONE, STACKELEMENT, STACKSETELEMENT (stack cluster)
- 0D37: CDBCHUNKLAST, CDBCHUNKEND, CDBCHUNKFIND (chunk walkers)
- 450F: HEAPCREATEFRAME, HEAPGETHANDLER, HEAPRELEASEFRAME (heap)
- 4D27: RESMGRENUM (resource)
- 3397: CDBLOCKVALUE (same selector as earlier CDBQUERYVALUE family)
- 381F: CDBSETPLERR
- 4477: LPSTRTODOUBLE
- 44FF: VALUECHECK
- 4267: VALUENEWLAYEREX
- 443F: VALUEATOL
- 2A8F: ANSILSTRNCMPI
- 29AF: ?SENDPROC@TBXBOBJ@@ (ActiveX)
- 449F: VALUENEWTIME

Useful vs noise

Most of this batch is value conversion and stack machinery. Not on the book-open or paint path.

Worth keeping:

- CDBCHUNKFIND / CDBCHUNKEND / CDBCHUNKLAST (0D37) — chunk walkers. These walk the property/record chunks. Related to the 14 00 00 reader question.
- CDBLOCKVALUE (3397) — same selector as the earlier CDBQUERYVALUE / CDBSENDVALUEEX family. Value locking.
- OBJFILEOPEN (3147) — file open path.
- ?SENDPROC@TBXBOBJ@@ (29AF) — ActiveX/OLE send. Side quest.

Skip for now:

- All STACK* (4537) — stack machine.
- All VALUE* convert/as/check (4457, 44FF, 443F, 4267, 449F) — value type conversion.
- HEAP* (450F) — heap frames.
- ANSILSTRNCMPI (2A8F) — string compare.
- RESMGRENUM (4D27) — resource enum.
- LPSTRTODOUBLE (4477) — string to double.
- CDBSETPLERR (381F) — error set.

The chunk walkers (CDBCHUNK*) are the closest thing to a record reader in this batch. They are not the 14 00 00 reader, but they walk the property chunks that sit inside the page records.

Callers and first instructions (this batch, interesting ones)

CDBCHUNKFIND (0D37:0146)

- First: mov ax,347F / enter 0008,00 / push ds / mov ds,ax / mov ax,word ptr 12[bp] / sub ax,0001 / sbb cx,cx / not cx
- Callers visible: 4857:0C68 (????), 4857:090B (call near), 3187:0210 (call far), 322F:0577 (cmp ax,268), 322F:0E1E (jmp word)

CDBCHUNKEND (0D37:0000)

- First: mov ax,347F / enter 0004,00 / push si / push ds / mov ds,ax / mov word ptr -02[bp],0 / cmp word ptr 0C[bp],00 / jne 003E
- Callers visible: 3187:0210 (call far), 0D37:2B77 (????), 0D37:12BC (????), 0D37:247E (????), 0D37:0AE5 (????), 0D37:07FB (????), CDBCHUNKFIND (jb 0x0297)

CDBCHUNKLAST (0D37:0334)

- First: mov ax,347F / enter 0010,00 / push si / push ds / mov ds,ax / xor ax,ax / mov word ptr -04[bp],ax / mov word ptr -06[bp],ax
- Callers visible: 4857:090B (call near), 4857:0B23 (call dword), 3187:0210 (call far), CDBCHUNKEND (leave)

CDBLOCKVALUE (3397:02CA)

- First: mov ax,347F / enter 0002,00 / push ds / mov ds,ax / mov al,byte ptr 0B[bp] / shr al,02 / sub ah,ah / sub ax,001A
- Callers visible: 4857:4FF6 (????), 4857:0C68 (call dword), 4857:42B8 (????), 4857:090B (call near), 3187:0210 (call far), 31B7:0073 (call near), 31B7:00F9 (call near)

OBJFILEOPEN (3147:177C)

- First: mov ax,347F / enter 025A,00 / push di / push si / push ds / mov ds,ax / mov word ptr -04[bp],...

- Callers visible: 322F:3252 (jmp 0x31FC), 3D6F:0416 (call near), 3D6F:0F79 (call near), 3D6F:0E8E (call far), 3247:0219 (call near), 3247:25A7 (call far), OBJGETFILE, OBJFREESEG

?SENDPROC@TBXBOBJ@@ (29AF:0000)

- First: mov ax,347F / push bp / mov bp,sp / push ds / mov ds,ax / mov ax,word ptr 0E[bp] / cmp ax,3014 / jne 0014
- Callers visible: ?GETPROC@TBXBOBJ@@, 29AF:033CE, ?SETPROC@TBXBOBJ@@, SENDPROC@TBXBOBJ@@ (self), 29AF:0206D, ?SETPROC@TBXBOBJ@@

Matching note

For next boot, match these by function name + first instruction bytes (mov ax,347F is the common entry, then the enter size differs). The 347F is this boot's DGROUP. It will change. The enter size and the next instructions are stable.

Callers and first instructions (batch 2026-10-15 18:16, interesting ones)

CDBCLASSTABLEGETBEHAVIOR (44F7:121A)

- First: mov ax,347F / enter 0008,00 / push ds / mov ds,ax / cmp word ptr 08[bp],00 / je 129A
- Callers visible: 3197:08A3, 3197:0C3F, 3197:105C, 3CC7:0379, 44F7:026A, 44F7:3FEF
- Note: Unbreaking this let the splash page run. Hit during book open / class-table setup.

CDBVALUENEWBOOKREF (4267:0000)

- First: mov ax,347F / enter 0004,00 / push ds / mov ds,ax / mov ax,word ptr 08[bp] / or ax,word ptr 06[bp]
- Callers visible: 3467:19AD, 3467:1333, 3467:0B0F, 322F:0E1E, 322F:180D

VALUENEWGRAPHIC (4267:1494)

- First: mov ax,347F / enter 0008,00 / push ds / mov ds,ax / mov al,byte ptr 11[bp] / and al,FC
- Callers visible: 3467:1934, 3467:1176, 3467:0B0F, 322F:0E1E, 0DC7:1C82, 516F:044B

VALUENEWBOOK (4267:0D90)

- First: mov ax,347F / enter 0006,00 / push ds / mov ds,ax / mov word ptr -06[bp],0
- Callers visible: 3D6F:0EDF, 3467:0369, 3467:19AD, 3467:1333, 3467:0B0F, 322F:0E1E

GETDIBCOLORCOUNT (35D7:053A)

- First: mov ax,347F / enter 0006,00 / push ds / mov ds,ax / les bx,dword ptr 06[bp]
- Callers visible: PATTERNDITHER_8BIT, CREATEDITHEREDHEADER_8BIT, GETDIBSIZE, CREATEDIBMASK (self-cluster)

HUGEGETDATA (33BF:0D6C)

- First: mov ax,347F / enter 0018,00 / push si / push ds / mov ds,ax / mov word ptr -04[bp],0
- Callers visible: 33BF:1140, 33BF:02AB, 33BF:0118, 33BF:2CF2, 33BF:0F64, 33BF:0A64, 33BF:0AAD

OBJREALLOCNEAR (3147:08D6)

- First: mov ax,347F / enter 0018,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0E[bp]
- Callers visible: 322F:170F, 409F:0890, 409F:2241, OBJNEWNEAREX, OBJFILECOPY, OBJGETHDRLONG, OBJFREENEAR

Matching note

Same as prior batches: mov ax,347F is this boot's DGROUP and will change. Match on enter size and the next 2-3 instructions. CDBCLASSTABLEGETBEHAVIOR is the splash-gate observation for this batch.

Callers and first instructions (batch 2026-10-15 18:22, interesting ones)

CDBCREATEINSTANCE (480F:0000)

- First: mov ax,347F / enter 0002,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0E[bp]
- Callers visible: 4EDF:0067, 0D67:87A2, 322F:170F, 40A7:167A, 40A7:1628, 3DE7:029B, 315F:0231

CDBVALUEBOOKFROMSTRING (4267:03F4)

- First: mov ax,347F / enter 0018,00 / push si / push ds / mov ds,ax / push word ptr 08[bp]
- Callers visible: 4267:1E8F, CDBVALUENEWGRAPHICREF, CDBVALUELAYERFROMSTRBOOK, VALUENEWBOOK, VALUENEWGRAPHIC

CDBVALUEGRAPHICFROMSTRING (4267:0AA0)

- First: mov ax,347F / enter 0074,00 / push si / push ds / mov ds,ax / mov word ptr -44[bp],0
- Callers visible: CDBVALUELAYERFROMSTRBOOK, VALUENEWBOOK, CDBVALUENEWGRAPHICREF, CDBVALUELAYERFROMSTRBOOK

CDBVALUELAYERFROMSTRBOOK (4267:05F8)

- First: mov ax,347F / enter 004C,00 / push si / push ds / mov ds,ax / mov word ptr -2C[bp],0
- Callers visible: CDBVALUENEWLAYERREF, CDBVALUENEWGRAPHICREF, CDBVALUELAYERFROMSTRBOOK, VALUENEWBOOK, VALUENEWLAYER
- Note: User goofed on CDBVALUELAYERFROMSTRING; two screenshots of this function.

CDBVALUERESOURCEFROMSTRING (4267:08B2)

- First: mov ax,347F / enter 004A,00 / push ds / mov ds,ax / mov word ptr -2C[bp],0
- Callers visible: CDBVALUEBOOKTOSTRING, CDBVALUENEWBOOKREF, CDBVALUENEWLAYERREF, CDBVALUENEWGRAPHICREF, CDBVALUELAYERFROMSTRBOOK, VALUENEWBOOK, VALUENEWRESOURCE

Matching note

Same as prior batches: mov ax,347F is this boot's DGROUP and will change. Match on enter size and the next 2-3 instructions.

Callers and first instructions (batch 2026-10-15 18:32, app now paging)

GETDIBSIZE (35D7:05DA)

- First: mov ax,347F / enter 0008,00 / push ds / mov ds,ax / les bx,dword ptr 06[bp]
- Callers visible: 35D7:113E (PATTERNDITHER_8BIT), CREATEDITHEREDHEADER_8BIT, 35D7:5EC2, 35D7:4FE5, 35D7:5134, 35D7:197E

VALUEARRAYCHKDIMS (4357:03D8)

- First: mov ax,347F / enter 0022,00 / push ds / mov ds,ax / mov al,byte ptr 0B[bp]
- Callers visible: 3467:1333, 3467:0B0F, 322F:0E1E, 4DEF:013B, 4D87:00ED, 516F:52BD, 322F:1689

VALUENEWRESOURCE (4267:0EFC)

- First: mov ax,347F / enter 000E,00 / push di / push si / push ds / mov ds,ax / cmp word ptr 0A[bp],00
- Callers visible: 4D87:0102, 0ED7:1E5B, 4D87:01C72, 0ED7:1E5B, 4C0F:1C72, 518F:4776, 322F:180D

RESMGRGETREFCNT (4D27:2838)

- First: mov ax,347F / enter 0008,00 / push ds / mov ds,ax / mov bx,word ptr 0C[bp]
- Callers visible: 4D87:0102, 0ED7:1E5B, 4C0F:1C72, 518F:4776, 322F:180D, RESMGRREPLACE

Status note

After this batch the application is up and paging through the book without re-breaking. Splash gate and early constructors have been passed.

Callers and first instructions (batch 2026-10-15 18:39, menu break)

COLORHLSTORGB (44B7:0B84)

- First: mov ax,347F / enter 0034,00 / push di / push si / push ds / mov ds,ax / mov ax,word ptr 08[bp] / or ax,word ptr 06[bp] / jne 0BBE / push 0000
- Callers visible: 516F:2ED4, 44B7:1FF7, VALUENEWCOLORTABLE, VALUENEWCOLOR
- Behavior: fired when menu was drawn. Unbreaking returned to the menu. HLS-to-RGB color helper, supporting graphics. Not a gate.

Callers and first instructions (batch 2026-10-15 18:42, Lesson Exit)

Successful exit. Cleanup paths only.

CDBUNREGISTERVBXFORM (4437:0068)

- First: mov ax,347F / enter 0002,00 / push ds / mov ds,ax / mov bx,word ptr 08[bp] / mov byte ptr ss:[bx],0 / push bx / push word ptr 06[bp] / push 0001 / call far ptr 4857:02FC
- Callers visible: 3197:03F8, 31EF:1DFD, DISPATCHINPUT, CALLNEXTHOOKEX

VBXDESTROYVIEW (4437:02F0)

- First: mov ax,347F / enter 000E,00 / push ds / mov ds,ax / les bx,dword ptr 06[bp] / mov word ptr -04[bp],bx / mov word ptr -02[bp],es / mov word ptr es:02[bx] / mov ax,word ptr es:06[bx] / or ax,word ptr es:04[bx]
- Callers visible: 31EF:1DFD, DISPATCHINPUT, CALLNEXTHOOKEX

CDBDESTROYINSTANCE (480F:040A)

- First: mov ax,347F / push bp / mov bp,sp / push ds / mov ds,ax / push word ptr 08[bp] / push word ptr 06[bp] / push 0003 / call far ptr 4857:02FC
- Callers visible: 31EF:1DFD, DISPATCHINPUT, CALLNEXTHOOKEX, 315F:2094, 315F:1194

Status note

These fire during Lesson Exit. Supporting cleanup (VBX and instance teardown). Not open/render keepers.

Callers and first instructions (batch 2026-10-15 18:44-18:51, restart)

Segment changed on restart. Object/heap/value init paths. Supporting only.

HEAPREGISTER (343F:0000)

- First: mov ax,2977 / enter 0014,00 / push si / push ds / mov ds,ax / cmp word ptr 11AE,0000 / jne 004C / push 2042 / push 0000 / push 002E

VALUEINIT (44CF:0000)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / les bx,dword ptr 06[bp] / sub ax,ax / mov word ptr es:B0[bx] / mov word ptr es:AE[bx] / mov word ptr es:08[bx]

VALUENEWARRAY (395F:0000)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / push word ptr 16[bp] / push word ptr 14[bp] / push word ptr 12[bp] / push word ptr 10[bp] / push word ptr 0E[bp]

TBHEAPALLOC (343F:0840)

- First: mov ax,2977 / enter 0014,00 / push di / push si / push ds / mov ds,ax / mov ax,word ptr 06[bp] / add ax,0004 / cmp ax,word ptr 06[bp] / jae 085A

HEAPPOINTERTOVALUE (343F:0EE2)

- First: mov ax,2977 / enter 0004,00 / push ds / mov ds,ax / mov cx,word ptr 06[bp] / mov word ptr -04[bp],cx / mov ax,word ptr 08[bp] / mov es,ax / mov ax,word ptr -02[bp] / mov cx,ax

CDBINIT (37A7:0000)

- First: mov ax,2977 / enter 00C8,00 / push di / push si / push ds / mov ds,ax / mov bx,word ptr 06[bp] / mov byte ptr ss:[bx],0 / push bx / call far ptr 2A9F:0000

OBJINIT (4357:0000)

- First: mov ax,2977 / enter 0016,00 / push ds / mov ds,ax / cmp word ptr 034C,0000 / jne 0054 / call far ptr 34F7:0E58 / mov word ptr -08[bp],dx / or dx,ax / je 002D

VALUENEWSTRING (3397:0000)

- First: mov ax,2977 / enter 0004,00 / push di / push ds / mov ds,ax / cmp word ptr 06[bp],00 / jne 0028 / mov ax,word ptr 0A[bp] / or ax,word ptr 08[bp]

CDBCREATEINSTANCE (4387:0000)

- First: mov ax,2977 / enter 0002,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0E[bp] / mov byte ptr ss:[bx],0 / cmp word ptr 018C,0000 / jne 0028 / mov bx,word ptr 0E[bp]

CDBREFVALUE (3B6F:0304)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / cmp word ptr 06[bp],00 / jne 0326 / cmp word ptr 08[bp],04 / jne 0326 / mov ax,word ptr 06[bp]

CDBFILENEW (3B5F:0028)

- First: mov ax,2977 / enter 0022,00 / push ds / mov ds,ax / push word ptr 0E[bp] / push word ptr 0C[bp] / push 0001 / call far ptr 3877:02FC / or ax,ax / jne 0046

OBJNEWSEG (4867:06E6)

- First: mov ax,2977 / enter 000E,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0C[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0A[bp] / mov word ptr 2862,ax / mov word ptr -02[bp],ax

OBJLOCK (4867:0308)

- First: mov ax,2977 / enter 0006,00 / push ds / mov ds,ax / push word ptr 0C[bp] / push word ptr 0A[bp] / push word ptr 08[bp] / push word ptr 06[bp] / call far ptr 3AB7:0000 / or ax,ax

OBJNEWNEAR (4867:0552)

- First: mov ax,2977 / enter 0008,00 / push di / push si / push ds / mov ds,ax / mov bx,word ptr 0E[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0C[bp] / mov dx,ax

LOOKUPSTDPROPBYNAME (337F:010A)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / push word ptr 08[bp] / push word ptr 06[bp] / push word ptr 19FC / push word ptr 19FA / push cs

OBJTOUCH (4867:0AD4)

- First: mov ax,2977 / enter 0024,00 / push si / push ds / mov ds,ax / mov ax,word ptr 0A[bp] / or ax,word ptr 08[bp] / jne 0AF0 / xor ax,ax / cwd

OBJOPTOOH (4867:01DA)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / push word ptr 0A[bp] / push word ptr 08[bp] / push word ptr 06[bp] / push word ptr 06[bp] / push cs

OBJOOTOOH (4867:0176)

- First: mov ax,2977 / enter 0014,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0C[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0A[bp] / sub bx,bx / mov es,ax

OBJNEWNEAREX (4867:0606)

- First: mov ax,2977 / enter 0022,00 / push si / push ds / mov ds,ax / mov bx,word ptr 10[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0E[bp] / sub bx,bx / mov es,ax

OBJFREENEAREX (4867:0E36)

- First: mov ax,2977 / enter 0016,00 / push si / push ds / mov ds,ax / mov word ptr -04[bp],0 / mov bx,word ptr 0C[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0A[bp] / sub bx,bx

SENDPROC@TBXUSER (33DF:0000)

- First: mov ax,2977 / enter 0004,00 / push ds / mov ds,ax / sub ax,ax / mov word ptr -02[bp],ax / mov word ptr -04[bp],ax / mov ax,word ptr 0E[bp] / sub ax,000A / je 004E

OBJPRINCIPALOO (4867:0A40)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / push word ptr 0A[bp] / push word ptr 08[bp] / push word ptr 06[bp] / push cs / call near ptr OBJPRINCIPALOP

OBJPRINCIPALOP (4867:0A5C)

- First: mov ax,2977 / enter 0012,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 08[bp] / sub bx,bx / mov es,ax

SENDPROC@TBXBASE (59DF:0000)

- First: mov ax,2977 / enter 002E,00 / push ds / mov ds,ax / mov ax,word ptr 0E[bp] / cmp ax,3012 / je 0064 / ja 0026 / sub ax,000A / je 0038

HCACHEPURGE (3787:0690)

- First: mov ax,2977 / enter 0038,00 / push ds / mov ds,ax / cmp word ptr 17E2,0000 / jne 06A4 / jmp near ptr 07EA / mov ax,word ptr 17DC / or ax,word ptr 17DA / jne 06B0

SETPROC@TBXBASE (59DF:00F4)

- First: mov ax,2977 / enter 0004,00 / push ds / mov ds,ax / mov bx,word ptr 14[bp] / xor ax,ax / mov byte ptr ss:[bx],7 / cwd / pop ds / leave

OBJUNLOCK (4867:0432)

- First: mov ax,2977 / enter 004A,00 / push ds / mov ds,ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 08[bp] / mov word ptr -4A[bp],ax / sub bx,bx / mov es,ax

OBJARENACHECK (4397:0AE2)

- First: mov ax,2977 / enter 002A,00 / push ds / mov ds,ax / mov word ptr -02[bp],0 / mov ax,word ptr 14[bp] / sub cx,cx / mov word ptr -06[bp],cx / mov word ptr -04[bp],ax / mov bx,cx

OBJGETFILE (4867:0094)

- First: mov ax,2977 / enter 0014,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 08[bp] / sub bx,bx / mov es,ax

SENDPROC@TBXLAYER (344F:0000)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / mov ax,word ptr 0E[bp] / sub ax,302E / je 0048 / sub ax,006A / je 005E

SETPROC@TBXLAYER (344F:0156)

- First: mov ax,2977 / push bp / mov bp,sp / push ds / mov ds,ax / mov ax,word ptr 0E[bp] / sub ax,401B / je 0194 / sub ax,000F / je 01AC

OBJGETHDRLONG (4867:0242)

- First: mov ax,2977 / enter 0002,00 / push si / push ds / mov ds,ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 08[bp] / mov word ptr 2862,ax / mov word ptr -02[bp],ax

RESMGRSENDMESSAGE (3817:1C82)

- First: mov ax,2977 / enter 0012,00 / push si / push ds / mov ds,ax / sub ax,ax / mov word ptr -10[bp],ax / mov word ptr -12[bp],ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],ax

CDBCLEARBOOKREFS (3B5F:10A6)

- First: mov ax,2977 / enter 00B8,00 / push ds / mov ds,ax / mov bx,word ptr 0A[bp] / mov byte ptr ss:[bx],0 / lea ax,-2E[bp] / push ss / push ax / call far ptr 3877:0000

CDBCREATEBOOKREF (3B5F:0F5C)

- First: mov ax,2977 / enter 000E,00 / push ds / mov ds,ax / mov bx,word ptr 0C[bp] / mov byte ptr ss:[bx],0 / push bx / push word ptr 0A[bp] / push 0001 / call far ptr 3877:02FC

CDBGETVALUEEX (3B6F:0D80)

- First: mov ax,2977 / enter 0038,00 / push ds / mov ds,ax / mov word ptr -12[bp],0 / sub ax,ax / mov word ptr -14[bp],ax / mov word ptr -16[bp],ax / mov word ptr -18[bp],ax / mov word ptr -10[bp],0

OBJLOCKNEAR (4867:03D6)

- First: mov ax,2977 / enter 000E,00 / push ds / mov ds,ax / mov bx,word ptr 0C[bp] / mov byte ptr ss:[bx],0 / mov ax,word ptr 0A[bp] / sub bx,bx / mov es,ax / mov ax,word ptr es:0000

CDBVALUENEWGRAPHICREF (33EF:02D4)

- First: mov ax,2977 / enter 0024,00 / push ds / mov ds,ax / lea ax,-02[bp] / push ax / push word ptr 10[bp] / push word ptr 0E[bp] / push word ptr 0C[bp] / push word ptr 0A[bp]

Status note

Object management, heap, and value creation. Supporting infrastructure for book open. Not open/render keepers. Segment changed on restart.

Bochs selectors, boot of 2026-10-20

> Lookup. One boot. Dead after the next reboot. If this file and `TBdecompile/record/` disagree, the record wins.

The debugger was Bochs 3.1, outside a Win98 SE guest. Offsets match the Ghidra listing. Selectors do not. Appendix L says how to rebuild this table.

An unloaded segment shows a shared stub at base `0x01000000`. Do not break there. Match a loaded segment by size: the LDT limit is about the segment length minus one. Code 23 is length `0x5B6E`, limit `0x5B7F`.

On this boot, `selector = (ldt_index << 3) | 7`. NE segments 2 through 49 used index `1360 − N`. Segment 50 and up, including data segment 199, used `1877 − N`. That pattern is a hint from one boot. Check it by size.

`info ldt` takes a decimal index, not a selector. Index of selector `S` is `S >> 3`.

Ghidra	Selector	Role
`1008`	`2A77`	C runtime. DOS read `1008:0466`.
`1048`	`2A37`	Book open. `2174`.
`10B0`	`29CF`	Class dispatcher. Message `0xE` fixup.
`10F8`	`2987`	Second class dispatcher. No `0xE` case.
`1160`	`291F`	File read. `02CC`.
`1168`	`2917`	`HUGEGETDATA`. Not on the object read.
`1170`	`290F`	Property record table.
`1178`	`2907`	Object loader. `08B2`.
`1180`	`28FF`	Message dispatch. `02CE`, `03C8`, `031E`.
`1188`	`391F`	Entry read. `04DA`, `08D6`, `02BC`.
`11A8`	`38FF`	Block allocator.
`11B0` / `11B8`	`38F7` / `38EF`	Demand-load front end.
`1340`	`3767`	Reader object. First word `0x412`.
`1630`	`3477`	`DAT_1630_*`.

Heap selectors move. Re-read them from the data segment. This boot: file table `3E47` (LDT index 1992), handle table `3E67` (index 1996), reader objects `3BFF`. The traced block was `48EF` (index 2333).

Appendix E — Numeric and System Tables

Decimal/hex, ASCII, and the Win16 ordinals used when a trace names KERNEL, USER, or GDI by number.

Date: 16 October 2026

Decimal/hex is 0 through 255. ASCII is 0 through 127. Neither table is cut off early. The Win16 rows are the 15 October appendix, in the same order, one ordinal per row. Numbers restart at 1 for each library.

Decimal to hexadecimal

Dec	Hex	Dec	Hex	Dec	Hex	Dec	Hex
0	00	1	01	2	02	3	03
4	04	5	05	6	06	7	07
8	08	9	09	10	0A	11	0B
12	0C	13	0D	14	0E	15	0F
16	10	17	11	18	12	19	13
20	14	21	15	22	16	23	17
24	18	25	19	26	1A	27	1B
28	1C	29	1D	30	1E	31	1F
32	20	33	21	34	22	35	23
36	24	37	25	38	26	39	27
40	28	41	29	42	2A	43	2B
44	2C	45	2D	46	2E	47	2F
48	30	49	31	50	32	51	33
52	34	53	35	54	36	55	37
56	38	57	39	58	3A	59	3B
60	3C	61	3D	62	3E	63	3F
64	40	65	41	66	42	67	43
68	44	69	45	70	46	71	47
72	48	73	49	74	4A	75	4B
76	4C	77	4D	78	4E	79	4F
80	50	81	51	82	52	83	53
84	54	85	55	86	56	87	57
88	58	89	59	90	5A	91	5B
92	5C	93	5D	94	5E	95	5F
96	60	97	61	98	62	99	63
100	64	101	65	102	66	103	67
104	68	105	69	106	6A	107	6B
108	6C	109	6D	110	6E	111	6F
112	70	113	71	114	72	115	73
116	74	117	75	118	76	119	77
120	78	121	79	122	7A	123	7B
124	7C	125	7D	126	7E	127	7F
128	80	129	81	130	82	131	83
132	84	133	85	134	86	135	87
136	88	137	89	138	8A	139	8B
140	8C	141	8D	142	8E	143	8F
144	90	145	91	146	92	147	93
148	94	149	95	150	96	151	97
152	98	153	99	154	9A	155	9B
156	9C	157	9D	158	9E	159	9F
160	A0	161	A1	162	A2	163	A3
164	A4	165	A5	166	A6	167	A7
168	A8	169	A9	170	AA	171	AB
172	AC	173	AD	174	AE	175	AF

Dec	Hex	Dec	Hex	Dec	Hex	Dec	Hex
176	B0	177	B1	178	B2	179	B3
180	B4	181	B5	182	B6	183	B7
184	B8	185	B9	186	BA	187	BB
188	BC	189	BD	190	BE	191	BF
192	C0	193	C1	194	C2	195	C3
196	C4	197	C5	198	C6	199	C7
200	C8	201	C9	202	CA	203	CB
204	CC	205	CD	206	CE	207	CF
208	D0	209	D1	210	D2	211	D3
212	D4	213	D5	214	D6	215	D7
216	D8	217	D9	218	DA	219	DB
220	DC	221	DD	222	DE	223	DF
224	E0	225	E1	226	E2	227	E3
228	E4	229	E5	230	E6	231	E7
232	E8	233	E9	234	EA	235	EB
236	EC	237	ED	238	EE	239	EF
240	F0	241	F1	242	F2	243	F3
244	F4	245	F5	246	F6	247	F7
248	F8	249	F9	250	FA	251	FB
252	FC	253	FD	254	FE	255	FF

ASCII (0–127)

Controls are shown by name. The landscape one-page card remains `TBdecompile/ASCII_lookup.pdf`.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	1	01	SOH	2	02	STX	3	03	ETX
4	04	EOT	5	05	ENQ	6	06	ACK	7	07	BEL
8	08	BS	9	09	HT	10	0A	LF	11	0B	VT
12	0C	FF	13	0D	CR	14	0E	SO	15	0F	SI
16	10	DLE	17	11	DC1	18	12	DC2	19	13	DC3
20	14	DC4	21	15	NAK	22	16	SYN	23	17	ETB
24	18	CAN	25	19	EM	26	1A	SUB	27	1B	ESC
28	1C	FS	29	1D	GS	30	1E	RS	31	1F	US
32	20	SP	33	21	!	34	22	"	35	23	#
36	24	$	37	25	%	38	26	&	39	27	'
40	28	(	41	29	)	42	2A	*	43	2B	+
44	2C	,	45	2D	-	46	2E	.	47	2F	/
48	30	0	49	31	1	50	32	2	51	33	3
52	34	4	53	35	5	54	36	6	55	37	7
56	38	8	57	39	9	58	3A	:	59	3B	;
60	3C	<	61	3D	=	62	3E	>	63	3F	?
64	40	@	65	41	A	66	42	B	67	43	C
68	44	D	69	45	E	70	46	F	71	47	G
72	48	H	73	49	I	74	4A	J	75	4B	K
76	4C	L	77	4D	M	78	4E	N	79	4F	O
80	50	P	81	51	Q	82	52	R	83	53	S
84	54	T	85	55	U	86	56	V	87	57	W
88	58	X	89	59	Y	90	5A	Z	91	5B	[
92	5C	\	93	5D	]	94	5E	^	95	5F	_
96	60	`	97	61	a	98	62	b	99	63	c
100	64	d	101	65	e	102	66	f	103	67	g
104	68	h	105	69	i	106	6A	j	107	6B	k
108	6C	l	109	6D	m	110	6E	n	111	6F	o
112	70	p	113	71	q	114	72	r	115	73	s

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
116	74	t	117	75	u	118	76	v	119	77	w
120	78	x	121	79	y	122	7A	z	123	7B	{
124	7C	vertical bar	125	7D	}	126	7E	~	127	7F	DEL

Win16 system ordinals

Library	Ordinal	Name
KERNEL	1	InitTask
KERNEL	2	ExitWindows
KERNEL	3	WaitEvent
KERNEL	4	PostEvent
KERNEL	5	SetPriority
KERNEL	6	LockCurrentTask
KERNEL	7	UnlockCurrentTask
KERNEL	8	GetCurrentTask
KERNEL	9	GetCurrentPDB
KERNEL	10	GetDOSEnvironment
KERNEL	11	GetInstanceData
KERNEL	12	SetInstanceData
KERNEL	13	GetExePtr
KERNEL	14	GetProcAddress
KERNEL	15	MakeProcInstance
KERNEL	16	FreeProcInstance
KERNEL	17	GetModuleHandle
KERNEL	18	GetModuleUsage
KERNEL	19	GetModuleFileName
KERNEL	20	GetWindowsDirectory
KERNEL	21	GetSystemDirectory
KERNEL	22	GetFreeSpace
KERNEL	23	GetTempDrive
KERNEL	24	GetTempFileName
KERNEL	25	Catch
KERNEL	26	Throw
KERNEL	27	LocalAlloc
KERNEL	28	LocalReAlloc
KERNEL	29	LocalFree
KERNEL	30	LocalLock
KERNEL	31	LocalUnlock
KERNEL	32	LocalSize
KERNEL	33	LocalFlags
KERNEL	34	LocalCompact
KERNEL	35	LocalNotify
KERNEL	36	GlobalAlloc
KERNEL	37	GlobalReAlloc
KERNEL	38	GlobalFree
KERNEL	39	GlobalLock
KERNEL	40	GlobalUnlock
USER	1	MessageBox
USER	2	GetFocus
USER	3	SetFocus
USER	4	GetActiveWindow
USER	5	SetActiveWindow
USER	6	GetCapture
USER	7	SetCapture
USER	8	ReleaseCapture

Library	Ordinal	Name
USER	9	GetWindowTask
USER	10	IsWindow
USER	11	IsChild
USER	12	IsWindowVisible
USER	13	IsWindowEnabled
USER	14	IsIconic
USER	15	IsZoomed
USER	16	ClientToScreen
USER	17	ScreenToClient
USER	18	WindowFromPoint
USER	19	ChildWindowFromPoint
USER	20	GetWindowWord
USER	21	SetWindowWord
USER	22	GetWindowLong
USER	23	SetWindowLong
USER	24	GetClassWord
USER	25	SetClassWord
USER	26	GetClassLong
USER	27	SetClassLong
USER	28	GetDesktopWindow
USER	29	GetParent
USER	30	SetParent
USER	31	GetWindow
USER	32	GetNextWindow
USER	33	GetTopWindow
USER	34	GetWindowText
USER	35	SetWindowText
USER	36	GetWindowTextLength
USER	37	GetClientRect
USER	38	GetWindowRect
USER	39	AdjustWindowRect
USER	40	AdjustWindowRectEx
GDI	1	GetStockObject
GDI	2	SelectObject
GDI	3	DeleteObject
GDI	4	GetObject
GDI	5	GetCurrentObject
GDI	6	UnrealizeObject
GDI	7	CreatePen
GDI	8	CreatePenIndirect
GDI	9	CreateSolidBrush
GDI	10	CreateHatchBrush
GDI	11	CreatePatternBrush
GDI	12	CreateBrushIndirect
GDI	13	CreateBitmap
GDI	14	CreateBitmapIndirect
GDI	15	CreateCompatibleBitmap
GDI	16	CreateDiscardableBitmap
GDI	17	CreateDIBitmap
GDI	18	CreateDIBPatternBrush
GDI	19	GetBitmapBits
GDI	20	SetBitmapBits
GDI	21	GetBitmapDimension
GDI	22	SetBitmapDimension
GDI	23	GetTextExtent

Library	Ordinal	Name
GDI	24	GetTextExtentEx
GDI	25	GetTabbedTextExtent
GDI	26	TabbedTextOut
GDI	27	ExtTextOut
GDI	28	TextOut
GDI	29	GetTextMetrics
GDI	30	GetTextFace
GDI	31	GetTextAlign
GDI	32	SetTextAlign
GDI	33	GetTextCharacterExtra
GDI	34	SetTextCharacterExtra
GDI	35	GetTextColor
GDI	36	SetTextColor
GDI	37	GetBkColor
GDI	38	SetBkColor
GDI	39	GetBkMode
GDI	40	SetBkMode

Appendix F — Confirmed Function Names

The rename table from the 15 October lookups, carried forward whole.

Date: 16 October 2026

A name in this table is the name attached in the decompile. It is not, by itself, a claim that the function runs on book open. Module roles are chapter 17 of the contents.

Module	Function	Name
BAS	FUN_1408_0376	CDBPAINTPOLYGON
BAS	FUN_13e8_00e6	CDBPAINTPOLYLINE
BAS	FUN_10a0_0558	CDBPAINTARC
BAS	FUN_10a0_0458	CDBPAINTARCALT
BAS	FUN_1020_03e4	CDBPAINTRECTANGLE
BAS	FUN_1008_01dd	CDBSCALETWIPSTOSCREEN
BAS	FUN_12d0_03ec	CDBSETPROPERTY
BAS	FUN_1408_00d8	CDBSETPOINTARRAY
BAS	FUN_12d0_086c	CDBSETUPPAINTRECT
BAS	FUN_1008_03f6	CDBSETSCRIPTPROPERTY
BAS	FUN_1070_1774	CDBVALIDATEPROPERTY
BAS	FUN_1070_1b3c	CDBCHECKVERTICES
BAS	FUN_1230_2f1c	CDBPAINTELLIPSE
BAS	FUN_13d8_0066	CDBPAINTELLIPSEALT
BAS	FUN_1230_2278	CDBPAINTROUNDRECT
BAS	FUN_1230_26f4	CDBPAINTROUNDRECTALT
BAS	FUN_13e0_015a	CDBPAINTROUNDRECTALT2
BAS	FUN_1230_2b5a	CDBPAINTARC2
BAS	FUN_10e0_208e	CDBSETMULTIPROPERTY
BAS	FUN_10e8_33e8	CDBSETMULTIPROPERTY2
BMP	FUN_1078_05c2	CreateDibBitmap
BMP	FUN_1090_0558	CreateDibBitmapAlt
BMP	FUN_1098_0874	CreateDibBitmapAlt2
BMP	FUN_10a0_16ee	CreateDibBitmapAlt3
BMP	FUN_1048_0116	BitBltBmp
BMP	FUN_1048_1250	BitBltBmp2
BMP	FUN_1048_2c5e	BitBltBmp3
BMP	FUN_1080_0194	BitBltBmp4
BMP	FUN_1010_18a4	SelectPaletteBmp
BMP	FUN_1030_03e4	GetDcBmp
BMP	FUN_1080_0548	SelectPaletteBmp2
BMP	FUN_1000_0031	Lz77Expand
BMP	FUN_1000_0162	Lz77ExpandCaller
BMP	FUN_1000_0752	Lz77ExpandCaller2
CMP	FUN_1010_0ee6	LoadCompilerResource
CMP	FUN_1038_0dea	LockCompilerResource2
CMP	FUN_1040_0970	ExpandTextCore
CMP	FUN_1010_0070	CompilerStrLen
CMP	FUN_1010_1774	CompilerStrCpy
CMP	FUN_1010_1a28	CompilerStrCat
CMP	FUN_1038_06f8	CompilerStrHelper
FLT	FUN_1028_14be	FltBitBlt
FLT	FUN_1040_071a	FltBitBlt2
FLT	FUN_1028_0390	FltSendMessage
FLT	FUN_1028_2316	FltSendMessage2
FLT	FUN_1028_26ac	FltSendMessage3

Module	Function	Name
FLT	FUN_1010_0e98	FltAlloc
FLT	FUN_1010_20b6	FltAlloc2
FLT	FUN_1000_093b	FltStrCpy
JPG	FUN_1008_c9ca	JpgAlloc
LNL	FUN_1018_04a6	LnlAlloc
LNL	FUN_1050_043a	LnlLock
LNL	FUN_1070_203e	LnlAlloc2
LNL	FUN_10d8_097e	LnlLock2
LNL	FUN_1040_344c	LnlStrCpy
LNL	FUN_1040_40f0	LnlStrLen
LNL	FUN_1090_0130	LnlStrCat
MM	FUN_1070_03f4	MmSendMessage
MM	FUN_1088_0200	MmSendMessage2
MM	FUN_1088_0642	MmSendMessage3
MM	FUN_1088_09f0	MmCreateWindow
MM	FUN_1048_00d0	MmAlloc
MM	FUN_1098_006a	MmAllocStr
MM	FUN_1098_0134	MmStrCpy
NET	FUN_1038_01b4	NetSendMessage
NET	FUN_1038_0484	NetSendMessage2
NET	FUN_1040_0000	NetSendMessage3
NET	FUN_1008_17c8	NetCreateWindow
NET	FUN_1008_0862	NetAlloc
NET	FUN_1008_136c	NetAlloc2
NET	FUN_1020_0410	NetStrCpy
NET	FUN_1020_0a10	NetStrCpy2
RCR	FUN_1008_158c	RcrBitBlt
RCR	FUN_1008_164c	RcrBitBlt2
RCR	FUN_1008_170e	RcrBitBlt3
RCR	FUN_1008_077a	RcrSendMessage
RCR	FUN_1008_0e94	RcrSendMessage2
RCR	FUN_1008_11aa	RcrSendMessage3
RCR	FUN_1008_08e2	RcrAlloc
RCR	FUN_1008_2190	RcrAlloc2
RCR	FUN_1008_aa62	RcrStrCpy
RTM	FUN_10e0_0050	PaintRectangle
RTM	FUN_1398_05c6	PaintRectangleAlt
RTM	FUN_1070_0b32	BitBltHelper
RTM	FUN_10d0_2d4e	BitBltHelper2
RTM	FUN_10d0_2fa0	BitBltHelper3
RTM	FUN_10e0_0dea	BitBltHelper4
RTM	FUN_1130_32ce	BitBltHelper5
RTM	FUN_1010_1a0a	SendMessageHelper
RTM_BOOK	FUN_1398_05c6	PaintPageRect
UTL	FUN_1000_093b	UtlAlloc
UTL	FUN_1018_1472	UtlLock
UTL	FUN_10b8_0eac	UtlAlloc2
UTL	FUN_10c0_028c	UtlLock2
UTL	FUN_1008_0504	UtlStrCpy
UTL	FUN_10b8_0000	UtlStrLen
UTL	FUN_10c8_001e	UtlStrCat
VBX	FUN_1008_12b2	VbxCreateWindow
VBX	FUN_1020_13fe	VbxBitBlt
VBX	FUN_1038_1bd8	VbxAlloc
VBX	FUN_1058_0534	VbxAlloc2

Module	Function	Name
VBX	FUN_1058_0910	VbxBitBlt2
VBX	FUN_1058_0ac6	VbxAlloc3
W16	FUN_1010_1914	W16Alloc
W16	FUN_1058_0f14	W16Lock
W16	FUN_1058_2568	W16Alloc2
W16	FUN_10b0_0060	W16Lock2
W16	FUN_1030_003a	W16StrCpy
W16	FUN_10b0_022c	W16StrLen
W16	FUN_10c0_013a	W16StrCat
W16	FUN_10c0_0310	W16StrHelper

Appendix G — SoftICE

The command card for the Windows 98 sessions. SoftICE and the Watcom debugger do not share a syntax. Do not type these lines into WDW. Appendix H is that debugger.

Date: 16 October 2026

SoftICE field work as the standing host closed on 11 October 2026. The kit that loaded was `SI405w9x` plus ICEDUMP 6.021, loader `w9x/4.05.334/icedump.exe`, and only after `winice.dat` listed the exports of both `kernel32.dll` and `user32.dll`. The night of 15 October still needed this card. The book-open read below is that night. The host was the Win98 VM. XP NTVDM is not a clean 16-bit host.

G.1 The screen

Ctrl-D pops SoftICE over Windows. Ctrl-D again returns to Windows if the machine is running. While it is popped, Windows is stopped.

The screen is four regions, top to bottom, and you can turn each one on or off:

Region	What it is	Command that toggles it
Register	AX BX CX DX, SI DI BP SP, DS ES SS CS IP, and the flags	`WR`
Data	Hex on the left, ASCII on the right. This is where a dump lands.	`WD`
Code	The disassembly at CS:IP. The current instruction is highlighted.	`WC`
Command	One line at the bottom. History with the up arrow.	always there

There is no Locals pane and no Calls pane. The stack is memory at SS:SP. The frame, after `ENTER`, is memory at SS:BP. If you want a call chain you walk return addresses yourself, or you use the Watcom Calls window (Appendix H). SoftICE will not draw it for you.

`MAP` output is a list of segments for one module. The segment numbers in that list are hex. The selector beside each segment is this boot only.

G.2 Commands that were actually used

Clear before you set. A breakpoint left over from the polygon painter (`GDI!POLYGON`, or BAS `1408:0376`) freezes SoftICE as soon as a page scrolls.

```
BC *
BL
```

`BC *` clears every breakpoint. `BL` lists what is still set. `BC n` clears one, by the number `BL` printed. `BD n` disables one without deleting it. `BE n` enables it again.

Break on an export. The ordinal form survives a reboot. A selector does not.

```
EXP TB80BAS
BPX TB80BAS.Ordinal_8
BPX TB80BAS!OBJFILEOPEN
```

`EXP` lists the exports. Ordinal 8 is `CDBFILEOPEN` on every boot. The `!` is SoftICE's module mark. Watcom uses `@` for the same idea. `BAS(82)` and `BAS(83)` are not addresses. SoftICE rejects them.

Break on a live instruction, and do something when it hits:

```
MAP TB80BAS
BPX <selector>:0376 DO "D SS:BP-4 L 4; D DS:30AC L 40"
```

`MAP TB80BAS` prints the live selector for each NE segment. You read the selector for the segment you want, then you type that selector. `DO` runs the quoted commands and, unless one of them stops, continues. `D` is a dump. `L` is the length. `SS:BP-4` is the saved word just under the frame. `DS:30AC` is the point buffer the polygon painter copies into.

Run and step:

Command	Effect
`G`	Go. Run until a breakpoint or a fault.
`T`	Trace. Step into a `CALL`. F8 is the same key.
`P`	Proceed. Step over a `CALL`. F10 is the same key.
`P RET`	Run until the next `RET` or `RETF` in this frame. This is how you leave a function without tracing every instruction inside it.
`U <addr>`	Unassemble at an address without moving CS:IP.
`R`	Show the registers. `R AX=1` sets one.

Dump:

Command	Effect
`D <addr>`	Dump bytes at a far address.
`D <addr> L <n>`	Dump `n` bytes.
`DB`, `DW`, `DD`	Bytes, words, dwords. A word dump is the one that matches a stack slot.
`D SS:BP+6`	The first stack argument after a far frame. See Appendix I for why `+6`.

`/DUMP`, from ICEDUMP, writes the SoftICE console unless the dump experts are set. It does not write the address you think you named. A file that starts with the command log, or with the DOS device chain (`CON`, `AUX`, `PRN`), is not a book. A dump is real when it contains book strings at offsets already known. `page.bin01511` did: `ASYM_TpID`, `Page#`, matching `9e954d01.tbk` at `0xB8B51`.

Do not search all of memory for a byte pattern. A wide search for `80 65` GPFed the VM.

G.3 What not to step into

A GPF in this host is usually a segment that is not present, or a far call into `KERNEL` that you traced into.

- Do not `T` into an unloaded NE segment. `MAP` the module first. If the segment is not in the list, it is not in memory. Set the breakpoint and `G` until the program loads it.
- Do not `T` a far `CALL` into `KERNEL`, `USER`, or `GDI` unless that segment is already present. Use `P`.
- The CMP expand host died in `TB80W16` at `001c:00000015` on the bytes `9A 9C 06 57 2A` (a far call, segment not present). That fault is parked. It is not the book-open path.
- `TBCBT` segment 7 holds the cipher exports. A module list that showed segments `01`, `0E`, `02`, `0B`, `08`, `0D`, `04`, `03` had not loaded segment 7. Breaking there before it is mapped faults. Searching memory for the leftover constants `0xA01` and `0x300` finds the call site while segment 7 is still unloaded.

If SoftICE will not pop, or the VM is wedged, the fallback on that VM is DEBUG.COM. It is a smaller language. `T` traces. `D` dumps. There is no `P`, no `BPX`, no `DO`, and no module name. `Q` quits DEBUG. Do not type SoftICE lines into it.

G.4 The 15 October book-open read

This is the path that was on the screen, from the call that asks for 4 bytes down to the buffer those 4 bytes were not.

The stable names, in order:

1 `CDBFILEOPEN` is BAS ordinal 8. Break there on a lesson open. Do not break ordinal 51. Ordinal 51 is `CDBSENDCREATE`, the author create path. A lesson does not call it.

2 From there the open goes through `CDBCREATEBOOKREF`, then `FUN_1048_1cc4`, then `FUN_1048_2174`, then `OBJFILEOPEN`.

3 `OBJFILEOPEN` calls `FUN_1160_03de`. That function is the read wrapper. On that boot the call site was `36E7:201B` and the callee was `36F7:03DE`.

Those selectors died at the next reboot. The NE numbers did not. `36E7` was NE segment `2F` (Ghidra `1170`). `36F7` was NE segment `2D` (Ghidra `1160`). The check is `(NE index − 1) × 8 + 0x1000`. Segment `0x2D` is 45 decimal, and `(45 − 1) × 8 + 0x1000 = 0x1160`. The breakpoint that survives the reboot is not a selector and not a segment number. It is `BPX TB80BAS!OBJFILEOPEN` or `BPX TB80BAS.Ordinal_8`, then `T` into the `CALL`. `BPX BAS(2D):03DE` is the notation already rejected. `MAP` gives the selector. The offset `03DE` is the part you copy.

At the call site the bytes were:

```
36E7:2019  PUSH 04
36E7:201B  CALL 36F7:03DE
```

`PUSH 04` is the length. Four bytes. The far pointer to the buffer is already on the stack under it.

`FUN_1160_03de` begins with `ENTER 0022,00`. Until that instruction has executed, SS:BP is still the caller's frame. Dumping `SS:BP+6` at the call site dumps the caller's locals. That was the first wrong dump of the night. Step in with `T` once so CS:IP lands on the callee, `T` again past `ENTER`, and only then dump.

```
T
T
DW SS:BP+6 L 1
DW SS:BP+8 L 2
```

On that hit, `[BP+6]` was `0004`. The far pointer at `[BP+8]` was `3DD7:9086`. Dumping that address:

```
DB 3DD7:9086 L 4
DB 3DD7:9070 L 30
```

The four bytes were `FC B5 BD B0`. That is an internal pointer sitting in the buffer, not a page header. A page header that is the book signature is `03 4A 42 4F` (`JBO`, with a length byte in front). A record header that is the one being hunted starts `14 00`. Neither of those was in the four bytes. The wider dump showed `4F 46` (`OF`) nearby, at `9092`. Nearby ASCII is not the bytes you just read. Read the four at the pointer, then stop.

Inside `FUN_1160_03de` there are two ways to satisfy the read:

- At offset `048F` it calls `FUN_1258_00a8`, the buffered path.
- The next test, at `049A`, is `JZ 0516`. When the buffer already held the bytes, that jump was taken.
- Offset `04F3` is the raw DOS read (INT 21h, AH=`3F`). It was skipped. Setting a breakpoint only on `04F3` and watching it never hit is the correct observation. It is not a broken breakpoint.

So a 4-byte read that returns `FC B5 BD B0` is the wrapper doing its job on something that is not yet a record. The next useful hit is the same function again, on a later call, when the length is larger than 4 or the buffer starts with `14 00` or with `03 4A 42 4F`. Do not single-step the whole of `FUN_1258_00a8` looking for a compare against `0x14`. Chapter 4 already closed the static candidates. The live test is the buffer about to become an object.

G.5 A session, in order

```
BC *
MAP TB80BAS
BPX TB80BAS!OBJFILEOPEN
G
```

Open the lesson. When it breaks:

```
T
```

only if the highlighted instruction is the `CALL` you mean to enter. Otherwise `P`. When CS:IP is inside `FUN_1160_03de` and past `ENTER`:

```
DW SS:BP+6 L 1
DW SS:BP+8 L 2
DB <segment>:<offset> L 10
```

Write down the length and the first bytes. `G` to the next hit. If the machine faults, `BC *` before you set anything else.

G.6 What transfers to Watcom

The discipline transfers. The words do not.

- Break a stable export, not a selector copied from last boot.
- Dump the frame only after `ENTER`.
- Step over `KERNEL`. Do not trace into a segment `MAP` did not list.
- A 4-byte buffer that is not `14 00` is data. Record it and continue.
- Ordinal 51 is the wrong function.

Appendix H has the Watcom words for each of those.

Appendix H — The Watcom Debugger

WDW, the Open Watcom windowed debugger, on the same Windows 98 image. This is the host used after 11 October 2026. It is not SoftICE with different punctuation.

Date: 16 October 2026

ToolBook's NE images have no Watcom debug info. WDW can see export names. It cannot see types, locals, or line numbers. The Locals window stays empty on `TB80BAS` and `TB80RTM`. You read the stack the same way Appendix I describes, or you do not learn the arguments.

H.1 The screen

WDW is a Windows program. It does not pop over the machine the way SoftICE does. The lesson runs in another window. When a breakpoint hits, that window freezes and WDW comes forward.

The Windows menu opens the panes that matter. Leave the ones you are not using closed. A full set of them hides the instruction you meant to read.

Pane	What it is for
Assembly	The bytes at CS:IP. This is the pane that matches Ghidra. Source will be blank.
Registers	AX through IP, and the flags. CS:IP is the instruction in the Assembly pane.
Functions	Export names for the module you have open. Right-click a name to break on it. This works when the command line will not resolve the name.
Break	The breakpoints that exist. Resume is a property of a row in this list.
Calls	The return addresses above the current frame. Chapter 12's spine was read out of this pane.
Log	Status lines. `log <file>` copies this pane to disk.
Memory / data	A hex dump. Use it when you have stopped. The log does not capture it.
Locals, Watch, Source	Empty on these NE images. Opening them does not create symbols.

The command line is a pane of its own, usually along the bottom. The keys that match the words:

Key	Word	Effect
F5	`go`	Run until a breakpoint.
F8	`trace`	Step into a call.
F10	`step`	Step over a call.

If a key does nothing, type the word. Do not invent a SoftICE key. There is no Ctrl-D. `P`, `T`, `BPX`, `BC *`, and `!` are SoftICE. Typed here they are errors, or they are the wrong program.

H.2 Commands

Module mark. SoftICE writes `TB80BAS!OBJFILEOPEN`. WDW writes `TB80BAS@OBJFILEOPEN`. The `@` is not optional when more than one module exports a similar name.

Break, then resume. `resume on` is not a flag you put on the creation line. It is a property of a breakpoint that already has a number.

```
break TB80BAS@OBJFILEOPEN
break 1 resume on
```

`BL` is not the list command here. The Break pane is the list. The number in `break 1` is the row in that pane.

If `@` fails, the DLL is loaded but WDW did not bind the export to the command line. Two ways that worked:

1 Functions pane, find the name, right-click, Break. Then in the Break pane turn Resume on.

2 Use the live selector from a hit you already trust.

```
break 44CF:0DE8
break 1 resume on
```

That selector is dead after a reboot. The export name is not.

Log, then continue. WDW has no “trace every call.” SoftICE’s `BPX … DO "…"` and WinDbg’s `wt` do not exist here. You name the functions. Each hit writes a status line and, with resume on, does not stop.

```
log c:\tbk\trace.log
break OBJFILEOPEN resume on
break CDBCHUNKFIND resume on
break CDBCHUNKLAST resume on
```

`log >> c:\tbk\trace.log` appends. `log` with no file stops logging. Close the log before you copy it off the VM. An open log is still growing.

`break <symbol> do <cmd>` runs a command on the hit. `hook <event> <cmd>` runs a command on a window event. Hook events are window events. They are not function entry. Do not use a hook to find the record reader.

The log line is a status line. It is not the registers and it is not the buffer. To see bytes, leave resume off on that one breakpoint, stop, and read the Memory pane at the address in DS:DX or at SS:BP+8. Then `go`.

H.3 What the three traces contained

These were full passes with resume on, named exports only. Counts are hits, not functions.

Trace	Hits	What fired	What did not
W16	11	`GETPLATFORMREF`, `IS32BITENV`, `GETRUNTIMELOCALIZEDDLL`, `ACXINITIALIZETOOLBOOKSUPPORT`, `ACXOLEINITIALIZE`, the 32-bit thunk (`LOADLIBRARYEX32W`, `GETPROCADDRESS32W`)	A book open. A record walk.
UTL	53	String and task helpers, `PLUGINSVCINIT`, `TBK.INI` profile reads, metrics, `GHEAPCREATE`	A record walk. The type-name table lives in UTL and was not what those hits were doing.
BAS	173	`VALUETOSTRING`, the `VALUENEW*` family, `OBJFILEOPEN` (the frame was `enter 025A`), `STACKGETSIZE`, `STACKPUSH`, `CDBCHUNKLAST`, `CDBCHUNKFIND`, `CDBCHUNKEND`, `RESMGR*`	A compare of a far pointer against `0x14`. Ordinal 51.

`OBJFILEOPEN` firing is the file-open path. It is not, by itself, the reader. The buffered read under it is `FUN_1160_03de`, and the raw DOS path at offset `04F3` is skipped when the buffer already has the bytes. That is the same fact as Appendix G.4, watched from the other debugger.

Another pass over the same W16 and UTL exports will produce the same log. Do not schedule one.

H.4 The Calls window is one boot

On 15 October the Calls pane showed the same frames on almost every stop during book open and the page walk. Selectors from that pane:

Selector:offset	Note
`3497:03EF`	Early frame.
`3497:4A6E`	The step point that was planned off this frame.
`3197:0379`	
`3167:0CB1`	Near call.
`3167:14BC`	Far call.
`3247:02BF`	Near.
`3247:05B7`	Near.
`3247:0B7A`	Far.

The full table for that boot is `TBdecompile/maps/wdw-to-ghidra.md`. It is a lookup for those selectors. It is not a map of the next boot. Match an offset to Ghidra. Remap the selector with the Functions pane or with a fresh break on the export.

H.5 Where to break next

1. `break TB80BAS@CDBFILEOPEN` on a lesson open. Resume on, so the open finishes far enough to load the book.
2. Then a stop, not a log, on `CDBCREATEPAGEREF` or `CDBSETVALUEEX`. Export names. Not the selectors in the table above.
3. When an object exists, dump the word at object `+0x0F` and the file offset the read just returned. That pair is the reader's output. The function that produced it is the reader.
4. Do not break ordinal 51. Do not break `GDI!POLYGON` and leave it set. The second of those is a SoftICE lesson, and it still applies: a paint breakpoint you forget to clear will stop you on every scroll.

H.6 A session, in order

```
log c:\tbk\open.log
break TB80BAS@CDBFILEOPEN
break 1 resume on
go
```

Open one lesson. When the log shows `CDBFILEOPEN` and the book is up, stop the machine (Break, or a second breakpoint you left with resume off). In the Calls pane, write down CS:IP and the first return address. In the Memory pane, dump the buffer pointer you get from Appendix I's frame, not from a selector copied out of this appendix.

```
log
```

That closes the file. Copy `open.log` off the VM before the next run overwrites it.

H.7 Faults, same as SoftICE

WDW will happily trace a far call into a segment that is not present. The result is the same GPF. `step` over `KERNEL`, `USER`, and `GDI`. Do not `trace` them. If the Assembly pane shows a selector that `MAP` would not have listed, that instruction is not yours to enter.

The CMP expand fault at `TB80W16` `001c:00000015` is the same parked fault as in Appendix G. It is not this spine.

Appendix I — Reading the Assembly

Ghidra's C is a guess at the calling convention, the width of each value, and which register is which argument. The bytes are the program. When the C and a live dump disagree, the dump wins. When the C and the disassembly disagree, the disassembly wins.

Date: 16 October 2026

This is the minimum needed to check a listing while stopped in SoftICE or WDW. It is not a general 8086 manual.

I.1 An address is two numbers

A far address is `segment:offset`. In real mode the physical address is `segment × 16 + offset`. In the Win98 protected-mode sessions the segment number is a selector. The CPU, not you, turns it into a base. Two different selectors can point at the same NE segment on two boots. The offset inside the segment does not change.

Ghidra's name `FUN_1160_03de` means offset `03DE` in the Ghidra segment whose base is `1160`. That base is not a live selector. NE segment `N` is usually Ghidra base `0x1000 + (N − 1) × 8`, with gaps where a segment is not code. Segment 1 is `1000`. Segment 2 is `1008`. Segment `0x2D` (45) is `1160`. Segment `0x2F` (47) is `1170`. Appendix D is the whole table.

A symbol map line `1160:03DE FUN_1160_03de` is this same pair. SoftICE and WDW do not load that file. You use it on paper.

I.2 Bytes are little-endian

The dump shows bytes in memory order. A word is stored low byte first.

Bytes in the dump	As a word	As text
`04 00`	`0004`	length 4
`4A 42`	`424A`	the characters `J` then `B` if you read bytes, not the word
`03 4A 42 4F`		`03`, then `JBO`
`14 00`	`0014`	a record type byte `0x14`, then a zero
`FC B5 BD B0`		the four bytes from the 15 October buffer

Read a word off a dump by swapping the two bytes. Read a signature by not swapping. `4F 46` at the next address is the characters `OF`. It is not part of the dword you already read at the previous address. The 15 October wider dump (`9070` for `0x30` bytes) showed `4F 46` at `9092` and `FC B5 BD B0` at `9086`. Those are different objects that happen to sit near each other.

I.3 The frame after ENTER

A far call pushes the return address as CS then IP, then the callee runs. ToolBook's compiled functions almost all start with `ENTER nn,00`. That instruction pushes BP, sets BP to SP, and subtracts `nn` from SP for locals.

After `ENTER` has executed, for a far callee:

Location	What it is
`[BP+0]`	Saved BP.
`[BP+2]`	Return IP.
`[BP+4]`	Return CS.
`[BP+6]`	First argument.
`[BP+8]`	Second argument.

Location	What it is
`[BP−2]` downward	Locals. Negative offsets are locals. Positive offsets past 4 are arguments.

A near call does not push CS. The first argument is then at `[BP+4]`. If you are not sure which you entered, look at the call instruction in the caller. `CALL 36F7:03DE` is far (it has a segment). `CALL 0CB1` with no segment is near. The Calls pane in WDW labels them. The code bytes do too: a far direct call is `9A`, a near direct call is `E8`.

Dump the frame only after `ENTER`. Before that instruction, BP still belongs to the caller, and `[BP+6]` is the caller's argument or a local. That was the wrong dump at `36E7:201B` on 15 October. The call had not been entered.

Two words that form a far pointer are pushed segment last, offset first, so that after the call the offset sits at the lower address. In the 4-byte read, the length was the last thing pushed (`PUSH 04`), so it is the first argument, at `[BP+6]`. The buffer pointer is the next argument: offset at `[BP+8]`, segment at `[BP+0A]`. Dumping `DW SS:BP+8 L 2` prints offset then segment, which is the order you need for `DB segment:offset`.

Ghidra often prints this as `param_1`, `param_2`, and a `local_4` that is actually the pointer. The names are positions. They are not types. `undefined2` means "two bytes, untyped." `undefined4` is often a far pointer the decompiler did not split. `unaff_CS` and `unaff_SS` mean "this register was live and the decompiler does not know who set it." Treat those as unknown, not as arguments you can pass.

I.4 Instructions that showed up

Enough to read a basic block. Sizes: a byte is 8 bits, a word is 16, a dword is 32.

Instruction	What it does	How it looks wrong in C
`MOV r, r/m`	Copy.	An assignment. Fine.
`PUSH imm` / `PUSH r`	Put a word on the stack.	A function argument. The order is right to left, so the last `PUSH` is the first argument.
`POP r`	Take a word off.	Often deleted from the C and folded into a return value.
`CALL far`	Push CS:IP, go.	A function call. The decompiler may invent the callee's signature.
`ENTER nn,00`	Make the frame in I.3.	The opening `{` of the function. The C will not show it.
`LEAVE`	Undo `ENTER`.	The closing of the function.
`RET` / `RETF`	Near or far return. `RETF n` also pops `n` bytes of arguments.	`return`.
`CMP a, b`	Set flags from `a − b`. Does not store the result.	An `if`. Look at the jump, not at the `CMP` alone.
`JZ` / `JNZ`	Jump if the compare was equal / not equal. Also written `JE` / `JNE`.	Which branch is the `if` body. Ghidra sometimes swaps them and then negates the test.
`ADD` / `SUB`	Arithmetic that is stored.	`+` / `−`.
`IMUL`	Signed multiply. `IMUL AX, imm16` multiplies AX by that immediate.	A scale. `IMUL AX, 0x14` is "times 20," which is a stride, not a coincidence.
`LEA`	Address, not a load.	`&` of something.
`INT 21` with AH=`3F`	DOS read. BX is the handle, CX is the length, DS:DX is the buffer.	A call the decompiler may name `DOS3CALL` or leave as a thunk.

`P` / `step` executes a `CALL` and stops on the next instruction in the caller. `T` / `trace` stops on the first instruction of the callee. Use `T` only when you mean to read that callee. A `CALL` into `KERNEL` traced with `T` is how the session GPFed.

I.5 One block, read as bytes

This is the call that asked for 4 bytes. Selectors are that boot.

```
36E7:2019  6A 04           PUSH 04
36E7:201B  9A DE 03 F7 36  CALL 36F7:03DE
```

`6A 04` pushes the sign-extended byte 4. `9A` is a far direct call. The next four bytes are offset then segment, little-endian: `DE 03` is offset `03DE`, `F7 36` is segment `36F7`. Ghidra shows the same call as `FUN_1160_03de(...)` and invents the

argument list. The length is the only thing this instruction pushes. Anything else the C lists was pushed earlier.

Inside the callee, after the buffer path returns, the branch that skipped the DOS read was:

```
048F   CALL FUN_1258_00a8
049A   JZ 0516
04F3   ; raw DOS read, not taken
```

`JZ` means “the zero flag is set, so skip ahead.” On that hit the flag was set, so `04F3` never ran. A C rendering that always shows both the buffered call and the DOS call as unconditional is wrong about control flow. The jump is the control flow.

I.6 A stride is a multiply, not a struct

The loop that started this search, at `35C7:02C2`, does this:

```
MOV  AX, [BX+SI+0E]
IMUL AX, 14
ADD  BX, AX
CMP  BYTE PTR [BX], 14
JNZ  <back>
```

Read it in words. Take the word at `[BX+SI+0x0E]`. Multiply by `0x14` (20 decimal). Add that to BX. Compare the byte at BX with `0x14`. If it is not `0x14`, do it again.

So each step moves 20 times some count, and the byte that must be `0x14` is the first byte of the thing landed on. That is a record stride of 20 bytes and a type byte of `0x14`. It does not require the three bytes `14 00 00` to sit in the file as a magic string the compiler compares in one instruction. A search for the immediate dword `14 00 00 00` can miss a compare of one byte against `0x14`.

Ghidra will often turn this into a `while` with casts through `undefined` and a multiply you have to find. The five instructions are the clearer statement. If the C does not multiply by 20 and then test a byte, the C is not this loop.

I.7 How the C lies, with the forms in these listings

Check these before you believe a body.

- The signature is invented. `__stdcall16far` is a good default for this compiler. The number and width of `param_N` is not. Count the `PUSH` instructions in the caller, and count the bytes a `RETF n` pops. Those two counts are the argument list.
- A far pointer is split. `param_1` as `undefined4`, then `(int)param_1` and `(undefined2)((ulong)param_1 >> 0x10)`, is offset and segment. In the debugger that is one `segment:offset`, not two integers you add.
- Locals are stack slots. `local_4`, `local_6`, `local_8` are `[BP-4]`, `[BP-6]`, `[BP-8]`. If the C uses `local_8` as a pointer and also as a flag, one of those uses is a register reuse the decompiler failed to split.
- **`unaff_CS` passed as an argument** means the decompiler lost a segment override or a live CS. Do not go looking for a source variable of that name. There is not one.
- **A thunk that shows `??`** is an ordinal import whose module was not bound in that Ghidra project. The instruction is still a far call. The export tables in Appendix A are where the ordinal gets a name. The C will not grow the name on its own.
- A switch on a byte (`if (local_4[0] == 0x38)` and a ladder of constants) is a type byte or a message. The constants are data. Write them down in hex before you accept the decompiler’s decimal.
- Dead code after a jump. Ghidra sometimes prints a block and then a comment that it is unreachable (`Removing unreachable block`). Do not trace that block live. It is not on the path.

- Absence. A function missing from an HTML or C export may be bytes the loader never turned into a function. That is an undisassembled run, not a deleted function. Force-disassemble recovers `ENTER` (`C8`) sites and a few prologues. It does not name them, and it does not prove they run on book open.

I.8 Check one claim, then stop

Pick one sentence of C. Find the offset Ghidra printed for it. In the debugger, `U` that offset (SoftICE) or set CS:IP's pane on it (WDW). You are looking for one of three outcomes.

1. The bytes match the sentence. Keep the sentence. Write the offset in the margin, not a paraphrase.
2. The bytes are a jump the C treated as certain. Keep the jump. Throw out the sentence.
3. The address is not code. You are in data, or in a segment that was not loaded. Do not name it.

The 4-byte buffer `FC B5 BD B0` is outcome 1 for the sentence "the wrapper reads the length at `[BP+6]` into the buffer at `[BP+8]`." It is outcome 2 for any sentence that says those four bytes are a record header. They are not `14 00`. They are not `JBO`. The wrapper is confirmed. The reader is still the function that, on a later hit, fills a buffer whose first byte is `0x14` and whose stride is `0x14`.

Do not promote a `FUN_` name to a purpose because the C compiled. Confirm the bytes, or leave the name as a location.

Appendix J — OpenScript

The bytecode, the keyword pool, the handlers that have actually been expanded, and the subset a clean-room player implements. Chapters 6 and 14 are the claim list. This appendix is the working description.

Date: 16 October 2026

OpenScript in an 8.1 book is not stored as the text you type in Instructor. The handler name is often plain text. The body is a compressed token stream. `TB80CMP` compiles and expands that stream. `TB80BAS` evaluates expressions. Neither module contains the keyword pool as a C string table. A search of BAS for `buttonClick` finds nothing. That is not evidence the scripts are missing.

J.1 Who owns it

Piece	Where	What it is
Keyword pool	`TB80CMP` NE resource `0x77`, type 100, id 119	The names the expander prints. 15776 bytes.
Second emit routine	Resource `0x78`, id 120	15616 bytes. An early dump counted 980 strings. It is not a second language.
Property names	`COMPILERPROPINFOFROMINDEX` locks resource `0x70` (id 112, 3872 bytes, 213 strings). The ids the books use are also BAS resource type `0x2D1`, id `0x2D0`.	`bounds` is `0x4004`. Not the keyword pool.
Expand	`FUN_1040_0970` in CMP. The rename list calls it `ExpandTextCore`.	The byte classes below are this function.
Compile	`COMPILERCOMPILETEXT`, CMP ordinal 1	Not used by a lesson open.
Expand export	`COMPILEREXPANDTEXT`, CMP ordinal 3	The stable breakpoint if a compile test is run.
Execute / evaluate	`CFUNEXECUTE` ordinal 8, `CFUNEVALUATE` ordinal 9	
Arithmetic	`FUN_1008_2d31` in BAS, reached from the switch at `1008:2DC7`	148 cases. FPU: `FLD`, `FSTP`, `FCLEX`. Not the record reader.
Small runtime words	`TB80RTM` file `0xAACAD`	`handle`, `get`, `set`, `notifybefore`, `notifyafter`, `true`, `false`, `to`. Not the compiler table.

Do not treat every locked resource near `0x70` as the keyword list. `0x70` is properties. `0x77` is keywords.

An early string dump of `0x77` counted 992 strings. The decoder that produced the 76 Logic Families handlers counts 994 keywords. Use 994. The offset table starts at byte 2 plus count times 4. A loader that used count times 2 was wrong before the back-reference bug was even found. The 994 names have not been locked against a script compiled in Instructor 8. A name in that table is a string in the resource. It is not, by itself, proof that NIDA emits it.

J.2 The byte classes

From `FUN_1040_0970`. A byte is classified once. The numeric ranges overlap, so a second pass that says “everything above `0xBF` is a back-reference” will also eat the whitespace opcodes.

Byte	Meaning
`0x00`–`0x7F`	Literal character. This is how a quoted page id survives.
`0x80`–`0xBF`	Keyword.
`0xC0`–`0xDF`	Back-reference, after the whitespace classes have been taken out. Copy from the compressed input, not from the output.
`0xE0`–`0xEF`	CR, LF, and spaces.
`0xF0`–`0xFE`	Spaces.
`0xFF`	Repeat.

Keyword index, inside `0x80`–`0xBF`:

```
index = byte & 0xF
```

```
if bit 4 is set:
    index = (index << 8) | next_byte
if bit 5 is clear:
    append a space after the keyword
```

Bit 4 and bit 5 are bits of the opcode, not of the keyword text. A space is part of the encoding. It is not a separate literal unless the script had a space the keyword rule did not already emit.

Back-reference, as fixed on 10 October 2026:

```
if bit 4 is clear:
    length = (byte & 0xF) + 3
    src = (pos + 1) + (-4 - next_byte)
else:
    length = (byte & 0xF) + 4
    src = (pos + 1) + (-1 - word_offset)
copy length bytes from input[src]
```

`pos` is the position of the opcode in the compressed buffer. `src` can fall before the slice you passed the decoder. On `tb80r.sbk`, reading those from the whole book dropped the `?` marks from 14256 to 66. The 66 that remain are question marks in the script, not failed copies. A windowed decoder will mark a real back-reference as garbage. That is the decoder's window, not a different encoding.

J.3 Two bugs that produced the wrong language

`tbworkingpapers/tbwp0001/openscript_notes.md` begins by saying the decoder emits `italic`, `third`, `fourth`, and `systime`, and that the bytes might not be OpenScript. That opening is superseded by the same file's later sections. Do not cite it.

Two independent mistakes:

1. The keyword offset table was read as count times 2. Keywords came from the wrong row. `systime` appeared where the script says something else.
2. Bytes `0xC0`–`0xFF` were copied from the output stream. That produced `keySlash` and truncated words.

After both fixes the same bytes are:

```
system lastbook
  lastbook = name of this book
  system nextpage
  nextpage = name of this page
  go to page "P20001" of book "nida81.tbk"
end
```

Other handlers that are real source, not Latin-1 of the bytecode: `send next`, `send previous`, `go to page "P400"`, `go to page "P310" of book "nida81.tbk"`.

Keywords live in `TB80CMP`. BAS has no `handle` or `buttonClick` strings. RTM's eight words at `0xAACAD` are not a substitute table. There is no separate NIDA keyword resource in `nc81int.dll` or `nc81rec.dll`. Those DLLs are trainer and session calls.

J.4 How a handler sits in the file

A lesson book stores the name as plain text and compresses the body.

```
To Handle EnterPage
<compressed body>
```

The capitalization is not stable. Logic Families has 46 handlers whose text is `TO HANDLE buttonClick`, 29 whose text is `To Handle ButtonClick`, and 1 `To Handle EnterPage`. That is the 76. An extractor that required a plain `END` found 22 of them. 76 is the count after the input-stream fix. The corpus tool stops at the first standalone `end` and drops what follows. That is the tool. It is not a claim that `COMPILEREXPANDTEXT` itself stops there.

`tb80r.sbk` compresses the prefix too. There is almost no plain `to handle` in the file. A scan that decodes windows and keeps those whose output starts with `to handle` or `to get` recovered 33 handlers. The encoding rules match the lesson books. The histogram does not: more `0xFF` and `0xE0`, fewer `0xBF` and `0xC0`-range copies. An early pass that only searched for the plain prefix reported zero handlers in `tb80r.sbk`. That pass is superseded for this one file.

Book	Handlers	Storage
`9e954d01.tbk`	76, on 39 of 129 pages	Plain prefix, compressed body.
`nida81.tbk`	100	All `buttonClick`. Same encoding. The shell is a book.
`tb80r.sbk`	33	Prefix compressed. Reader book: help, bookmarking, logging, path manager, popup page manager. Not the engine.
`nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`	0 decoded	High-byte runs do not decode as this bytecode. Early pattern counts (61, 17, 94, 62) were not handlers.
`tb80anm.sbk`, `tb80hyp.sbk`	0 decoded	Strings and menus.

“No script in any system book” is too strong for `tb80r.sbk` and still true, as decoded text, for the four `nc81*` books. Those four are resource containers: the object-type name table and menus. The shell behavior that looks like a system book is `nida81.tbk` plus `nc81int.dll` and `nc81rec.dll`.

A `linkDLL` is not a system book. `TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, and `TBCBT` add commands only when a handler links them. `TBWIN` is `INIT_TBKWIN` and `TBM_EVALUATE`. None of those walks page records.

J.5 What the decoded handlers do

Logic Families is a menu of jumps. It is not a self-running program. Navigation is not “a quoted page id and nothing else.” P20000 of `nida81.tbk` goes to P20001 because the `buttonClick` handler says so.

Page	What was read
P900	Lesson menu. TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` is the IBM PC Compatible Computer line. `buttonClick` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Assessment. Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Section end. Menu returns to P900.
P1290	Quiz. Correct choice 5408. Again → P1292. Next → P1300.

A retry page returns to the quiz whose `again` field names it. `initializeRecs` on the shell clears responses when a session loads. That sentence is inferred from the shell script. It has not been watched as a call into `nc81rec.dll`.

The words the courses actually use:

`go to`, `send`, `send next`, `send previous`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, `name of this book`, `name of this page`, `system`, and a quoted page or book name.

One decoded `enterPage` starts `system logical __import`, sets `lastbook` to `name of this book`, sets `nextpage` to `"p10"`, and branches on `__import`. Another button is a comment `--{Go to next page}` and then `send next`.

`9e954d01.tbk` at file `0xB1A96` names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and the script calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`. Those initialize names have not been watched as

a call log. The order in `toolbook.lst` has not been watched live.

A TBX `.script` file is the same handler as plain text. It is a playback path for the export. It is not a reader of the `.tbk`. The export of this lesson has 302 `.script` files. The binary has 76 decoded handlers in the lesson plus 100 in the shell. Those counts are not required to match. The export splits what the binary stored as one compressed body, and it includes pages the 8.1 file does not have (P440 and P2920).

J.6 Expressions

`FUN_1008_2d31` stays in BAS segment `1008`. It looks for names such as `log`, then uses the FPU. The clean-room evaluator copies the operations that switch reaches:

`+`, `-`, `*`, `/`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`.

Anything else is left as the original text. The function is not the record reader and not a paint path. An earlier engine treated the right-hand side of an assignment as an opaque string. The named operations above are the part that changed.

J.7 What the clean-room player does and does not do

Accepts, as of the player that ships with this record:

- `send NAME to TARGET`
- a `get` that is a call
- `forward to TARGET`
- `break NAME`
- `PROP of TARGET`
- the navigation and assignment subset in J.5
- the arithmetic in J.6

Records, and does not evaluate: predicates such as `is not in`.

Ignores: `local` and `system` declarations. They decode. They do not allocate.

Does not apply: property writes, to a live object. Setting `bounds of` something does not move a widget.

Not implemented, or only sketched: nested property expansion, opcodes past the common subset, and `forward` into a handler that lives in a system book.

The CMP expand host that died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present) is parked. It is not the NIDA open path. Do not single-step that far call to learn a keyword.

J.8 The compile that would lock the table

Instructor 8 can compile OpenScript. It cannot write TBX. TBX starts in 9.5. The way to get bytecode out of 8.1 is to save a book and expand the handler.

1 On the Win98 or WinXP VM, open Instructor 8. Make a new book. Add a button. Paste:

```
to handle buttonClick
  go to page "P100"
end buttonClick
```

1 Save as `compile-test.tbk`. Copy it off the VM.

2 Expand with the decoder that already finds `to handle` / `end`, or break `COMPILEREXPANDTEXT` (CMP ordinal 3) if the expand is done inside Instructor.

3 A second script with `if` / `else` / `send` / `step` covers the rest of the common subset. A token that does not resolve is a gap in the 994-row table.

That test is written down. It has not been run. It does not trace the author UI. The author binaries (`TB80EDT`, `TB80.EXE`) are not in the archive.

J.9 What this appendix will not pretend

- The 994 strings are not printed here. They have not been sifted against a known compile. Printing them would repeat the symbol-map mistake.
- A high-byte run in an `nc81*` book is not a handler until it decodes to `to handle` or `to get`.
- `CDBCHUNKFIND` and the comma / CR / LF walker are text chunkers. They are not this expander.
- Ordinal 51 is not involved. A lesson open does not call it, and OpenScript does not create the object the script is attached to.

Appendix K — How to disassemble

This is the how-to. It is the procedure this project uses, and it is the reasoning under that procedure: how a listing decides that a byte is an instruction, how a function gets a name, how a name is not yet a purpose, and how a purpose is not yet a description of the operation. Ghidra is the tool. The theories are about the work, and they apply the next time the tool is something else.

Date: 16 October 2026

Appendix I is how to check the C against the bytes. This appendix is how the project file gets to a C listing you can check, and how you decide what a function is before you are willing to name it. A menu path that was not written down in the record is not invented here. A theory is marked as a theory. A number that was counted is marked as counted.

Three jobs are always in front of you, and they are not the same job.

Job	The question	What a wrong answer looks like
Name	What will you type the next time you search?	A plausible word on a function you have not read.
Purpose	What question does this function answer for a caller?	The name of the one API it happens to call.
Operation	What does the body do, in order?	The C, trusted because it compiled into C.

A rename script does the first job, badly, if you let it do the other two. A decompiler does the third job, partially, and will happily invent a sentence for the second. The record's grades apply to all three. A pattern is not Known. A body that was read is Known as an operation. A purpose is Thought until a second caller, or a stop in a debugger, agrees with the sentence.

K.1 What is loaded

`toolbookunpacker` produces an honest NE image. It clears the Optloader self-load flag. It does not patch the entry table. That patch is an optional post-step beside the unpacker.

Stock Ghidra mis-reads two fields in that table. The entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image, on the current set, with entry segment bytes at or above 128. The other modules do not need the post-step for that reason. An honest BAS image loads in the patched Ghidra 12.1.4 build. The unpacker does not rewrite those bytes. The Ghidra patch does.

Do not "fix" the image so that a stock Ghidra will open it by rewriting the entry table on disk and then treating that rewritten file as the program. The post-step is for the loader. The bytes of the functions are the unpacked image.

The packed originals are the wrong file to disassemble. On those files every segment but the stub has the relocation flag clear. The unpacked images are the ones with the relocating segments. A beautiful listing of the packed file is a listing of the wrong bytes. Chapter 20 counted this for the module-reference table. The same rule applies to every later listing.

Thought. A signed 8-bit read of a segment number turns 128 and above into a negative value. A loader that then rejects a non-positive segment drops those entries. They never become places to start disassembly. `ne_cbenttab` is the length of the entry table in the NE header. A loader that does not use that length is not walking the table the header describes. Both defects remove seeds. They do not alter a single opcode. That is why the patch belongs on the loader and not on the image.

K.2 Three addresses, and none of them is optional

Ghidra's `CodeN` is not the NE segment number. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps. `FUN_1160_03de` is offset `03DE` in the Ghidra segment whose base is `1160`. That base is not a live selector.

What you are holding	What it is
`maps/bas-ne-to-ghidra.md`	TB80BAS NE segment to Ghidra space. Use this one.
`maps/rtm-ne-to-ghidra.md`	TB80RTM, the same kind of table.
`maps/wdw-to-ghidra.md`	One boot's selectors. Dead after the next boot. Match the offset. Remap the selector.
`maps/html-segment-index.md`	Bases from one HTML export. If it disagrees with the three maps above, those three win.

A symbol-map line `1160:03DE` is the same pair. SoftICE and WDW do not load that file. The `.map` dumps are unsifted decompiler names. They were taken out of this edition. They stay on disk under `TBdecompile/20261015/20261012/`.

Theory: an address is a triple, or it is not an address. File segment, Ghidra base, live selector. Write all three when you have them, or write the one that is stable and say which one it is. The offset inside the NE segment is the stable half. The selector moves every boot. The Ghidra base is an invention of the loader, useful only inside that project file.

Mixing them is how a note becomes unverifiable. "The function at `4D27:2838`" is a live selector from one boot. "The function at `1028:0466`" may be a Ghidra pair. They are not comparable until the map says the Ghidra base that `4D27` was that day. Chapter 7 is the same rule for the cipher segment that a SoftICE module list had not loaded. The code was not absent. The segment was not mapped.

When you copy an address out of a Calls window, copy the offset and then do the map. Do not promote a selector into a `FUN_` name. `FUN_1028_0466` means the Ghidra space. If you did not do the map, you do not have that name.

K.3 What Ghidra is actually doing

Ghidra does not read the file from the first byte to the last and turn each byte into an instruction. That other method is a linear sweep. A linear sweep is what you get from a tool that assumes the whole range is code, in order, with no gaps and no data mixed in. It is the wrong default for a Win16 segment. A segment is a bag. It holds functions, jump tables, padding, and sometimes an immediate that happens to look like a prologue.

Ghidra's disassembler is a work list. The loader plants seeds. A seed is an address where an instruction is allowed to start. The entry point is a seed. Each exported entry the loader understood is a seed. The start of a code block is sometimes a seed. Everything else waits.

The disassembler takes a seed and asks the processor specification what instruction begins there. The specification for x86 is a SLEIGH file. SLEIGH matches the bytes against constructor patterns and, for each match, emits both the assembly text and a short program in pcode. Pcode is a small language of loads, stores, copies, adds, branches, calls, and returns. The assembly line is for you. The pcode is for the decompiler. They are produced together. They are not two independent opinions.

Once the instruction exists, the disassembler asks where execution can go next, and it puts those addresses on the work list.

The instruction	What is added to the work list
An ordinary instruction	The next address. This is fall-through.
A conditional jump	The target, and the fall-through.
A direct jump	The target only.
A direct call	The target, as a place a function may start. Fall-through of the call is also code, in the caller.

The instruction	What is added to the work list
A return, a return-far, or a halt	Nothing.
An indirect jump or an indirect call	Nothing, unless some other analyzer later proves a target.

The last row is the whole of the undisassembled-run problem. Recursive descent never walks into a function that nothing points at with a direct target. A Win16 program is full of those functions. They are reached by a far pointer built in two registers, by a function pointer stored in an object, by an export the loader dropped, or by a call that goes through a register. The bytes sit in the segment. The HTML prints `??`. The function is not missing from the program. It is missing from the work list.

This is the sentence to keep. The end of auto-analysis means the work list and the analyzer queue are empty. It does not mean every byte was considered. An empty queue is the analyzer having no further theory it is willing to act on. Undefined bytes are the bytes no theory claimed. Treat them as unread, not as data, until you have a positive reason they are data.

A positive reason is one of: a relocation or a reference that uses the address as a pointer to bytes, a table whose stride you have checked, a string the program actually loads, or a run of zeros used as alignment between two functions you already trust. "It was still undefined when analysis finished" is not a positive reason.

K.4 Why the walk stops, in more detail

A return stops the walk because the specification says the instruction does not fall through. That is correct for a real `RET` (`C3`) or `RETF` (`CB`). It is also how a mistaken decode hides the next function. If the byte before a real function is decoded as a return, the next function is never reached by fall-through. It will be reached only if something else names it. The prologue pass in K.6 exists because of this. It treats the byte after a real return as a candidate, and only as a candidate.

An indirect call stops the walk of the callee, not of the caller. The caller continues after the call. The callee is whoever was in the register or in the memory operand. Ghidra will not guess. You should not guess either, not from the call site alone. The call site tells you a pointer was used. The body, once you have found it, tells you what ran. Chapter 20 is the example already in the record: `CDBGETVALUEEX` locks, calls the code pointer in the `0x76` slot, and unlocks. The decompiler can show the call. It cannot show you which function sat in the slot on a given page. That is a runtime fact. A name for the slot's callee, chosen from the call site, would be a fiction.

A computed jump stops both directions if you are unlucky. The bytes after the jump may be a table of offsets, not code. If you force those bytes to disassemble, you manufacture instructions out of offsets, and those fake instructions fall into the real cases and split them. The practice is: when an indirect jump's targets are a dense table of words, define the table as words before you disassemble past it. This project has not needed that step to name the functions in the record. The rule is here so the next hole is not "fixed" by sweeping it.

Overlapping decodes are a different failure. x86 instructions are not a fixed size. The same bytes can be an instruction starting at address A and a different instruction starting at address A+1. Ghidra commits to one start. A seed that lands one byte off will consume the real instruction as raw material and produce a function that almost makes sense. The check is the one in Appendix I: the bytes, not the C. A function whose first instruction is not a prologue you recognize, and whose callers do not actually target that address, is a candidate for clearing and re-seeding one byte away. Clearing is destructive. Do it to the one function. Do not clear the segment.

K.5 Seeds, and the entry table as a theory of the program

The entry table is the program's own list of seeds. Each entry is an ordinal, a segment, an offset, and a flag that says whether the entry is exported by name. The resident and nonresident name tables are the program's own list of names for some of those seeds. An export name is the strongest name you will ever have. The linker published it. Another module's import table may cite it. You do not rename an export because a pattern found a better word. You may add a plate comment. The export name stays.

Internal functions have no such gift. They are reached by calls, and the calls are the only publication. A function with no call and no export is unpublished. It may still be real. It may be a candidate the prologue pass invented. Those two possibilities are why a created function is not yet a confirmed function.

Theory: the import tables are a purpose statement written by the linker. RTM's imports from BAS are `CDB*`, `VALUE*`, `STACK*`, `HEAP*` / `TBHEAP*`, and `RESMGR*`, plus `OBJBOOKVERSION`. That list is what RTM was willing to ask BAS to do. It is not a list of what BAS contains. BAS contains the `OBJ*` layer, and RTM does not link it except for that one version call. A theory of purpose that starts from "what do the other modules import?" will find the book API and will miss the object file. Both facts are in chapter 20. The method is the general one: read the import table as the cross-module purpose, and do not expect it to name the helpers inside the module.

Theory: an export that no import names is not dead. The EXE may call it by ordinal. An author tool may call it. The module may call it itself. `CDBSENDCREATE` is the case already settled. It is an export. It is the author path. A lesson open does not call it. "Present in the export table" and "on the open path" are different sentences. The first is settled by the NE file. The second is settled by a trace, or by reading the callers that the trace names.

Count the seeds after the load. The number of exported entries you can parse from the NE file, by hand or by the unpacker, should match the number of entry symbols Ghidra created. A shortfall on BAS, before the patch, is the signed-segment defect. A shortfall after the patch is a new defect, and it should stop the session. Do not begin naming in a project whose entry table is short. You would be naming a program with the doors closed.

K.6 Holes

A hole is a run of undefined bytes between two instructions, or between an instruction and the end of a block. Holes are the unread part of the listing. The HTML export of a hole is `??`. A search that only reads the HTML will report the hole as absent. Chapter 10 already says that absence from the HTML is not evidence.

Holes have a few shapes, and the shape is the first question, before any name.

Shape	Likely meaning	What to do
A few bytes, `90` or `CC` or zeros, between a `RET` and a prologue	Alignment	Leave them. Do not make them a function.
A run that starts with `C8` (`ENTER`)	A function the work list never reached	The first forced pass, below.
A run that starts with `55 8B EC` or the export prologue	The same, for a compiler that does not use `ENTER`	The second forced pass.
A run of word-sized values, dense, after an indirect jump	A jump table	Define words. Do not disassemble.
A run that contains the bytes `14 00` as an immediate, a `MOV CX, 0x14`, or a displacement	Not a record parser	Chapter 4. Thirteen such runs were counted. None of them is the far-pointer compare that builds an object.
A long run with no prologue and no table	Unread	Leave it unnamed. Forcing it because it is long manufactures code.

The `14 00 00` search is the cautionary case for this whole appendix. The bytes were found. They were found in the wrong instruction class. A theory that says "the object is three words of `0x14`" will light up every immediate, every short move, and every jump that happens to encode that displacement. The disassembler is not on your side here. It will decode those bytes as whatever the seed made them. Your job is to ask which instruction contains them. An immediate inside a call is a call. A displacement is a branch. A compare against a far pointer is the thing the theory was

about. Only the last one would have mattered, and it was not in the image that was searched.

K.7 Forcing the listing to finish

Two Script Manager passes force the listing to finish. Run them in this order. They do not name anything, and they do not touch code that is already defined.

1 The `C8` pass. Walk undefined bytes that are `C8` (`ENTER`) and create a function there.
2 The prologue pass. Walk undefined bytes. Create a function at `55 8B EC`, at the Win16 export prologue `45 55 8B EC 1E 8E D8`, at the byte after a real `C3` or `CB`, and at every NE entry that has no function. Accept a real `RET` or `RETF` before treating the following byte as code. Create functions only at entry-like symbols.

Run the `C8` pass first. The prologue pass is what runs second. Reversing them defines the wrong spans.

`ENTER` (`C8 iw 00`, in the common form) is the instruction a compiler emits for "make a frame of this many bytes." It is rare as the first byte of a data table and not impossible. The pass can be wrong. The protection is the rule that already exists: do not redefine a function that already exists, and do not treat the new function as named. After the pass, a new function that has no `RET` or `RETF` before the next defined range is a suspect. Clear that one. Do not clear the pass.

Theory: why the order matters. The prologue pass is greedy about the byte after a return. If it runs first, it can start a function one return too early and swallow an `ENTER` that belonged to the next function. That `ENTER` is then an interior byte. The `C8` pass, running second, sees a defined byte and walks past it. The function boundary is wrong, and the second pass cannot repair it because of the rule against redefining. Running `C8` first plants the seeds that are the most specific. The prologue pass then fills only what is still undefined. Specific seeds first, general seeds second. That is the same discipline as the entry table itself: trust the program's own list before you trust a pattern.

Thought, on the export prologue. `45 55 8B EC 1E 8E D8` is `INC BP`, `PUSH BP`, `MOV BP, SP`, `PUSH DS`, `MOV DS, AX`. Windows stack walking has long treated a saved `BP` whose low bit is set as a far frame. `INC BP` before the push is what sets that bit. A function that begins this way was written to be entered from outside the module, or at least to look like one. It is a better seed than a bare `PUSH BP`. It is still only a seed. The purpose is not "exported helper" until an entry or a far call says so.

The near prologue `55 8B EC` is `PUSH BP`, `MOV BP, SP`. Every compiler emits it, including compilers emitting the body of a function that is nothing but a comparison. It is the weakest of the seeds. Use it because the alternative is a hole. Do not use it as a reason to name.

K.8 SLEIGH, pcode, and the C you are reading

The bytes do not become C. The bytes become an instruction. The instruction becomes pcode. The decompiler simplifies pcode and prints something that looks like C. Four representations, and only the first one is the program.

Representation	Who produces it	What it is allowed to be
Bytes	The unpacked image	The evidence.
Assembly	SLEIGH, from the bytes	A reading of one instruction. Wrong if the seed was off by one.
Pcode	SLEIGH, with the assembly	The effects of that instruction: loads, stores, arithmetic, branches.
C	The decompiler, from simplified pcode	A hypothesis about a whole function, under a set of type guesses.

Simplification is why the C does not match the assembly line for line, and why that mismatch is not automatically a bug. Copy propagation removes a register that was only a temporary. A test and a branch become `if`. A flag set by `CMP` is folded into the comparison, so the `CMP` itself may not appear as a statement. Dead-code removal drops an instruction the decompiler decided could not matter. When the decompiler is right, the C is shorter than the assembly and easier to read. When it is wrong, the C is shorter than the assembly and missing the only instruction that mattered.

Appendix I is the procedure for that fight. This appendix only adds the reason you will have the fight: the C is downstream of two guesses, the guess of where instructions start and the guess of the types of the values those instructions move. Fix the guess. Do not edit the C in the notes and leave the database alone. The next export will overwrite your correction with the same guess.

Theory: a consistent lie is one cause. `SENDMESSAGE` was mangled at 321 sites in one RTM listing, and `LSTRLEN` at 282. The same nonsense at every site is not 321 separate misreadings. It is one signature, or one pointer size, applied everywhere the analyzer propagated it. The response is to stop, fix the type or the calling convention, and decompile again. The response is not to "correct" the arguments in the notes, site by site, while the database still holds the bad signature. The next person will export HTML and undo the corrections without knowing they were corrections.

Far pointers are the usual cause on this image. A far pointer is two words, segment and offset. A type system that wants one pointer will glue them, split them, or leave both as `undefined2`. The listings are full of `undefined2` and of values carried as a pair of words. That is what an untyped 16-bit value looks like after decompilation. It is not, by itself, evidence of a structure. A structure is a layout you have checked against more than one allocation site. Chapter 4's `0x76` and chapter 19's heap header are structures because the allocation and the field writes agree. Two `undefined2` parameters are not a structure. They are a far pointer the decompiler declined to name.

K.9 The string magnet

Ghidra finds runs of ASCII and defines them as strings. A later analyzer looks at immediates and, when an immediate equals the address of something already defined, it plants a reference. The decompiler then prints the name of that something instead of the number.

This is often what you want. A call that passes the address of an error string should show the string. It is also how a number becomes a fiction. The Ghidra address space for these modules is flat: segment bases at `1000`, `1008`, and so on, eight apart. An offset in one segment can be numerically equal to the address of a string that lives at that offset in a low segment. The analyzer does not know about the segment the instruction was using. It knows the number. The number matched. The reference is planted.

Chapter 4 already has the case. The string arithmetic in one decompilation is Ghidra matching constants against `s_CTbkSimpleOLEDispatch_CallMethod`. It is not a string compare in the program. The same magnet produced `s_DeleteBSTR` beside a call that is not `DeleteBSTR`. When a string name appears in a place that does not load a string, ignore the name and read the immediate. The immediate is the operand. The string is the analyzer's comment, printed in the slot where the operand should be.

Practice. If the instruction is `PUSH` of an immediate, and the called function is a string function, the magnet is probably right. If the instruction is an arithmetic compare, a function-pointer load, or a `GetProcAddress` ordinal that the decompiler rendered as a string, the magnet is a suspect. `GetProcAddress` in the `VBX*` wrappers takes an offset into a table of export names (`VbxInit`, `VbxNewBook`, and the rest). There the string is real, and it was checked against the file, not against the decompiler's label. That is the standard: a string claim is Known when the bytes at that file offset are those characters. A string claim that exists only as a Ghidra label is not yet a claim.

K.10 Theories of a name

A name is a key. It is not a paragraph. You will type it into a search box hundreds of times. A wrong key is worse than `FUN_`, because a search for the wrong key returns the wrong function with confidence, and a search for `FUN_` at least looks unfinished.

These are the ways a name can be earned. They are not interchangeable. A later way does not promote an earlier guess. It replaces it, or it fails to.

1. The binary named it. Export tables, import tables, and the strings passed to `GetProcAddress` when those strings have been checked in the file. This is Known. You do not improve `CDBFILEOPEN`. You can describe it.

2. The body matches an idiom you have already confirmed on another function. A GDI call sequence that is the same as `PaintRectangle`, a `GLOBALALLOC` of a size you have already tied to a structure, a `SENDMESSAGE` whose message number you have looked up. This is a candidate. On 11 October 2026, 1,918 of 10,991 functions were nameable this way, 17.5 percent. Confirmed renames after the later digs were 112. The gap is unreviewed candidates, not unknown programs. A candidate that has not been reviewed is not a name. The rename scripts in `TBdecompile/rename/` are what was applied. They rename only when the old name is still `FUN_…`. They do not invent the rest.

3. A unique constant selects it. An error code, a magic, a size that appears at one allocation and one set of field writes. `0x1F7A` is the out-of-indexes failure. `0xCCDD` is the heap magic in the model, because it was the magic in the reading. A constant that appears in fifty functions selects nothing. A constant that appears in one function, in a compare or a store and not as a displacement, is a reason to read that function. It is not yet a name.

4. The callers agree. Every caller you have read passes the same kind of argument and uses the result the same way. This is a theory of purpose leaking into the name. It becomes a name only after the body agrees. Callers lie by omission. They show you the use on one path.

5. A debugger stop put it on a path you care about. The function ran during open, or during paint, or during a script push. You now know a purpose on that path. You do not know the other paths. `TBHEAPCLEARCRITSECT` was a stop on the open path. It was not the last stop before the window. A name that says "the function that opens the book" would have been false. A note that says "it ran during open" is true.

6. A pattern matcher said so. The weakest source. Coverage by module, pattern only: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6. CMP is low because most of it is the compiler, not a wrapper around a Windows API. A useful CMP name is `COMPILEREXPANDTEXT`. A name on a function that only moves a token is not. Naming another thousand `StrCpy` wrappers will not find the record reader. That was a counting goal in an 11 October note, not a research goal.

Theory: names should be boring. `PaintPolygon` is a good name if the body paints a polygon. `HandleObject` is a bad name even if the body handles an object, because the search box cannot tell it from the hundred other functions that also handle an object. Prefer the verb and the operand the body actually uses. Prefer the export's own stem (`CDB`, `OBJ`, `VALUE`, `RESMGR`, `TBX`, `VBX`) when the function is part of that family, so a search for the stem finds the family. Do not invent a new stem. The record has been careful about this: `RESMGR` type words are the words in the binary, not words chosen because they sound like resource kinds.

Theory: a thunk takes its name from the callee, never the other way around. The `TBX*` wrappers that are one call and a return exist so one calling convention can reach another. Their purpose is the call. Name them as the wrapper. Do not rename the callee to match the wrapper, or every internal caller of the real function changes meaning in your notes.

Theory: leave it unnamed. An unnamed function is easier to correct than a plausible wrong name. The discomfort of a `FUN_` in a chapter is useful. It marks the sentence as a sentence about an address. The moment the function is named, later chapters will treat the name as a word in the program. Chapter 11 had to undo a symbol-map line that called `FUN_11a8_04f6` by the export name `OBJFILEOPEN`. The export is a different offset. A wrong name in a map survives longer than a gap.

K.11 Theories of a purpose

Purpose is the smallest sentence that is still true of every caller you have read. It is not the nicest sentence that is true of the caller you like.

Role. Ask what the caller could not do if this function returned immediately. That is the role. If the answer is "it could not allocate," the role is allocation, even if the function also stores a magic. If the answer is "nothing, on this path," you have not found a purpose. You have found a call that might be a no-op, a probe, or a path you do not understand. Write that down as the state of the reading. Do not promote it to a verb.

Layer. Purpose lives in a layer. The layers in this program, as the imports and the bodies have sorted them, are roughly: the book API (`CDB*`), the object file (`OBJ*`), the script heap and the value stack (`HEAP*` / `TBHEAP*`, `STACK*`, `VALUE*`), the resource session (`RESMGR*`), paint, the script bytes, the VBX host (`VBX*`, which mostly calls `TB80VBX.DLL`), and the C++ edge (`TBX*`). A function that calls `BitBlt` and also allocates is a painter that allocates. Name and describe it in the paint layer. Do not file it under the heap because the heap call is the one you recognized.

Failure. The error a function stores is often a better purpose-seed than the success path. A function that returns null and stores `0x1F7A` exists, in part, to refuse a request the heap cannot satisfy. The success path is "here is a value." The failure path is "the index space is exhausted." Both belong in the sentence. A purpose written only from the success path will send the next reader looking for a parser when the interesting behavior is the refusal.

Frame. The frame size is a hint, not a name. `ENTER` of a handful of bytes, followed by one call, is a thunk or a setter. A frame of a few hundred bytes that calls no imported API is a walker of a buffer, or a parser, or a function with a large local table. You cannot tell those apart from the size. You can tell that it is not a one-line wrapper, so a wrapper-pattern name would be a lie. Use the frame to choose which functions are worth a long reading. Do not use it to title them.

Path. A debugger stop assigns a purpose on that path only. Open, paint, script, author, save. A function on the open path may also be on the save path and do a different half of the work. `CDBFILEOPEN` and `OBJFILEOPEN` are not the same function and not the same path into the file. Chapter 20. A purpose sentence should name the path it was observed on, or it should say "the body, regardless of path" and mean that the body was what was read.

The unpublished helper. Most of the program is unpublished. Helpers exist because the compiler split a function, or because a size check was shared, or because far calls are expensive and a near helper sits in the same segment. Their purpose is often "the three instructions the caller did not want to repeat." That is a real purpose. It does not deserve a grand name. A plate comment with the caller's name is enough until a second caller, in another subsystem, forces a real word.

Theory: purpose is stable under renaming, and renaming is not stable under purpose. You can change your mind about the purpose and the bytes do not care. You cannot change the purpose and keep the name without lying to your future search. When the purpose sentence changes, change the name in the same pass, or change the name back to `FUN_` until the sentence settles. A stale name is a purpose claim you no longer believe.

K.12 Theories of an operation

Operation is the ordered effect of the body. It is the only one of the three jobs that the bytes can settle by themselves.

Read it in this order. Skipping a step is how a helper gets the name of its caller.

1. Span. Where does the function start, and where does it return? One function, one span. A span that falls into the next prologue is two functions. A span that starts one byte late is not a function yet. Check the bytes at the start against the prologue you think you seeded.
2. Entry. Export ordinal, near call, far call, or a pointer. The entry decides the calling convention you are about to assume. A far export and a near helper do not use the stack the same way. If you assume one convention for both, the C arguments are a fiction from the first parameter on.
3. Saved registers. `BP` frame or not. `DS` reloaded or not. `SI` and `DI` saved or not. This tells you which values can outlive a call the function makes. It also tells you whether the export prologue ran.

4 Calls you already trust. List them by the names that are already Known. Ignore `FUN_` callees on the first pass, except to count them. A function is often explained by the three Known calls it makes and by nothing else.

5 Stores that outlive the call. A write to a field of an object, a store of an error code, a store through a pointer argument. These are the effects a caller can observe. Locals that die at the `RET` are not part of the purpose, though they may be the mechanism.

6 The return value. Which register, which flag, and whether the caller tests it. A function whose callers never test the return is not "returning the object" just because `AX` holds a pointer at the `RET`. Look at one caller.

7 Only then a sentence. One sentence of operation, graded Known if you read the body, and one sentence of purpose, graded Thought until something else agrees.

Theory: the operation is a trace through the body, not a summary of the name. Write the stores and the calls first, in order, and write the English second. If you write the English first you will select the instructions that match the English and call the rest compiler noise. Compiler noise is where the off-by-one lives. Chapter 19's heap is the example to imitate. The sentence came after the sizes, the tags, and the error codes. The tags were not invented to make the sentence round.

Theory: a decompiler artifact is an operation you must delete from the sentence. `*(uint *)0x0` in a listing is the decompiler losing a register. A string name that is not a string load is the magnet. An argument list that is identical nonsense at every call to one API is the bad signature. These may stay in the HTML. They do not stay in the chapter. The chapter says they are artifacts, or the chapter does not mention them.

K.13 The order of searches that does not lie

When you do not know what a function is, search in an order that cannot flatter you. Each step is allowed to promote a function to "worth reading." None of them is allowed to name it except the first.

1 Exports and imports. If it has a published name, stop. Read the body to describe it. Do not rename it.

2 Checked strings. File offsets you have dumped, not labels in the listing. A `GetProcAddress` name, an error string, a window class.

3 Unique constants, in the right instruction. Compare, store, allocate. Not a displacement. Not the immediate of an unrelated call.

4 Callers of a Known function. Who calls `VALUENEWLONG` tells you who builds a long. It does not tell you that those callers are parsers.

5 Callees of the function you are reading. Three Known callees are a hypothesis. Zero Known callees means you are in a desert. Read more carefully, or come back with a better seed. Do not name the desert after the nearest town.

6 Prologue shape. Near, far, `ENTER`, export prologue. This chooses the convention for the C. It does not choose the name.

7 Pattern lists. Last. And only as a queue of functions to review, never as a list of names to apply in bulk because the count looks low.

Reverse this order and the work feels faster. You will produce names. They will be the names of the patterns, which are the names of the Windows API, which are not the names of the program. UTL at 41.1 percent pattern-nameable is the trap. A high score means the module is a wrapper. Wrappers are the easy part. The reader, if it is anywhere, is in the part the patterns do not cover. CMP at 5.6 percent is the more honest number.

Theory: search for the failure first. On a function you suspect is interesting, find the store of a nonzero error or the early return. The success path is longer and full of moves. The failure path is short and says what the function refuses to do. Refusals are specific. Success is often "copy the arguments somewhere." The record reader, whatever it is, will refuse malformed input somewhere. A search that only looks for successful parses will find every copy loop in the module.

K.14 What not to name, and what not to conclude

Do not name a function because the neighborhood is named. Neighbors share a segment because of the linker, not because of a theme. A paint helper and a string helper can sit next to each other if they were compiled in the same file. The segment is not a chapter.

Do not name a function from one caller. The caller has a purpose. The callee may be shared.

Do not conclude that a function is absent because the HTML does not contain it. Force the listing, or say that you searched the HTML and not the image.

Do not conclude that a byte sequence is a structure because it appears in a hole. Ask the instruction class.

Do not conclude that a function runs on open because it is an export, or that it does not run because a trace you stopped early did not reach it. `TBHEAPCLEARCRITSECT` was not the last stop. A later stop does not erase the earlier one. An earlier stop does not crown a function.

Do not "correct" mangled arguments in prose while the database still has the bad type. Decompile again.

Do not disassemble the packed file and call it the program.

Do not mix a live selector with a Ghidra base in the same sentence without the map.

Do not treat a counting goal as a research goal. 1,918 candidates was a count. The reader was not going to be candidate number 1,919.

Ghidra has a function-ID library that hashes a body and proposes a name from a catalogue of known libraries. These modules are not in that catalogue. A hit, if you ever turn the analyzer on, is a coincidence of instructions, not a port of some other program's function into ToolBook. Leave function ID off, or treat a hit as a joke you can look at later. It is not a source of names. This is advice. The project did not need a hit from that library to do the readings in the record, and it should not acquire one.

K.15 Scripts, exports, and the HTML as a snapshot

A readable decompile is not one rename of the project. Each pass has a thing it is allowed to change.

Pass	What it may do	What it must not do
1. Finish the listing	The two passes in K.7	Name a function. Redefine a function that already exists.
2. Mark candidates	Note a body that matches a pattern	Treat the pattern as the name.
3. Confirm a name	Rename when the body matches, by hand or by a script in `TBdecompile/rename/`	Rename a function whose body does not match. Invent names for the functions that do not match.
4. Replace the reading copy	Export HTML again, with the date in the filename	Search yesterday's HTML for a name you applied today. That file still says `FUN_`.

The HTML export is a snapshot of the database, not the database. Searches you can repeat belong on the snapshot only if you name the snapshot. A claim that "the listing does not contain X" must say which file, which date, and whether the forced passes had been run. Otherwise the claim is about your export settings.

The scripts are idempotent on purpose. A second run finds no `FUN_` left to rename, so it does nothing. That is the property you want from any later script. A script that renames whatever is at an address, regardless of the current name, will overwrite a correction you made by hand and will do it quietly. Refuse that script. The address might have been wrong. The hand correction might have been the only careful reading in the segment.

Java and Python rename scripts live in `TBdecompile/rename/`. Copies of the nameable TSV files live under `TBWorldview/Modules/tb80*_knownnames/`. The TSV is the candidate list. It is not the script. Running the TSV as

if it were the script is how a pattern becomes a name without a review.

Ghidra `CodeN` is not the NE segment number. Use `TBdecompile/maps/` before typing a segment into a debugger. The same map is what a script should use if it is going to mention a segment in a comment. A script that writes `Code47` into a note has created a name that dies when the project is reloaded into a different address layout. Write the NE segment, or write the offset only.

K.16 One session

1. Unpack. Apply the entry-table post-step only for an image that needs it. BAS does. The others, on this set, do not.
2. Open the image in the patched Ghidra 12.1.4 build. Confirm it is the 16-bit language. A 32-bit load of a 16-bit segment will look busy and be nonsense from the first call.
3. Let analysis run. Do not treat the end of analysis as a finished listing. Count entry symbols against the NE entry table. Stop if the count is short.
4. Run the `C8` pass, then the prologue pass. Spot-check one new function for a real return. Clear the one that does not have one.
5. Check one export you already trust, such as `CDBFILEOPEN` (ordinal 8), against `maps/` before you trust the segment name on the screen.
6. If one imported API is mangled at every site, stop. Fix the signature. Decompile again. Do not start the naming pass on top of a bad convention.
7. Read one function by the order in K.12. Write the operation first. Write a purpose sentence second, and mark it as a theory.
8. Rename that one function, or run the rename scripts for a family you have already reviewed. Do not run a pattern list as if it were the script.
9. Export a new HTML if that is the copy you search. Keep the old file. It is the trail of what the listing used to say.
10. If the C and the bytes disagree, Appendix I wins. The decompiler does not get a vote against the bytes.

Come back when a chapter has used a `FUN_` name in a claim. The cleanup for that function is: confirm the span is a real function, read the body, and only then replace the name. Do not rename the neighborhood because the neighborhood is unnamed.

K.17 A method, applied to functions the record already settled

These are not new findings. They are the method above, run backwards on names that are already in the record, so the next unnamed function has a shape to imitate.

`CDBFILEOPEN`, ordinal 8. The binary named it. The job is description, not renaming. The description in chapter 20 is that it does not call `OBJFILEOPEN` by name. The decompile reaches the object file through unnamed helpers. The purpose of the export is "open," because that is what RTM links it for. The operation includes a path that is not a direct call. Both sentences can be true. A theory that said "the open export is the open routine, so it must be the one that reads the header" would have collapsed them, and would have sent the search to the wrong function.

`VALUENEWLONG`. The name is the binary's, or the family's. The operation is the construction of a long, a 4-byte block, tag `0xB`, on the script heap. The purpose is "make a script value that is a long." It is not "parse a record," even if a record parser would have to call it. The constant `0xB` is a tag, earned by being the second argument at the allocation, not by being a number that appears somewhere in the function. That is the difference between a unique constant used in the right instruction and a byte that merely occurs.

`VBXINIT`. The export name is a seed for a search, not the end of one. The body loads `TB80VBX.DLL` and calls `GetProcAddress` for `VbxInit`. The operation is a forwarder. The purpose is "start the VBX host that lives in the other

module." A theory that said "VBXINIT constructs a VBX object" would be wrong. The view object, `0x11C` bytes and the magic `0xABC1`, is `VBXNEWVIEW`, and that one is implemented in BAS. The method separates them because the bodies are different, not because the names share a stem. The stem made them neighbors in the search box. The bodies decided which one allocates.

The thing that is still `FUN_`. Apply the same steps and stop where the evidence stops. If you have a span and no trusted callee, you have an operation you have not finished, and you do not have a purpose. Leave the name. A chapter may cite the address. The appendix you are reading is the reason that citation is allowed to look unfinished.

K.18 How to read the names Ghidra already printed

Before you add a name, read the names the tool invented. They are a classification, not a caption. Each prefix is a theory the database is willing to state in public.

Prefix	What the database is claiming	What it is not claiming
`FUN_`	A function symbol at this Ghidra address. The digits are the base and the offset.	That anyone has read it. That the span is right.
`LAB_`	A branch target inside a function.	A function. Do not rename a `LAB_` as if it were a callee.
`DAT_`	A data symbol at an address.	A type. `DAT_` is "something lives here." The size is a separate guess.
`s_`	A string the ASCII pass defined, with a mangled prefix of the characters.	That any instruction loads it. The magnet may have glued it on later.
`switchD_`	A jump the decompiler decided was a switch.	That the jump table was checked by hand.
`unaff_`	A register used in the function and not written by the function. Incoming.	That it is junk. It is often a parameter the signature missed.
`extraout_`	A register a call wrote, which the declared return type does not include.	That the call failed to return. The type is short of the truth.
`in_`	An incoming register the convention accepted as a parameter.	That the convention is the one the compiler used.
`local_`	A stack slot.	That the source had a local variable with a meaning.
`param_`	A stack slot the decompiler thinks the caller passed.	That the caller pushed it. Check the call site.

`unaff_SS` and `unaff_CS` are the usual 16-bit artifacts. The stack segment and the code segment are real, and they are almost never "the argument." An `unaff_BX` or `unaff_DX` beside a call that does not push enough words is a different matter. Watcom and other compilers of this era could pass arguments in registers. This project has not established which compiler built which module, and this appendix will not invent one. The operation-level question does not need the vendor's name. Look at the call site. If the caller loads `AX` and `DX` and then calls, and the callee consumes `AX` and `DX` without reading them from the stack, those registers are the arguments. The decompiler will call them `unaff_` until the signature says otherwise. Promoting them to `param_` is a type edit, in the database, followed by a fresh decompile. Writing "the first argument is BX" in a note, while the HTML still says `unaff_BX`, is a correction that the next export will not remember.

`extraout_DX` next to a function that returns a far pointer is the high half of the pointer, sitting outside a return type that was declared as a single word. The fix is the return type, not a paragraph about `DX`. After the type is a far pointer, a new decompile should fold the pair. If it does not, the call is not returning a far pointer and the theory was wrong. That is a cheap experiment. Do it before the theory reaches a chapter.

`param_` slots that the caller never pushes are the mirror image. The decompiler assumed a stack convention and invented homes for values that arrived in registers, or it assumed too many arguments because an earlier signature said so. One bad signature on an imported API does this at every site. Section K.8 already said to stop and fix it. The prefix is how you notice. If every call to one import has the same extra `param_`, the import is the bug.

`local_` variables with an address of the form `local_4`, `local_6`, climbing by two, are the frame. Their number should be in the same neighborhood as the `ENTER` immediate or the `SUB SP, n` you can see in the bytes. If the C has twelve locals and the prologue subtracts four, the decompiler has invented stack slots out of registers. Trust the prologue. Appendix I.

K.19 What the analyzer queue is willing to believe

Auto-analysis is a queue of theories, each allowed to act, each ordered by a priority you do not see in the HTML. When the queue is empty, the tool stops. It does not mean the theories were true. It means none of them had a further action it was willing to take.

The theories, in the order they matter for this kind of image, are roughly these. The names here are the ideas, not a menu path. A menu path that was not written down is not invented.

Disassemble from seeds. The work list of K.3. Empties when every direct target has been turned into an instruction or has been refused.

Find strings. Runs of printable bytes become `s_` symbols. This runs early, which is why the magnet in K.9 has something to stick to. A string found in a code hole may be a real string that a function returns, or it may be bytes that happened to be printable. A string with no reference after analysis is a candidate for "not a string." Do not delete it on the first session. A later seed may be the function that loads it.

Create functions at call targets. A direct call is a strong seed. The callee gets a function symbol even if you never asked. This is why exports and internal callees acquire `FUN_` names without the two forced passes. It is also why a bad decode that contains a byte pattern looking like a direct call will spawn a function at a nonsense address. The child function is not evidence. The call instruction is the evidence. Check the bytes of the call.

Search for prologues in what is still undefined. Ghidra has its own version of this idea, and a more aggressive variant that the tool itself treats as dangerous because it false-positives. The `C8` pass and the prologue pass are a narrower form of the same idea: specific bytes, undefined ranges only, no redefinition. Prefer them to a switch that sweeps every hole. A sweep is how a jump table becomes code.

Recover switches. One analyzer asks the decompiler which jumps look like switches and then defines the tables. When it is right, a `switchD_` name appears and the cases become a body you can read. When it is wrong, a table of offsets has been consumed and the real cases are split. Treat `switchD_` as a theory. The check is the same as any table: the bytes are word-sized, dense, and in range for the segment. If they are not, clear the switch and define words.

Stack and types. Frame size, locals, parameters, then the propagation of any type you have applied. This is the pass that turns one corrected `SendMessage` into 321 readable call sites, or into 321 sites that are wrong in a new way. After a type change, this is the pass you are re-running, whether you think of it that way or not. Budget a fresh export. Do not compare the new C to your memory of the old C. Compare it to the bytes.

Constant references. The magnet. Last in spirit even when the priority number says otherwise, because it depends on strings and data existing already.

The queue will not do the following, and you should not expect it to.

It will not read a far pointer built in two registers and disassemble the target. It will not follow `GetProcAddress`. It will not apply NE relocations that the image has not already applied. It will not notice that two `FUN_` bodies are the same pattern of moves. It will not know that ordinal 8 is the open you care about. Those are your seeds, your maps, and your names.

Theory: a tool's refusal is information. Ghidra stops at an indirect call because it does not have a target. That refusal is more honest than a linear sweep, which would have invented a target by falling into whatever bytes came next. When the listing shows a hole after a return, the tool is saying "I will not pretend." The forced passes are you deciding, on a narrow pattern, to pretend a little. Keep the pretense narrow. The moment the pattern is "any undefined byte," you have thrown away the refusal that made the listing trustworthy.

K.20 Calling conventions are a theory of the stack

A calling convention is a contract between the call site and the body. The decompiler has to assume one in order to print parameters. If the assumption is wrong, every parameter in that function is wrong, and every sentence you write about "the first argument" is about a stack slot the caller may not have pushed.

On a Win16 module the contracts in play are at least these. Which one a given function uses is a question for the bytes, not for the vendor's manual.

Contract	What the call site does	What the return does	What the decompiler gets wrong if it assumes another
Near, caller cleans	Pushes words, then a near call. The caller adds to SP after the call.	RET with no immediate.	It will show the callee popping arguments the callee does not pop.
Near, callee cleans	Pushes words, then a near call. The caller does not adjust SP.	RET n.	It will show the arguments as locals of the caller, still on the stack.
Far, callee cleans	Pushes words, then a far call.	RETF n.	A near assumption splits one far return into a near return and a stray immediate.
Registers	Loads AX, DX, BX, CX and calls, with few or no pushes.	Values in AX or in AX:DX.	It prints unaff_ and invents param_ slots nobody pushed.
Far pointer return	The caller uses both AX and DX.	AX is the offset, DX is the segment, or the reverse. Check one caller before you pick.	It keeps extraout_DX forever if the return type is a single word.

The practice at one function is short. Look at three call sites if you have them, or at one if you have only one. Count the pushes between the previous call and this one. Count the RET immediate, in bytes, not in words: RET 4 pops four bytes, which is two near words. If the pushes and the RET immediate disagree, you are not looking at a simple contract. Someone passed extra words, or the decompiler combined two calls, or the span is wrong. Do not paper over a disagreement with a name.

RETF with no immediate means a far function whose caller cleans, or a far function with no stack arguments. Both are real. The export prologue of K.7 is a hint that the function expected to be entered from outside, which on Windows is often far and callee-cleaned. The hint is not the contract. The RET immediate is the contract.

Theory: fix the convention at the function, then at the import, and never in the prose. A custom convention on one function stays on that function. A convention on an imported API is stored with the external symbol, and every caller in the module inherits it. That is the lever behind the 321 identical SENDMESSAGE sites. It is also the lever behind a single internal function whose C is nonsense while its neighbors are fine. Change the smaller thing first. If one function is wrong, edit that function's signature. If every call to one import is wrong, edit the import. Then decompile again and read the bytes of one site before you trust the other three hundred.

K.21 Relocations, ordinals, and other numbers that are not yet addresses

A relocating segment is a segment the loader is prepared to patch. The NE relocation table lists places in the segment where a segment number, an offset, or an imported ordinal has to be filled in when the segment is mapped. The unpacked image is the right file to disassemble. It is not, by that fact alone, a file in which every immediate is the runtime address.

Before you treat a far target as a place you can open in the listing, know which of these you are holding.

What you see	What it might be
A far call whose target is a FUN_ you can click	A direct call the disassembler already followed. The seed worked.
A far immediate that is not a code address in this project	A fixup that was not applied, a segment number stored as an immediate, or the magnet.
A call through a register	Not an address. Do not invent the callee from the register's last store unless that store is a direct load of a known function.

What you see	What it might be
An import by ordinal	A function in KERNEL, USER, GDI, or another module. Appendix E is the ordinal list this manual carries for the three system libraries. Resolve it before you describe the call. "Calls KERNEL" is not an operation. "Calls `GETMODULEHANDLE`" is, once the ordinal says so.
`GetProcAddress` of a checked string	A runtime bind. The callee is the string, in the module that was loaded. `VbxInit` is the case already checked. The decompiler will not disassemble into `TB80VBX.DLL` because of that call. You open the other module.

Theory: a number in a listing is an address only after you have said which address space. Offset in the segment, Ghidra base plus offset, live selector plus offset, ordinal, file offset. Five spaces. The relocation table is how the program itself translates between some of them at load time. If you skip it, you will eventually add two numbers from different spaces and get a `FUN_` that is not in the file. The maps exist so that translation is written down once. Use them. Do not do the arithmetic in your head and then forget which side you started on.

Ordinals deserve a second warning. An import recorded as "KERNEL, ordinal 49" does not contain the letters of the function's name anywhere in the module. A string search will not find it. The only honest resolution is the ordinal table, checked against a source you trust, which for the system libraries is Appendix E of this manual. A function whose only interesting call is an unresolved ordinal is a function you have not started to read. Resolve the ordinal first. The body will be shorter afterwards, because the call will mean something and the moves around it will turn into setup for that call.

K.22 A segment is sometimes a compilation unit and usually is not

Chapter 20 counted 29 `OBJ*` exports. Twenty-six of them sit in one NE segment, Ghidra `1170`. That neighborhood means something, because the prefix was already a name and the linker piled one body of code into one segment. The inference ran from the names to the segment. It does not run backwards.

Theory: count the prefixes before you infer a theme. Take the exports of one segment, which are the names the binary is willing to defend. If they share a stem, the segment is probably one compilation unit or one class, and a helper in that segment is a candidate for the same layer. If the exports of the segment have three stems, the segment is a bag the linker filled, and a helper's neighbors tell you nothing. Internal `FUN_` symbols do not vote. They have no stem until you invent one, and inventing one in order to make the count come out is the fallacy the theory exists to block.

The practical consequence is where you spend a long reading. A segment whose exports are all `VALUE*` is a place to study values. A segment whose exports are mixed, with a hole in the middle, is a place to apply K.6 and K.7 and then leave. The hole is not "the rest of the value code" because it shares a segment with a value export. The hole is unread.

The same count is the right way to think about the C++ edge. `TBX*` names are a stem. A function with that stem that is one call and a return is a thunk, and its purpose is the callee. A function with that stem that has a real body is the place the work happens. The stem got you the list. The body length, the frame size, and the trusted callees sort the list. Sorting a stem by body length is a better afternoon than naming every `FUN_` in the segment the stem lives in.

K.23 Three witnesses, and which one is allowed to outvote

You will have, for any function you stay with, up to three witnesses. They disagree. The disagreement is the work.

The bytes, disassembled with a span you are willing to defend. This witness outvotes the other two on the question of what the function does. It cannot tell you the path that reaches the function on a Tuesday. It can tell you the function refuses a null pointer. If the bytes and the C disagree, keep the bytes. Appendix I is that rule written as a procedure.

The decompiler, which is a reader of pcode under type guesses. It outvotes your memory of a function you have not opened in a month. It does not outvote the bytes. It is allowed to suggest a structure. It is not allowed to settle one. Use

it to find the calls and the stores, then verify the calls and the stores against the assembly before they enter a sentence graded Known.

A debugger stop, SoftICE or the Watcom debugger, on a path you provoked. This witness outvotes the other two on the question of whether the function ran on that path. It does not outvote them on what the function does on a path you did not take. A stop during open is not a description of save. A function that did not stop during a short trace is not absent. The trace may have been stopped first. Chapter 19's note that `TBHEAPCLEARCRITSECT` was a stop, and not the last one, is the shape of an honest debugger sentence: it ran, here is where, and the window came later.

Write the witness into the sentence. "The body stores `0x1F7A`" is the bytes. "The C shows a call to `VALUENEWLONG`" is the decompiler, and it needs the bytes behind it before it is Known. "It was on the stack during open" is the debugger. A sentence that mixes them without saying which is which cannot be graded, and a claim that cannot be graded should not be in the record.

Theory: do not use one witness to decorate another. The bad habit is to take a debugger stop, find the nearest named function in the HTML, and describe that function's C as what ran. The stop gave you a selector and an offset. The map gives you a Ghidra address, if you do the map. The bytes at that address are what ran. The nearest named function may be the next export, a hundred bytes away, and it may be a function the stop did not reach. Distance in the listing is not proximity in time. The Calls window is the better debugger tool when you can get it, because it names the return address, which is a caller, which is a fact about the path. Even then, map the selector before you search.

K.24 False friends, functions that look like the thing you want

Every long disassembly grows a set of functions that look like the goal. This program already has a collection. They are listed here so the next one is recognized as a member of the collection, not as a discovery.

It looks like	It is	Why the resemblance happens
A parser of the page record	`VALUENEWLONG`, and the other `VALUENEW*` constructors	They allocate a typed value. A parser would call them. They are not the parser.
The open routine, one function	`CDBFILEOPEN` plus unnamed helpers plus `OBJFILEOPEN`	The export name says open. The operation is a chain.
The reader, because the bytes `14 00 00` are present	An immediate, a `MOV CX, 0x14`, or a displacement	The search did not ask the instruction class.
The last step before the window	Whichever heap call the trace had reached when you looked	The trace had not finished.
A VBX object constructor	`VBXINIT`	The name says init. The body is `GetProcAddress`. The allocator is `VBXNEWVIEW`.
A string compare	An arithmetic instruction the magnet decorated with `s_`	The immediate matched an address in a flat space.
The record reader, because it is unnamed and large	A compiler pass, a painter, or a thunk cluster	CMP is mostly unnamed because it is the compiler. Large and unnamed is the normal state of this image.

Theory: the goal-shaped function is usually the support act. The function you can find by a pattern is the function the pattern was built from. Patterns are built from Windows APIs and from the exports, because those are the names you already had. The function that implements a private format does not call a distinctive API. It moves bytes, compares a tag, and calls the support act. When a search returns a perfect match, you have found the support act. Start from its callers, not from its name. The callers that are still `FUN_` are the list. The callers that are exports are the door. The private function is the one in between that failed every pattern, which is why the pattern search did not return it.

This is not a claim that the record reader has been located by elimination. Elimination on 10,991 functions is not a method. It is a claim about where not to stop. Stopping at the first function that allocates, opens, or contains a familiar constant is how the support act gets the name of the feature.

K.25 Repeatability, the trail, and the database

A listing is a database with a date. The HTML is a print of it. The record cites the print, or it cites the bytes. It should say which.

The practices that make a later session able to check this one:

Keep the previous HTML. A claim that a name was absent is a claim about a file. If you overwrite the file, the claim cannot be checked, and it also cannot be distinguished from a claim about the program. The trail directories under `TBdecompile/` are that habit already. Do not tidy them into the manual. Do not delete them because the new export is prettier.

Date the filename of every export you intend to search. Searching an undated `TB80BAS.DLL.html` is how yesterday's `FUN_` and today's name get mixed in one sentence.

Prefer a script that refuses to rename a function whose name is no longer `FUN_`. The refusal is the audit. A script that always writes the name will erase a hand correction and leave no trace except the new name, which looks deliberate.

Do not clear a segment and re-disassemble it because one function is wrong. You will lose every type you applied and every span you forced, and the queue will refill the segment with the same theories as the first time, plus whatever the aggressive options happen to do. Clear the one function. If the bytes need to go back to undefined so the `C8` pass can see them, clear the code units of that span only, and know that you have done it.

Write plate comments for theories, and names for operations you have finished. A plate can say "thought: parser." A name `ParsePage` cannot be unread by a search. When the thought dies, the plate is edited and the search does not still find `ParsePage` in every chapter that copied it. This is the same rule as the record's grades. Thought is allowed to be wrong. A name is a kind of Known, whether you meant it to be or not, because later readers will treat it as one.

Theory: the database is not the program, and the manual is not the database. The program is the unpacked image. The database is one reading of it, with seeds and types and names layered on. The manual is the set of sentences that survived. When you are about to write a sentence, ask which of the three it is about. A sentence about the program can be checked against the bytes in a year, on a different machine, with a different Ghidra. A sentence about the database can be checked against the dated HTML. A sentence about the manual is a sentence about what you believed. All three are allowed. They are not interchangeable, and only the first one remains true if the database is lost.

K.26 A longer pass over one function, as a template

What follows uses `HEAPVALUETOPOINTER`, which chapter 19 already settled, as a template for the notes you take on a function that is not settled. The point is the shape of the note. The facts are the chapter's.

Span and entry. It is an export in the heap family, so the name is the binary's and the entry is a published one. You do not rename it. You do not look for a prologue in order to decide it is a function. The entry table already did that, once the loader was patched.

Operation, from the bytes the chapter records. The index is the low 10 bits of the value (`value & 0x3FF`), scaled by 2, and used to read a selector word in the heap. The offset is the other half of the value. WDW's first instructions on that export were `and si, 03FF` and `add si, si`, which is the debugger witness agreeing with the body. The sentence of operation is: a script value is not a pointer; it is an index and an offset, and this function turns them back into a pointer.

What the decompiler will do to it. It will be tempted to print the mask as a bitwise and, which in this case is faithful. It may also attach a string or a `DAT_` to one of the small constants if the magnet finds a match. `0x3FF` as a string would be the magnet, and you would ignore it. The `and` and the shift are the operation. They are simple enough that the C and the bytes should agree. If they do not, the span is wrong or the language size is wrong. That agreement, on a function

this small, is a good calibration of the project file before you trust it on a function that is not small.

Purpose, and the purpose it does not have. The purpose is "recover the address of a script value." The purpose is not "parse a record," not "allocate," and not "the inverse of every constructor." `HEAPPOINTERTOVALUE` is the other direction, and chapter 19 is careful that the low 10 bits of that direction were not a clean reading. A template note copies that caution instead of forcing the two functions to be perfect mirrors. When the reverse direction is muddy, say so. Do not tidy it so the pair looks designed.

What you would have done if the name had been `FUN_`. Mask and scale of a field is a unique-constant search (`0x3FF` on a value, in an `AND`, not as a displacement). The callers would be every function that holds a script value and needs a pointer, which is too many to name from. The purpose sentence would stay Thought until the heap chapter's other functions agreed: the constructors store the same shape the pointer function recovers. Agreement across several functions is what would promote it. One function with a mask is only a function with a mask.

Use that last paragraph as the bar. A single body can give you an operation. A purpose you are willing to put in the record should survive contact with a second function, or with a stop on a path. If it cannot, it is a plate comment.

K.27 Mistakes this listing has already paid for

They are collected so they are one list, not a set of surprises distributed through the chapters.

The packed file was the wrong image, because the relocation flag was clear on every segment but the stub. Disassemble the unpacked image.

Stock Ghidra dropped BAS entries whose segment byte was at or above 128, and it did not honor `ne_cbenttab`. The patch is on the loader. The image stays honest.

`CodeN` was treated as an NE segment number. It is not. The step is 8, with gaps, from `1000`.

A symbol-map line called a `FUN_` by an export's name. The export was a different offset. The map line is not the function.

The bytes `14 00 00` were found, in the wrong instruction class, thirteen times. The far-pointer compare was not among them.

`TBHEAPCLEARCRITSECT` was taken for the last thing before the window because it was the stop in hand. A later stop was `VALUEFREE`.

`CDBSENDCREATE` was taken for the open path because it is an export and its name says create. It is the author path.

The string `s_CTbkSimpleOLEDispatch_CallMethod` was taken for a compare. It was the magnet.

`SENDMESSAGE` and `LSTRLEN` were read site by site as if each mangled argument list were a separate fact. Each list was the same bad signature.

A pattern count was treated as progress toward the reader. 1,918 candidates did not move the reader.

None of these required a new kind of mistake. Each one was a witness used outside its authority: the wrong file, the wrong address space, the wrong instruction class, a stop that was not the last stop, an export that was not a path, a label that was not a load, a type that was not the program, a count that was not a finding. The theories in this appendix are the authorities written down. A new mistake will be a witness used outside them in a way the list has not named yet. When that happens, add the case. Do not add a name for it.

K.28 The bytes that move the work list

The work list is not a metaphor. It is the consequence of a handful of opcodes. When you are looking at a hole, or at a function that seems to end too soon, these are the bytes that caused it. The numbers are the instruction, not a Ghidra name.

Bytes	Instruction	What the work list does
`E8 iw`	Near call, relative	The caller falls through. The target is a new seed.
`9A ow sw`	Far call, absolute offset and segment	The caller falls through. The target is a seed only if that segment is in this image and the offset lands on code. A far call into another module is an import, not a seed in this file.
`FF /2`	Near call, indirect	The caller falls through. No callee is added.
`FF /3`	Far call, indirect	The same. This is the object slot and the function pointer.
`E9 iw`	Near jump, relative	No fall-through. The target is a seed.
`EB ib`	Short jump	The same, with a one-byte displacement. A displacement of `14` is one of the ways the bytes `14 00` appear without being a record.
`EA ow sw`	Far jump, absolute	No fall-through. Often a thunk into another segment.
`C3`	Near return	The walk stops.
`C2 iw`	Near return, pop `iw` bytes	The walk stops. `iw` is the contract of K.20.
`CB`	Far return	The walk stops.
`CA iw`	Far return, pop `iw` bytes	The walk stops. The export that forgot to be a seed often sits immediately after one of these.
`C8 iw 00`	`ENTER` of `iw` bytes, nesting level 0	Not a branch. A seed only if you plant one. The first forced pass.
`55 8B EC`	`PUSH BP` / `MOV BP, SP`	Not a branch. The weak prologue.
`45 55 8B EC 1E 8E D8`	The export prologue of K.7	Not a branch. A stronger seed, still not a name.

`/2` and `/3` are the ModR/M reg field, not a second opcode byte you can search for as the literal `02`. Searching the file for `FF 1E` or whatever the ModR/M happens to be will find some of the indirect calls and miss the rest. Do not do that search and call it a census of function pointers. Ask the disassembler, after the seeds are in, which calls have no target. That list is the list of unpublished callees. It is the closest thing this method has to a table of contents for the unread part of the module.

A far call's segment word is in the instruction. If that segment word is a fixup, the bytes in the file may not be the segment you will see at runtime. K.21. If the disassembler already turned the call into a `FUN_` in this project, the fixup was either applied or was unnecessary. If the call is still an immediate pair of words, you are looking at a number, not at a function, until the map or the relocation entry says otherwise.

Theory: the hole after a `CB` or a `CA` is the function the entry table was supposed to name. That is the ordinary shape of a Win16 segment: return, then the next function, with no call inside the segment pointing at it, because the callers are in other segments and the pointer was built at runtime or the call is far and the seed was dropped. The prologue pass that starts at the byte after a real return is doing this theory with a narrow pattern. When you look at such a hole by hand, do not ask "what calls this?" first. Nothing in the already-disassembled code may call it. Ask whether the bytes are a prologue. If they are, plant the seed. The callers will be found later, as far calls and as stops, or they will not be found. Absence of a near caller is not absence of a function.

K.29 When to stop reading

A function can be read forever. The decompiler will keep offering a local you have not explained. The rule for stopping:

You may stop when you can write the operation as the stores and the trusted calls, in order, and the bytes of those stores and calls match the C. You do not need to explain every `local_`. You need to explain every write that outlives the return, and every call whose name you are going to repeat in a chapter.

You should stop, and come back later, when the next fact you need is a convention you have not fixed, a segment you have not mapped, or an ordinal you have not resolved. More staring will not produce it. The sentence you write on the way out is "unread past the call to ordinal N" or "unread past the far immediate," which is a precise kind of unfinished. It is more useful than a guessed purpose, because the next session knows where to start.

You should stop immediately when you notice you are about to type a name in order to feel finished. That impulse is the 1,919th pattern. The function will still be at its address tomorrow. The wrong name will be in the search box until someone is brave enough to delete it.

K.30 Food for the next hole

A few questions, to ask of the next undefined run before anyone names it. They are questions, not findings.

Is this run undefined because the work list never reached it, or because a previous pass defined the bytes before it as data and stopped? The first is a missing seed. The second may be correct.

Does the first byte match `ENTER`, the near prologue, or the export prologue? If none of them, what positive reason is there to treat the run as code at all?

If you disassemble it, does a real return occur before the next thing you already trust? If not, the span is wrong or the run is not code.

Which of the three addresses are you about to write down? If you cannot say "NE segment" or "Ghidra base" or "selector from the boot of this date," do not write the number.

Which of the three jobs are you about to do? If you are about to invent a name in order to have a purpose, stop. Write the operation you can actually see, or write nothing.

Does the function store an error? Start there.

Does every call to one imported API in this hole look mangled in the same way as the rest of the module? Then the hole is not the problem. The signature is.

Does the interesting constant appear as a compare, a size, a store, or a displacement? Only the first three are about the constant. The fourth is about a branch.

Who imports this module, and do they import this entry? If they do not, the function can still be the one. It is not the one because of the import table. It is the one, if it is, because of the body.

Would a wrong name be worse than no name? It would. The search box has a long memory.

None of these questions names the record reader. They are the filter in front of the next claim that something is the record reader. A claim that survives the filter is a candidate for the record. A claim that skips the filter is a pattern, and this program has already produced 1,918 of those.

Appendix L. Bochs

Date: 2026-10-20

The command card for the host that opened the lesson. Chapter 7.6 is the claim. Chapter 4.6 is what the stops showed. `maps/bochs-20261020.md` is that boot's selectors. This appendix is how to do it again.

L.1 Why this host

Bochs runs the Win98 guest and holds the debugger outside it. A break does not need SoftICE, and it does not need a program inside Windows. Registers, the stack, and memory are what the CPU holds. The cost is speed.

L.2 This machine

Item	Value
Build	`~/bochs/bochs/build-dbg/bochs` (3.1, internal debugger). `build-rfb` is the VNC build and was not used.
Config	`~/bochs/vm/win98.bxrc`
Disk	`~/bochs/vm/win98.img` (2 GB flat, cylinders=4063, heads=16, spt=63)
Guest	Win98 SE, 128 MB, Pentium model, ips=50000000, i440fx, Cirrus VGA, PS/2 mouse
BIOS	`~/bochs/bochs/bios/BIOS-bochs-latest` and `/usr/share/bochs/VGABIOS-lgpl-latest`
Dumps	`~/bochs/vm/dump*/`

Start:

```
~/bochs/bochs/build-dbg/bochs -f ~/bochs/vm/win98.bxrc -q -unlock -debugger -dbglog ~/bochs/vm/debug.log
```

`-debugger` opens `<bochs:N>` in the terminal. `c` runs the guest. Ctrl+C returns to the prompt. `-unlock` clears a stale lock left by a killed Bochs. `-dbglog` saves what the debugger prints, including a long `info ldt`.

The guest window and the terminal are separate. Click the window for the mouse and the keyboard. Ctrl+F10 toggles the mouse. Read the terminal before the window. A missing BIOS is a panic there, not a Windows error.

L.3 Files into the guest

```
cd ~/<folder>
genisoimage -o ~/files.iso -J -R .
```

Point `ata0-slave` in `win98.bxrc` at that ISO with `status=inserted`, then restart. Copying a file back out of the guest has not been needed. The `genisoimage` flags were not re-tested this session.

L.4 Commands that worked

Command	What it does
`vb 0xSEL:0xOFF`	Break at a live selector:offset.
`info break`	List breakpoints. The numbers are what `bpd` and `bpe` use.
`bpd N` / `bpe N`	Disable or enable breakpoint N.
`c`	Continue. One command per line.
`r`	Registers. A 16-bit read is the low half: AX, DX, not EAX.
`print-stack N`	N stack words. Addresses are linear.
`info ldt A B`	LDT entries by decimal index. `2333 2333`, not `0x48ef>>3`.
`x /20xb ADDR`	Bytes at a linear address. Not selector:offset.
`writemem "file" ADDR LEN`	Save guest memory. The folder must already exist. Stops at the first unmapped page.
`source file`	Run a command file. Absolute path.

Not used, so not trusted: `del N`, `lb`, `trace on`, `show call`, `loadmem`. There is no debugger command for a full physical RAM dump. The GUI Suspend hung. Do not use it. Kill with `kill $(pgrep -n bochs)` and start again with `-unlock`.

L.5 Reading a stop

Bochs prints `Breakpoint K`, the selector:offset, and the instruction.

At a function entry the first two stack words are the return address: offset, then segment. The arguments follow, nearest the top first. That order matches a Ghidra `__stdcall16far` signature.

```
print-stack 24
  STACK 0x80477ac0 [0x0a60]   return offset
  STACK 0x80477ac2 [0x38f7]   return segment (11B0 this boot)
  STACK 0x80477ac4 [0x0c36]   first argument
```

A far pointer comes back in DX:AX. A word comes back in AX. To catch the return, break on the instruction after the call. `28FF:02E5` was the instruction after the `03C8` lookup on this boot.

L.6 Selectors after a reboot

Windows assigns an LDT selector when the NE segment is loaded. The numbers change every boot. Only a segment that has been touched has a real entry.

1. Start with `-dbglog`. Open the lesson so TB80BAS loads.
2. `info ldt`. Match Ghidra segments by size, not by the index formula.
3. Confirm with a relocation. A far call in the file has the Ghidra selector (`0x10B0` and the rest). The same word in the loaded bytes has the live selector.

The table for the boot of 2026-10-20 is `maps/bochs-20261020.md`. Do not reuse it.

L.7 One object load

Set these only after the lesson has been opened once, so the segments exist. Then open the page. The selectors are the 20 October boot.

```
vb 0x2a37:0x2174   book open
vb 0x2907:0x08b2   reader creator     handle, 0x096c, err
vb 0x391f:0x04da   entry read         reader far, 2, entry far, slot, err
vb 0x391f:0x08d6   read setup
vb 0x291f:0x02cc   file read          len, dest off, dest seg, pos lo, pos hi, file, err
vb 0x28ff:0x02ce   send message       0, 0, msg, class word, block, handle, err
vb 0x28ff:0x03c8   handler lookup
vb 0x28ff:0x02e5   lookup returned    DX:AX is the handler
vb 0x2907:0x0aeb   after message 0xE
```

Every object hits several of these. `bpd N` silences a stop that is already understood. The handle (`0x0C1E`, `0x0C36`, `0x0C4E`) names the object. Those three are `0x18` apart.

To dump the record: at `08D6`, note the block selector. After `02CC` returns, `info ldt N N` for that selector's decimal index, then `writemem` the base for `limit + 1` bytes.

The on-disk copy of entry index `I` is at file `0x10D6 + 24 * I`.

L.8 Pitfalls

What happened	What to do
Bochs will not start. Files locked.	Add `-unlock`.
Panic reading a ROM image.	The BIOS path in the config is wrong. Read the terminal.
Guest window stays small.	This X build does not scale. Not solved.
Mouse stuck on the zoom tool.	That was the desktop magnifier. Click the guest. Ctrl+F10.
`syntax error at print-stack`	More than one command was pasted. One line at a time.
`writemem` writes nothing.	The folder is missing, or the address is unmapped. `mkdir` first. `info ldt` first.
Suspend hangs.	Do not use it. Kill Bochs. Restart with `-unlock`.
The dump is the wrong segment.	The first data dump used LDT index 1161. The data segment was `3477` this boot.
The bytes look right and the selector is wrong.	Selectors changed. Section L.6.

L.9 Still to capture

Each of these needs the lesson opened again, so the selector map comes first.

1 The dispatcher `03C8` returns for handle `0x0C36`. The return already captured was handle `0x0C1E`, which is not the 412-byte record.

2 Twenty-four bytes of the live entry. On this boot that was `x /24xb` at the `3E67` base plus `0x0C36`. The file's last four bytes are `00 21 25 00`. They have not been seen in memory.

3 The file record at slot `0x096C` and file-table offset `0x0AE0`, with the book open, including the class-table pointer at `+0x163`.

4 The class table that pointer names.